

ECE 408



目录

第 1 讲 Introduction	1
1.1 课程框架与技术背景	1
1.2 课程运行框架、作业流程与学术规范	3
1.3 从摩尔定律时代到后登纳德时代	10
1.4 中央处理器与图形处理器的设计取向	15
1.5 图形处理器的起源、统一计算设备架构与应用扩张	20
1.6 并行化工作流与核心性能挑战	26
1.7 阿姆达尔定律：整体加速比的串行上界	30
1.8 综合框架与关键结论	34
第 2 讲 Introduction to CUDA C and Data Parallel Programming	37
2.1 核心内容	37
2.2 从线程到大规模数据并行	37
2.3 CUDA 的主机-设备执行模型	40
2.4 线程层次结构与一维索引映射	41
2.5 向量加法：从顺序循环到线程网格	44
2.6 核函数实现、启动配置与边界检查	47
2.7 函数限定符与 CUDA 编译流程	50
2.8 异步执行与错误检查	52
2.9 完整、可检查的向量加法程序	54
2.10 例题与完整推导	57
2.11 常见错误、检查清单与公式表	58
2.12 综合框架与关键结论	61
第 3 讲 CUDA Parallel Execution Model: Multidimensional Grids and Data	62
3.1 核心内容	62
3.2 多维执行层次与索引系统	64
3.3 多维启动配置与数据布局	68
3.4 实例一：RGB 彩色图像转灰度	72
3.5 实例二：邻域平均模糊	76
3.6 实例三：矩阵与矩阵乘法	80
3.7 例题与完整推导	85
3.8 统一方法与关键结论	88
第 4 讲 Compute Architecture and Scheduling	92
4.1 核心内容	92
4.2 GPU 计算架构与线程块驻留	93
4.3 线程块内同步、跨线程块同步与可伸缩性	96
4.4 线程束、处理块与单指令多线程执行	99
4.5 控制发散与有效执行通道	102
4.6 线程束调度与延迟隐藏	105
4.7 占用率、资源约束与线程块大小	108

4.8	运行时资源查询与例题	112
4.9	架构、调度与性能机制的统一关系	115
第 5 讲	CUDA Memory Model	117
5.1	课程安排与核心内容	117
5.2	从处理器执行到 CUDA 存储层次	118
5.3	基本矩阵乘法：从公式到线程映射	124
5.4	访存量、算术强度与 Roofline 性能分析	130
5.5	例题解析与知识归纳	136
第 6 讲	Data Locality and Tiled Matrix Multiply	140
6.1	课程事项与核心内容	140
6.2	数据局部性与分块矩阵乘法	142
6.3	线程块同步与正确执行顺序	148
6.4	边界条件与一般尺寸矩阵	151
6.5	分块收益与资源约束	158
6.6	共享内存接口与分块思想的推广	162
6.7	关键结论与公式汇总	165
第 7 讲	DRAM Bandwidth and other Performance Considerations	167
7.1	课程安排与核心内容	167
7.2	DRAM 组织与带宽	168
7.3	Warp 访存与共享内存分块	172
7.4	线程粒度、指令调度与同步	179
7.5	优化权衡与例题	187
第 8 讲	Convolution Concept; Constant Cache	194
8.1	课程安排与核心内容	194
8.2	卷积的计算结构与边界处理	195
8.3	缓存层次与常量存储器	200
8.4	分块的数据复用与三种策略	204
8.5	分块核函数与索引推导	207
8.6	综合算例与关键结论	212

第 1 讲

Introduction

1.1 课程框架与技术背景

内容概览

课程运行框架包括课程目标、资源平台、作业流程、评分方式、计算环境以及学术诚信要求。单核频率不再持续快速增长之后，现代计算系统转向**并行性**(*parallelism*)、**异构性**(*heterogeneity*)与**处理器专用化**(*processor specialization*)，图形处理器也成为这一转向中的核心加速器。

相关硬件趋势与软件责任构成以下因果链：

晶体管密度持续增加 + 频率缩放趋于停滞

⇒ 更多晶体管无法自动转化为更快的单线程执行

⇒ 多核、向量化与专用加速器成为主要性能路径

⇒ 软件必须显式暴露并行性，并控制负载、访存与串行瓶颈。

1.1.1 课程的技术定位

课程名称为应用并行编程，核心对象是**大规模并行处理器**(*massively parallel processor*)，主要以支持统一计算设备架构的英伟达图形处理器为实验平台。课程并不把处理器结构本身作为最终目标；架构知识只学习到足以解释程序性能的深度。换言之，处理器结构、编程接口、并行算法模式和性能测量在本课程中始终服务于同一个目标：把一个具有足够并行性的计算映射到实际硬件上，并使映射后的程序获得可解释、可复现且可扩展的性能。

课程目标包含以下五类技术内容：

- 并行编程的基本模型与术语；
- 常见并行算法的原则与模式；

- 编程接口、工具链与实现技巧；
- 影响性能的处理器的结构特征与约束；
- 以人工智能负载为代表的高价值应用。

关键结论

并行程序的质量不能只用“能否运行”衡量。一个合格的并行实现至少同时满足四项要求：结果正确、性能足够高、代码可维护、算法与实现能适应后续硬件代际。仅通过增加线程数得到一次性的速度提升，并不等于获得了可扩展的并行程序。

1.1.2 课程形成背景与计算平台脉络

美国国家超级计算应用中心的图形处理器系统建设构成了本课程的计算平台背景。课程教师长期从事高性能计算、计算加速器以及机器学习系统研究。三代具有代表性的系统分别是：2008年建设的加速集群使用较早期的图形处理器；2018年建设的面向人工智能的集群使用 V100 图形处理器；2024年建设的 DeltaAI 系统采用基于 GH200 超级芯片的 Grace-Hopper 节点。课程实验实际使用较早的 Delta 超级计算机，而不是 DeltaAI。

图形处理器属于更大的系统演化趋势：高性能计算系统逐步从以通用中央处理器为绝对中心的结构，转向由中央处理器负责控制与串行工作、由一个或多个加速器承担大规模并行计算的异构结构。

1.1.3 课程规则与技术导论的逻辑关系

课程运行规则与技术导论看似是两组独立内容，实际存在直接联系：

- 课程要求在 Delta 上编译并通过作业调度系统运行，是因为真实超级计算机由大量用户共享，计算资源必须被隔离和调度。
- 实验强调自己编写代码，是因为期中考试要求在纸面上完成与实验相近的程序片段；依赖自动代码生成无法形成独立实现能力。
- 项目同时评价功能与性能，是因为并行程序的价值来自正确性与性能的共同满足。
- 硬件代际快速变化，因此并行程序需要具备可移植性、可扩展性和可维护性，不能只适用于某一张图形处理器。

评估并行化方法时需要同时回答两个问题：第一，该方法为什么能提高吞吐量；第二，该方法在真实共享系统、未来硬件和完整应用中是否仍然成立。

1.2 课程运行框架、作业流程与学术规范

1.2.1 授课形式与支持体系

课程以线下授课为主，课堂同时录制并上传至学校媒体平台。实验又称**机器问题 (Machine Problem, MP)**，项目由学生在课外独立或按指定团队形式完成；两次期中考试均在线下进行。助教团队分别承担实验、项目和答疑工作，办公时间从第二周开始，其时间表由课程管理平台公布。

课程支持体系由多个平台共同构成。每个平台只承担一部分功能，因此应按任务类型选择入口。

平台	主要用途	使用要点
课程管理平台 (Canvas)	教学大纲、课件、录像链接、课程文件、项目报告与成绩	课件通常在上课前上传；最终成绩在此汇总。 打开课程页面 。
学校媒体平台 (Illinois Media Space)	课堂录像	录像发布到课程专用频道。 打开录像频道 。
代码托管平台 (GitHub)	实验与项目的基础代码、开发仓库和最终代码提交	先同步发布代码，再在 Delta 上编辑、编译和运行，最后把待评分版本提交到仓库。 打开发布仓库 。
在线评测平台 (PrairieLearn)	实验配套测验与概念问题	应在代码部分完成之后作答，因为部分问题依赖实际运行结果。
课程讨论平台 (Campuswire)	公告、学生问答、教师与同学共同讨论	面向班级显示时可采用匿名形式；不能把它当作提交受保护代码的渠道。 打开讨论区 。
课程人工智能问答系统 (course AI chatbot)	课程规则、统一计算设备架构概念、接口、编译与运行错误等问题	推荐先询问该系统；若结果不充分，再使用讨论区或办公时间。链接以课程管理平台的当期入口为准。

1.2.2 教材与技术文档

首选教材为 Wen-mei Hwu、David Kirk 与 Izzat El Hajj 编写的《大规模并行处理器编程：实践方法》(*Programming Massively Parallel Processors: A Hands-on Approach*) 第五版，2026 年出版。第四版于 2022 年出版，也被课程接受，并可通过学校订阅在线阅读。第三版及更早版本由于内容老化，不建议使用。

统一计算设备架构的主要文档包括：

- 《统一计算设备架构 C 编程指南》(*CUDA C Programming Guide*);
- 《统一计算设备架构 C++ 最佳实践指南》(*CUDA C++ Best Practices Guide*)。

Delta 环境安装 CUDA 13.2，文档参考涉及 13.2 与 13.3 两个版本。阅读文档时需核对文档版本与安装版本；实际使用时应以课程管理平台的当期公告为准。

1.2.3 典型周节奏与提交时间

前三周的示例安排如下：

表 1.2 前三周典型节奏

周次	课堂内容	课外任务与截止项
第 1 周	周二：第 1 讲“导论”；周四：第 2 讲“统一计算设备架构导论”	配置代码仓库与计算机访问权限。
第 2 周	周二：数据并行模型；周四：统一计算设备架构内存模型	实验 0：设备查询示例。
第 3 周	周二：继续讲解内存模型；周四：性能考虑	实验 1：向量加法示例。

部分周次会用项目检查点替代实验提交；项目被拆分为三个检查点，并采用逐步完成的交付方式。

所有实验和项目里程碑的正式截止时间均为**美国中部时间周五晚 8:00**。课程同时提供三天默认宽限期：在周一晚 8:00 之前提交仍可获得满分，且不需要提前通知。宽限期的制度含义必须准确理解：

- 正式截止时间仍是周五晚 8:00，周一不是新的常规截止时间；
- 如需超过三天的额外延长，必须在周五晚 8:00 之前给负责该作业的助教发邮件；

- 周五晚 8:00 之后提出的额外延长请求不予考虑，是否批准由课程团队根据特殊情况决定；
- 周一晚 8:00 时仓库中的代码被视为最终版本。即使较早提交过正确版本，评分也会取该时刻检出的仓库状态。

需要特别注意

共享超级计算机可能因系统维护或队列拥堵而延迟作业启动。宽限期为这类不可控情况提供缓冲，但它不能替代提前开始。若大量学生在周五集中提交，等待计算节点本身就可能消耗数小时。

1.2.4 评分结构与考核方式

课程总评由三部分组成：

表 1.3 总评构成

组成	权重	具体规则
期中考试	50%	两次期中考试各占 25%；第一次约覆盖前 11 讲，第二次约覆盖其余讲次。课程没有期末考试；期中考试为三小时纸笔考试，并要求手写与实验相近的代码。
实验	25%	自动数据集通过情况约占实验成绩的 90%，同时考查配套问题的正确性；最低的一个计分实验成绩被舍弃；实验 0 和实验 1 不计入总评。实验数量与各项权重以课程团队发布的详细量规为准。
项目	25%	演示、功能与编码风格约占项目分数的 50%；在完整功能成立的前提下，性能约占另外 50%。详细量规将在之后公布。

项目形式包括两种：一种围绕卷积神经网络若干层的实现；另一种由两至三人组队，实现大语言模型或变换器中的若干层。无论选择哪类项目，功能完整性都先于性能评分。一个输出错误但运行很快的程序不构成高性能实现，因为其计算语义已经改变。

纸笔考试与实验之间形成直接约束。实验中出现的程序结构可能在考试中以手写代码形式再次出现，因此完成实验的目标包含两层：一是获得正确输出，二是建立不依赖代码生成器或他人实现的独立编程能力。

1.2.5 实验的端到端 workflow

实验采用下列端到端流程：

1. 从课程发布仓库取得基础代码，并按说明把发布内容合并到个人仓库；
2. 在 Delta 的登录节点上编辑与编译代码；
3. 通过作业调度系统把程序提交到计算节点执行，不能在登录节点上直接运行重计算任务；
4. 根据正确性测试与运行结果继续调试，直到代码满足作业要求；
5. 将最终版本提交到代码托管仓库，供评分流程检出；
6. **代码部分完成后**，再回答在线评测平台中的配套问题。

最后一步的顺序不是形式要求。实验问题往往询问实际输出、性能现象或实现细节；在程序尚未完成时作答，会把推测当成测量结果，也更容易忽略题目中隐含的前提。

1.2.6 Delta 超级计算机与作业调度

Delta 是由美国国家超级计算应用中心运行、通过高等科学与工程研究计算环境分配资源的超级计算机。课程使用其中配备 A40 图形处理器的子系统。四卡计算节点规格如下。

NCSA Delta Supercomputer

I

- Delta is a dedicated, ACCESS-allocated resource designed by HPE and NCSA, delivering a highly capable GPU-focused compute environment for GPU and CPU workloads.



4-Way A40 GPU Compute Node Specs	
Specification	Value
Number of nodes	100
GPU	NVIDIA A40
GPUs per node	4
GPU Memory (GB)	48 DDR6
CPU	AMD Milan
CPU sockets per node	1
Cores per socket	64
Cores per node	64
Clock rate (GHz)	~ 2.45
RAM (GB)	256

ELECTRICAL & COMPUTER ENGINEERING

GRAINGER ENGINEERING

图 1.1 Delta 四卡 A40 子系统

表 1.4 四卡 A40 计算节点规格

项目	数值
节点数量	100
图形处理器型号	英伟达 A40
每节点图形处理器数量	4
每卡显存	48 GB
中央处理器	AMD Milan
每节点中央处理器插槽数	1
每插槽核心数	64
每节点核心总数	64
近似时钟频率	2.45 GHz
主存容量	256 GB

Delta 是多用户共享系统。用户首先连接到**登录节点 (login node)** 或**头节点 (head node)**，在这里管理文件、编辑代码和编译程序。实际计算通过**作业调度系统 (job scheduler)** Slurm 提交到**计算节点 (compute node)**。调度器根据请求的资源、队列状态与系统策略决定作业何时启动。

该共享系统具有以下运行特性：

- 用户可能登录到多个登录节点中的任意一个；这些节点共享文件系统与调度队列，因此登录节点编号变化不影响文件可见性。

- 系统繁忙时，作业可能等待数分钟、数小时，项目级大任务甚至可能等待更久。
- 课程作业通常请求一张图形处理器，而一个计算节点包含四张 A40，因此同一节点上最多可同时存在若干用户作业。
- 调度器对分配给不同作业的图形处理器进行隔离；一个作业不能使用分配给另一个作业的设备。
- 登录节点面向许多并发用户，不能把它当作绕过调度器的计算节点。

关键结论

在共享超级计算环境中，“程序运行时间”与“从提交到拿到结果的墙钟时间”不是同一个量。前者是作业实际执行时间，后者还包含排队等待。课程截止时间约束的是完整 workflow，因此排队风险必须通过提前提交来管理。

1.2.7 Delta 账户与登录步骤

账户建立需要经过高等科学与工程研究计算环境和 Delta 两层系统，不能在截止日前临时完成。具体步骤如下：

1. 在[账户注册页面](#)创建用户；已有账户者继续使用原用户名，不应重复注册。
2. 新建账户时把“伊利诺伊大学厄巴纳-香槟分校”选作[身份提供方 \(Identity Provider\)](#)，以学校凭据完成认证。
3. 记录由系统分配的用户标识，并通过[课程表单](#)提交给课程团队。
4. 等待课程团队把用户加入课程资源组，再等待 Delta 端完成账户创建。典型等待时间为 24–48 小时，完整传播过程可能更长。
5. 收到确认邮件后，通过安全外壳协议连接：

```
ssh yourusername@login.delta.ncsa.illinois.edu
```

6. 登录需要 Duo 双因素认证；随后克隆课程仓库并按仓库说明配置环境。

1.2.8 课程人工智能问答系统的技术机制

课程问答系统采用**检索增强生成 (Retrieval-Augmented Generation, RAG)**。它不只依赖语言模型内部参数回答问题，而是先从课程材料中检索相关片段，再把这些片段作为上下文交给语言模型。

建模说明

其基本数据流可表示为

课程文档 → 分段 → 嵌入向量 → 向量数据库，
学生问题 → 问题嵌入 → 相似片段检索，
→ 上下文拼接 → 大语言模型 → 回答与来源指向。

其中，嵌入向量 (embedding) 用数值向量表示文本语义，向量数据库 (vector database) 用于相似性检索，大语言模型 (large language model) 根据检索上下文生成回答。向提示词中放入大量课程上下文称为**上下文填充 (context stuffing)** 或**超长提示构造 (mega-prompting)**。系统可通过应用程序编程接口调用模型，也可使用本地运行的开源模型。

检索增强只能提高回答与课程材料的一致性，不能保证每个答案都正确。系统可能遗漏题目隐含假设、检索到不完整片段，或在生成过程中引入错误。课程明确规定：学生对最终代码与答案承担责任，不能把“问答系统给出了该答案”作为重新评分的依据。

1.2.9 允许与禁止的协作和人工智能使用

课程允许概念层面的讨论，也鼓励学生利用问答系统学习接口与技术；同时严格禁止把实现责任转移给他人或人工智能工具。边界可按下表理解。

允许	禁止
与同学讨论思路、算法方向、所用函数以及调试策略；接受修过课程者的口头建议。	查阅往年作业实现或网络上发布的本课程作业答案。
询问统一计算设备架构接口的含义，例如内存复制函数的作用、内核函数的调用方式。	要求人工智能系统直接生成某个实验或项目的实现代码。
询问编程技术，例如共享内存数组如何声明、运行时如何指定动态共享内存大小。	复制超过一行左右的非平凡代码；让他人提供代码同样会受到处罚。

允许	禁止
询问并行编程概念与性能优化原理，例如分块为什么能改善全局内存带宽。	阅读他人代码或人工智能生成的代码后，再离开原文“独立重写”；课程把这种行为也视为复制。
询问编译错误、运行时错误的含义，并学习系统化调试方法。	把完整程序粘贴给问答系统，让系统代替自己定位并修复错误。
进行期中复习与一般课程规则查询。	把实验测验题直接粘贴给问答系统，或在考试中给予、接受帮助。
逐行理解并按当前问题改写课堂明确提供的代码示例。	故意规避实验要求，或只做表面变量改名、语句重排来掩盖复制。

“非平凡代码”通常指超过一行左右的实现。代码相似性检测不只比较变量名，也能识别重排、改名等表面变化。允许引用课堂示例的前提是逐行理解并针对当前程序进行实质性改写，而不是整段搬运。

学术不端的后果为：第一次在相应作业或考试上记零分，并通过学校的 FAIR 系统向学院报告；重复违规将导致课程自动不及格。提供代码者与接收代码者都可能受到相同处罚，因为相似性检测并不能可靠判断谁最先写出该实现。

需要特别注意

课程对人工智能工具的限制不是“完全禁止使用”。允许用途集中在概念、接口、错误解释与复习；禁止用途集中在替代学生完成受评分实现。判定边界时应问：该工具是在帮助理解问题，还是在产生本应由学生独立完成的答案或代码？

1.3 从摩尔定律时代到后登纳德时代

1.3.1 两条经常被混淆的缩放规律

现代并行计算的硬件动因必须从两条历史趋势区分开来。

表 1.6 摩尔定律与登纳德缩放的作用区分

概念	核心含义	对性能的历史影响
摩尔定律 (Moore's Law)	制造特征尺寸不断缩小，使单位面积可容纳的晶体管数量约每 18–24 个月翻倍。它是经验性“定律”，也可视为产业目标。	提供更高的元件密度，使芯片能够容纳更多功能单元、缓存和控制逻辑。它本身并不等价于单线程性能翻倍。
登纳德缩放 (Dennard Scaling)	随器件尺寸缩小，晶体管能够在更高时钟频率下工作，是推动频率上升的物理基础。	使单个处理器核心能够更快执行同一指令流，是早期“相同代码自动变快”的重要来源。

大众叙述常把处理器性能增长统称为“摩尔定律”，但历史上的快速性能增长实际来自摩尔定律与登纳德缩放的共同作用。晶体管数量增加提供了更复杂的处理器结构，时钟频率提高则直接缩短许多指令的执行时间。两者叠加后，通用计算机在约三十年内表现出大致每 18 个月翻倍的性能趋势。

1.3.2 “自动性能增长”对软件的影响

在这一时期，软件常能在不改变算法与并行结构的情况下获得显著加速。一个今天运行过慢的串行程序，经过一到两个处理器代际后，只需重新编译或迁移到新机器，就可能明显变快。由此形成了三项长期软件预期。

1.3.2.1 同一份软件随硬件代际自动加速

程序不需要显式使用更多线程，也能受益于更高频率、更好的流水线与更强的单核结构。在这种历史环境中，“代码不够快，等待两年”一度可以成立；它不是现代系统中的可靠开发策略。

某些程序较早暴露了这一策略的边界。若程序主要受内存访问延迟 (*memory-access latency*) 限制，例如部分稀疏、非结构化或图算法，处理器算术速度提高并不会按相同比例缩短运行时间。此类程序需要的数据访问不规则，缓存难以持续命中，因此内存系统可能先成为瓶颈。

1.3.2.2 内存从稀缺资源变为多数应用中的充足资源

芯片密度增加也扩大了可用内存容量。早期程序常需要显式把数据分块，在磁盘与主存之间来回搬运；随着系统能够提供数百吉字节乃至更大容量，许多应用可以把完整数据集常驻内存。这一变化直接改变了算法设计：过去为节省内存而使用的复杂外存策略，在新的容量条件下可能不再必要。

1.3.2.3 软件可移植性成为默认期待

如果相同程序可以在多个处理器代际上继续运行并自动加速，那么软件稳定性和可移植性(*portability*)自然具有很高价值。应用代码可以长期演进，而不必每次为了新硬件彻底重写。可扩展且可移植的软件能够跨越多个硬件代际；现代并行软件必须通过更主动的设计才能维持这种寿命。

1.3.3 专用计算机在早期为何难以长期保持优势

专用处理器可以针对某类计算取得初始性能优势，但在通用处理器性能约每 18 个月快速增长的时期，该优势可能在数年内被下一代通用微处理器追平。专用芯片的设计、制造与软件生态成本很高，如果其目标市场不足以支撑持续迭代，就难以与通用处理器的规模经济竞争。

这种现象称为“杀手级微处理器的冲击”(Attack of the Killer Micros)：许多试图领先于摩尔定律与登纳德缩放组合速度的计算机公司，最终被快速演化的通用微处理器压缩了生存空间。图形处理器之所以比许多其他专用加速器更容易形成长期生态，也与这一背景有关。

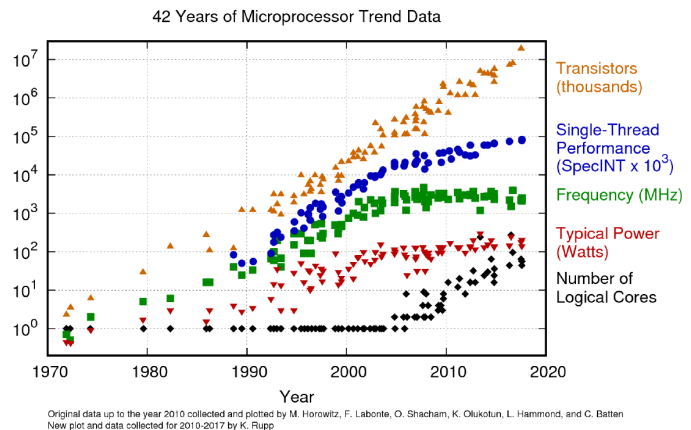
1.3.4 轻松性能增长的结束

约在 2005–2006 年，登纳德频率缩放停止延续原有趋势。过去可以忽略的漏电流(*leakage current*)变得重要，功耗与散热约束使时钟频率无法继续按历史速度增加。处理器频率大体停留在约 2.5 GHz to 3.5 GHz 的范围；近二十年后，频率数量级仍没有发生根本变化。

与此同时，晶体管数量仍在增加，尽管摩尔定律本身也在放缓。于是出现了新的设计矛盾：芯片可以容纳更多晶体管，却不能简单地把它们全部用于提高一个核心的时钟频率。图 1.2 中，晶体管数量继续上升，而频率曲线在 2000 年代中期后明显趋平；逻辑核心数量随后开始增长。

End of Easy Performance Gains

- Dennard (Frequency) Scaling ended around 2005-2006
 - Previously neglected effect, leakage current, became important
 - Clock speeds stagnated at 2.5-3.5 GHz
 - Almost 20 years later, clock speeds are nearly the same
- But the expectation of doubling of performance was strong
 - And Moore's "Law", though slowing down, is still providing higher component density
- Maintaining performance expectation required doing something different
 - Instruction parallelism (vectors)
 - Multiple cores per "processor"
 - Specialization, especially to vectors/streams/GPUs



Slides courtesy of Prof. William Gropp, NCSA Director



NCSA | NATIONAL CENTER FOR SUPERCOMPUTING APPLICATIONS

23

图 1.2 单纯依靠频率获得性能增长的时代结束

关键结论

登纳德缩放结束后的关键变化是：硬件仍可能提供更多计算资源，但串行程序不会自动使用这些资源。新增晶体管被组织为更多核心、更多向量执行单元或专用加速器之后，软件必须暴露与之匹配的并行工作，才能把硬件规模转化为应用性能。

1.3.5 后登纳德时代的三条主要性能路径

维持性能增长主要有三种方向。

1.3.5.1 指令级与向量并行性

向量执行 (vector execution) 让一条指令对多个数据元素执行相同运算。其基本思想是把规则、重复的数据运算打包，使同一控制流驱动多个算术通道。后续图形处理器设计继承并扩大了这种“相同操作作用于不同数据”的思想。

1.3.5.2 多核处理器

当单个核心无法继续大幅提高频率时，可以在同一处理器中放置多个核心。每个核心执行自己的指令流，共享部分缓存、内存与互连资源。多核设计提高了芯片可并行执行的工作量，但单线程程序通常仍只使用一个核心，因此程序需要被改写为多线程或多进程形式。

现代处理器可包含上百个核心。即使硬件核心数从 1 增长到 100，一个只含单一串行指令流的程序也不会自动获得 100 倍加速。核心数量是可用资源，不是无条件性能保证。

1.3.5.3 处理器专用化

通用中央处理器追求“几乎所有代码都能运行，并在多数工作负载上获得尚可到优秀的表现”。专用处理器则主动放弃一部分通用性，把更多芯片面积用于某类操作或数据流。处理器专用化包括以下方向：

- 使用更简单的轻量级核心并增加核心数量，例如高性能计算中的蓝色基因系统与至强融核；
- 针对向量、流式数据或图形运算构建专用结构；
- 在移动芯片中同时配置中央处理器、图形处理器和神经网络处理单元 (Neural Processing Unit, NPU)，分别承担通用控制、图形与特定机器学习运算；
- 在超级计算机中由通用中央处理器协调，由图形处理器承担大规模数值计算。

1.3.6 并行性与异构性的共同转向

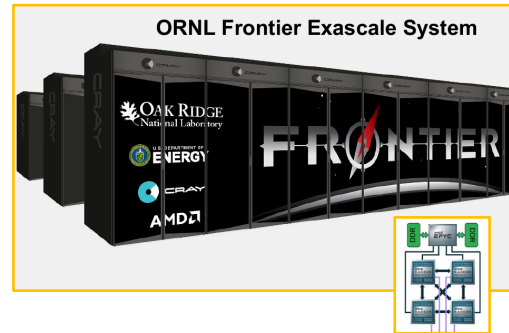
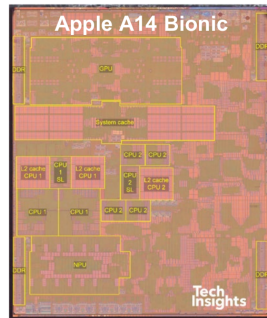
异构计算 (*heterogeneous computing*) 是指同一系统包含多种结构不同、性能特征不同的处理器，并把应用的不同部分映射到最合适的设备。它与并行性相互依赖：专用加速器往往通过大量并行执行单元取得吞吐量优势，而应用必须提供足够规则的并行任务才能利用这些单元。

移动芯片 Apple A14 Bionic 与橡树岭国家实验室 Frontier 百亿亿次级系统都采用加速器架构，说明这种结构从最小规模的移动设备一直延伸到最大规模的超级计算机。

The Trend is Across the Board



- From the smallest to the largest, computing today involves accelerators architectures



ELECTRICAL & COMPUTER ENGINEERING

GRAINGER ENGINEERING

图 1.3 加速器架构贯穿移动设备与超级计算机

这一转向改变了软件责任分配。在早期“自动性能增长”环境中，硬件升级可以替软件完成大量加速工作；在后登纳德时代，硬件只提供并行资源与专用能力，程序员必须识别可并行部分、选择可扩展算法、组织数据并消除瓶颈。

1.4 中央处理器与图形处理器的设计取向

1.4.1 先区分延迟与吞吐量

比较中央处理器和图形处理器之前，必须先区分两个性能指标。

- 延迟 (latency)** 描述单个任务从开始到完成所经历的时间。若任务在时刻 t_{start} 开始、在 t_{finish} 完成，则

$$L = t_{\text{finish}} - t_{\text{start}}$$

延迟越低，单个请求越快得到结果。

- 吞吐量 (throughput)** 描述单位时间内完成的任务数量。若时间区间 Δt 内完成 N 个任务，则平均吞吐量为

$$R = \frac{N}{\Delta t}$$

吞吐量越高，系统在一段时间内处理的总工作越多。

低延迟与高吞吐量并不等价。一个流水线可以让单个任务经过多个阶段，因此单任务延迟较长；当流水线填满后，各阶段同时处理不同任务，系统仍可在每个周期完成一个或多个结果，从而获得高吞吐量。

建模说明

设一条理想流水线包含 d 个阶段，时钟频率为 f 。单个任务的流水线延迟约为 d/f ，但流水线达到稳态后，每条执行通道的理想结果产生速率可以接近 f 。增加流水线深度可能不降低单任务延迟，却能支持更高的稳态工作速率，从而形成图形处理器“长延迟、高吞吐量”的组合。

1.4.2 中央处理器：面向低延迟的设计

中央处理器采用面向延迟的设计 (*latency-oriented design*)，主要优化少量指令流的快速完成，其芯片面积通常分配给以下结构。

1.4.2.1 少量但功能强的算术逻辑单元

算术逻辑单元 (*Arithmetic Logic Unit, ALU*) 执行整数、逻辑及部分数值运算。中央处理器核心包含较少但较复杂的执行单元，目标是降低单条指令或短指令序列的完成时间。复杂的乱序执行与多种执行端口也属于同一低延迟取向。

1.4.2.2 较大的缓存层次

主存访问延迟远高于算术操作。中央处理器投入大量芯片面积构建多级**缓存** (*cache*)，把频繁访问的数据保留在靠近执行单元的位置。缓存命中时，原本较长的主存访问被转换为较短的缓存访问，从而降低单线程停顿。

1.4.2.3 复杂控制逻辑

中央处理器使用复杂控制结构减少依赖与分支带来的等待：

- **分支预测** (*branch prediction*) 提前猜测控制流方向，尽量避免处理流水线因等待分支结果而空闲；
- **数据转发** (*data forwarding*) 把某条指令刚产生的结果直接送给后续相关指令，减少等待数据写回再读取的时间；

- 适度多线程用于隐藏相对较短的延迟，但线程数量通常远少于图形处理器。

中央处理器还通常采用较高时钟频率。综合这些机制，它非常适合控制密集、分支复杂、串行依赖明显或任务规模较小的程序部分。

1.4.3 图形处理器：面向高吞吐量的设计

图形处理器采用**面向吞吐量的设计**(*throughput-oriented design*)，主要优化大量相似任务的总完成速率。

1.4.3.1 大量较小的算术单元

图形处理器把更多芯片面积用于计算通道，因而能同时保持大量算术操作在执行流水线中。单个操作或单个线程的延迟可能高于中央处理器，但当工作规模足够大时，所有线程的总完成时间可以显著缩短。

1.4.3.2 深流水线与海量线程

图形处理器通过深流水线提高稳态吞吐量，并维护大量可执行线程。当一组线程等待长延迟事件，例如全局内存数据返回时，调度器可以选择另一组已经具备操作数的线程执行。

这个机制称为**延迟隐藏**(*latency hiding*)。它没有缩短那次内存访问本身的物理延迟；它通过让其他工作占用原本会空闲的执行周期，减少延迟对总体吞吐量的影响。若程序没有足够多的独立线程，或所有线程同时等待同一瓶颈，延迟就无法被有效隐藏。

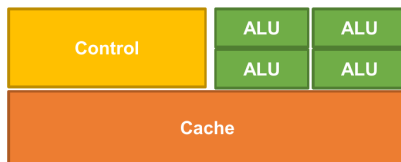
1.4.3.3 相对较小的缓存与较简单的控制

图形处理器也包含缓存，但相对于中央处理器，它把更大比例的面积用于算术单元、寄存器与线程执行资源。控制结构也偏向规则执行，因为最有效的工作负载通常让许多线程对不同数据执行相同或相近的指令序列。其时钟频率通常适中，性能主要来自并行宽度与总吞吐量，而不是极高频率。

Processor Design Approaches



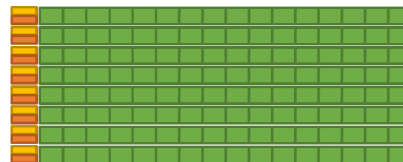
CPU: Latency-Oriented Design



- A few powerful **ALUs**
 - Reduced operation latency
- Large **caches**
 - Convert long latency memory accesses to short latency cache accesses
- Sophisticated **control**
 - Branch prediction to reduce control hazards
 - Data forwarding to reduce data hazards
- Modest multithreading to hide short latency
- High clock frequency

ELECTRICAL & COMPUTER ENGINEERING

GPU: Throughput-Oriented Design



- Many small **ALUs**
 - Long latency, high throughput
 - Heavily pipelined for further throughput
- Small **caches**
 - More area dedicated to computation
- Simple **control**
 - More area dedicated to computation
- Massive number of threads to hide the very high latency!
- Moderate clock frequency

GRAINGER ENGINEERING

图 1.4 中央处理器的低延迟取向与图形处理器的高吞吐量取向

1.4.4 芯片面积分配反映了不同目标

图 1.4 的方块图不是精确的晶体管布局，而是面积预算的概念图。中央处理器把大量面积用于缓存和复杂控制，以减少少量线程的等待；图形处理器把更多面积复制为大量计算通道，以提高规则并行工作负载的总处理速率。

这种权衡可以从机会成本理解。若增加缓存和控制逻辑，就会减少可放置的算术通道；若增加算术通道，就必须接受较少的单线程优化资源。两类处理器都不是绝对“更快”，它们针对不同目标函数优化。

两张芯片照片可以进一步说明粒度差异：第十代英特尔酷睿处理器 (10th Gen Intel Core processor) 在 14 nm 工艺下包含 10 个中央处理器核心；英伟达 GK110 在 28 nm 工艺下标有 2880 个统一计算设备架构核心。较老工艺的 GK110 仍能容纳数量大得多的“核心”，正是因为这里的统一计算设备架构核心远小于完整中央处理器核心。该对比用于说明结构粒度，不能仅按数字推断两张芯片的通用性能。

表 1.7 中央处理器与图形处理器的高层对比

维度	中央处理器	图形处理器
主要目标	降低单个线程或请求的延迟	提高大批相似任务的总吞吐量
算术资源	少量、复杂、功能强的执行单元	大量、较小、深流水化的执行单元
缓存	较大，用于降低平均访存延迟	相对较小，把更多面积留给计算资源
控制	分支预测、数据转发等复杂机制	偏向规则控制和大规模线程调度
并发方式	少量强线程，适度多线程	海量线程，通过切换隐藏长延迟
典型优势部分	串行依赖、复杂控制、低延迟响应	数据并行、规则数值计算、高吞吐批处理

1.4.5 “核心”一词在两类处理器中并不等价

核心(*core*)在两类处理器中具有不同的结构粒度和语义。

中央处理器核心是相对完整的处理单元，通常包含多个执行单元、寄存器、至少一级私有缓存和复杂控制逻辑。它能够独立推进一个或多个通用指令流。

英伟达所称的**统一计算设备架构核心(CUDA core)**则是更细粒度的算术数据通路，可以近似理解为流式处理器中的一个算术执行单元。它不具备中央处理器核心那样完整的缓存与控制结构。因此，“10 个中央处理器核心”与“2880 个统一计算设备架构核心”不能按核心数量直接比较；两者的粒度、能力和组织方式完全不同。

1.4.6 高性能系统通常同时使用中央处理器与图形处理器

异构系统中的任务划分原则是：

- 对延迟敏感、难以并行化的串行部分由中央处理器执行；
- 对吞吐量敏感、含有大量相互独立工作的并行部分由图形处理器执行。

两类处理器在各自擅长的代码上可能相差十倍以上。该数值是概念性数量级说明，不是对任意程序的固定保证。实际加速取决于可并行比例、数据传输、内存访问、分支规则性、问题规模和实现质量。

一个典型异构应用的执行过程是：中央处理器读取与准备数据、决定计算阶段和发起图形处理器任务；图形处理器执行规模最大的并行计算；结果再被中央处理器用于后处理、控制或输出。两种设备之间的协同本身会引入数据移动与同步开销，因此正确的任务划分不仅

要比较计算速度，还要判断把任务迁移到加速器是否足以抵消这些额外成本。

关键结论

中央处理器与图形处理器的根本差异可以概括为：中央处理器用更多硬件资源缩短少数线程的关键路径，图形处理器用更多并行执行资源扩大单位时间内完成的工作总量。图形处理器编程的首要条件不是“有很多算术”，而是“有很多能够同时推进、且执行结构足够规则的算术”。

1.5 图形处理器的起源、统一计算设备架构与应用扩张

1.5.1 图形工作负载天然具有大规模数据并行性

图形处理器最初为图形渲染而设计，因此名称中的“图形”并不是历史偶然。屏幕包含大量像素，许多像素可以在相互独立或弱依赖的条件下执行相近的颜色、几何与纹理计算。把相同计算作用于大量不同数据元素，正符合高吞吐并行硬件的设计取向。

2007 年以前，图形处理器主要通过图形应用程序编程接口使用，例如**开放图形库 (OpenGL)** 与 **直接三维图形接口 (Direct3D)**。研究者若想让图形处理器完成非图形数值计算，必须把问题重新表述为图形流水线能够接受的形式，即把一般计算改写成“为某个像素着色的函数”，再把渲染输出解释为所需数值结果。

这种方法证明了图形处理器具有通用数值计算潜力，但编程过程高度间接，算法表达受图形接口约束。它促使硬件与软件接口共同演化。

GPU Origins



- GPUs were originally built for graphics
 - Hence the name, Graphics Processing Unit
 - Graphics workloads process a massive number of pixels in parallel
- Before 2007, the only way to program GPUs was using graphics APIs
 - e.g., OpenGL, Direct3D
 - Using GPUs for other than graphics meant reformulating a computation as a function that paints a pixel
- In 2007, NVIDIA released CUDA
 - A programming interface for using GPUs in a general-purpose way
 - Requires extensions to the GPU architecture



ELECTRICAL & COMPUTER ENGINEERING

GRAINGER ENGINEERING

图 1.5 图形处理器从图形接口走向通用计算的历史转折

1.5.2 2007 年的统一计算设备架构转折

2007 年，英伟达发布**统一计算设备架构** (*Compute Unified Device Architecture, CUDA*)。它为图形处理器提供面向通用计算的编程接口，使程序员可以直接表达线程、内核函数、设备内存与数据传输，而不必把数值问题伪装成图形渲染。

这一转折通常称为**通用图形处理器计算** (*General-Purpose Computing on Graphics Processing Units, GPGPU*)。它包含两方面变化：

- 软件层面提供适合 C/C++ 程序员使用的语言扩展、运行时接口与工具链；
- 硬件层面增加或调整支持通用加载、存储、控制与数值计算的结构，使图形处理器不再只适合固定图形流水线。

统一计算设备架构 C/C++ 程序通常同时包含中央处理器代码和图形处理器代码：中央处理器端创建数据、分配设备资源并发起计算；图形处理器端执行由大量线程组成的**内核函数** (*kernel*)。这种主机端与设备端的协作构成统一计算设备架构程序的高层执行模型。

1.5.3 为什么沿用图形处理器，而不是从零设计另一种吞吐处理器

先进芯片的设计、验证、制造与软件生态建设成本极高，需要足够大的销量分摊固定成本。完全新建一种大规模并行处理器，即使技术上可行，也必须同时建立市场、编译器、工具、库和开发者生态。

图形处理器在并行计算进入主流之前已经拥有庞大的游戏与图形市场。这个既有装机基础带来三项先发优势：

1. 规模化销量能够分摊芯片研发与制造成本；
2. 图形工作负载已经推动硬件形成大规模并行执行能力；
3. 既有产品迭代、驱动与开发工具可以逐步扩展到科学计算和人工智能，而不必从零建立完整生态。

因此，图形处理器的成功来自技术结构与市场结构的共同作用。它既具备适合吞吐计算的硬件基础，也拥有其他专用加速器往往缺乏的持续商业规模。

1.5.4 数据中心与人工智能推动通用图形处理器计算扩张

英伟达 2019–2024 年按市场部门划分的收入曲线反映了通用图形处理器计算的扩张。图中数据中心收入在后期快速上升，这一变化与人工智能驱动的训练和推理需求密切相关。

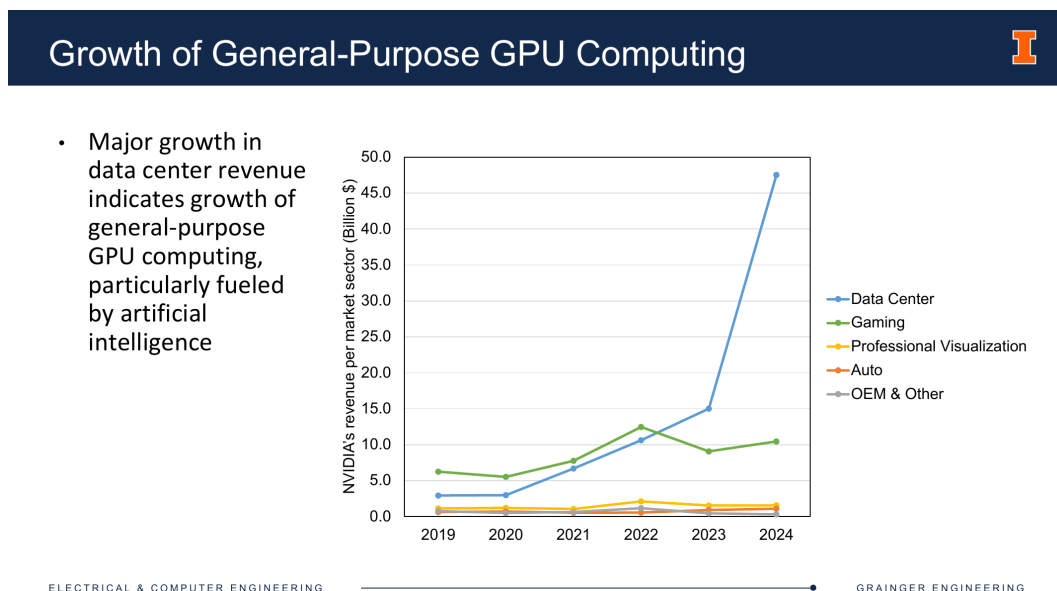


图 1.6 数据中心收入增长与通用图形处理器计算的扩张

该图反映了应用重心的变化：图形处理器仍服务于游戏与可视化，但数据中心、高性能计算和人工智能已成为其重要使用场景。模型训练需要执行大规模矩阵与张量运算；模型推理则把类似计算转化为持续服务需求。两者都可产生规模巨大且结构相对规则的并行工作。

1.5.5 人工智能计算需求的数量级增长

若干机器学习系统所需的浮点运算数量表明，从早期模型到大型语言与多模态模型，计算量跨越了许多数量级，最高约为 10^{24} 次浮点运算。

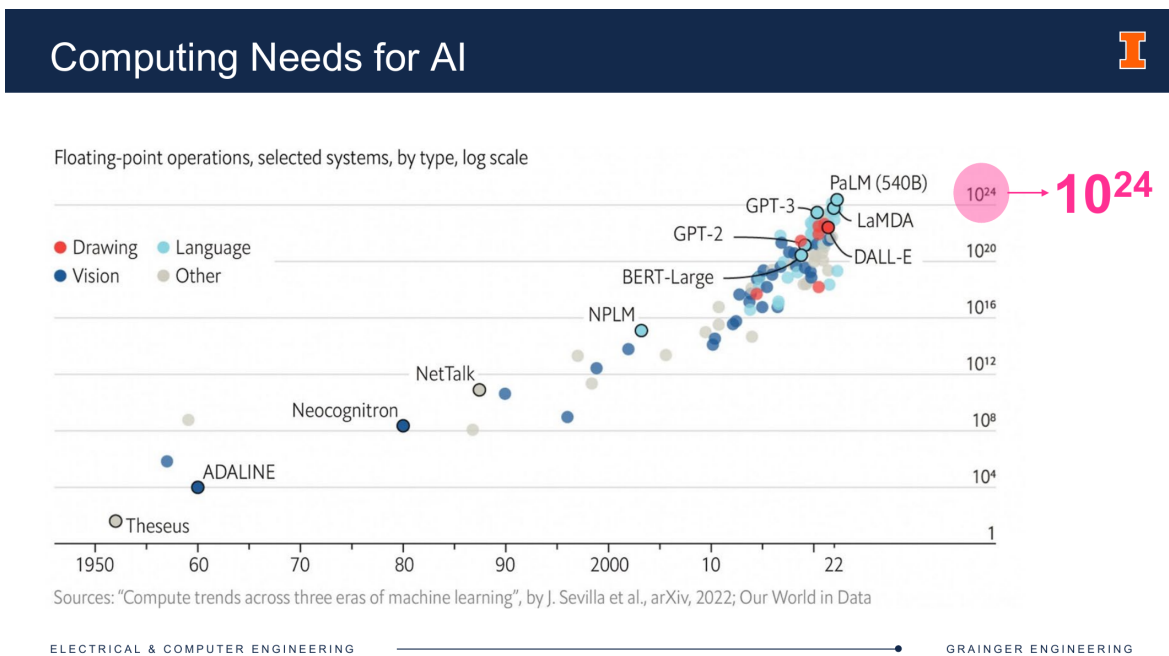


图 1.7 人工智能系统计算需求增长趋势

对数坐标上的纵向小幅移动代表乘法级增长。例如从 10^{16} 增长到 10^{20} 不是增加四个单位，而是增加 10^4 倍。大模型计算需求的这种数量级变化，使单纯依靠单核性能提升不可行，也使高吞吐加速器、多设备并行与大规模系统调度成为必要条件。

1.5.6 超级计算机中的加速器普及

全球超级计算机五百强榜单依据大规模线性代数基准对系统进行排序，通常每半年更新一次，分别对应年中和年末的主要超级计算会议。2026 年 6 月榜单的前五名中，第一名使用自定义多核处理器；第二至第五名分别明确使用 AMD Instinct MI300A、AMD Instinct MI250X、英特尔数据中心图形处理器 Max 和英伟达 GH200 超级芯片。这个样本说明，顶级系统通常依靠大量加速器获得主要浮点计算能力。

Prevalence of GPUs in Supercomputers



• June 2026 Top-500 List

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	LineShine - LingKun, LX2 304C 1.55GHz, LingQi, Kylin OS, Shenzhen Cloud Computing Center Co., Ltd. National Supercomputing Centre in Shenzhen (NSCS), China	13,789,440	2,198.40	2,735.82	42,220
2	El Capitan - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL, United States	11,340,000	1,809.00	2,821.10	29,685
3	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory, United States	9,066,176	1,353.00	2,055.72	24,607
4	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory, United States	9,264,128	1,012.00	1,980.01	38,698
5	JUPITER Booster - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, Bull EuroHPC/FZJ, Germany	4,801,344	1,000.00	1,226.28	15,794

<https://www.top500.org>

图 1.8 2026 年 6 月全球超级计算机五百强前五名

中央处理器仍负责操作系统、作业管理、控制流和部分串行计算；加速器负责高密度并行数值工作。要让几十年前为串行处理器编写的科学代码利用这些系统，必须重构算法、数据布局 and 并行分解。硬件峰值只是潜在能力，应用性能来自软件对硬件的有效映射。

1.5.7 并行编程框架之间的关系

统一计算设备架构是并行编程体系中的一种平台；其他常见框架分别提供跨设备编程、指令式卸载、分布式通信或共享内存并行等抽象。

表 1.8 常见并行编程框架

框架	主要目标	与统一计算设备架构的关系
统一计算设备架构	英伟达图形处理器的通用并行编程平台	本课程的主要编程环境。
开放计算语言 (OpenCL)	面向多类处理器与加速器的开放并行编程接口	提供另一类图形处理器并行编程框架。
开放加速器编程指令 (OpenACC)	通过编译指令把循环与数据区域卸载到加速器	可用于较高层次的加速器编程，本课程不以它为主线。
消息传递接口 (Message Passing Interface, MPI)	分布式内存系统中进程之间的显式通信	可与统一计算设备架构组合，在多个计算节点和多张图形处理器之间分配工作。
开放式多处理 (Open Multi-Processing, OpenMP)	共享内存系统中的线程级并行与编译指令	可与统一计算设备架构组合，协调单节点内的中央处理器线程及多设备工作。

单张图形处理器只解决一个设备内部的并行执行。若应用要扩展到多个节点，就需要额外机制完成进程间数据交换、任务划分和同步；消息传递接口承担这类分布式协调。若应用在单个共享内存节点内使用多个中央处理器线程或多个加速器，开放式多处理可以参与上层协调。统一计算设备架构、消息传递接口和开放式多处理因此可以处于同一应用的不同层次，而不是相互排斥的替代方案。

关键结论

图形处理器成为通用加速器的完整原因链为：图形渲染提供天然大规模数据并行性；游戏市场提供持续销量与研发投入；统一计算设备架构把硬件能力暴露为通用编程模型；科学计算与人工智能又提供了规模更大的规则并行工作负载。四个条件共同形成了今天的图形处理器计算生态。

1.6 并行化工作流程与核心性能挑战

1.6.1 从“程序太慢”到可验证的并行实现

当已有程序运行过慢时，需要找出耗时部分，把适合的部分放到图形处理器上，并持续优化直到性能满足目标。并行化应形成测量驱动的闭环，而不能从“直接重写全部代码”开始。

1.6.1.1 建立正确的串行基线

首先保留一个能够产生正确结果的基线版本，并记录代表性输入上的运行时间。基线提供两种参照：一是验证并行版本是否保持相同计算语义，二是定义加速比的分母。没有可靠基线，后续的“更快”可能来自减少计算、降低精度或产生错误结果。

1.6.1.2 性能剖析并识别热点

使用性能剖析 (*profiling*) 工具测量各函数、循环或阶段的时间占比，找出真正主导总运行时间的热点 (*hotspot*)。如果某段代码只占总时间的 1%，即使把它加速 100 倍，对整体性能贡献也很有限；第 1.7 节将用阿姆达尔定律严格表达这一点。

剖析还应区分计算、内存访问、数据传输和同步等待。某一函数“耗时长”不一定表示算术量大，也可能表示它在等待数据或共享资源。

1.6.1.3 判断原算法能否暴露可扩展并行性

有些算法可以直接把独立迭代分配给不同线程；另一些算法包含长依赖链，原始形式不适合并行执行。此时需要采用或重新设计可扩展算法 (*scalable algorithm*)。可扩展性意味着：问题规模和处理资源增加时，可同时利用的有用工作也能增加，而串行协调、通信与资源争用不会迅速吞噬收益。

因此，并行化可能只是实现层面的映射，也可能要求算法层面的替换。若现有算法不适合并行执行，程序员需要选择结构完全不同的算法。

1.6.1.4 映射工作并优化数据布局

把工作划分为大量线程后，还要决定数据如何存放、何时移动以及哪些线程访问哪些数据，其目标是优化数据排列以最大化局部性。**局部性 (locality)** 表示程序在较短时间内重复访问相近或相关数据的倾向。良好的数据布局可以减少高延迟访问、提高缓存或片上存储的复用，并让大量线程更有效地使用全局内存带宽。

1.6.1.5 测量驱动的性能调优

性能调优 (performance tuning) 是迭代过程：修改实现、测量、解释瓶颈，再决定下一项修改。每次优化必须同时检查正确性和性能，不能仅依据代码看起来“更并行”就断言加速成立。

完整 workflow 可概括为：

正确基线 → 性能剖析 → 热点与并行性分析 → 算法或实现重构，
→ 数据布局优化 → 测量与验证 → 继续迭代。

最后还要检查代码的可移植性、可扩展性和可维护性。一个依赖特定设备偶然参数、只在单一输入上有效的实现，可能在下一代硬件或更大数据集上失效。

1.6.2 挑战一：大规模并行要求规则性

图形处理器最有效的执行形态是许多线程在同一阶段执行相同或相近的操作，只是处理不同数据。这种要求称为**规则性 (regularity)**。规则性包含至少三个层面：

- 工作结构规则：线程执行路径相近，不出现少数线程承担大量额外步骤；
- 数据访问规则：线程以可预测方式访问数据，避免大量随机、冲突或低复用访问；
- 阶段推进规则：线程组能在相近时间完成当前阶段，减少等待与闲置。

规则性不是说所有线程必须处理完全相同的数据，而是说硬件可以用共享的控制与调度机制高效驱动大量相似工作。若每个线程都执行截然不同的控制流，图形处理器复制大量算术通道的优势就难以兑现。

1.6.3 挑战二：负载均衡

设一个并行阶段包含 n 个任务，第 i 个任务的执行时间为 T_i 。如果下一阶段必须等所有任务完成，则该阶段的完成时间为

$$T_{\text{phase}} = \max_{1 \leq i \leq n} T_i. \quad (1.1)$$

式 (1.1) 表明，最快线程提前完成并不能缩短阶段时间；最慢线程决定何时能够继续。这就是**负载均衡** (*load balance*) 的核心。

Parallel Computing Challenges



- Massive Parallelism demands Regularity
- Load Balance
 - The total amount of time to complete a parallel job is limited by the thread that takes the longest to finish



ELECTRICAL & COMPUTER ENGINEERING

GRAINGER ENGINEERING

图 1.9 规则并行与负载不均衡

为了量化不均衡，可以定义一个解释性指标

$$E_{\text{load}} = \frac{\sum_{i=1}^n T_i}{n \max_i T_i}, \quad 0 < E_{\text{load}} \leq 1. \quad (1.2)$$

若所有任务耗时相同，则 $E_{\text{load}} = 1$ ；若少数任务远慢于其余任务，该比值会降低，表示大量并行资源在等待最慢任务时处于空闲状态。式 (1.2) 是用于理解负载不均衡的解释性指标；性能也可以用其他指标量化。

负载不均衡可能来自输入数据差异、分支路径差异、任务划分粒度过粗或某些任务争用更多资源。解决方向是重新分割任务、细化工作粒度或选择更规则的算法，使各执行单元承担的有效工作接近。

1.6.4 挑战三：全局内存带宽的理想值与实际值

全局内存带宽 (global memory bandwidth) 表示设备内存与处理器之间单位时间能够传输的数据量。硬件规格给出的峰值带宽是在理想访问模式下的上限，应用实际获得的**有效带宽 (effective bandwidth)** 可写为

$$B_{\text{eff}} = \frac{D_{\text{useful}}}{T}, \quad (1.3)$$

其中 D_{useful} 是完成目标计算所需的有效数据量， T 是相应时间。若程序重复搬运数据、访问模式分散或不能充分并行发起内存请求，则 B_{eff} 可能远低于峰值。

水坝与吸管的类比可以区分“理想”与“现实”：设备可能具备很宽的总带宽，但单个程序若组织不当，只能利用其中很小一部分。提高有效带宽依赖正确的内存模型、数据复用和访问模式。

1.6.5 挑战四：冲突访问导致串行化与延迟

当大量线程同时访问某个受限或关键资源时，请求必须排队，原本并行的工作被迫部分**串行化 (serialization)**。国际机场入境检查可以类比这种瓶颈：许多航班可以同时到达，但若所有旅客必须通过数量有限的检查窗口，检查阶段就成为窄口，前面的并行到达能力无法转化为端到端吞吐量。

Parallel Computing Challenges (cont'd)

- Global Memory Bandwidth
 - Ideal vs. Reality
- Conflicting Data Accesses Cause Serialization and Delays
 - Massively parallel execution cannot afford serialization
 - Contentions in accessing critical data causes serialization

Ideal



Reality





ELECTRICAL & COMPUTER ENGINEERING

GRAINGER ENGINEERING

图 1.10 全局内存带宽与关键资源争用

从系统角度看，争用会产生两类损失：

1. 请求排队，单个访问的等待时间增加；
2. 等待资源的线程不能推进，执行通道可能空闲，整体吞吐量下降。

海量并行系统对串行化尤其敏感。若只有一个线程，某个小的串行步骤只是该线程的一部分；若有成千上万线程同时汇聚到同一关键资源，该步骤会把大量并行执行压缩为一条队列。同步、共享数据更新与内存访问设计都必须控制这种汇聚效应。

1.6.6 四类挑战之间的耦合

规则性、负载均衡、带宽和争用不是相互独立的问题。一个不规则算法可能同时造成：不同线程工作量差异、数据访问失去局部性、多个线程争用热点数据，以及难以隐藏的等待。相反，规则的数据并行结构通常更容易均匀划分工作，也更容易形成高效访存。

关键结论

并行化的目标不是把串行语句机械地复制到更多线程，而是构造一种硬件能够持续推进的工作形态：线程数量足够多，执行路径足够规则，任务负载相近，数据访问能够利用带宽，并且不存在把全部线程重新压回串行队列的关键资源。

1.7 阿姆达尔定律：整体加速比的串行上界

1.7.1 问题设定与符号

考虑一个固定规模的应用，其原始串行执行时间为 t 。把原始时间按能否被目标并行化方法加速分成两部分：

- 可并行部分占比为 p ，原始耗时为 pt ；
- 不可并行部分占比为 $1 - p$ ，原始耗时为 $(1 - p)t$ ；
- 可并行部分经过并行化后获得局部加速比 s ，因此其新耗时为 pt/s 。

这里 $0 \leq p \leq 1$ ， $s \geq 1$ 。该模型假设问题规模固定，并暂时不单独计入内核启动、数据传输、同步与通信等额外开销。

1.7.2 总执行时间的推导

不可并行部分没有被加速，因此仍需 $(1 - p)t$ ；可并行部分缩短为 pt/s 。新程序的总执行时间为

$$T_{\text{new}} = (1 - p)t + \frac{pt}{s} = \left(1 - p + \frac{p}{s}\right)t. \quad (1.4)$$

整体加速比定义为原始执行时间与新执行时间之比：

$$S(p, s) = \frac{t}{T_{\text{new}}} = \frac{t}{\left(1 - p + \frac{p}{s}\right)t} = \boxed{\frac{1}{1 - p + \frac{p}{s}}}. \quad (1.5)$$

原始时间 t 在分子与分母中约去，因此在这一固定比例模型中，整体加速比只由可并行比例 p 和局部加速比 s 决定。

1.7.3 并行部分无限快时的上界

即使可并行部分被理想化为瞬间完成，即 $s \rightarrow \infty$ ，仍有

$$\lim_{s \rightarrow \infty} S(p, s) = \frac{1}{1 - p}. \quad (1.6)$$

式 (1.6) 是阿姆达尔定律最重要的含义：整体加速比由未被加速的串行比例限制。可并行部分继续变快时，它在总时间中的占比越来越小，最终总时间几乎全部由串行部分构成。

Amdahl's Law



- In general, if an application has:
 - t sequential execution time
 - p fraction of execution that is parallelizable
 - s speedup achieved on the parallelizable part
- What is the overall speedup of the application?

$$t_{parallel} = (1 - p) * t + \frac{p * t}{s} = \left(1 - p + \frac{p}{s}\right) * t$$

$$speedup = \frac{t_{sequential}}{t_{parallel}} = \frac{t}{\left(1 - p + \frac{p}{s}\right) * t} = \frac{1}{1 - p + \frac{p}{s}} \xrightarrow{s \rightarrow \infty} \frac{1}{1 - p}$$

ELECTRICAL & COMPUTER ENGINEERING

GRAINGER ENGINEERING

图 1.11 阿姆达尔定律推导

1.7.4 示例：90% 可并行，局部加速 1000 倍

例题

原程序串行运行时间为 $t = 100\text{ s}$ ，可并行比例为 $p = 0.9$ ，可并行部分在图形处理器上获得 $s = 1000$ 的加速。求新执行时间与整体加速比。

串行部分原本占 10%，因此保持为

$$(1 - p)t = (1 - 0.9) \times 100\text{ s} = 10\text{ s}.$$

可并行部分原本耗时 90 s，加速 1000 倍后为

$$\frac{pt}{s} = \frac{0.9 \times 100\text{ s}}{1000} = 0.09\text{ s}.$$

所以

$$T_{\text{new}} = 10\text{ s} + 0.09\text{ s} = 10.09\text{ s},$$

$$S = \frac{100\text{ s}}{10.09\text{ s}} \approx 9.91.$$

尽管局部加速比达到 1000，整体加速比仍只有约 9.91。若把可并行部分变为无限快，整体

上界也只是

$$S_{\max} = \frac{1}{1 - 0.9} = 10.$$

1.7.5 串行比例对上界的高度敏感性

表 1.9 不同可并行比例对应的理想整体加速上界

可并行比例 p	串行比例 $1 - p$	$s \rightarrow \infty$ 时的上界 $1/(1 - p)$
0.50	0.50	2
0.90	0.10	10
0.95	0.05	20
0.99	0.01	100
0.999	0.001	1000

要获得数量级更高的整体加速，串行比例必须按同样数量级下降。理想情况下，若希望整体加速达到 S_{target} ，至少需要

$$1 - p \leq \frac{1}{S_{\text{target}}}.$$

例如，100 倍加速要求串行比例不超过 1%；有限的局部加速和额外开销会使实际要求更严格。许多大数据集应用的可并行比例可以超过 99%，因此超过 100 倍的加速在合适条件下可以实现。

1.7.6 为什么先做性能剖析

假设某函数占总时间的比例为 p ，对该函数的任何局部优化都受式 (1.5) 约束。因此，性能剖析的作用不是只找“看起来复杂”的代码，而是确定哪些部分具有足够大的时间占比。优化低占比部分，即使局部加速很高，整体收益也会被阿姆达尔定律压缩。

反过来，当并行部分已经极快时，原先不起眼的串行初始化、数据准备、输出或同步会成为新的热点。性能优化因此具有阶段性：每消除一个瓶颈，时间分布都会改变，需要重新剖析，而不能永久沿用最初的热点排序。

1.7.7 模型边界与现实开销

实际异构程序通常还包含图形处理器内核启动、中央处理器与图形处理器之间的数据移动、同步、资源争用和调度开销。若把这些额外时间记为 T_{overhead} ，则更现实的表达为

$$S_{\text{real}} = \frac{t}{(1-p)t + \frac{pt}{s} + T_{\text{overhead}}}. \quad (1.7)$$

式 (1.7) 在基本模型中加入了显式开销。它说明，即使 p 很大，问题规模过小也可能无法抵消固定开销；提高局部内核速度之后，数据移动与同步可能主导总时间。这与第 1.3 式所描述的有效带宽问题以及资源争用问题相互关联。

关键结论

阿姆达尔定律给出的优化顺序是：先找到占比最大的可加速部分，同时持续削减不可并行与协调开销；局部加速达到很高后，必须重新测量，因为瓶颈会迁移到之前占比很小的部分。整体性能永远由完整执行路径决定。

1.8 综合框架与关键结论

1.8.1 硬件演化到软件责任的完整链条

硬件演化与软件责任构成以下统一框架：

1. 摩尔定律长期提高晶体管密度，登纳德缩放长期提高时钟频率；两者共同形成早期单核性能快速增长。
2. 约在 2005–2006 年，漏电流、功耗与散热约束使频率增长趋于停滞，但晶体管数量仍能继续增加。
3. 新增晶体管被组织为多核、向量执行资源和专用加速器。现代系统因此具有显著并行性与异构性。
4. 中央处理器面向低延迟，适合串行依赖和复杂控制；图形处理器面向高吞吐量，适合大量规则且相互独立的计算。
5. 图形处理器原有的游戏市场和图形并行结构提供了经济与技术基础；统一计算设备架构进一步把它转化为通用计算平台。

6. 应用必须显式暴露并行性。性能依赖算法可扩展性、线程规则性、负载均衡、数据布局、有效带宽和争用控制。
7. 任意局部加速最终都受完整应用中未加速部分限制，阿姆达尔定律给出这一上界。

1.8.2 选择中央处理器或图形处理器时的判断顺序

面对一个待加速代码段，可以按下列问题逐步判断：

1. 它在完整程序中占多少时间？占比过低时，局部优化的整体收益受限。
2. 是否存在大量相互独立或弱依赖的任务？任务数量必须足以填充图形处理器。
3. 各任务是否执行相同或相近的操作？高度不规则控制流会降低并行执行效率。
4. 每个任务的工作量是否接近？最慢任务会决定并行阶段完成时间。
5. 数据访问是否规则、具有局部性，并能利用全局内存带宽？
6. 把数据移动到图形处理器并启动内核的开销，是否小于并行计算所节省的时间？
7. 实现能否在更大输入和后续硬件上保持正确性、性能与可维护性？

只有当这些问题的总体答案支持加速器映射时，图形处理器才可能产生端到端收益。对于串行依赖明显、任务规模很小或数据移动占主导的代码，中央处理器可能更合适。

1.8.3 高频概念辨析

容易形成的误解	准确理解
摩尔定律等同于处理器性能翻倍。	历史性能增长来自晶体管密度、频率缩放和处理器结构改进的共同作用；摩尔定律主要描述晶体管密度趋势。
处理器核心数量增加会自动加速串行程序。	串行指令流通常只使用一个核心；程序必须暴露并行任务并承担调度、通信和同步成本。
图形处理器的单个线程一定比中央处理器线程快。	图形处理器优化大批线程的总吞吐量，单线程延迟可能更高。
统一计算设备架构核心与中央处理器核心可以按数量直接比较。	两者粒度不同；统一计算设备架构核心更接近细粒度算术执行通道。
硬件峰值内存带宽就是程序能够获得的带宽。	实际有效带宽取决于数据布局、访问模式、复用、并发程度和争用。

容易形成的误解	准确理解
并行部分加速 1000 倍会让整个程序加速 1000 倍。	整体加速由 $S = 1/(1 - p + p/s)$ 决定，未加速部分最终成为上界。
问答系统使用课程材料，因此答案必然正确。	检索增强生成只能改善相关性；检索遗漏、隐含假设与生成错误仍可能导致错误答案。

1.8.4 “高性能”的操作性定义

高性能需要放在完整软件工程目标中理解。并行实现应满足：

- **功能正确**：与参考语义一致，能够通过规定的数据集与问题检查；
- **端到端加速**：完整应用时间缩短，而不是只报告某个局部内核的理想时间；
- **可解释**：能够用并行度、访存、负载、串行比例等因素解释性能变化；
- **可扩展**：问题规模或硬件资源增加时仍能利用更多并行能力；
- **可维护与可移植**：实现结构清晰，不依赖难以复现的偶然参数，并能适应硬件代际变化。

这五项要求也解释了项目评分为何同时关注功能、编码风格和性能。单独追求某一项会产生失衡：正确但极慢的实现没有利用加速器价值；快速但错误的实现改变了问题；高度手工特化但不可维护的实现难以跨代延续。

1.8.5 阅读与操作任务

需要完成以下事项：

- 阅读教材第 1 章；
- 阅读《统一计算设备架构编程指南》第 1.1 节“导论”；
- 尽快创建高等科学与工程研究计算环境账户并取得 Delta 访问权限；
- 配置个人代码仓库，完成课程发布代码的同步流程；
- 在账户可用后验证安全外壳协议登录、编译节点与计算节点作业提交流程。

关键结论

软件可持续性要求算法与库具备可扩展性和可移植性，从而跨越多代硬件；其价值来自对并行结构和性能瓶颈的清晰建模，而不是依赖某一代设备的偶然峰值。

第 2 讲

Introduction to CUDA C and Data Parallel Programming

2.1 核心内容

内容概览

- 数据并行计算 (*data-parallel computing*) 的基本概念;
- CUDA C 编程接口 (*CUDA C programming interface*) 的基本特性。

2.2 从线程到大规模数据并行

2.2.1 线程是计算的基本单位

线程 (*thread*) 是一条正在执行的顺序计算。线程包含程序或指令序列、指出当前执行位置的程序计数器 (*program counter, PC*), 以及执行上下文 (*context*), 例如内存状态与寄存器状态。单个线程内部仍按顺序执行; 并行性来自许多线程在硬件上同时推进。

线程的最小心智模型

设线程 T 的状态写成

$$T = (P, PC, R, M_{\text{visible}}),$$

其中 P 是程序, PC 是下一条待执行指令的位置, R 表示该线程可见的寄存器状态, M_{visible} 表示该线程可以访问的内存状态。CUDA 中同一核函数 (*kernel*) 产生的线程执行相同程序 P , 但它们看到的线程索引不同, 因此访问的数据位置不同。

2.2.2 “多个线程”与“海量线程”

传统多核 CPU 往往运行数量相对有限、行为较复杂的线程；GPU 的设计目标则是让数量极大的线程并发执行。当线程数量达到成千上万甚至更多时，任意而频繁的线程间通信会迅速造成等待与串行化。因此，最容易映射到 GPU 的工作负载通常具备以下特征：

- 线程之间的依赖很少；
- 每个线程能够独立确定自己的输入与输出；
- 大量线程执行相同或相近的计算；
- 线程之间即使合作，也尽量限制在较小的局部范围内，例如同一线程块。

CUDA 线程仍可通过**共享内存**(*shared memory*)、**原子操作**(*atomic operation*)与**屏障同步**(*barrier synchronization*)进行合作。设计并行程序时，应先寻找独立工作，再加入必要合作。

2.2.3 任务并行与数据并行

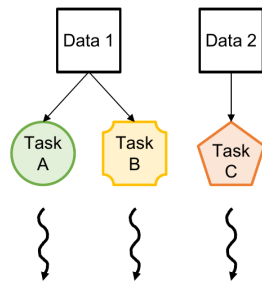
任务并行(*task parallelism*)把不同操作分配给不同执行单元；**数据并行**(*data parallelism*)则让大量执行单元在不同数据上执行相同操作。GPU 最擅长后者，因为大型数据集常能释放出极高的并行度。

Types of Parallelism



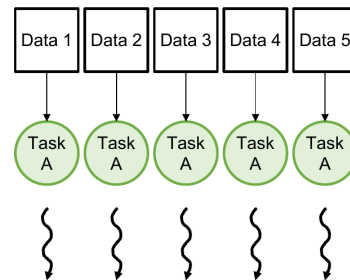
Task Parallelism

- Different operations performed on same or different data
- Usually, a modest number of tasks unleashing a modest amount of parallelism



Data Parallelism

- Same operations performed on different data
- Potentially massive amounts of data unleashing massive amounts of parallelism
 - Most suitable for GPUs



ELECTRICAL & COMPUTER ENGINEERING

GRAINGER ENGINEERING

图 2.1 任务并行 (task parallelism) 与数据并行 (data parallelism) 的对比

可用一个统一的数学形式描述数据并行。设输入数据为

$$X = \{x_0, x_1, \dots, x_{n-1}\},$$

每个输出元素由同一个函数 f 产生：

$$y_i = f(x_i), \quad 0 \leq i < n.$$

若 y_i 的计算不依赖尚未生成的 y_j ，也不修改其他线程负责的输出位置，则所有 y_i 可以并行计算。

2.2.3.1 彩色图像转灰度图像

每个像素具有红、绿、蓝三个分量 (r_i, g_i, b_i) 。常用的亮度近似为

$$L_i = 0.299r_i + 0.587g_i + 0.114b_i.$$

对任意两个不同像素 $i \neq j$ ， L_i 的计算不读取或修改像素 j 的输出，因此像素转换彼此独立。顺序循环

for each pixel: $L_i \leftarrow f(r_i, g_i, b_i)$

可以被解释成“一个线程对应一次循环迭代”。这正是 CUDA 入门中最重要的循环并行化 (*loop parallelism*) 视角。

判断数据并行时必须问的三个问题

1. 每次迭代写入的位置是否唯一？
2. 每次迭代读取的数据是否已经可用，而非由其他并行迭代临时产生？
3. 改变不同迭代的执行顺序，是否仍能得到相同结果？

三个答案均为“是”时，通常可以直接采用一线程一元素的映射。

2.3 CUDA 的主机-设备执行模型

2.3.1 一个源程序，两类执行位置

CUDA 程序是异构应用 (*heterogeneous application*)：同一个工程同时包含在 CPU 上运行的主机代码 (*host code*)，以及在 GPU 上运行的设备代码 (*device code*)。主机代码通常负责串行或并行度较低的工作，例如输入输出、内存管理与核函数调度；设备代码负责数据量大、并行度高的计算阶段。

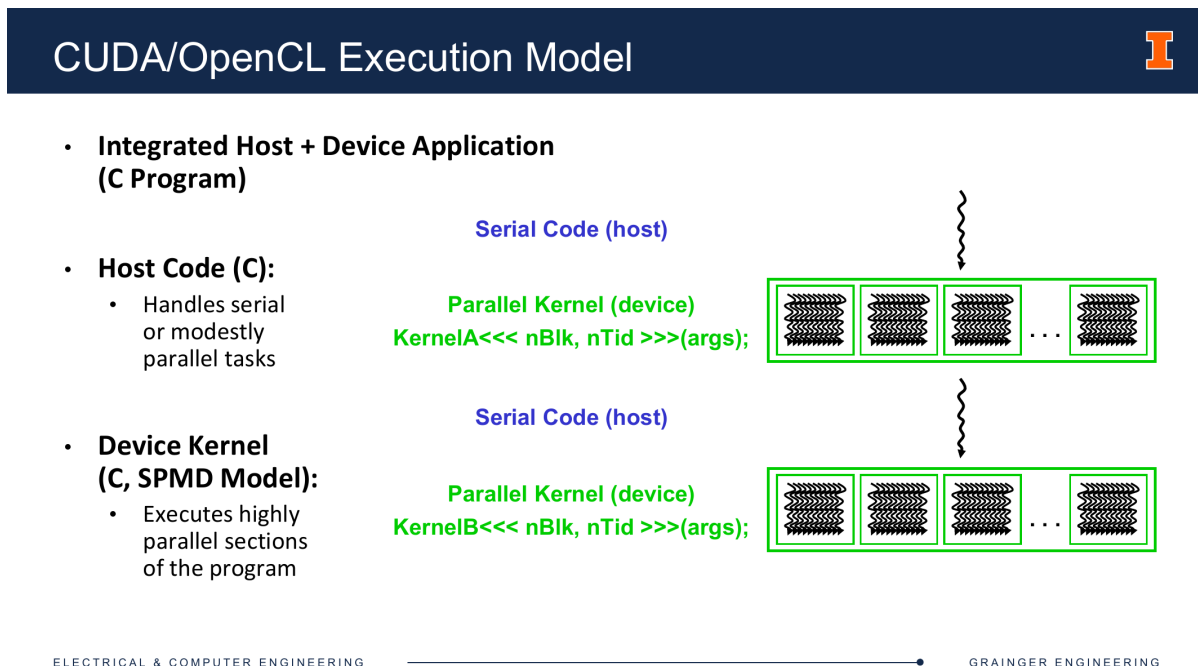


图 2.2 CUDA/OpenCL 的集成式主机-设备执行模型

一个典型的时间顺序可以写成：

主机准备数据 → 启动核函数 A → 主机继续执行或等待 → 启动核函数 B → 取回结果。

2.3.2 核函数与单程序多数据模型

核函数(kernel)是由许多 GPU 线程执行的设备函数。一次核函数启动 (**kernel launch**) 创建一个线程网格。网格中的所有线程运行同一份核函数代码，但每个线程使用自己的索引选择不同数据。这种编程方式称为**单程序多数据 (Single Program Multiple Data, SPMD)**。

SPMD 的“同一程序”并不意味着所有线程在每一时刻必然完成相同工作。线程可以依据索引进行条件判断；只是它们的代码文本相同。最简单的向量加法核函数逻辑是

$$C[i] = A[i] + B[i],$$

其中线程索引 i 决定该线程访问哪一对输入元素。

主机与设备的职责边界

对向量加法而言，主机代码负责准备 A 、 B 、 C ，把输入移动到设备可访问的位置，配置线程组织并启动核函数；设备线程只负责执行元素级加法。把数据移动、调度和计算区分开，是阅读任何 CUDA 程序的第一步。

2.4 线程层次结构与一维索引映射

2.4.1 网格、线程块与线程

一次核函数启动创建一个**网格(grid)**。网格由若干**线程块(thread block)**组成，每个线程块再由若干**线程(thread)**组成。对于一维网格：

- `gridDim.x`：网格在 x 方向包含的线程块数；
- `blockIdx.x`：当前线程块在网格中的索引；
- `blockDim.x`：每个线程块在 x 方向包含的线程数；
- `threadIdx.x`：当前线程在线程块内部的索引。

Thread Blocks: Scalable Cooperation



- Threads within a block cooperate via **shared memory, atomic operations** and **barrier synchronization** (to be covered later)
- Threads in different blocks cooperate less.

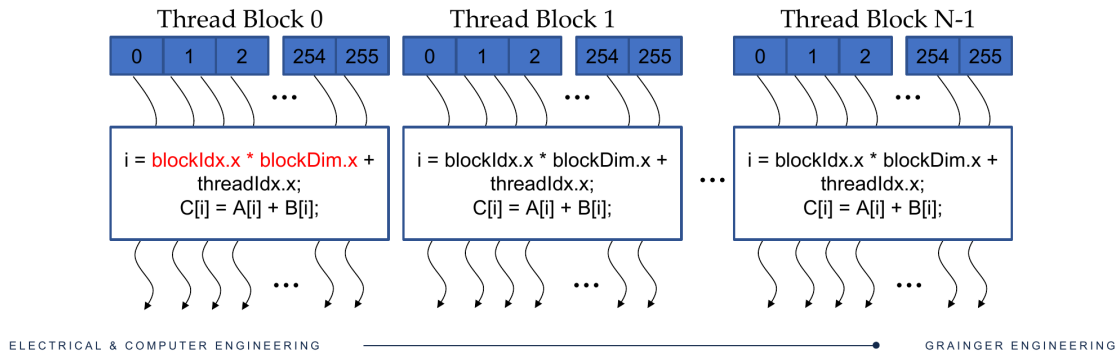


图 2.3 线程块内部可进行较强合作，不同线程块之间的合作更受限制

同一线程块内的线程可通过共享内存、原子操作与屏障同步进行局部合作。不同线程块必须能够以任意顺序调度，因此各线程块通常应设计为彼此独立。这个限制换来了**可扩展性 (scalability)**：同一核函数可以在执行资源不同的 GPU 上运行，而无需把线程块绑定到特定硬件位置。

2.4.2 全局线程索引的推导

设每个线程块有 $T = \text{blockDim.x}$ 个线程，当前线程块索引为

$$b = \text{blockIdx.x},$$

线程在线程块内的局部索引为

$$t = \text{threadIdx.x}.$$

在当前线程块之前共有 b 个完整线程块，每个线程块贡献 T 个线程，所以当前线程块之前一共有 bT 个线程。把局部偏移 t 加上，得到全局一维索引

$$i = bT + t$$

即

```
int i = blockIdx.x * blockDim.x + threadIdx.x;
```

2.4.2.1 覆盖范围证明

对固定的线程块 b ，局部索引满足 $0 \leq t < T$ ，因此该线程块产生的全局索引范围是

$$bT \leq i \leq bT + T - 1.$$

相邻线程块 $b + 1$ 的起始索引为

$$(b + 1)T = bT + T,$$

恰好接在前一线程块的末尾之后。因此所有线程块共同形成连续、不重叠的一维索引区间。

例：线程块大小为 256

若 $T = 256$ ：

线程块0： $i = 0, 1, \dots, 255$,

线程块1： $i = 256, 257, \dots, 511$,

线程块3中的线程0： $i = 3 \times 256 + 0 = 768$.

由此得到的全局索引与图示中的线程分配一致。

2.4.3 线程索引是“数据归属规则”

索引公式本身只是编号。真正的并行设计问题是：编号为 i 的线程应该负责哪一块数据？对一维向量加法，最自然的规则是线程 i 负责元素 i 。对二维图像、卷积、归约或扫描，线程到数据的映射会更复杂，但方法相同：先定义工作单元，再用内置索引计算该工作单元的位置。

关键结论

给定 B 个线程块、每块 T 个线程：

$$\text{启动线程总数} = BT.$$

每个线程拥有自己的自动局部变量副本；因此若核函数中声明一个局部变量 i ，整个网格生命周期中最多会为 BT 个线程各产生一个逻辑副本。

2.5 向量加法：从顺序循环到线程网格

2.5.1 顺序基线与并行分解

向量加法（vector addition）是数据并行程序中的 “Hello, World!”。给定长度均为 n 的向量 A 与 B ，目标是计算

$$C[i] = A[i] + B[i], \quad 0 \leq i < n.$$

顺序 C++ 实现用一个循环依次完成 n 次加法：

```
1 void vecAddCPU(const float* A, const float* B, float* C, int n) {  
2     for (int i = 0; i < n; ++i) {  
3         C[i] = A[i] + B[i];  
4     }  
5 }
```

并行化时，不需要保留这个循环的顺序执行结构。把第 i 次循环迭代定义为一个独立工作单元，再把工作单元 i 分配给全局线程 i ：

$$\text{线程 } i \longleftrightarrow C[i] = A[i] + B[i].$$

因此，原来的循环被整个线程网格替代。并行化前应先回答 “每个线程究竟独立负责什么工作”，因为这一选择会同时决定线程数量、数据布局与后续优化空间。

唯一写入者映射

输出元素 $C[i]$ 由唯一线程 i 负责写入。不同线程不会同时修改同一输出位置，因此不需要原子操作或同步。

2.5.2 CPU 与 GPU 的分离内存空间

经典的离散 GPU 系统模型由彼此分离的主机与设备内存空间组成：CPU 连接主机内存 (*host memory*)，GPU 连接设备全局内存 (*device global memory*)。主机指针默认指向主机地址空间，设备指针指向设备地址空间；在基础 CUDA 编程模型中，二者不能被对方代码直接解引用。

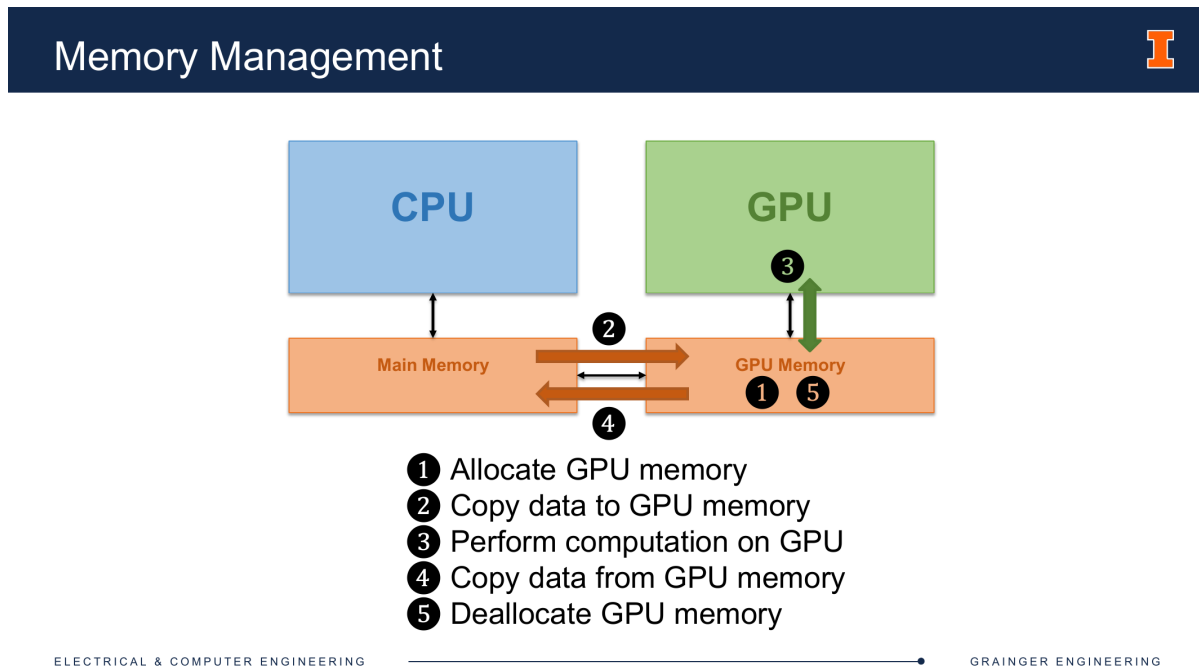


图 2.4 设备内存管理的五个阶段

完整数据流由五步构成：

1. 在设备端分配空间；
2. 把输入数据从主机复制到设备；
3. 在设备端启动核函数并计算；
4. 把结果从设备复制回主机；
5. 释放设备端空间。

即使系统提供**统一内存** (*unified memory*) 或某些物理共享内存机制，数据仍需要在合适时机出现在 GPU 可高效访问的位置。显式分配与显式复制能够清楚呈现数据移动的成本与依赖关系。

2.5.3 设备内存管理 API

2.5.3.1 分配与释放

CUDA API	作用与关键参数
<code>cudaMalloc(void **devPtr, size_t size)</code>	在设备全局内存中分配 <code>size</code> 字节，并把起始地址写入 <code>*devPtr</code> 。返回值类型为 <code>cudaError_t</code> 。
<code>cudaFree(void *devPtr)</code>	释放由 <code>devPtr</code> 指向的设备内存区域。

对 n 个 `float` 元素，字节数应写成

$$S = n \cdot \text{sizeof}(\text{float}).$$

实际代码宜用 `size_t` 保存 S ，避免大规模数组下的整数溢出。

```
size_t bytes = static_cast<size_t>(n) * sizeof(float);
float *A_d = nullptr, *B_d = nullptr, *C_d = nullptr;

cudaMalloc(reinterpret_cast<void**>(&A_d), bytes);
cudaMalloc(reinterpret_cast<void**>(&B_d), bytes);
cudaMalloc(reinterpret_cast<void**>(&C_d), bytes);
// ... use the allocations ...
cudaFree(A_d);
cudaFree(B_d);
cudaFree(C_d);
```

设备指针的语义

`A_d` 的数值是设备地址。主机代码可以把它传给 CUDA 运行时 API 或核函数，但不应在普通主机表达式中写 `A_d[i]`。设备端数据访问应发生在核函数中。

2.5.3.2 主机-设备数据复制

`cudaMemcpy` 的接口为

```
cudaError_t cudaMemcpy(void* dst, const void* src,
                      size_t count, cudaMemcpyKind kind);
```

第四个参数明确复制方向：

枚举常量	方向
<code>cudaMemcpyHostToHost</code>	主机到主机 (host to host)
<code>cudaMemcpyHostToDevice</code>	主机到设备 (host to device)
<code>cudaMemcpyDeviceToHost</code>	设备到主机 (device to host)
<code>cudaMemcpyDeviceToDevice</code>	设备到设备 (device to device)

向量加法的输入与输出复制分别为

```
cudaMemcpy(A_d, A_h, bytes, cudaMemcpyHostToDevice);
cudaMemcpy(B_d, B_h, bytes, cudaMemcpyHostToDevice);
// kernel launch
cudaMemcpy(C_h, C_d, bytes, cudaMemcpyDeviceToHost);
```

读代码时可先检查 `dst` 与 `src` 是否和复制方向一致。例如, `cudaMemcpyDeviceToHost` 要求目标指针位于主机端、源指针位于设备端。

2.6 核函数实现、启动配置与边界检查

2.6.1 核函数声明与线程工作

核函数使用 `__global__` 限定符声明。每个线程执行同一函数体, 并根据内置索引计算自己的元素编号:

```
1 __global__ void vecAddKernel(const float* A_d,
2                             const float* B_d,
3                             float* C_d,
4                             int n) {
5     int i = blockIdx.x * blockDim.x + threadIdx.x;
6     if (i < n) {
7         C_d[i] = A_d[i] + B_d[i];
8     }
9 }
```

`i` 是自动局部变量 (*automatic local variable*)。逻辑上, 每个线程拥有自己的 `i`, 一个线程对它的赋值不会改变其他线程的副本。

2.6.2 三重尖括号与启动参数

主机通过三重尖括号语法 (*triple-chevron syntax*) 启动核函数：

```
vecAddKernel<<<numBlocks, threadsPerBlock>>>(A_d, B_d, C_d, n);
```

第一个执行配置参数是网格中的线程块数，第二个参数是每个线程块的线程数。二者的位置不能互换。

2.6.3 向上取整除法的推导

设数据元素数为 n ，每个线程块有 T 个线程。若每个线程只处理一个元素，需要的线程块数量为

$$B = \left\lceil \frac{n}{T} \right\rceil.$$

在整数算术中常写为

$$B = \left\lfloor \frac{n + T - 1}{T} \right\rfloor,$$

对应代码

```
int numBlocks = (n + threadsPerBlock - 1) / threadsPerBlock;
```

证明如下。令 $n = qT + r$ ，其中 $0 \leq r < T$ 。

- 若 $r = 0$ ，则 n 可被 T 整除，公式得到 $B = q$ ；
- 若 $r > 0$ ，则至少还需要一个线程块，公式得到 $B = q + 1$ 。

因此该整数表达式与向上取整完全等价。

Thread Assignment for vecAdd ($n = 1000$, block size = 256) I

```
vecAddKernel<<<ceil(n/256.0), 256>>>(...)
int i = blockIdx.x * blockDim.x + threadIdx.x;
if (i < n) C[i] = A[i] + B[i];
```

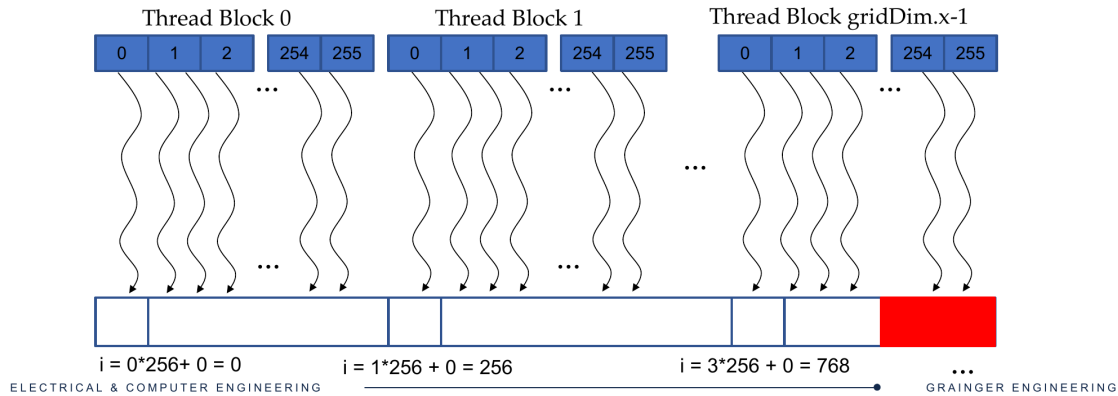


图 2.5 $n = 1000$ 、线程块大小为 256 时的线程分配

2.6.4 为什么必须做边界检查

总启动线程数为

$$N_{\text{launch}} = BT.$$

由于 B 经过向上取整，通常有 $N_{\text{launch}} \geq n$ 。多出的线程数

$$E = BT - n$$

满足 $0 \leq E < T$ 。这些额外线程没有对应的数据元素，必须在访问数组前检查

```
if (i < n) { /* valid work */ }
```

否则它们可能越界读取或写入设备内存。

例： $n = 1000$, $T = 256$

$$B = \left\lceil \frac{1000}{256} \right\rceil = 4, \quad N_{\text{launch}} = 4 \times 256 = 1024.$$

前 1000 个线程处理有效元素，最后 24 个线程满足 $i \geq 1000$ ，被边界条件屏蔽。

2.6.5 向量加法的主机端骨架

把内存管理与核函数启动串联起来，得到如下流程：

```
1 void vecAdd(const float* A_h, const float* B_h,  
2           float* C_h, int n) {  
3     size_t bytes = static_cast<size_t>(n) * sizeof(float);  
4     float *A_d = nullptr, *B_d = nullptr, *C_d = nullptr;  
5  
6     cudaMalloc(reinterpret_cast<void**>(&A_d), bytes);  
7     cudaMalloc(reinterpret_cast<void**>(&B_d), bytes);  
8     cudaMalloc(reinterpret_cast<void**>(&C_d), bytes);  
9  
10    cudaMemcpy(A_d, A_h, bytes, cudaMemcpyHostToDevice);  
11    cudaMemcpy(B_d, B_h, bytes, cudaMemcpyHostToDevice);  
12  
13    const int threadsPerBlock = 256;  
14    const int numBlocks = (n + threadsPerBlock - 1)  
15                        / threadsPerBlock;  
16    vecAddKernel<<<numBlocks, threadsPerBlock>>>(A_d, B_d, C_d, n);  
17  
18    cudaMemcpy(C_h, C_d, bytes, cudaMemcpyDeviceToHost);  
19  
20    cudaFree(A_d);  
21    cudaFree(B_d);  
22    cudaFree(C_d);  
23 }
```

这段代码展示了结构，但尚未加入系统化的错误检查。下一轮将补齐可运行版本，并解释异步启动、编译流程与错误传播。

2.7 函数限定符与 CUDA 编译流程

2.7.1 三类函数限定符

CUDA C++ 用函数限定符 (*function qualifier*) 说明函数从哪里调用、在哪里执行。

限定符	可调用位置	执行位置	含义
<code>__host__</code>	主机 (host)	主机 (host)	普通主机函数；省略限定符时的默认情况。
<code>__global__</code>	主机，或受支持的设备端启动	设备 (device)	核函数；调用时创建新的线程网格，返回类型必须为 <code>void</code> 。
<code>__device__</code>	设备 (device)	设备 (device)	由核函数或其他设备函数调用，不创建新网格。

同一个普通函数也可以同时标记为

```
__host__ __device__ float add(float a, float b) {
    return a + b;
}
```

编译器会为主机和设备分别生成版本。调用端仍需满足各自执行环境的限制。

2.7.2 从.cu 源文件到 CPU/GPU 机器代码

CUDA 源文件通常使用 `.cu` 扩展名，并交给 **NVIDIA CUDA 编译器** (*NVIDIA CUDA Compiler, NVCC*)。NVCC 根据函数限定符区分主机代码与设备代码：

1. 主机部分交给常规 C/C++ 编译器，生成 CPU 目标代码；
2. 设备部分先生成并行线程执行中间表示 (*Parallel Thread Execution, PTX*)；
3. PTX 在安装或运行阶段由**即时编译器** (*just-in-time compiler, JIT compiler*) 转化为目标 GPU 的设备机器码，常以 SASS 表示。

Compiling A CUDA Program

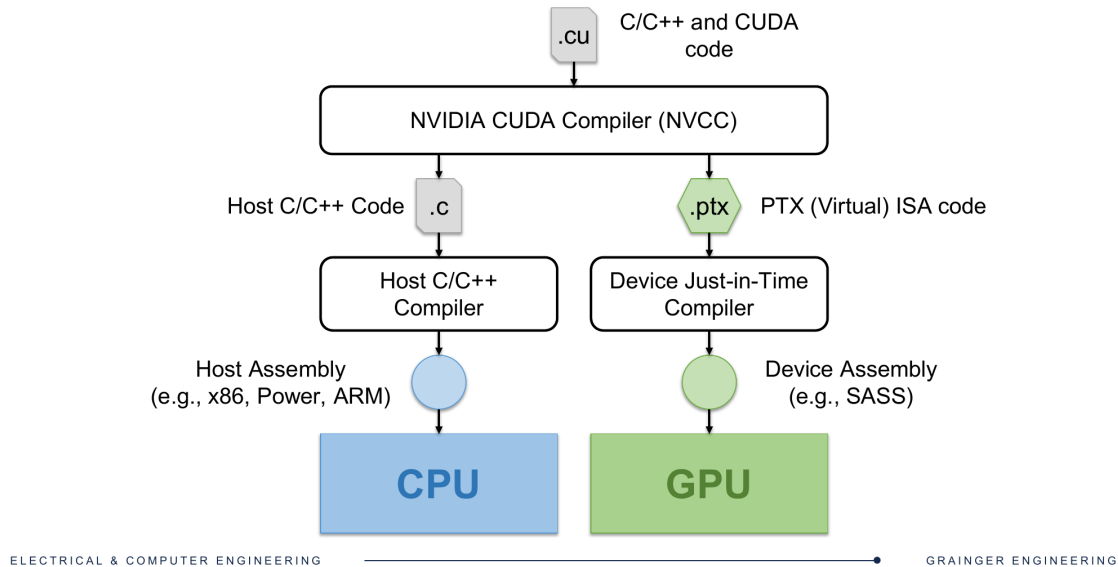


图 2.6 CUDA 程序的主机代码与设备代码编译路径

一个典型编译命令为

```
nvcc -O2 -std=c++17 vec_add.cu -o vec_add
```

其中 `-O2` 请求优化，`-std=c++17` 选择 C++ 语言标准。构建脚本或作业框架可以进一步封装这一步。

2.8 异步执行与错误检查

2.8.1 核函数启动默认是异步的

一次核函数启动通常先把工作提交给 GPU，然后让主机线程继续执行。此行为称为**异步调用** (*asynchronous call*)。因此，下面的 CPU 计时方式主要测到“提交请求”的时间，不一定覆盖 GPU 完成计算所需的时间：

```
start_timer();
vecAddKernel<<<B, T>>>(A_d, B_d, C_d, n);
stop_timer();           // usually too early
```

要让主机等待此前提交的设备工作完成，可调用

```
cudaDeviceSynchronize();
```

该函数提供主机与设备之间的**同步点** (*synchronization point*)，并返回此前异步工作中暴

露的错误状态。

Asynchronous Kernel Calls



- By default, kernel calls are asynchronous
 - Useful for overlapping GPU computations with CPU computations
- Use the following API function to wait for the kernel to finish

```
cudaError_t cudaDeviceSynchronize()
```

 - Blocks until the device has completed all preceding requested tasks
- Better ways of timing kernel execution:
 - Use CUDA events and stream synchronization
 - Use a profiler
 - We will cover these concepts later

ELECTRICAL & COMPUTER ENGINEERING

GRAINGER ENGINEERING

图 2.7 异步核函数调用与 `cudaDeviceSynchronize()`

当核函数后紧跟设备到主机的 `cudaMemcpy` 时，复制不能在产生结果的核函数完成之前得到最终输出。更精确的性能测量通常使用 **CUDA 事件(event)**，并在需要时配合**流同步(stream synchronization)** 或**性能分析器(profiler)**。

2.8.2 两类错误：启动错误与执行错误

绝大多数 CUDA 运行时 API 都返回 `cudaError_t`。`cudaSuccess` 表示成功。其他错误码可交给 `cudaGetErrorString`，转换为可读消息。

核函数调用本身没有普通返回值，因此常分两步检查：

1. `cudaGetLastError()`：检查非法启动配置等立即可见的**启动错误(launch error)**；
2. `cudaDeviceSynchronize()`：等待执行完成，并暴露越界访问等**异步执行错误(asynchronous execution error)**。

```
vecAddKernel<<<numBlocks, threadsPerBlock>>>(A_d, B_d, C_d, n);  
CUDA_CHECK(cudaGetLastError());  
CUDA_CHECK(cudaDeviceSynchronize());
```

简化示例与工程代码的区别

用于突出概念的短代码通常会省略错误检查，也可能包含简化写法。工程代码应检查内存分配、数据复制、核函数启动与核函数执行的返回状态。

2.9 完整、可检查的向量加法程序

以下完整示例把线程索引、内存管理、核函数启动、同步和错误检查串联起来。代码使用 CUDA 运行时 **API**(*CUDA Runtime API*), 并用宏统一检查错误。

```
1  #include <cuda_runtime.h>
2
3  #include <cmath>
4  #include <cstdio>
5  #include <cstdlib>
6  #include <vector>
7
8  #define CUDA_CHECK(call) \
9      do { \
10         cudaError_t err__ = (call); \
11         if (err__ != cudaSuccess) { \
12             std::fprintf(stderr, "CUDA error at %s:%d: %s\n", \
13                 __FILE__, __LINE__, cudaGetErrorString(err__)); \
14             std::exit(EXIT_FAILURE); \
15         } \
16     } while (0)
17
18 __global__ void vecAddKernel(const float* A_d,
19                             const float* B_d,
20                             float* C_d,
21                             int n) {
22     const int i = blockIdx.x * blockDim.x + threadIdx.x;
23     if (i < n) {
24         C_d[i] = A_d[i] + B_d[i];
25     }
26 }
```

```
28 void vecAddGPU(const float* A_h, const float* B_h,
29               float* C_h, int n) {
30     if (n <= 0) {
31         return;
32     }
33
34     const size_t bytes = static_cast<size_t>(n) * sizeof(float);
35     float *A_d = nullptr, *B_d = nullptr, *C_d = nullptr;
36
37     CUDA_CHECK(cudaMalloc(reinterpret_cast<void**>(&A_d), bytes));
38     CUDA_CHECK(cudaMalloc(reinterpret_cast<void**>(&B_d), bytes));
39     CUDA_CHECK(cudaMalloc(reinterpret_cast<void**>(&C_d), bytes));
40
41     CUDA_CHECK(cudaMemcpy(A_d, A_h, bytes, cudaMemcpyHostToDevice));
42     CUDA_CHECK(cudaMemcpy(B_d, B_h, bytes, cudaMemcpyHostToDevice));
43
44     constexpr int threadsPerBlock = 256;
45     const int numBlocks =
46         (n + threadsPerBlock - 1) / threadsPerBlock;
47
48     vecAddKernel<<<numBlocks, threadsPerBlock>>>(A_d, B_d, C_d, n);
49     CUDA_CHECK(cudaGetLastError());
50     CUDA_CHECK(cudaDeviceSynchronize());
51
52     CUDA_CHECK(cudaMemcpy(C_h, C_d, bytes, cudaMemcpyDeviceToHost));
53
54     CUDA_CHECK(cudaFree(A_d));
55     CUDA_CHECK(cudaFree(B_d));
56     CUDA_CHECK(cudaFree(C_d));
57 }
```

```
59 int main() {
60     constexpr int n = 1 << 20;
61     std::vector<float> A(n), B(n), C(n);
62
63     for (int i = 0; i < n; ++i) {
64         A[i] = static_cast<float>(i);
65         B[i] = 2.0f * static_cast<float>(i);
66     }
67
68     vecAddGPU(A.data(), B.data(), C.data(), n);
69
70     for (int i = 0; i < n; ++i) {
71         const float expected = A[i] + B[i];
72         if (std::fabs(C[i] - expected) > 1.0e-5f) {
73             std::fprintf(stderr, "Mismatch at %d\n", i);
74             return EXIT_FAILURE;
75         }
76     }
77
78     std::printf("Vector addition passed for n=%d.\n", n);
79     return EXIT_SUCCESS;
80 }
```

2.9.1 程序结构

1. `main` 在主机端创建并初始化三个向量；
2. `vecAddGPU` 为三个向量分配设备内存；
3. 两次主机到设备复制把输入送入 GPU；
4. 启动配置根据 n 自动计算线程块数；
5. 每个设备线程计算一个 $C[i]$ ，越界线程被条件屏蔽；
6. 主机等待核函数完成并检查错误；
7. 设备到主机复制取回结果，随后释放设备资源；
8. `main` 在 CPU 上验证每一个结果元素。

正确性与性能是两个层次

本程序首先保证结果正确、资源被释放、错误能被报告。它仍会为每次函数调用重新分配内存并往返复制数据；对于包含多个 GPU 阶段的真实程序，通常应让数据在设备端保留更久，减少重复分配与传输。

2.10 例题与完整推导

2.10.1 例题一：读取核函数启动配置

给定

```
kernel<<<VECTOR_N, ELEMENT_N>>>(d_C, d_A, d_B, ELEMENT_N);
```

三重尖括号中的第一个参数是线程块数，第二个参数是每块线程数。因此

$$\text{每块线程数} = \text{ELEMENT_N}$$

且

$$\text{网格总线程数} = \text{VECTOR_N} \times \text{ELEMENT_N}.$$

变量名不会改变语法位置的含义；即使命名不理想，也必须按 CUDA 启动规则解释。

2.10.2 例题二：每线程处理 8 个元素

已知向量长度 $n = 16000$ ，每个线程处理 $k = 8$ 个输出元素，线程块大小 $T = 256$ 。

第一步，求逻辑上需要的工作线程数：

$$L = \left\lceil \frac{n}{k} \right\rceil = \frac{16000}{8} = 2000.$$

第二步，求线程块数：

$$B = \left\lceil \frac{L}{T} \right\rceil = \left\lceil \frac{2000}{256} \right\rceil = 8.$$

第三步，求实际启动线程数：

$$N_{\text{launch}} = BT = 8 \times 256 = 2048.$$

因此答案为

$$2048 \text{ 个线程}.$$

其中 48 个线程没有完整工作。实现线程粗化 (*thread coarsening*) 时，线程 q 可令起始元素为 kq ，并对其负责的每个元素分别检查是否小于 n 。

2.10.3 例题三：局部变量有多少份

核函数启动 $B = 512$ 个线程块，每块 $T = 256$ 个线程。总线程数为

$$BT = 512 \times 256 = 131072.$$

若核函数中声明一个普通自动局部变量，则每个线程拥有自己的逻辑副本，因此整个执行生命周期中共有

131072 份

线程私有版本。这里计算的是逻辑变量实例数；编译器可能把变量放入寄存器，也可能因资源压力使用局部内存，但线程私有语义不变。

2.10.4 特殊长度输入

$$B = \left\lceil \frac{4097}{256} \right\rceil = 17, \quad N_{\text{launch}} = 17 \times 256 = 4352, \quad E = 4352 - 4097 = 255.$$

该例达到单个线程块大小下“额外线程数”的最大可能值 $T - 1$ ，说明边界检查不是罕见特例，而是通用模板的一部分。

2.11 常见错误、检查清单与公式表

2.11.1 常见错误

错误	后果与修正
把 $\lll B, T \rrr$ 的顺序记反	第一个参数是线程块数，第二个参数是每块线程数。
写成 $\text{ceil}(n/T)$ ，但 n, T 都是整数	整数除法先截断， ceil 已无作用。改用 $(n+T-1)/T$ 或浮点除法后取整。
遗漏 $\text{if } (i < n)$	最后一个线程块中的额外线程可能越界访问。
在主机端解引用设备指针	主机普通代码不应直接访问设备地址；通过 CUDA API 或核函数使用。

错误	后果与修正
不检查 CUDA 返回码	内存不足、非法配置或设备执行错误会被延迟发现，调试困难。
直接用 CPU 计时包围核函数启动	异步启动导致测到的主要是提交开销。计时结束前必须同步，后续宜使用 CUDA 事件。
假设线程块按编号顺序执行	CUDA 可按任意顺序调度线程块；不同线程块应在没有全局同步的情况下保持独立。
忘记释放设备内存	长时间运行或共享 GPU 环境中会造成资源泄漏与后续分配失败。

2.11.2 写一维 CUDA 核函数前的检查清单

1. 明确一个线程负责的输出元素或数据片段；
2. 写出全局索引公式 $i = bT + t$ ；
3. 计算 $B = \lceil n/T \rceil$ ；
4. 对不整除情况加入边界检查；
5. 明确每个指针位于主机还是设备；
6. 检查分配、复制、启动、执行与释放的错误状态；
7. 不对不同线程块的执行先后作假设；
8. 验证结果后再讨论性能。

2.11.3 核心公式表

问题	公式
一维全局线程索引	$i = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}$
需要的线程块数	$B = \lceil n/T \rceil = \lfloor (n + T - 1)/T \rfloor$
实际启动线程数	$N_{\text{launch}} = BT$
额外线程数	$E = BT - n$, 且 $0 \leq E < T$
k 元素/线程的逻辑线程数	$L = \lceil n/k \rceil$
k 元素/线程的线程块数	$B = \lceil L/T \rceil$
n 个 <code>float</code> 的字节数	$S = n \cdot \text{sizeof}(\text{float})$
局部变量逻辑副本数	网格总线程数 BT

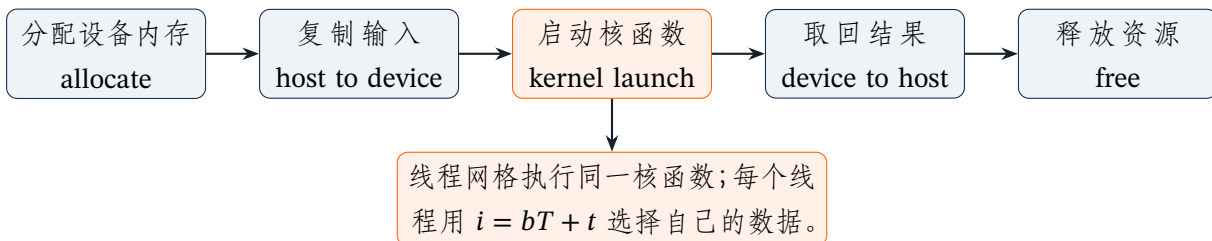
2.11.4 术语速查

中文术语	英文标准名称
数据并行	data parallelism
任务并行	task parallelism
线程 / 线程块 / 网格	thread / thread block / grid
核函数 / 核函数启动	kernel / kernel launch
单程序多数据	Single Program Multiple Data (SPMD)
主机 / 设备	host / device
主机内存 / 设备全局内存	host memory / device global memory
边界检查	boundary check
向上取整除法	ceiling division
异步调用 / 同步点	asynchronous call / synchronization point
函数限定符	function qualifier
即时编译	just-in-time compilation (JIT compilation)

2.12 综合框架与关键结论

数据并行 CUDA 程序的基本结构可以归纳为以下完整链条：

1. 从数据依赖出发，定义每个线程的独立工作；
2. 用主机代码管理数据与调度，用设备核函数执行高并行计算；
3. 用网格-线程块-线程层次和 $i = bT + t$ 建立执行到数据的映射；
4. 用显式内存管理完成分配、传输、计算、回传与释放；
5. 用向上取整启动和边界检查覆盖任意长度输入；
6. 用 NVCC 编译主机与设备代码，并正确处理异步执行与错误状态；
7. 先验证正确性，再进行性能优化。



关键公式

$$i = \text{blockIdx.x} \cdot \text{blockDim.x} + \text{threadIdx.x}, \quad B = \frac{n + T - 1}{T},$$

if ($i < n$) 保护边界, $\langle\langle B, T \rangle\rangle$ 依次表示线程块数与每块线程数.

第 3 讲

CUDA Parallel Execution Model: Multidimensional Grids and Data

3.1 核心内容

内容概览

- 理解 CUDA 线程的多维逻辑组织；
- 了解如何在核函数中使用循环等控制结构。

3.1.1 并行线程数组与单程序多数据模型

CUDA 核函数启动后，由一个线程网格执行。网格中的所有线程运行同一份核函数代码，但每个线程都能读取一组与自身位置有关的只读内建变量，从而确定自己负责的数据位置。该执行方式属于单程序多数据 (single-program, multiple-data, SPMD) 模型：

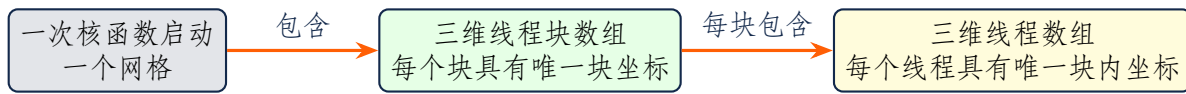
同一程序 + 不同线程标识 $\implies$ 不同数据上的并行实例。

定义

网格是一次核函数启动所创建的全部线程块的逻辑集合；线程块是能够使用块内共享机制进行协作的一组线程。因而，一次常规核函数启动形成两级层次：

网格 $\longrightarrow$ 线程块 $\longrightarrow$ 线程。

每个网格内的线程块具有相同的块尺寸，每个线程块内的线程执行相同的核函数代码。



未使用的维度尺寸取 1；所有线程执行同一核函数，并通过坐标选择不同数据。

图 3.1 CUDA 的逻辑执行层次

3.1.2 块内协作与块间独立性

线程块既是逻辑索引单位，也是协作边界。块内线程可通过共享内存 (shared memory)、原子操作 (atomic operation) 和屏障同步 (barrier synchronization) 等机制协作。不同线程块不共享块内资源，通常主要通过全局内存 (global memory) 交换数据，因此块间协作更受限制。

这一约束直接解释了线程块设计的两个基本原则：

- 同一块内的线程应对应一组存在局部数据关系、未来可能需要协作的工作项；
- 各线程块应尽量能够独立完成工作，使硬件可以按任意次序调度线程块。

3.1.3 多维组织的目的

CUDA 将网格和线程块统一表示为至多三维的逻辑数组。即使应用只使用一维或二维，未使用的维度仍存在，其尺寸取为 1。多维组织的主要价值是让线程坐标直接对应数据坐标：

- 一维向量通常使用全局下标 i ；
- 二维图像或矩阵通常使用行列坐标 (r, c) ；
- 三维体数据通常使用坐标 (z, y, x) 或 (d, r, c) 。

这种对应关系降低了地址计算的复杂度，也使边界判断与任务划分更清晰。多维网格并未改变存储器本身的一维线性地址空间；它提供的是更自然的逻辑坐标系。逻辑坐标仍需在线程内部转换为线性内存偏移。

关键结论

多维线程组织解决的是“线程负责哪个逻辑数据元素”；行主序等数据布局解决的是“该逻辑元素位于线性内存的哪个位置”。二者需要连续完成：先求全局逻辑坐标，再求线性地址。

3.2 多维执行层次与索引系统

3.2.1 四组只读内建变量

每个正在执行的线程都可读取四组 CUDA 只读内建变量。它们共同给出“网格有多大、当前块在哪里、块有多大、当前线程在块内哪里”。

表 3.1 多维线程组织中的四组内建变量

变量	含义	三个分量及取值范围
<code>gridDim</code>	网格在各方向上的线程块数量	<code>x</code> 、 <code>y</code> 、 <code>z</code> ；每个分量都是相应方向上的块数
<code>blockIdx</code>	当前线程块在网格中的坐标	$0 \leq \text{blockIdx}.d < \text{gridDim}.d$ ，其中 $d \in \{x, y, z\}$
<code>blockDim</code>	每个线程块在各方向上的线程数量	<code>x</code> 、 <code>y</code> 、 <code>z</code> ；三者乘积为每块线程总数
<code>threadIdx</code>	当前线程在线程块内部的坐标	$0 \leq \text{threadIdx}.d < \text{blockDim}.d$ ，其中 $d \in \{x, y, z\}$

其中，`gridDim` 与 `blockDim` 描述尺寸，`blockIdx` 与 `threadIdx` 描述位置。对一维配置，未使用的 `y, z` 维尺寸为 1，对应索引只能取 0；对二维配置，未使用的 `z` 维尺寸为 1。

三维尺寸示例

若

$$\text{gridDim} = (8, 3, 2), \quad \text{blockDim} = (5, 4, 3),$$

则网格含有

$$8 \times 3 \times 2 = 48$$

个线程块，每块含有

$$5 \times 4 \times 3 = 60$$

个线程，总启动线程数为

$$48 \times 60 = 2880.$$

块坐标的最大值为 (7, 2, 1)，块内线程坐标的最大值为 (4, 3, 2)。

3.2.2 一维全局索引

仅使用线程的块内下标 `threadIdx.x` 无法唯一标识整个网格中的线程，因为不同块内可以存在相同的块内下标。设每块包含 $B_x = \text{blockDim.x}$ 个线程，当前块编号为 $b_x = \text{blockIdx.x}$ ，当前线程在块内的编号为 $t_x = \text{threadIdx.x}$ 。当前块之前共有 b_x 个完整线程块，因此它们共覆盖 $b_x B_x$ 个工作项；再加上块内偏移 t_x ，得到一维全局工作下标

$$i = b_x B_x + t_x.$$

对应的 CUDA 表达式为

代码 3.1 一维全局工作下标

```
1 unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
```

若网格在 x 方向有 $G_x = \text{gridDim.x}$ 个块，则启动线程覆盖的全局下标范围为

$$0 \leq i < G_x B_x.$$

数据长度通常并不恰好等于 $G_x B_x$ ，因此实际内存访问仍需检查 i 是否小于数据长度。

3.2.3 二维全局坐标

二维数据通常用行列坐标 (r, c) 表示。CUDA 习惯让 x 方向对应列，让 y 方向对应行。设

$$\begin{aligned} b_x &= \text{blockIdx.x}, & B_x &= \text{blockDim.x}, & t_x &= \text{threadIdx.x}, \\ b_y &= \text{blockIdx.y}, & B_y &= \text{blockDim.y}, & t_y &= \text{threadIdx.y}. \end{aligned}$$

则线程负责的全局行列坐标为

$$r = b_y B_y + t_y, \quad c = b_x B_x + t_x$$

即

代码 3.2 二维线程坐标到全局数据坐标的映射

```
1 unsigned int row = blockIdx.y * blockDim.y + threadIdx.y;
2 unsigned int col = blockIdx.x * blockDim.x + threadIdx.x;
```

这两个式子彼此独立： x 方向只决定列坐标， y 方向只决定行坐标。其几何意义与一维情形相同，都是“前面完整块的跨度加上块内偏移”。

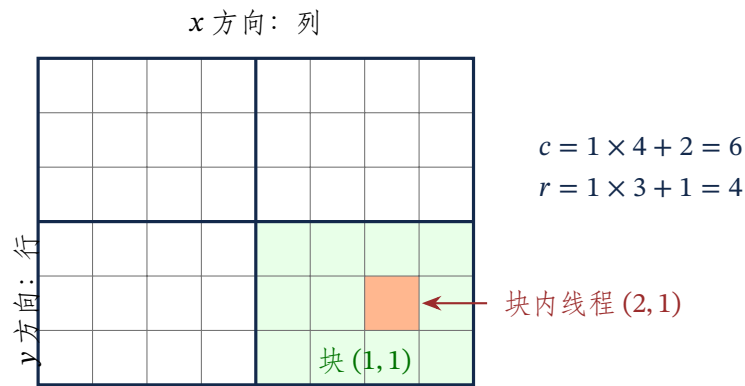


图 3.2 二维索引示例: 块尺寸为 4×3 , 高亮线程的全局坐标为 $(r, c) = (4, 6)$

关键结论

在二维图像或矩阵核函数中, 最常见的坐标约定是

$$x \leftrightarrow \text{列}, \quad y \leftrightarrow \text{行}.$$

因此, 宽度与 x 方向相关, 高度与 y 方向相关。将二者交换会导致转置式访问、错误边界判断或越界。

3.2.4 三维全局坐标

对体数据或三维张量, 可逐维应用同一构造。若把 z 方向解释为深度, 则

$$\begin{aligned} c &= \text{blockIdx.x} \text{blockDim.x} + \text{threadIdx.x}, \\ r &= \text{blockIdx.y} \text{blockDim.y} + \text{threadIdx.y}, \\ d &= \text{blockIdx.z} \text{blockDim.z} + \text{threadIdx.z}. \end{aligned}$$

每个线程因此对应一个逻辑体素 (d, r, c) 。各维度的计算互不干扰, 边界条件也应逐维检查。

3.2.5 块数、每块线程数与总线程数

记网格尺寸为

$$(G_x, G_y, G_z) = (\text{gridDim.x}, \text{gridDim.y}, \text{gridDim.z}),$$

块尺寸为

$$(B_x, B_y, B_z) = (\text{blockDim.x}, \text{blockDim.y}, \text{blockDim.z}).$$

则

$$\begin{aligned} N_{\text{blocks}} &= G_x G_y G_z, \\ N_{\text{threads/block}} &= B_x B_y B_z, \\ N_{\text{threads,total}} &= G_x G_y G_z B_x B_y B_z. \end{aligned}$$

这些乘积计算的是启动的线程数量。有效工作项数量由数据尺寸决定，边界处多出的线程会通过条件判断退出。

三维逻辑坐标的线性化

多维线程索引可逐维使用；在需要给全部启动线程构造一个线性序号时，可进一步线性化。令

$$N_x = G_x B_x, \quad N_y = G_y B_y,$$

并用 (g_z, g_y, g_x) 表示线程的三维全局坐标，则按 x 最快变化的顺序，线性线程坐标可写为

$$q = (g_z N_y + g_y) N_x + g_x.$$

该式只是坐标表示的补充。处理二维数据时，直接保留 (r, c) 通常更清晰，无需先构造 q 再反解行列。

3.2.6 每次核函数启动拥有独立配置

同一应用中的多个核函数可以使用彼此独立的网格尺寸与块尺寸。配置属于某一次核函数启动，不是整个程序的固定属性。图像处理阶段可以采用二维组织，向量阶段可以采用一维组织，体数据计算可以采用三维组织。只要每次启动时传入的配置与该核函数的工作项形状一致即可。

3.3 多维启动配置与数据布局

3.3.1 使用 **dim3** 指定块与网格尺寸

核函数启动语法中的两个配置参数分别表示网格尺寸和线程块尺寸：

代码 3.3 二维网格的典型启动配置

```
1 dim3 numThreadsPerBlock(32, 32);  
2  
3 dim3 numBlocks(  
4     (width + numThreadsPerBlock.x - 1) / numThreadsPerBlock.x,  
5     (height + numThreadsPerBlock.y - 1) / numThreadsPerBlock.y);  
6  
7 kernel<<<numBlocks, numThreadsPerBlock>>>(/* kernel arguments */);
```

内建类型 **dim3** 用三个分量保存 x, y, z 方向的尺寸。上例只显式给出两个分量，因此 z 分量取 1。核函数内部读到的 **blockDim** 等于启动时传入的 **numThreadsPerBlock**，读到的 **gridDim** 等于 **numBlocks**。

二维数据的启动配置遵循以下对应关系：

$$\begin{aligned} B_x &= \text{线程块宽度}, & B_y &= \text{线程块高度}, \\ G_x &= \text{水平方向块数}, & G_y &= \text{竖直方向块数}. \end{aligned}$$

其中 $B_x B_y$ 是每块线程数， $G_x G_y$ 是网格中的块数。

3.3.2 向上取整除法

若数据宽度为 W ，每块在 x 方向覆盖 B_x 个元素，则所需块数必须满足

$$G_x B_x \geq W.$$

取满足该条件的最小整数，得到

$$G_x = \left\lceil \frac{W}{B_x} \right\rceil, \quad G_y = \left\lceil \frac{H}{B_y} \right\rceil.$$

对正整数，可用整数运算实现为

$$\left\lceil \frac{W}{B_x} \right\rceil = \frac{W + B_x - 1}{B_x},$$

右侧除法按整数除法截断。

整数向上取整公式为何成立

令 $W = qB_x + r$ ，其中 $0 \leq r < B_x$ 。

- 若 $r = 0$ ，则 $(W + B_x - 1)/B_x = q + (B_x - 1)/B_x$ ，整数除法结果为 q ；
- 若 $r > 0$ ，则 $W + B_x - 1 = (q + 1)B_x + (r - 1)$ ，整数除法结果为 $q + 1$ 。

两种情形分别对应整除与非整除，恰好等于 $\lceil W/B_x \rceil$ 。

3.3.3 覆盖矩形与多余线程

向上取整保证所有有效元素都被覆盖，但通常会启动一部分落在数据范围外的线程。以宽 $W = 76$ 、高 $H = 62$ 的图像和 16×16 线程块为例：

$$G_x = \left\lceil \frac{76}{16} \right\rceil = 5, \quad G_y = \left\lceil \frac{62}{16} \right\rceil = 4.$$

因此启动 $5 \times 4 = 20$ 个线程块，共有

$$20 \times 16 \times 16 = 5120$$

个线程，而图像只有

$$76 \times 62 = 4712$$

个像素。多出的 $5120 - 4712 = 408$ 个线程必须通过边界条件排除。

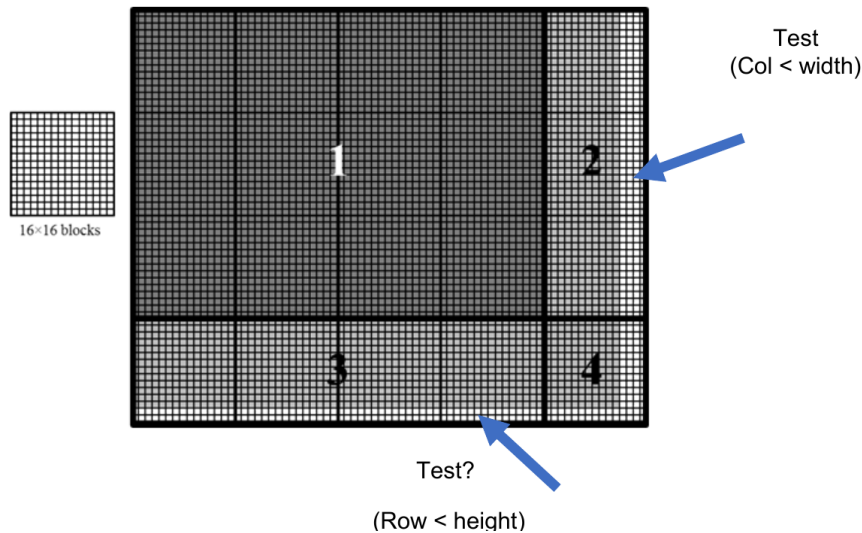


图 3.3 76 × 62 图像由 16 × 16 线程块向上取整覆盖。右侧、底部以及右下角区域包含越过图像边界的线程。

对应的二维有效性条件为

代码 3.4 二维输出坐标的边界保护

```
1 if (row < height && col < width) {
2     // All output memory accesses using (row, col) are valid here.
3 }
```

需要特别注意

向上取整只保证“线程数量足够”，不保证每个线程都对应有效数据。只要数据尺寸可能不是块尺寸的整数倍，就必须在访问内存前检查每个相关维度。

3.3.4 二维数据的行主序布局

C 与 C++ 按行主序 (row-major order) 存放普通二维数组：同一行的元素在线性内存中连续，整行存完后才存下一行。设矩阵或图像共有 H 行、 W 列，元素的逻辑坐标为 (r, c) ，其中

$$0 \leq r < H, \quad 0 \leq c < W.$$

第 r 行之前共有 r 个完整行，每行含 W 个元素，因此该行起点的线性偏移为 rW ；再加上行内偏移 c ，得到

$$\text{index}(r, c) = rW + c.$$

若数组首地址为 p 、元素类型为 T ，对应字节地址为

$$\text{addr}(r, c) = p + (rW + c) \text{sizeof}(T).$$

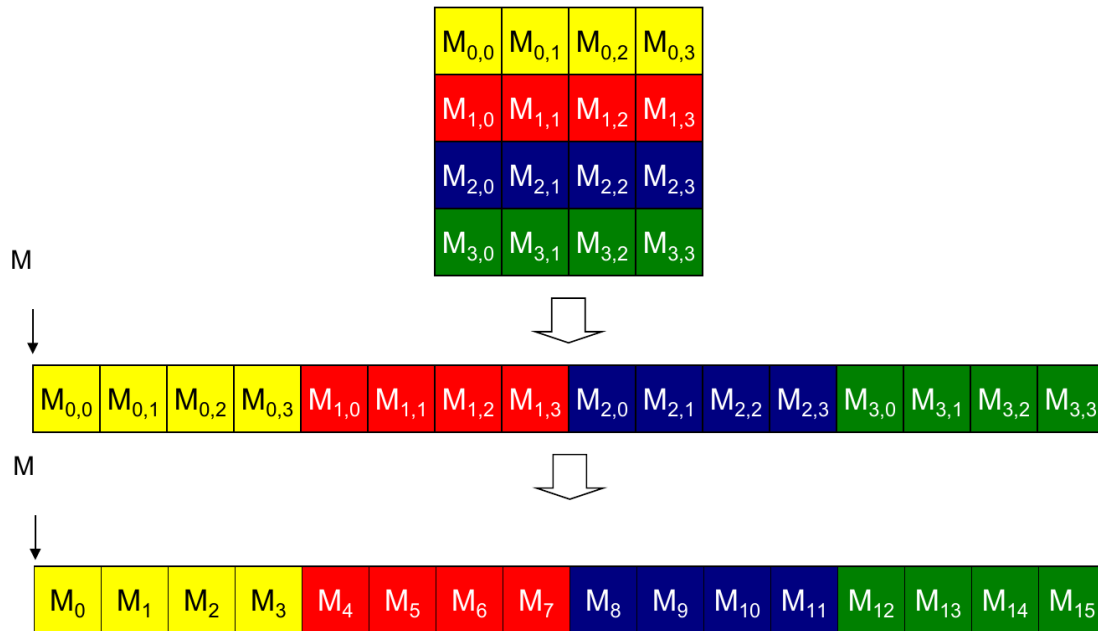


图 3.4 4×4 矩阵按行主序展开为一维连续序列。同行元素相邻，下一行紧随上一行。

从二维坐标到线性下标

对宽度 $W = 4$ 的矩阵，元素 $M_{2,1}$ 位于第 2 行、第 1 列，采用从 0 开始的下标。其线性下标为

$$\text{index}(2, 1) = 2 \times 4 + 1 = 9.$$

这意味着它是从线性下标 0 开始计数的第 10 个元素。

行主序公式还直接给出两个局部关系：

$$\text{index}(r, c + 1) = \text{index}(r, c) + 1,$$

说明同一行相邻列在内存中相邻；而

$$\text{index}(r + 1, c) = \text{index}(r, c) + W,$$

说明同一列相邻行之间相隔 W 个元素。

3.3.5 三维数据的线性化补充

若三维数据尺寸为深度 D 、高度 H 、宽度 W ，并采用 x 对应列、 y 对应行、 z 对应深度的约定，则可先将 (d, r, c) 看成第 d 个二维平面中的位置。每个平面含 HW 个元素，因此

$$\text{index}(d, r, c) = (dH + r)W + c = dHW + rW + c.$$

该式是二维行主序的逐层扩展。核函数仍应先验证

$$0 \leq d < D, \quad 0 \leq r < H, \quad 0 \leq c < W,$$

再使用线性下标访问内存。

关键结论

二维核函数的标准地址链条为

$$(\text{blockIdx}, \text{threadIdx}) \longrightarrow (r, c) \longrightarrow rW + c \longrightarrow \text{内存访问}.$$

每一步使用不同信息：线程层次给出逻辑坐标，数据布局给出线性偏移，边界条件保证该偏移有效。

3.4 实例一：RGB 彩色图像转灰度

3.4.1 数据并行分解

红绿蓝 (red-green-blue, RGB) 图像的每个像素包含三个颜色分量。三个独立的一维数组 **red**、**green** 和 **blue** 分别保存三个输入分量，数组 **gray** 保存灰度输出。四个数组的长度均为 WH ，其中 W 为图像宽度， H 为图像高度。

每个输出像素只依赖同一位置的三个颜色分量，不依赖其他像素。因此可把输出空间直接作为并行工作空间：

$$\boxed{\text{一个线程} \longleftrightarrow \text{一个输出像素}}.$$

线程在二维网格中先求自己负责的 (r, c) ，再用 $p = rW + c$ 访问三个输入通道并写入一个输出值。

设某像素的红、绿、蓝分量分别为 R_p, G_p, B_p ，灰度计算为

$$\boxed{Y_p = \frac{3}{10}R_p + \frac{6}{10}G_p + \frac{1}{10}B_p}.$$

三个系数之和为 1，因此当三个输入均位于 $[0, 255]$ 时，浮点中间结果也位于该区间内。

单个像素的灰度计算

若某像素满足

$$R_p = 100, \quad G_p = 150, \quad B_p = 200,$$

则

$$Y_p = 0.3 \times 100 + 0.6 \times 150 + 0.1 \times 200 = 30 + 90 + 20 = 140.$$

该像素的三个输入值由一个线程读取，结果 140 由同一线程写入对应的灰度位置。

3.4.2 完整核函数

核函数使用三个独立通道数组与上述权重。线性下标 **pix** 在边界判断通过后计算，使边界保护与内存访问的对应关系更直接。

代码 3.5 带二维边界保护的 RGB 转灰度核函数

```

1 __global__
2 void rgb2gray_kernel(unsigned char* red,
3                     unsigned char* green,
4                     unsigned char* blue,
5                     unsigned char* gray,
6                     unsigned int width,
7                     unsigned int height)
8 {
9     unsigned int row = blockIdx.y * blockDim.y + threadIdx.y;
10    unsigned int col = blockIdx.x * blockDim.x + threadIdx.x;
11
12    if (row < height && col < width) {
13        unsigned int pix = row * width + col;
14
15        gray[pix] = red[pix]    * 3.0f / 10.0f
16                  + green[pix] * 6.0f / 10.0f
17                  + blue[pix]  * 1.0f / 10.0f;
18    }
19 }

```

后缀 **f** 表示单精度浮点字面量。乘加表达式先产生浮点中间结果，再在赋给 **unsigned char** 时转换为字节值。源实现未加入额外舍入步骤，因此若结果带小数部分，赋值转换会去除其小数部分。

3.4.3 启动配置

采用 16×16 线程块时，二维启动配置可写为

代码 3.6 RGB 转灰度核函数的二维启动配置

```

1 dim3 threadsPerBlock(16, 16);
2 dim3 numBlocks(
3     (width + threadsPerBlock.x - 1) / threadsPerBlock.x,
4     (height + threadsPerBlock.y - 1) / threadsPerBlock.y);
5
6 rgb2gray_kernel<<<numBlocks, threadsPerBlock>>>(
7     red, green, blue, gray, width, height);

```

该配置创建的矩形线程区域至少覆盖整个图像。右边界与下边界多出的线程仍会执行到条件判断，但不会读取或写入任何像素。

3.4.4 一次线程索引的完整追踪

考虑宽 $W = 76$ 、高 $H = 62$ 的图像，块尺寸为 16×16 。某线程满足

$$\text{blockIdx} = (3, 2), \quad \text{threadIdx} = (5, 7).$$

其全局列、行坐标分别为

$$c = 3 \times 16 + 5 = 53,$$

$$r = 2 \times 16 + 7 = 39.$$

因为 $53 < 76$ 且 $39 < 62$ ，该线程对应有效像素。线性下标为

$$p = 39 \times 76 + 53 = 3017.$$

于是该线程读取

$$\text{red}[3017], \quad \text{green}[3017], \quad \text{blue}[3017],$$

并写入 $\text{gray}[3017]$ 。

若同一块中的线程改为 $\text{threadIdx.y} = 15$ ，则行坐标变为

$$r = 2 \times 16 + 15 = 47,$$

仍然有效；而位于最后一行块中的部分线程会得到 $r \geq 62$ ，这些线程由外层条件排除。

关键结论

该实例包含一条完整的二维逐元素模式：

二维线程坐标

↓

像素行列坐标 (r, c)

↓

线性下标 $p = rW + c$

↓

三个输入读取与一个输出写入。

所有四次内存访问共享同一个有效性条件 $r < H \wedge c < W$ 。

3.5 实例二：邻域平均模糊

3.5.1 从逐元素映射到邻域映射

灰度图像模糊仍采用“一个线程生成一个输出像素”的并行分解，但一个输出不再只依赖一个输入位置。设模糊半径为

$$R = \text{BLUR_SIZE}.$$

输出像素 (r, c) 的名义邻域为

$$\mathcal{N}_R(r, c) = \{(i, j) \mid r - R \leq i \leq r + R, c - R \leq j \leq c + R\}.$$

若该邻域完全位于图像内部，其中包含

$$(2R + 1)^2$$

个输入像素。当 $R = 1$ 时，内部像素使用 3×3 邻域，共 9 个输入。

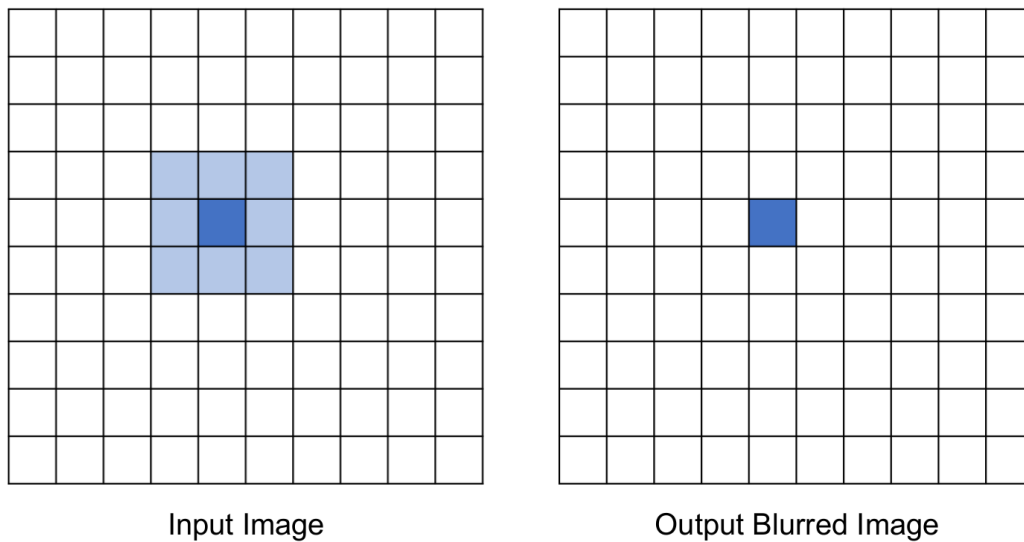


图 3.5 一个线程负责一个输出像素，并读取该位置周围的多个输入像素。图中半径为 1。

设图像有效区域为

$$\Omega = \{(i, j) \mid 0 \leq i < H, 0 \leq j < W\}.$$

真正参与边界像素计算的集合是

$$\mathcal{V}_R(r, c) = \mathcal{N}_R(r, c) \cap \Omega.$$

因此输出应定义为有效邻域的平均值：

$$B_{r,c} = \frac{1}{|\mathcal{V}_R(r, c)|} \sum_{(i,j) \in \mathcal{V}_R(r,c)} I_{i,j}.$$

3.5.2 核函数中的双重循环

每个线程知道自己的输出坐标 (`outRow`, `outCol`) 后，需要枚举邻域的所有候选输入坐标。二维邻域自然对应两层循环：外层改变输入行，内层改变输入列。

代码 3.7 未处理输入边界的基本邻域遍历

```

1 unsigned int sum = 0;
2
3 for (int inRow = outRow - BLUR_SIZE;
4     inRow < outRow + BLUR_SIZE + 1;
5     ++inRow) {
6     for (int inCol = outCol - BLUR_SIZE;
7         inCol < outCol + BLUR_SIZE + 1;
8         ++inCol) {
9         sum += image[inRow * width + inCol];
10    }
11 }
```

该代码只对完全位于图像内部的输出像素安全。即使输出坐标本身有效，邻域中的 `inRow` 或 `inCol` 仍可能越界。例如，左上角输出 (0,0) 在 $R = 1$ 时会尝试访问行 -1 或列 -1 。

3.5.3 两层边界条件

模糊核函数中存在两类不同的内存访问，因而需要两层保护：

1. **输出保护**：验证线程负责的输出坐标满足 $0 \leq \text{outRow} < H$ 且 $0 \leq \text{outCol} < W$ ，从而保证最终写入有效；
2. **输入保护**：对循环中每一个候选邻域坐标验证 $0 \leq \text{inRow} < H$ 且 $0 \leq \text{inCol} < W$ ，从而保证每次读取有效。

内存访问规则可形式化为：

每次内存访问都必须有对应的索引保护

对任意数组访问 `array[index]`，必须能够指出一条在该访问之前成立的条件，并由该条件推出 `index` 位于数组合法范围内。模糊核函数的输出写入由外层条件保护，输入读取由内层条件保护。

邻域索引使用有符号整数 `int`。这样，顶边和左边附近的候选坐标可以自然取负值，并由 `inRow >= 0` 与 `inCol >= 0` 排除。若这些变量使用无符号类型，减去模糊半径可能发生下溢，使边界逻辑难以按预期工作。

3.5.4 固定分母为何会产生错误

只跳过越界输入还不够。若始终除以名义邻域大小 $(2R+1)^2$ ，边界处没有被加入求和的像素仍被隐含地计入分母，相当于把缺失像素按 0 处理，输出会偏小。

当 $R = 1$ 时：

- 内部像素有 9 个有效邻居，分母为 9；
- 非角点边缘像素通常有 6 个有效邻居，分母应为 6；
- 角点像素有 4 个有效邻居，分母应为 4。

因此必须在累加有效像素时同步维护计数 `count`，最后使用 `sum / count`。

角点像素的有效邻域

考虑 3×3 灰度块

$$\begin{bmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \\ 70 & 80 & 90 \end{bmatrix}.$$

当 $R = 1$ 时，中心像素的九点平均为 50。左上角输出只使用

$$10, 20, 40, 50,$$

所以

$$\text{sum} = 120, \quad \text{count} = 4, \quad B_{0,0} = 120/4 = 30.$$

若错误地除以 9，则得到 13，明显把边界亮度压低。

3.5.5 完整核函数

代码 3.8 对有效邻域求平均的完整灰度模糊核函数

```
1 #define BLUR_SIZE 1
2
3 __global__
4 void blur_kernel(unsigned char* image,
5                  unsigned char* blurred,
6                  unsigned int width,
7                  unsigned int height)
8 {
9     int outRow = blockIdx.y * blockDim.y + threadIdx.y;
10    int outCol = blockIdx.x * blockDim.x + threadIdx.x;
11
12    if (outRow < height && outCol < width) {
13        unsigned int sum = 0;
14        unsigned int count = 0;
15
16        for (int inRow = outRow - BLUR_SIZE;
17             inRow < outRow + BLUR_SIZE + 1;
18             ++inRow) {
19            for (int inCol = outCol - BLUR_SIZE;
20                 inCol < outCol + BLUR_SIZE + 1;
21                 ++inCol) {
22                if (inRow >= 0 && inRow < height &&
23                    inCol >= 0 && inCol < width) {
24                    sum += image[inRow * width + inCol];
25                    ++count;
26                }
27            }
28        }
29
30        blurred[outRow * width + outCol]
31            = (unsigned char)(sum / count);
32    }
33 }
```

对任何通过外层条件的线程，中心输入(outRow,outCol)一定有效，因此 count 至少为 1，最终除法不会除以 0。

3.5.6 控制流与工作量

所有线程执行同一双重循环结构，但边界线程的内层条件只有部分迭代成立。对半径 R ，每个有效输出线程检查 $(2R + 1)^2$ 个候选位置；实际读取次数等于 $|\mathcal{V}_R(r, c)|$ 。这说明核函数内部可以包含普通控制结构：循环负责枚举一个线程的局部工作集，条件判断负责过滤无效位置。

关键结论

RGB 转灰度与模糊使用相同的输出并行分解，但单线程输入工作集不同：

计算	每线程输出	每线程输入读取
RGB 转灰度	1 个灰度像素	3 个同位置通道值
邻域模糊	1 个模糊像素	$ \mathcal{V}_R(r, c) $ 个邻域像素

线程数量由输出空间决定，单线程内部的循环由输入依赖范围决定。

3.6 实例三：矩阵与矩阵乘法

3.6.1 输出元素的定义

设方阵 $A, B \in \mathbb{R}^{N \times N}$ ，其乘积记为

$$C = AB.$$

采用从 0 开始的下标，输出元素 $C_{r,c}$ 是 A 的第 r 行与 B 的第 c 列的内积：

$$C_{r,c} = \sum_{k=0}^{N-1} A_{r,k} B_{k,c}.$$

固定 (r, c) 后，求和变量 k 枚举共享维度。不同输出位置写入 C 的不同元素，因此可以并行计算：

$$\boxed{\text{一个线程} \longleftrightarrow \text{一个输出矩阵元素}}.$$

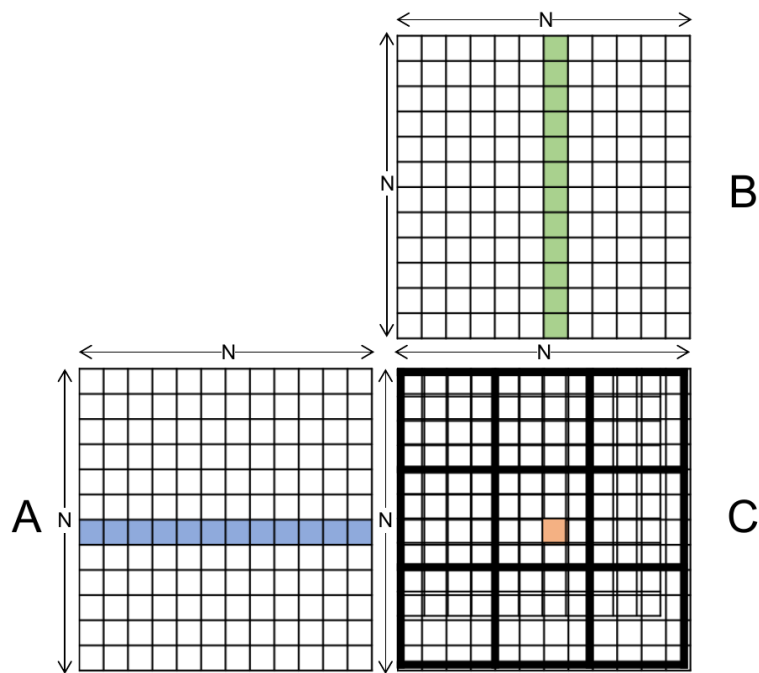


图 3.6 输出元素由 A 的一行与 B 的一列做逐项乘积并求和得到；二维线程网格覆盖输出矩阵。

这种分解与图像模糊具有相同的外层结构：线程坐标决定一个输出位置，线程内部再用循环遍历生成该输出所需的多个输入。区别在于，模糊遍历二维局部邻域，矩阵乘法遍历长度为 N 的共享维度。

3.6.2 简化方阵核函数

方阵情形可使用以下简化实现：

代码 3.9 简化的方阵乘法核函数

```

1 __global__
2 void mm_kernel(float* A, float* B, float* C,
3               unsigned int N)
4 {
5     unsigned int row = blockIdx.y * blockDim.y + threadIdx.y;
6     unsigned int col = blockIdx.x * blockDim.x + threadIdx.x;
7
8     float sum = 0.0f;
9     for (unsigned int k = 0; k < N; ++k) {
10         sum += A[row * N + k] * B[k * N + col];
11     }
12
13     C[row * N + col] = sum;
14 }

```

代码中三个线性下标分别对应

$$\begin{aligned}
 A_{r,k} &\longleftrightarrow A[\text{row} * N + k], \\
 B_{k,c} &\longleftrightarrow B[k * N + \text{col}], \\
 C_{r,c} &\longleftrightarrow C[\text{row} * N + \text{col}].
 \end{aligned}$$

该简化版本未包含边界条件。只有在启动线程区域恰好不超过 $N \times N$ 输出范围，或调用者另有保证时，所有访问才有效；一般实现必须显式处理边界。

3.6.3 一般矩形矩阵

矩阵不必为方阵。设

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}.$$

两者共享维度均为 K ，输出为

$$C = AB \in \mathbb{R}^{M \times N},$$

并满足

$$C_{r,c} = \sum_{k=0}^{K-1} A_{r,k} B_{k,c}, \quad 0 \leq r < M, \quad 0 \leq c < N.$$

这里三个矩阵的行主序宽度不同： A 的每行宽度为 K ， B 与 C 的每行宽度为 N 。因此正

确线性下标为

$$\text{index}_A(r, k) = rK + k,$$

$$\text{index}_B(k, c) = kN + c,$$

$$\text{index}_C(r, c) = rN + c.$$

把所有宽度都写成同一个 N 只适用于方阵情形。

带边界条件的矩形版本

将维度关系推广为 $M \times K$ 、 $K \times N$ 和 $M \times N$ ，并加入边界条件，可得到：

```

1  __global__
2  void mm_kernel_rect(float* A, float* B, float* C,
3                      unsigned int M,
4                      unsigned int K,
5                      unsigned int N)
6  {
7      unsigned int row = blockIdx.y * blockDim.y + threadIdx.y;
8      unsigned int col = blockIdx.x * blockDim.x + threadIdx.x;
9
10     if (row < M && col < N) {
11         float sum = 0.0f;
12         for (unsigned int k = 0; k < K; ++k) {
13             sum += A[row * K + k] * B[k * N + col];
14         }
15         C[row * N + col] = sum;
16     }
17 }
```

外层条件同时保护 A 的行索引、 B 的列索引与 C 的写入位置；循环条件 $k < K$ 保护共享维度上的两次读取。

3.6.4 网格尺寸由输出矩阵决定

由于每个线程生成一个 C 元素，线程网格应覆盖 $M \times N$ 输出空间。对块尺寸 (B_x, B_y) ，启动块数为

$$G_x = \left\lceil \frac{N}{B_x} \right\rceil, \quad G_y = \left\lceil \frac{M}{B_y} \right\rceil.$$

注意 x 方向对应输出列数 N ， y 方向对应输出行数 M ；共享维度 K 决定每个线程的循环长度，不决定二维网格的外部尺寸。

代码 3.10 矩形矩阵乘法的启动配置

```

1 dim3 threadsPerBlock(16, 16);
2 dim3 numBlocks(
3     (N + threadsPerBlock.x - 1) / threadsPerBlock.x,
4     (M + threadsPerBlock.y - 1) / threadsPerBlock.y);
5
6 mm_kernel_rect<<<numBlocks, threadsPerBlock>>>(A, B, C, M, K, N);

```

输出空间决定启动空间

若 A 为 5×3 , B 为 3×4 , 则 C 为 5×4 。采用 2×2 线程块时,

$$G_x = \left\lceil \frac{4}{2} \right\rceil = 2, \quad G_y = \left\lceil \frac{5}{2} \right\rceil = 3.$$

共启动 2×3 个块、24 个线程, 其中 20 个线程对应有效输出元素, 最后一行块中有 4 个线程因 $r \geq 5$ 被排除。每个有效线程执行 $K = 3$ 次乘加迭代。

3.6.5 单线程工作量与访问方式

每个输出元素的内积包含 K 次乘法和 $K - 1$ 次数学求和, 核函数循环执行 K 次累加。全部 MN 个输出元素的算术规模随 MNK 增长。

从行主序下标可直接观察两种输入访问:

- 当 k 增加 1 时, $A[\text{row} * K + k]$ 的下标也增加 1, 线程沿 A 的一行连续读取;
- 当 k 增加 1 时, $B[k * N + \text{col}]$ 的下标增加 N , 线程沿 B 的一列跨行读取。

相邻输出线程会重复需要 A 的同一行片段或 B 的同一列片段, 但简化核函数直接从全局内存逐次读取, 没有显式利用这些重用关系。因此, 该实现的主要问题是性能: 计算单元可能花费大量时间等待数据, 形成以内存访问为主的瓶颈。

关键结论

矩阵乘法的三个维度承担不同角色:

维度	作用
M	输出行数, 也决定网格的 y 向覆盖范围
N	输出列数, 也决定网格的 x 向覆盖范围
K	内积长度, 也决定每个线程的循环次数

混淆这些维度会直接造成错误的启动配置或错误的行主序地址。

3.7 例题与完整推导

3.7.1 统一计算框架

对三维网格尺寸

$$(G_x, G_y, G_z)$$

与三维块尺寸

$$(B_x, B_y, B_z),$$

依次计算

$$N_{\text{threads/block}} = B_x B_y B_z,$$

$$N_{\text{blocks}} = G_x G_y G_z,$$

$$N_{\text{threads,total}} = N_{\text{threads/block}} N_{\text{blocks}}.$$

未使用的维度取 1，不能把它们从概念上混成 0；尺寸为 0 会意味着不启动相应工作，而一维或二维配置的剩余维度实际为 1。

3.7.2 例题一：一维网格与二维线程块

题目

一个 CUDA 核函数启动 32 个线程块，每个线程块包含 32×32 个线程。求每块线程数与总启动线程数。

网格可写为

$$\text{gridDim} = (32, 1, 1),$$

线程块可写为

$$\text{blockDim} = (32, 32, 1).$$

因此每块线程数为

$$N_{\text{threads/block}} = 32 \times 32 \times 1 = 1024.$$

网格中的块数为

$$N_{\text{blocks}} = 32 \times 1 \times 1 = 32.$$

总线程数为

$$N_{\text{threads,total}} = 1024 \times 32 = 32768.$$

答案

每个线程块包含 1024 个线程；整个网格共启动 32768 个线程。

3.7.3 例题二：二维网格与二维线程块

题目

线程块配置为 16×16 ，网格尺寸为 32×16 个块。求每块线程数、块总数与整个网格的线程总数。

对应尺寸为

$$\text{blockDim} = (16, 16, 1), \quad \text{gridDim} = (32, 16, 1).$$

每块线程数为

$$N_{\text{threads/block}} = 16 \times 16 \times 1 = 256.$$

块总数为

$$N_{\text{blocks}} = 32 \times 16 \times 1 = 512.$$

因此总线程数为

$$N_{\text{threads,total}} = 256 \times 512 = 131072.$$

也可一次性验证：

$$(32 \times 16) \times (16 \times 16) = 131072.$$

答案

每块包含 256 个线程；网格含 512 个块；整个网格共启动 131072 个线程。

3.7.4 例题三：三维网格与三维线程块

题目

网格尺寸为 $8 \times 4 \times 2$ 个块，每个线程块尺寸为 $4 \times 8 \times 8$ 。求每块线程数与整个网格的线程总数。

每块线程数为

$$N_{\text{threads/block}} = 4 \times 8 \times 8 = 256.$$

网格中的块数为

$$N_{\text{blocks}} = 8 \times 4 \times 2 = 64.$$

总线程数为

$$N_{\text{threads,total}} = 256 \times 64 = 16384.$$

答案

每块包含 256 个线程；网格含 64 个块；整个网格共启动 16384 个线程。

3.7.5 综合例题：最后一个有效像素与第一个无效线程

考虑 76×62 图像，块尺寸为 16×16 ，因此

$$\text{gridDim} = (5, 4, 1).$$

图像最后一个有效像素坐标为

$$(r, c) = (61, 75).$$

取最后一个线程块

$$\text{blockIdx} = (4, 3),$$

并取块内线程

$$\text{threadIdx} = (11, 13).$$

其坐标为

$$c = 4 \times 16 + 11 = 75,$$

$$r = 3 \times 16 + 13 = 61.$$

因此该线程恰好负责最后一个有效像素，线性下标为

$$p = 61 \times 76 + 75 = 4711,$$

与长度 $76 \times 62 = 4712$ 的数组最后下标一致。

若只把块内 x 坐标增加 1，即取 `threadIdx.x=12`，则

$$c = 4 \times 16 + 12 = 76.$$

此时 $c < 76$ 不成立，该线程是最靠近右边界的无效线程之一。这个例子说明，线程的块内索引本身完全合法，并不意味着它对应的数据坐标合法；数据边界必须单独判断。

3.7.6 常见计算错误

- 把“网格中的块数”与“每块线程数”混为同一个量；
- 只乘二维图中显式写出的分量，却忘记未使用维度应乘以 1；
- 把 `gridDim` 当成线程总数，或把 `blockDim` 当成块总数；
- 认为启动线程总数必然等于有效数据元素数，忽略向上取整产生的边界线程；
- 在二维索引中把 x 当作行、 y 当作列，导致宽度与高度的判断互换。

3.8 统一方法与关键结论

3.8.1 从输出空间建立并行映射

三个实例都可用同一建模步骤构造。

1. **确定输出空间。** 向量输出是一维，图像与矩阵输出通常是二维，体数据输出可以是三维。
2. **确定单线程输出。** 一个线程生成一个输出元素，使不同线程写入互不重叠的位置。
3. **选择线程块形状。** 块的逻辑维度与输出坐标轴对应，二维情形通常令 x 对应列、 y 对应行。

4. 向上取整计算网格。每一维都用数据长度除以对应块尺寸并向上取整。
5. 计算全局逻辑坐标。逐维使用“块坐标乘块尺寸加块内线程坐标”。
6. 建立边界保护。输出写入与每一类输入读取都必须由可证明其下标有效的条件保护。
7. 线性化并计算。按行主序把逻辑坐标转换为线性偏移，再执行逐元素运算或线程内部循环。

二维输出核函数的通用骨架为

代码 3.11 二维逐输出元素核函数骨架

```

1  __global__
2  void kernel(/* arrays and parameters */, unsigned int width,
3             unsigned int height)
4  {
5      unsigned int row = blockIdx.y * blockDim.y + threadIdx.y;
6      unsigned int col = blockIdx.x * blockDim.x + threadIdx.x;
7
8      if (row < height && col < width) {
9          unsigned int out = row * width + col;
10
11          // Read the valid input working set for (row, col).
12          // Compute one output value.
13          // Write the result to output[out].
14      }
15  }

```

该骨架只规定输出映射。输入工作集由具体算法决定：可以是同位置的若干通道、一个局部邻域，也可以是一整行与一整列。

3.8.2 三个实例的结构对照

表 3.2 RGB 转灰度、模糊与矩阵乘法的并行结构

计算	输出空间	单线程输入工作集	线程内控制结构
RGB 转灰度	$H \times W$ 灰度图像	同一像素位置的红、绿、蓝三个分量	固定数量的乘加，无循环
邻域模糊	$H \times W$ 模糊图像	以输出像素为中心的有效邻域 $\mathcal{V}_R(r, c)$	两层循环与输入边界条件
矩阵乘法	$M \times N$ 输出矩阵	A 的第 r 行与 B 的第 c 列，共享长度为 K	一层长度为 K 的累加循环

三者的线程数量都由输出空间决定，而单线程工作量由生成一个输出所需的输入数量决定。这一区分有助于判断某个维度应放在网格中，还是应由线程内部循环遍历：

- 输出坐标通常映射到并行线程；
- 单个输出内部的归约维度通常映射到线程内循环。

3.8.3 关键公式汇总

一维全局下标: $i = \text{blockIdx.x} \text{blockDim.x} + \text{threadIdx.x},$

二维全局坐标: $r = \text{blockIdx.y} \text{blockDim.y} + \text{threadIdx.y},$
 $c = \text{blockIdx.x} \text{blockDim.x} + \text{threadIdx.x},$

二维行主序: $p = rW + c,$

二维块数: $G_x = \left\lceil \frac{W}{B_x} \right\rceil, \quad G_y = \left\lceil \frac{H}{B_y} \right\rceil,$

总启动线程数: $N_{\text{threads,total}} = (G_x G_y G_z)(B_x B_y B_z),$

矩形矩阵乘法: $C_{r,c} = \sum_{k=0}^{K-1} A_{r,k} B_{k,c}.$

3.8.4 正确性检查表

1. 网格的每个维度是否按对应输出尺寸向上取整？
2. 是否始终使用 x 对应列、 y 对应行的同一约定？
3. 全局坐标是否包含块偏移与块内偏移两部分？
4. 行主序公式中的宽度是否属于正在访问的那个数组？
5. 输出写入之前是否检查了所有输出坐标？
6. 每次输入读取之前是否检查了该输入访问涉及的全部坐标？
7. 邻域循环是否使用能够表示负候选坐标的有符号索引？

8. 边界处的平均值是否除以实际参与求和的元素数？
9. 矩阵乘法中，输出维度 M, N 与归约维度 K 是否各自使用在正确位置？

关键结论

多维网格的核心作用是把数据的几何结构直接编码到线程坐标中。正确核函数由三条关系组成：

线程层次 $\rightarrow$ 逻辑输出坐标, 逻辑坐标 $\rightarrow$ 线性内存地址, 有效性条件 $\rightarrow$ 安全访问.

循环与条件控制单个线程的局部工作集，不改变所有线程共享同一核函数代码这一单程序多数据模型。

第 4 讲

Compute Architecture and Scheduling

4.1 核心内容

内容概览

- 理解 GPU 架构与组织的基础；
- 理解线程调度、延迟容忍与硬件占用率；
- 区分驻留线程束与当前实际发射指令的线程束。

4.1.1 软件执行层次

CUDA 的软件执行层次由网格、线程块和线程组成。网格中的所有线程运行同一份核函数代码，每个线程通过线程索引和线程块索引选择自身负责的数据与工作。这种执行方式属于**单程序多数据** (*Single Program, Multiple Data, SPMD*) 模型。

线程块可以组织成一维、二维或三维形状。多维组织使线程索引能够自然对应多维数据，从而简化地址计算；维度本身并不会自动提高访存效率或计算性能。例如，二维图像既可以使用二维线程块，也可以使用一维线程块，后一种方式需要显式完成一维索引到二维坐标的映射。

关键结论

线程块的形状用于表达问题结构；硬件执行方式取决于线程块如何驻留到流式多处理器、如何拆分为线程束，以及线程束如何被调度。

4.1.2 线程块的三个基本语义

1. 线程块是资源分配与驻留的基本单位。同一线程块的全部线程被分配到同一个流式多处理器；一个线程块不会跨越多个流式多处理器执行。
2. 同一线程块内部可以高效协作。线程可以使用共享内存，并通过线程块级屏障进行同步。
3. 不同线程块的执行顺序没有保证。核函数必须在不依赖线程块调度顺序的前提下得到正确结果；同一程序由此能够在硬件并行度不同的 GPU 上透明伸缩。

4.1.3 软件层次到硬件层次的映射

软件执行层次与硬件执行资源之间的基本关系为

网格 → 线程块 → 流式多处理器驻留 → 线程束 → 处理块发射与执行。

线程束(warp)是硬件调度的基本单位。线程块驻留到某个流式多处理器后，其线程按照线性化线程索引划分为若干线程束；截至 Hopper 的所列 NVIDIA 架构均采用 32 线程的线程束。控制发散、延迟隐藏与占用率都以线程束为核心进行分析。

4.2 GPU 计算架构与线程块驻留

4.2.1 流式多处理器是主要硬件组织单元

一块 GPU 由多个流式多处理器(Streaming Multiprocessor, SM)组成。每个流式多处理器内部包含若干执行核心、控制资源和片上存储资源；多个流式多处理器通过更高层次的存储系统访问全局内存(global memory)。以 Hopper H100 的架构示意图为例，144 个流式多处理器、每个流式多处理器 128 个核心，对应

$$144 \times 128 = 18432$$

个核心。

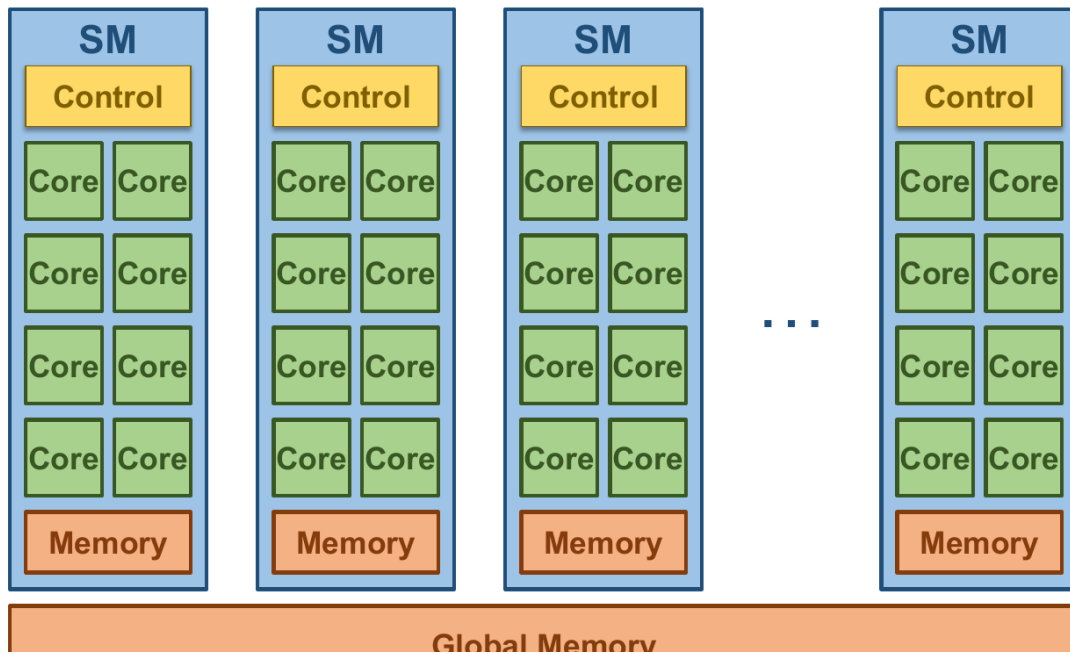


图 4.1 多个流式多处理器共享更高层次的全局内存；每个流式多处理器内部具有控制、执行核心和片上存储资源

定义

流式处理器 (Streaming Processor, SP) 或 CUDA 核心表示流式多处理器内部的标量执行核心。流式多处理器负责维护大量驻留线程的执行状态并调度线程束；流式处理器是实际完成相应算术工作的执行资源之一。两者不能用同一个“核心数”概念替代。

4.2.2 线程块以整体形式分配到一个流式多处理器

CUDA 线程不是逐个、独立地被分配到任意流式多处理器。硬件以**线程块粒度**完成驻留分配：一个线程块的全部线程都属于同一个流式多处理器，并在该流式多处理器上保持驻留直到线程块完成。一个线程块不能拆到多个流式多处理器上。

多个线程块可以同时驻留在同一个流式多处理器上，但前提是它们共同占用的硬件资源没有超过该流式多处理器的限制。尚未获得驻留资源的线程块处于等待状态；当已有线程块完成并释放资源后，等待线程块才可能获得驻留机会。

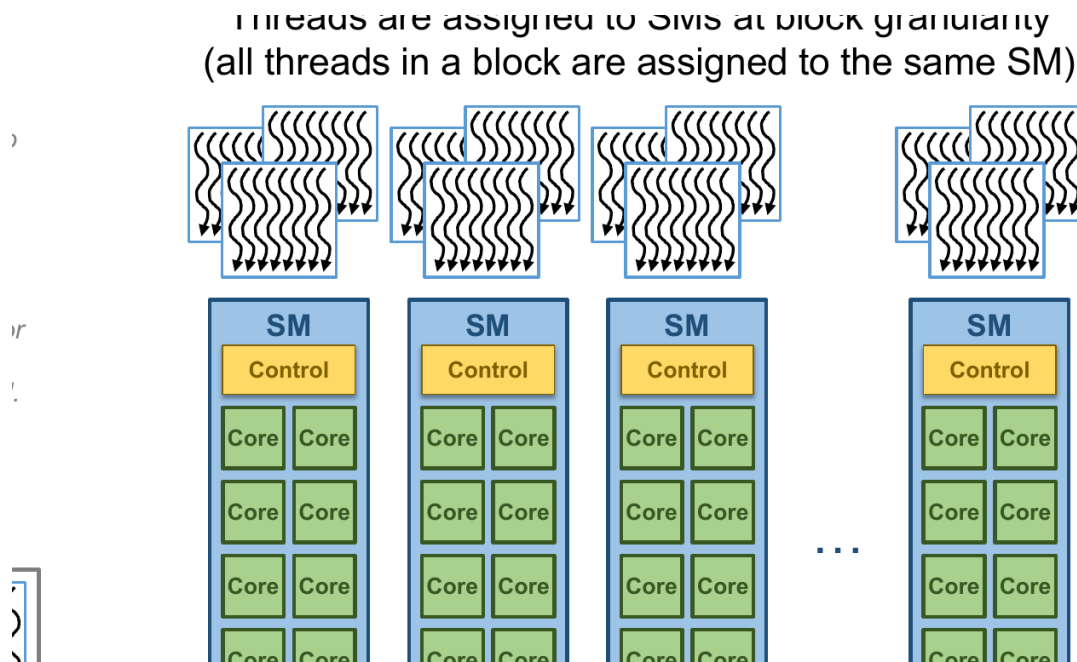


图 4.2 线程块以整体形式分配到流式多处理器；超出当前驻留能力的线程块必须等待资源释放

这里必须区分**驻留**和**正在执行**。假设某个流式多处理器能够容纳 2048 个驻留线程，并不意味着这 2048 个线程在同一个时钟周期中都各自占用一个物理执行核心。一个流式多处理器的物理执行核心数通常远小于可驻留线程数；驻留线程会进一步组成线程束，由调度器选择其中已经具备执行条件的线程束发射指令。

关键结论

“线程块被分配到某个流式多处理器”表示该线程块所需的执行状态和资源已在该流式多处理器上建立，并不表示线程块中的所有线程同时占用独立核心执行。

4.2.3 为什么同一线程块必须留在同一个流式多处理器

同一线程块中的线程依赖两类低开销协作资源：

- **共享内存 (shared memory)**: 物理资源位于流式多处理器上，但逻辑上按线程块分配；同一线程块中的线程可以访问该线程块的共享内存。
- **线程块级屏障**: 同一线程块的线程可以使用 `__syncthreads()` 等机制等待彼此达到同步点。

如果一个线程块被拆分到不同流式多处理器，这些低开销机制就失去统一的本地硬件基础。因此，线程块边界既是编程模型中的协作边界，也是硬件资源分配的重要边界。

不同线程块之间如果需要交换数据，通常必须依赖全局可见的存储与更高层次的同步机制。其成本和约束都显著高于线程块内部协作，因此 CUDA 的基本执行模型要求普通线程块之间不依赖具体的调度顺序。

4.2.4 驻留需要同时满足多类资源限制

线程块获得驻留位置之前，流式多处理器必须能够为整个线程块预留足够资源。主要约束包括：

- 每线程块最大线程数；
- 每流式多处理器最大驻留线程数；
- 每流式多处理器最大驻留线程块数；
- 寄存器文件容量；
- 共享内存容量；
- 最大驻留线程束数等架构限制。

因此，“线程数还放得下”只是必要条件之一。例如，一个线程块可能在线程数方面合法，但其寄存器或共享内存需求使得同一个流式多处理器只能驻留很少的线程块。后面的占用率分析将把这些约束统一起来。

4.3 线程块内同步、跨线程块同步与可伸缩性

4.3.1 线程块级屏障的语义

同一线程块内最基本的屏障同步由 `__syncthreads()` 提供。其核心语义是：到达该屏障的线程必须等待，直到线程块中所有尚未退出且应参与该屏障的线程都一致地到达同一个屏障实例，然后才能继续执行。

机制

线程块级屏障之所以可以高效实现，是因为同一线程块的全部驻留状态位于同一个流式多处理器中。屏障除了协调控制进度，还会对相关的共享内存和全局内存访问建立必要的顺序关系，使屏障之前的相关写入在屏障之后按照该同步语义被观察。

4.3.2 发散条件中的屏障为什么危险

下面的结构存在根本问题：

代码 4.1 发散条件中的两个不同屏障实例

```
1 if (predicate) {  
2     __syncthreads();  
3 } else {  
4     __syncthreads();  
5 }
```

表面上看，两条分支都出现了 `__syncthreads()`；但如果线程块中的不同线程沿不同分支前进，它们到达的可能是两个不同的静态屏障实例。这样无法保证全部参与线程在同一个屏障上会合，结果可能造成错误甚至死锁。

正确性要求可以概括为：对于某一个线程块级屏障实例，所有应参与的线程必须以兼容、可达且一致的方式到达它。屏障不能依赖会让同一线程块中的线程分成互不相交路径的控制结构。

需要特别注意

“每条分支里都写一个屏障”不等价于“所有线程到达同一个屏障”。判断同步是否正确，必须看线程是否会合到同一个屏障实例，而不是只看源代码中是否都出现了同名函数。

4.3.3 为什么线程块必须一次性获得完整驻留资源

线程块不能只让一部分线程先驻留，剩余线程等以后再分配。原因可以用屏障直接解释：假设线程块前半部分已经驻留并执行到屏障，而后半部分因为资源不足根本无法开始执行，那么前半部分会永久等待后半部分；同时前半部分占据的资源又可能阻止后半部分获得驻留机会，从而形成死锁。

所以，线程块只有在流式多处理器能够为**整个线程块**提供所需驻留资源时，才会被接纳。资源分配的整体性是线程块级同步语义能够成立的硬件条件之一。

4.3.4 普通线程块之间为何不能依赖执行顺序

如果不同线程块之间没有同步依赖，那么这些线程块可以：

- 以任意顺序开始；
- 在不同流式多处理器上并行执行；
- 在硬件资源较少时部分顺序执行；
- 在硬件资源较多时让更多线程块同时执行。

这形成了**透明可伸缩性** (*transparent scalability*)：同一份核函数代码可以在具有不同流式多处理器数量的 GPU 上运行，而正确性不依赖具体有多少线程块同时驻留。

关键结论

线程块执行顺序不受程序员控制，因此普通核函数中的正确性条件必须对所有可能的线程块调度顺序成立。

4.3.5 跨线程块屏障的死锁风险

若程序试图让整个网格中的线程块通过普通自制屏障互相等待，问题在于部分线程块可能尚未获得驻留。已经驻留的线程块先到达屏障并占用流式多处理器资源；未驻留的线程块却必须先运行到同一屏障才能解除等待。如果驻留资源被等待中的线程块占满，后续线程块就永远没有机会启动。

受约束的跨线程块同步机制主要包括：

- **协作组** (*cooperative groups*) 可以在满足特定驻留约束的条件下，支持线程块簇或整个网格范围的屏障；关键前提是相关参与者能够被同时调度到硬件上。
- 某些单向生产者—消费者同步可以通过更严格的执行结构建立先后关系，但普通线程块的调度顺序本身仍不能被程序假定。

因此，跨线程块同步属于比线程块内同步更高成本、约束更多的机制。除非算法明确需要并满足相应条件，否则应优先保持线程块之间的顺序独立性。

4.4 线程束、处理块与单指令多线程执行

4.4.1 线程块进一步划分为线程束

线程块驻留到流式多处理器后，硬件按照线程的线性化索引把它拆分成若干线程束(warp)。线程束大小由设备实现决定；截至 Hopper 的所列 NVIDIA 架构均采用 32 个线程。

若令线程束大小为 $W = 32$ ，线程块中线性化索引为 t 的线程属于

$$\text{warpId} = \left\lfloor \frac{t}{W} \right\rfloor.$$

因此，索引 0 ~ 31 属于线程束 0，索引 32 ~ 63 属于线程束 1，依此类推。一个包含 T_b 个线程的线程块需要

$$W_b = \left\lceil \frac{T_b}{W} \right\rceil$$

个线程束。

例题：三个 256 线程的线程块

每个线程块包含

$$256/32 = 8$$

个线程束。若三个这样的线程块同时驻留在同一个流式多处理器上，则共有

$$8 \times 3 = 24$$

个驻留线程束。这里的 24 表示可供调度的线程束状态数量，不表示 24 个线程束在同一时钟周期同时发射指令。

4.4.2 处理块与线程束调度器

流式多处理器内部的执行核心还会组织成若干**处理块 (processing block, PB)**。在简化的流式多处理器模型中，每个处理块具有自己的控制与线程束调度资源；线程束被调度到这些处理块上执行。

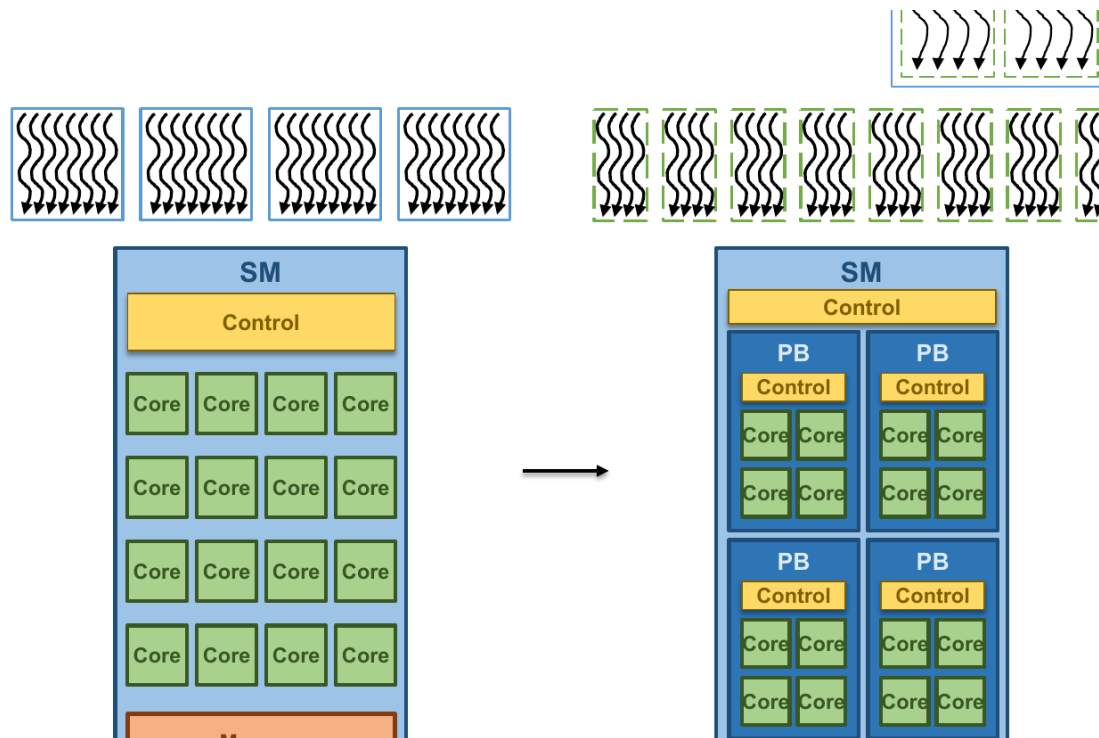


图 4.3 线程块先拆分成线程束；流式多处理器内部再由多个处理块及其控制资源调度这些线程束

以 Hopper H100 为例，128 个 FP32 核心组织成 4 个处理块，每个处理块包含 32 个 FP32 核心并共享相应的指令分派资源。该图是吞吐结构的简化表示，因为一个流式多处理器内还存在整数单元、加载/存储单元、FP64 单元与张量核心等其他执行资源。

4.4.3 从单程序多数据到单指令多线程

软件层次的**单程序多数据** (*Single Program, Multiple Data, SPMD*) 描述的是：大量线程运行同一个核函数程序，每个线程依据自己的索引和数据完成不同实例的工作。

硬件调度层次则使用**单指令多线程** (*Single Instruction, Multiple Threads, SIMT*)：当某个线程束被选中时，调度器为该线程束发射一条指令，各个活动线程在各自的数据和线程状态上执行这条指令。该组织与**单指令多数据** (*Single Instruction, Multiple Data, SIMD*) 具有相同的控制复用思想，即让多个执行通道共享指令获取和分派成本。

关键结论

SPMD 是程序员看到的核函数编程模型；SIMT 是线程束层次的硬件执行模型。二者的连接点是线程束：来自同一核函数的线程被成组调度，同一时刻的指令发射以线程束为单位。

这种结构的主要硬件收益是摊薄控制成本。若 32 个执行通道都要执行同一条指令，就不需要为每个通道复制完全独立的指令获取与分派逻辑。代价则是：如果线程束内部的线程需要走不同控制路径，执行通道不能始终保持同时有效，这就产生控制发散。

4.4.4 独立线程调度不改变性能分析的基本单位

从 Volta 开始，NVIDIA 引入**独立线程调度** (*independent thread scheduling*)。线程束中的线程可以维护更独立的执行状态，在发散后沿不同路径前进并在之后重新会合，不再要求所有线程在所有情况下表现为严格锁步。

性能分析仍把线程束视为关键单位，原因有两点：第一，执行资源和指令发射仍围绕线程束组织；第二，线程束内部的控制不一致依然会降低有效通道利用率。独立线程调度还意味着某些过去依赖隐含锁步行为的线程间协作必须使用更明确的同步机制。

4.4.5 驻留线程束、就绪线程束与实际发射线程束

对一个流式多处理器中的线程束，需要区分三个状态概念：

- **驻留线程束**：其线程状态、寄存器等资源已经分配在流式多处理器中；
- **就绪或可选线程束**：下一条指令所需操作数与执行条件已经满足，可以被调度器选择；
- **实际发射线程束**：当前获得发射机会、正在向执行资源送出指令的线程束。

占用率统计的是驻留规模，而不是“这一周期有多少线程束正在发射”。理解这一差别，是后续延迟隐藏和占用率分析的前提。

4.5 控制发散与有效执行通道

4.5.1 控制发散是线程束内部的问题

当同一线程束中的线程对分支条件得到不同结果，或执行不同数量的循环迭代时，就出现**控制发散** (*control divergence*)。线程束需要依次处理不同控制路径；处理某一路径时，只启用需要执行该路径的通道，其他通道处于非活动状态。

如果两个不同线程束分别选择不同路径，但每个线程束内部的 32 个线程都一致，则不存在该意义下的线程束内发散。因而判断发散时，关键问题是：**同一线程束的活动通道是否对当前控制条件意见一致。**

4.5.2 分支发散示例

考虑以下条件：

代码 4.2 线程束内的分支发散

```
1 if (threadIdx.x < 24) {  
2     A;  
3 } else {  
4     B;  
5 }  
6 C;
```

对第一个线程束，线程索引为 0 ~ 31。条件成立的线程有 24 个，条件不成立的线程有 8 个。因此执行路径 *A* 时有 24 个活动通道、8 个非活动通道；执行路径 *B* 时则相反。两条路径都完成后，线程束重新会合并共同执行 *C*。

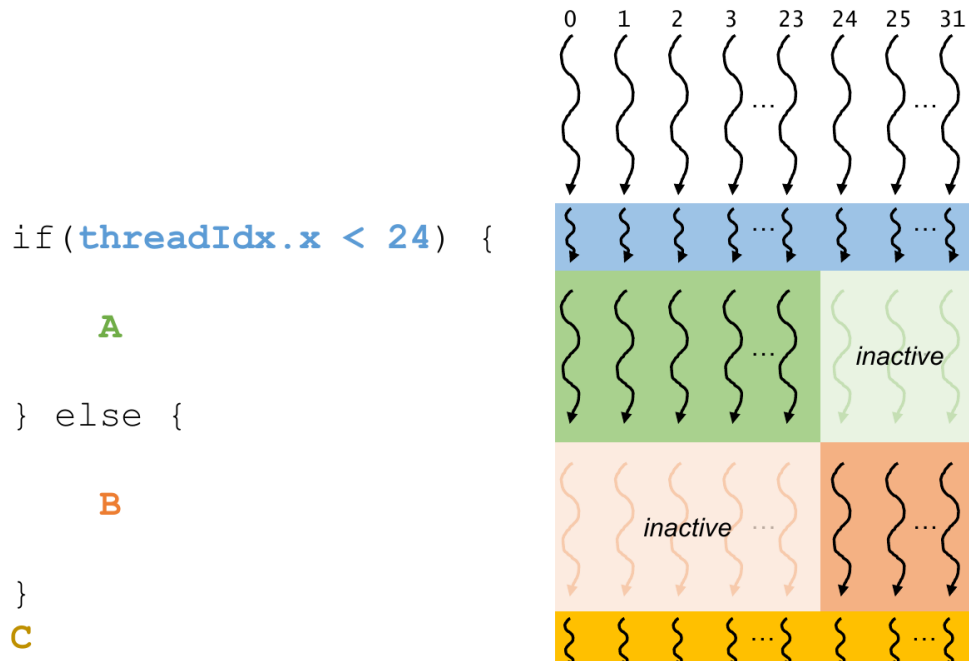


图 4.4 同一线程束中的线程在条件分支上分成两组时，两条路径对该线程束串行处理，未选择当前路径的通道保持非活动

4.5.3 活动通道比例与发散区间效率

活动通道比例 (active-lane fraction) 是一次 SIMT/SIMD 执行中启用的线程或执行通道所占比例。对于线程束大小 W ，某条路径上有 n 个活动线程时，活动通道比例为

$$f_{\text{active}} = \frac{n}{W}.$$

在上例中，路径 A 的活动通道比例是 $24/32 = 75\%$ ，路径 B 的活动通道比例是 $8/32 = 25\%$ 。若两条路径拥有相同数量的动态指令，则仅考虑分支区间时，两个串行阶段一共提供 $2W = 64$ 个通道-指令槽，其中实际有用的通道-指令数为 $24 + 8 = 32$ ，因此平均有效比例为

$$\eta = \frac{24 + 8}{2 \times 32} = 50\%.$$

若不同路径长度不同，可把这一思路写成加权形式。设路径 p 有 n_p 个活动线程，并执

行 I_p 条动态指令，则发散区间的通道利用率可写成

$$\eta = \frac{\sum_p n_p I_p}{W \sum_p I_p}.$$

该表达式只描述“每条路径串行执行、非参与通道关闭”造成的通道利用率，没有包含存储延迟、流水线吞吐或其他瓶颈。

4.5.4 循环也会产生控制发散

若每个线程的循环次数来自线程私有数据，例如

代码 4.3 每线程循环次数不同造成的发散

```
1 int N = a[threadIdx.x];
2 for (int i = 0; i < N; ++i) {
3     A;
4 }
```

则线程束中的线程可能在不同迭代结束。只要还有某个活动线程尚未达到自己的 N ，线程束就必须继续执行循环体；已经完成的线程通道则保持非活动。随着越来越多线程退出，活动通道比例会逐步下降。

因此，发散不仅来自显式的 **if/else**。任何依赖每线程数据的控制结构，包括循环退出条件，都可能让同一线程束内部的动态执行路径不同。

4.5.5 通过线程束粒度组织条件减少发散

一种减少分支发散的方法是让条件在一个线程束内部保持恒定。例如令条件依赖

$$\left\lfloor \frac{\text{threadIdx.x}}{\text{warpSize}} \right\rfloor,$$

则同一线程束内的线程具有相同商值。不同线程束仍然可以选择不同分支，但每个线程束内部不会因为该条件产生发散。

代码 4.4 线程束粒度的分支条件

```
1 if (threadIdx.x / warpSize > 2) {
2     // 路径 A
3 } else {
4     // 路径 B
5 }
```

这里仍有两个程序路径；优化点在于路径选择发生在线程束粒度，而不是在线程粒度。

4.5.6 边界检查中的少量发散通常不可避免

处理向量或图像时，问题规模经常不是线程块大小的整数倍。常见写法会向上取整线程块数量，然后在核函数中执行边界检查：

代码 4.5 向量尾部的边界检查

```
1 int i = blockIdx.x * blockDim.x + threadIdx.x;
2 if (i < N) {
3     // 有效工作
4 }
```

最后一个线程块中可能只有部分线程有效，因此最后若干线程束会出现发散。这类发散通常只影响计算尾部的一小部分线程束，代价有限。实际目标不是机械消除所有分支，而是避免让大量线程束长期处于严重的通道失活状态。

关键结论

减少控制发散的核心原则是让控制条件尽量在线程束内部保持一致。判断性能影响时，应关注发散路径的长度以及每条路径上的活动通道比例，而不是只统计源代码里有多少个分支语句。

4.6 线程束调度与延迟隐藏

4.6.1 线程束调度器选择可执行的线程束

流式多处理器同时维护多个驻留线程束。在线程束已有驻留状态之间切换发射机会，可近似视为**零开销线程束调度 (zero-overhead warp scheduling)**：当前线程束无法继续时，调度器可以迅速选择另一个已经就绪的驻留线程束，无需执行传统 CPU 线程上下文切换所需的完整保存与恢复。

线程束只有在下一条指令具备执行条件时才能成为候选对象，其中一个关键条件是所需操作数已经可用，即操作数已“ready for consumption”。

机制

每个驻留线程已经在流式多处理器的寄存器文件中占有相应状态；线程束还具有程序计数器等控制状态。在线程束之间切换发射机会时，不需要把整个线程上下文先写回内存再恢复。调度器主要是在已有驻留状态之间选择下一批要推进的线程束。

CUDA 不向程序员暴露具体的线程束优先级策略。程序可以依赖的是：满足执行条件的驻留线程束会被硬件调度以持续推进，而不能编写依赖某个固定线程束优先顺序的程序。

4.6.2 长延迟操作如何造成线程束停顿

算术指令所需操作数通常要先进入寄存器。若线程束执行高延迟的全局内存加载，而后续指令立即依赖加载结果，那么在数据返回之前，该线程束无法继续执行依赖指令，因而处于等待状态。

类似地，某些需要多个周期的执行资源也可能造成依赖停顿。关键并不在于停顿来源是内存还是计算，而在于：当前线程束的下一条指令暂时无法发射。

4.6.3 用其他线程束覆盖等待时间

延迟隐藏(*latency hiding*)的核心是把一个线程束的等待区间与其他线程束的有用执行重叠。设线程束 W_0 发出高延迟操作后必须等待；只要同一流式多处理器中存在另一个已经就绪的线程束 W_1 ，调度器就可以让 W_1 获得执行机会。若 W_1 之后也停顿，则继续寻找 W_2 、 W_3 等可执行线程束。

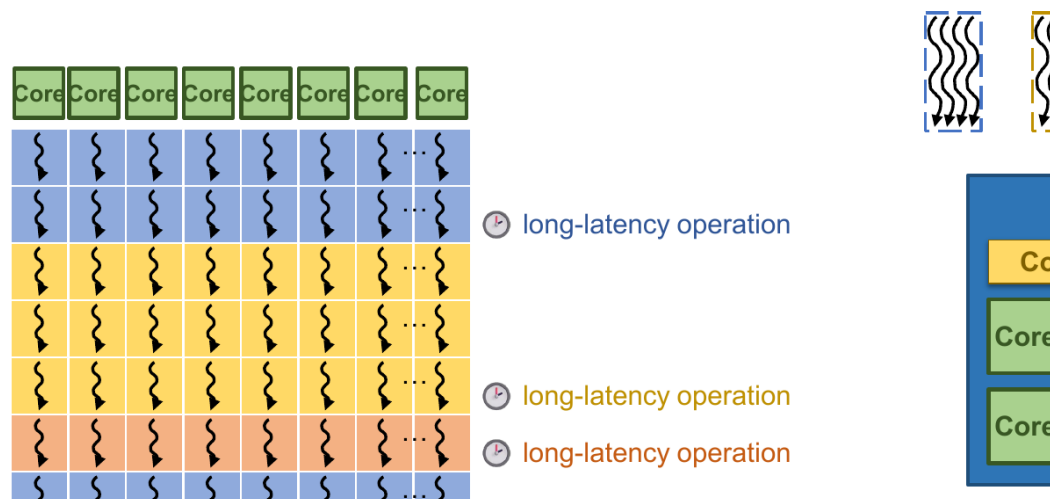


图 4.5 某个线程束遇到高延迟操作后，处理块可以选择其他就绪线程束继续执行，从而把等待时间与有用工作重叠

延迟隐藏不缩短一次全局内存访问本身需要的时间；它通过在这段时间内执行其他独立工作来减少执行资源的空闲时间。因而判断它是否有效时，应问“等待期间有没有别的线程束可以推进”，而不是只问“单次操作的延迟是多少”。

4.6.4 为什么驻留线程数远大于物理核心数

一个流式多处理器通常支持的最大驻留线程数明显大于其物理执行核心数，这使硬件能够保持较大的候选工作池。

假设流式多处理器中只有很少线程束。如果这些线程束同时发出长延迟加载，调度器就可能找不到任何就绪线程束，执行资源只能空等。若驻留线程束数量更多，则在部分线程束等待时，其他线程束已经准备好操作数的概率通常更高。

因此可以把关系概括为

更多驻留线程束 $\implies$ 更大的就绪候选集合 $\implies$ 更强的延迟隐藏机会。

这条关系解释了为什么占用率常常重要：占用率提高意味着流式多处理器上同时驻留的线程束更多，从而更可能在某个线程束停顿时找到替代工作。

4.6.5 延迟隐藏失效的典型情形

延迟隐藏只有在存在其他可执行线程束时才有作用。以下情况会使“增加线程束”无法继续解决问题：

- **内存带宽已经饱和。**更多线程束只会增加等待请求，不能提高已经受带宽上限约束的吞吐；
- **所有线程束都受到同类未解决依赖阻塞。**如果没有任何线程束的下一条指令已经就绪，调度器仍然无事可做；
- **缺少足够的指令级并行。**指令级并行 (*instruction-level parallelism*) 不足时，各线程束可能都很快进入依赖停顿，无法形成可交错执行的独立工作。

关键结论

延迟隐藏的必要条件不是“高占用率”这个数字本身，而是驻留工作中持续存在可发射的独立线程束。高占用率通常提高这一机会，但不能越过带宽、依赖或执行资源的硬瓶颈。

4.7 占用率、资源约束与线程块大小

4.7.1 占用率的定义

占用率 (occupancy) 描述一个流式多处理器上实际驻留的线程束数量，相对于该流式多处理器允许的最大驻留线程束数量的比例。记实际驻留线程束数为 W_{res} ，最大驻留线程束数为 W_{max} ，则

$$\mathcal{O} = \frac{W_{\text{res}}}{W_{\text{max}}}.$$

由于同一架构上的线程束大小固定，若只考虑线程数约束，也可以等价地用驻留线程数与最大驻留线程数之比理解。但正式定义以驻留线程束为中心更直接，因为调度器的候选单位就是线程束。

需要特别注意

占用率中的“驻留/活动”不等于“当前周期正在执行”。一个线程束可以已经驻留，却正在等待内存；也可以已经就绪，却暂时没有得到发射机会。占用率衡量的是可驻留并保持状态的工作池大小。

4.7.2 流式多处理器资源示例

不同 GPU 架构具有不同的资源参数。最大驻留线程数通常显著高于每个流式多处理器的核心数，同时还受到线程块数、寄存器和共享内存等约束。

GPU Name	K40	M40	P100	V100	A100	H100
Microarchitecture	Kepler	Maxwell	Pascal	Volta	Ampere	Hopper
Compute Capability	3.5	5.2	6.0	7.0	8.0	9.0
Number of SMs	15	24	56	80	108	132
Cores per SM	192	128	64	64	64	128
Max threads per SM	2048	2048	2048	2048	2048	2048
Max blocks per SM	16	32	32	32	32	32
Max threads per block	1024	1024	1024	1024	1024	1024
Registers per SM	64K	64K	64K	64K	64K	64K
Shared memory per SM	48KB	96KB	64KB	96KB	164KB	228KB

图 4.6 多代 GPU 的流式多处理器资源对比；红框突出每个流式多处理器的核心数与最大驻留线程数

GPU	K40	M40	P100	V100	A100	H100
微架构	Kepler	Maxwell	Pascal	Volta	Ampere	Hopper
计算能力	3.5	5.2	6.0	7.0	8.0	9.0
流式多处理器数量	15	24	56	80	108	132
每个流式多处理器核心数	192	128	64	64	64	128
最大驻留线程数/流式多处理器	2048	2048	2048	2048	2048	2048
最大驻留线程块数/流式多处理器	16	32	32	32	32	32
最大线程数/线程块	1024	1024	1024	1024	1024	1024
寄存器/流式多处理器	64K	64K	64K	64K	64K	64K
共享内存/流式多处理器	48KB	96KB	64KB	96KB	164KB	228KB

表 4.1 多代 GPU 的流式多处理器资源示例

4.7.3 把多重驻留限制写成统一形式

设一个线程块有 T_b 个线程，线程束大小为 W ，则每个线程块需要

$$W_b = \left\lceil \frac{T_b}{W} \right\rceil$$

个线程束。设设备限制分别为：最大驻留线程块数 $B_{\max}$ 、最大驻留线程数 $T_{\max}$ 、最大驻留线程束数 $W_{\max}$ 。若每个线程块还需要 R_b 个寄存器资源单位与 S_b 字节共享内存，而流式多处理器总量分别为 R_{SM} 和 S_{SM} ，则多重限制可概念化为

$$B_{\text{res}} \leq \min \left(B_{\max}, \left\lceil \frac{T_{\max}}{T_b} \right\rceil, \left\lceil \frac{W_{\max}}{W_b} \right\rceil, \left\lceil \frac{R_{\text{SM}}}{R_b} \right\rceil, \left\lceil \frac{S_{\text{SM}}}{S_b} \right\rceil \right).$$

该表达式用于统一理解各类驻留约束；实际硬件还可能按特定粒度分配寄存器或共享内存，因此精确驻留数应由设备属性与占用率工具计算。确定 B_{res} 后，驻留线程束数为

$$W_{\text{res}} = B_{\text{res}} W_b,$$

再代入占用率定义即可。

4.7.4 H100 风格限制下的三个线程块大小

以下例题采用一组简化限制：每线程块最多 1024 个线程，每流式多处理器最多 2048 个驻留线程、最多 32 个驻留线程块，并假设寄存器和共享内存不形成额外限制。

4.7.4.1 每块 256 个线程

线程数约束允许

$$\left\lfloor \frac{2048}{256} \right\rfloor = 8$$

个线程块，且 $8 < 32$ 。所以可驻留 $8 \times 256 = 2048$ 个线程，即 64 个线程束；在这一简化模型中占用率为

$$\mathcal{O} = \frac{64}{64} = 100\%.$$

4.7.4.2 每块 32 个线程

若只看线程数，需要

$$2048/32 = 64$$

个线程块才能填满线程容量，但设备最多只允许 32 个驻留线程块。因此实际只能驻留

$$32 \times 32 = 1024$$

个线程，也就是 32 个线程束，对应简化占用率

$$\mathcal{O} = \frac{32}{64} = 50\%.$$

这里的瓶颈是最大驻留线程块数。

4.7.4.3 每块 768 个线程

线程数约束允许

$$\left\lfloor \frac{2048}{768} \right\rfloor = 2$$

个线程块，驻留线程数为

$$2 \times 768 = 1536.$$

剩余 $2048 - 1536 = 512$ 个线程容量无法再容纳完整的第三个线程块，因此简化占用率为

$$\mathcal{O} = \frac{1536}{2048} = 75\%.$$

关键结论

线程块驻留必须以完整线程块为单位，所以资源利用会出现“装箱”效应。即使总资源还有余量，只要余量不足以容纳另一个完整线程块，这部分资源就不能被该核函数的额外线程块使用。

4.7.5 二维线程块大小：彩色转灰度核函数

另一个例题假设某 GPU 的每个流式多处理器最多驻留 1536 个线程和 8 个线程块，比较 8×8 、 16×16 与 32×32 三种二维线程块。

线程块形状	每块线程数	可驻留块数	驻留线程数	简化占用率
8×8	64	8	512	33.3%
16×16	256	6	1536	100%
32×32	1024	1	1024	66.7%

推导过程如下：

- 8×8 ：线程数允许 $1536/64 = 24$ 个线程块，但块数上限为 8，所以只能驻留 512 个线程；
- 16×16 ： $1536/256 = 6$ 个线程块，既不超过 8 块限制，又正好使用全部 1536 个线程容量；
- 32×32 ：每块 1024 线程，第二块会使线程数达到 2048，因此只能驻留一块。

在该题给定的简化限制下， 16×16 获得最高理论占用率。

4.7.6 占用率高并不保证性能最高

较高占用率通常有利于增加延迟隐藏机会，但占用率只在能够帮助隐藏延迟的范围内有用，并不单独保证性能。

原因与上一节一致：若内存带宽已经饱和、所有线程束都受共同依赖阻塞，或其他执行资源成为瓶颈，即使继续增加驻留线程束，也未必增加有效指令吞吐。因此，占用率最适合作为资源驻留与延迟隐藏能力的诊断量，而不是完整的性能模型。

4.8 运行时资源查询与例题

4.8.1 运行时查询设备属性

不同 GPU 架构的资源上限不同，因此程序不应把所有驻留限制都永久写死。CUDA 运行时提供 `cudaGetDeviceProperties` 查询设备属性：

代码 4.6 `cudaGetDeviceProperties` 函数原型

```
1 cudaError_t cudaGetDeviceProperties(cudaDeviceProp* prop, int device);
```

基本用法如下：

代码 4.7 查询设备线程资源上限

```
1 int devID = 0;  
2 cudaDeviceProp devProp;  
3 cudaGetDeviceProperties(&devProp, devID);  
4  
5 int maxThreadsPerBlock = devProp.maxThreadsPerBlock;  
6 int maxThreadsPerMultiProcessor =  
7     devProp.maxThreadsPerMultiProcessor;
```

设备属性结构还可以提供线程束大小以及其他架构限制。程序可以据此选择更合适的启动配置，而不是假设未来设备与当前设备拥有完全相同的线程数、线程块数或线程束限制。

4.8.2 占用率计算工具

NVIDIA Nsight Compute 提供占用率计算器 (*Occupancy Calculator*), 能够根据目标 GPU、每线程块线程数、每线程寄存器使用量、共享内存使用量等信息估算多处理器占用率。该工具替代了较早的 CUDA 占用率计算表格。

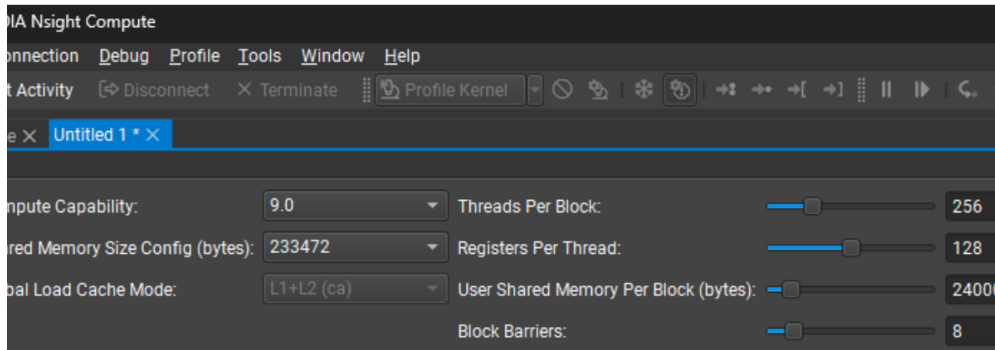


图 4.7 占用率计算器把线程块大小、寄存器和共享内存等静态资源需求映射为驻留线程束与线程块限制

占用率工具的价值在于快速识别明显的资源浪费或驻留瓶颈。它不能替代性能测量, 因为高占用率只说明潜在的延迟隐藏能力较强, 并不能说明实际瓶颈一定来自驻留线程束不足。

4.8.3 例题一：1536 线程与 4 线程块上限

某 CUDA 设备的每个流式多处理器最多驻留 1536 个线程, 并且最多驻留 4 个线程块。比较每块 256、384、576 个线程, 哪一种配置能在仅考虑这两个限制时填满线程容量?

完整推导

每块 256 线程:

$$\frac{1536}{256} = 6.$$

线程数允许 6 个线程块, 但块数上限只有 4, 因此实际最多驻留

$$4 \times 256 = 1024$$

个线程, 不能填满 1536 线程容量。

每块 384 线程:

$$\frac{1536}{384} = 4.$$

恰好需要 4 个线程块，既满足线程数限制，也满足线程块数限制，驻留线程数为

$$4 \times 384 = 1536.$$

因此该配置在题目给定的简化限制下达到满线程容量。

每块 576 线程：

$$\left\lfloor \frac{1536}{576} \right\rfloor = 2,$$

只能驻留

$$2 \times 576 = 1152$$

个线程；第三个线程块会使线程总数超过 1536，所以不能填满容量。

因此答案是 **384 线程/块**。

4.8.4 例题二：只知道流式多处理器和核心数，能否选出最佳配置

题目给定一个一维数组，每个元素由一个线程处理。目标 GPU 有 8 个流式多处理器，每个流式多处理器有 16 个流式处理器，问“最佳执行配置”是什么。

仅凭这些信息无法确定。流式多处理器数量和核心数只描述部分物理执行资源；至少还缺少：

- 每流式多处理器最大驻留线程数和最大驻留线程块数；
- 每线程块最大线程数；
- 寄存器与共享内存容量；
- 该核函数每线程寄存器使用量和每线程块共享内存使用量；
- 实际性能测量，用于判断瓶颈是否来自占用率、带宽或其他资源。

关键结论

从“核心数”直接推导线程块大小是不充分的。CUDA 的启动配置同时受到驻留限制和核函数资源足迹约束，而最高理论占用率也不等价于最高实测性能。

4.8.5 例题三：统一分支条件是否造成控制发散

考虑核函数：

代码 4.8 统一分支条件的核函数

```
1 __global__ void do_work(int q, int *A) {  
2     int result = 0;  
3     if (q < 5)  
4         result = threadIdx.x;  
5     A[threadIdx.x] = result;  
6 }
```

问题是：条件 $q < 5$ 是否造成控制发散？

答案是**不会**。变量 q 是统一传入的标量参数，同一线程束中的线程对该条件得到相同结果。因此整个线程束要么都执行赋值语句，要么都跳过赋值语句。虽然赋值结果使用了每线程不同的 threadIdx.x ，但这只意味着各线程处理不同数据，并不意味着控制路径不同。

更一般地说，控制发散要求同一线程束的活动线程对分支谓词得到不同结果。分支条件不必显式包含线程索引才可能发散；只要它依赖的每线程数据不同，就可能造成发散。反之，即使分支语句存在，只要谓词在线程束内一致，就没有线程束内控制发散。

4.9 架构、调度与性能机制的统一关系

4.9.1 从网格到执行通道的完整映射

从软件层次到硬件执行资源的完整映射链为

网格 → 线程块 → 流式多处理器驻留
→ 线程束 → 处理块调度 → 执行通道

线程块决定资源分配和协作范围；线程束决定调度与控制发散的粒度；处理块和执行资源决定线程束指令如何获得实际执行机会。把这三个粒度区分开，就能避免把“线程块同时驻留”“线程束被调度”和“线程在核心上执行”混为同一件事。

4.9.2 四个性能概念之间的因果链

1. **资源限制决定驻留规模。**线程数、线程块数、寄存器和共享内存等限制共同决定一个流式多处理器能容纳多少线程块与线程束。
2. **驻留规模决定占用率。**驻留线程束数与架构最大线程束数之比就是占用率。
3. **占用率影响延迟隐藏机会。**驻留线程束越多，某个线程束停顿时找到另一个就绪线程束的机会通常越高。
4. **控制发散降低单个线程束的有效通道利用。**即使有足够多驻留线程束，如果每个线程束内部大量通道因为分支而失活，执行资源的有效利用仍会下降。

因此，线程块大小不是一个只影响“线程有多少”的参数。它同时改变每块线程束数、流式多处理器可驻留线程块数、资源装箱效率和可用于延迟隐藏的线程束池；如果算法中的控制结构又与线程索引相关，还要进一步考虑线程束内部的发散模式。

4.9.3 概念边界总结

概念	主要粒度	回答的问题
线程块驻留	线程块/流式多处理器	一个线程块能否进入某个流式多处理器，以及同时能驻留多少块
线程束调度	线程束/处理块	哪个已经驻留且就绪的线程束现在获得指令发射机会
控制发散	单个线程束内部	同一线程束的活动线程是否沿不同控制路径执行
延迟隐藏	多个驻留线程束	当前线程束等待时，是否有其他就绪线程束可以使用执行资源
占用率	流式多处理器	驻留线程束池相对架构最大容量有多大
线程块内同步	单个线程块	同一线程块中的线程如何在共享资源上建立协作和屏障
跨线程块协作	多线程块/更高作用域	如何在不依赖普通线程块调度顺序的前提下建立受约束的协作

第 5 讲

CUDA Memory Model

5.1 课程安排与核心内容

5.1.1 学习目标与技术范围

CUDA 存储模型连接了两类问题：线程能够访问哪些数据，以及这些访问如何影响计算速度。基本矩阵乘法提供了一个完整例子：为输出元素分配线程，建立正确的地址映射，再根据浮点运算量与访存量分析性能。

学习目标为：了解 CUDA 线程可访问的各类存储器的基本特点；为 Lab 2 的基本矩阵乘法实现做准备；学习评估全局内存访问对性能的影响；了解 GPU 性能指标。对应的技术内容依次为存储层次、基本矩阵乘法，以及算术强度和 Roofline 性能模型。

5.1.2 实验、交流与课程项目

5.1.2.1 实验与支持渠道

以下安排对应 2026 年 9 月 8 日的课程信息。Lab 1 的截止时间为当周周五，9 月 11 日；需要先完成 Lab 0，Lab 1 不计入最终成绩。

课程问题通过 Campuswire 发布，课程工作人员只回复在 Campuswire 上发布的问题。提问前应先尝试自行查找答案，包括询问课程聊天机器人。答疑时间同时包含线下与线上形式，需要查看相应时间表。课后事项为阅读教材第 3 章、取得 Delta 账户、设置 GitHub，以及提交 Labs 0 和 1。

5.1.2.2 课程项目与暂定里程碑

两个项目方向都只实现推理的前向计算 (*inference / forward pass*)，区别在于实现对象与组织形式。

方向	实现内容	组织形式
CNN	LeNet-5 卷积神经网络中的卷积层	个人项目
GPT-2	GPT-2 模型中的各层	团队项目，每组 3-4 人

暂定时间为：Milestone 1 在第 7-8 周，Milestone 2 在第 10-11 周，Milestone 3 在第 15-16 周。
以上里程碑为暂定周次。

5.2 从处理器执行到 CUDA 存储层次

5.2.1 数据为什么必须先进入寄存器

5.2.1.1 冯·诺依曼模型中的计算与存储

冯·诺依曼模型 (*Von Neumann model*) 将计算机划分为处理单元、存储器、输入输出子系统和控制单元。处理单元 (*processing unit, PU*) 中的算术逻辑单元 (*arithmetic logic unit, ALU*) 执行算术与逻辑操作，寄存器堆 (*register file*) 保存正在参与运算的数据。存储器同时保存程序指令和数据；输入输出子系统 (*I/O subsystem*) 负责与外部环境通信；控制单元 (*control unit, CU*) 协调指令执行。

在这一执行模型下，运算的操作数位于寄存器中。存储器里的数据需要先装入寄存器，指令也需要从存储器取出并装入指令寄存器 (*instruction register, IR*)。因此，一条高级语言表达式中的“取值”和“计算”，在执行层面对应不同的工作。

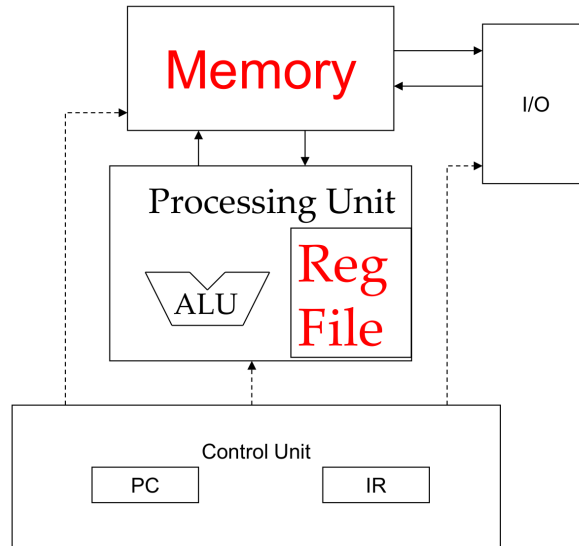


图 5.1 处理单元通过寄存器保存工作数据，控制单元协调存储、运算与输入输出。图中的 PC 为程序计数器，IR 为指令寄存器。

5.2.1.2 ADD 与 LOAD 的区别

指令处理可分为取指 (*fetch*)、译码 (*decode*)、执行 (*execute*) 和访存 (*memory*) 几个阶段。三类指令分别承担运算、数据传输与控制流功能。下面两条 LC-3 指令用于说明寄存器运算与数据装载的差别。

```
ADD R1, R2, R3
LDR R4, R6, #3
```

ADD R1, R2, R3 读取 R2 和 R3，进行加法，再把结果写入 R1。两个输入都已在寄存器中，其数据运算不需要再从数据存储器读取操作数。用寄存器内容 R_i 表示，可写为

$$R_1 \leftarrow R_2 + R_3.$$

LDR R4, R6, ##3 先读取 R6，将其内容加上偏移量 3，形成有效地址，再从该地址读出数据并写入 R4。若 $\text{Mem}[a]$ 表示地址 a 处的内容，则

$$a \leftarrow R_6 + 3, \quad R_4 \leftarrow \text{Mem}[a].$$

两条指令本身都需要被取出和译码；差别在于 LOAD 还显式访问其计算出的数据地址。这里的 3 保留 LC-3 示例中的地址偏移含义，不将其转换为 CUDA 数组的字节步长。

5.2.1.3 延迟与容量的差别

寄存器接近运算单元，适合保存当前工作值；主存容量大，适合保存完整数据集。简化的数量级对照为：寄存器访问约 1 个周期，内存访问约数百个周期；CPU 的寄存器数量为数百量级，GPU 的每个流式多处理器(*streaming multiprocessor, SM*)拥有 $O(10^4)$ 量级的寄存器，而内存容量可达到 GB 或更大。

从计算式看数据流

对于“从数组读取两个数并相乘累加”，数据先被读入可供运算使用的寄存器，再进行乘法和累加。反复使用已经取到的工作值，可以减少再次读取远端存储器的需求。CUDA 的不同存储空间，为线程私有工作值、块内协作和大规模数据集提供了不同的位置。

5.2.2 线程、线程块与设备的可见范围

存储空间可以从两个角度理解：作用域(*scope*)决定哪些线程能够访问同一份数据；物理位置(*physical location*)决定数据处于 SM 内部、设备内存，还是相应的缓存路径。变量的生命周期(*lifetime*)则表示这份存储随线程、线程块或应用程序存在的时间范围。

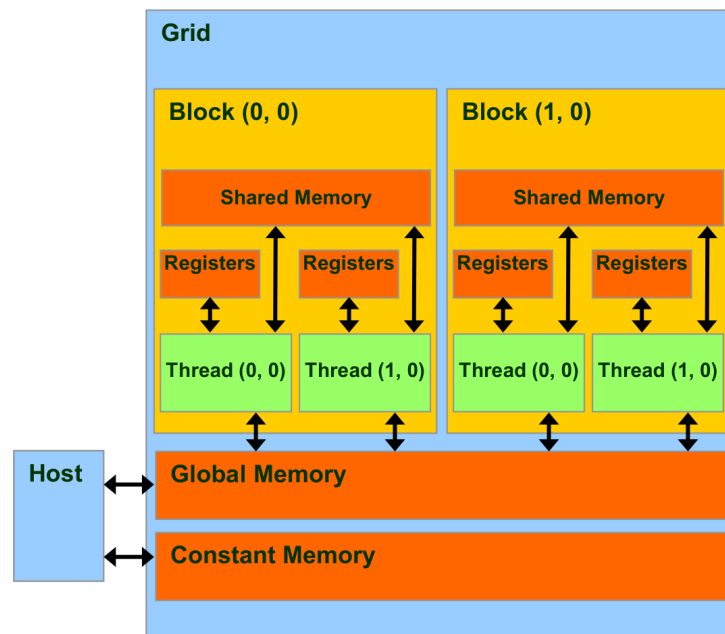


图 5.2 线程持有自己的寄存器，同一线程块的线程共同访问该块的共享内存；全局内存与常量内存对网格中的线程可见。

5.2.2.1 线程私有：寄存器与局部内存

寄存器(*registers*) 保存单个线程的私有工作值，访问延迟最低。例如，矩阵乘法中的累加器只需要由当前线程更新，适合用一个自动标量变量表示，其典型位置就是寄存器。

局部内存(*local memory*) 同样属于单个线程的私有地址空间，但物理上由设备内存支持，并通常经过 L1/L2 缓存。较大的自动对象，以及**寄存器溢出**(*register spill*) 所需的存储，都可能使用这一空间。这里的 *local* 描述的是线程私有的作用域，物理位置仍然可能远离运算单元。

这一区分直接影响对代码的理解：两个变量都只在一个线程中使用，并不代表它们具有相同的访问代价。自动标量通常使用寄存器；线程私有数组可能使用寄存器，也可能使用局部内存，具体分配由编译器处理。

5.2.2.2 块内共享：共享内存

共享内存(*shared memory*) 位于 SM 上，由同一**线程块**(*thread block*) 中的线程共同访问，生命周期与线程块对应。不同线程块各自拥有独立的共享内存存储。

共享内存适合承担块内协作和数据复用：某个线程把数据写入本块的共享内存后，其他需要该数据的块内线程可以从这份共享存储中读取。因而，当多个线程需要反复使用相同输入时，共享内存提供了显式组织复用的位置。共享内存分块矩阵乘法利用这一机制组织输入复用。

5.2.2.3 设备可见：全局内存与常量内存

全局内存(*global memory*) 对设备上的线程可见，适合保存主数据集，例如输入矩阵与输出矩阵。它支持线程读取和写入，容量大，但访问延迟高。代码中的全局地址访问也可能由缓存提供数据，因此其请求与最终到达设备内存的流量需要分开计数。

常量内存(*constant memory*) 保存设备线程只读的数据，并通过**常量缓存**(*constant cache*) 加速访问。**广播或一致访问**(*broadcast / uniform access*)，即多个线程读取相同的数据，是它特别适合的访问情形。全局内存和常量内存的可见范围都可以覆盖一个网格；对设备变量而言，其生命周期在声明表中列为应用程序级。

5.2.3 物理层次、缓存与访问代价

5.2.3.1 从 SM 到设备内存

在硬件视图中，每个 SM 内部具有寄存器、共享内存、L1 缓存与常量缓存；多个 SM 通过下一级 L2 缓存访问全局内存。**缓存 (cache)** 保存已访问数据的副本，使后续访问有机会在更接近计算单元的位置得到响应。局部内存与全局内存的访问流量都可以经过缓存。

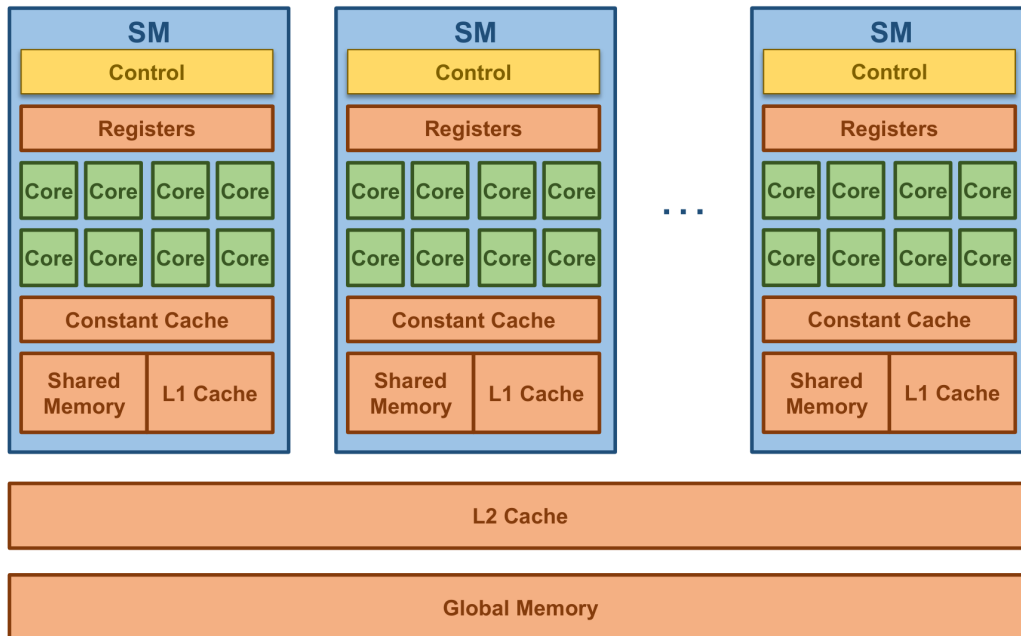


图 5.3 寄存器与块内共享存储位于 SM 内部，L2 缓存位于多个 SM 与全局内存之间。寄存器、共享内存与缓存分别承担不同的数据保存与复用任务。

表 5.1 CUDA 存储层次的作用域、位置与管理方式。

类型	作用域	典型位置	管理方式	主要用途
寄存器	线程	SM	编译器	私有工作值
局部内存	线程	设备内存及缓存	编译器	溢出、较大自动对象
共享内存	块	SM	程序员	块内协作与复用
全局内存	设备/网格	HBM / DRAM	程序员	主数据集
常量内存	设备/网格	DRAM 与 常量缓存	程序员	一致的只读数据
L1 / L2	缓存层	片上	主要由硬件	缓存全局/局部访问

这里“程序员管理”指需要在程序中安排相应存储及数据使用；寄存器与局部内存之间的自动变量分配则主要由编译器处理。

5.2.3.2 周期数的使用范围

简化的程序员视图给出寄存器约 1 周期、共享内存约 5 周期、全局内存约 500 周期，以及命中缓存的常量内存约 5 周期。硬件层次图另将共享内存和常量缓存标为数十周期、全局内存标为数百周期。

这些示意值用于建立层次之间的代价差别：寄存器访问最快，片上共享存储与缓存提供较低延迟，设备内存访问具有更高延迟。两个图的数值口径分别保留，不能从中得到某一指定 GPU 上各类访问的固定延迟。

5.2.4 声明限定符与变量生命周期

5.2.4.1 从声明读出存储空间

CUDA 类型限定符 (*type qualifiers*) 让变量声明表达其存储属性。下表中的设备变量声明对应设备侧存储；函数内没有限定符的自动变量则按线程独立产生。

表 5.2 变量声明、典型存储空间与生命周期。

声明形式	存储空间	作用域	生命周期
<code>__device__ int globalVar;</code>	全局	设备/网格	应用程序
<code>__device__ __constant__ int constantVar;</code>	常量	设备/网格	应用程序
<code>__device__ __shared__ int sharedVar;</code>	共享	块	块
<code>int localVar;</code>	通常为寄存器	线程	线程
<code>int localArr[N];</code>	寄存器或局部内存	线程	线程

声明中若已有 `__shared__` 或 `__constant__`，可以省略 `__device__`。

单独使用 `__device__` 的变量声明不能放在函数内部。表中的 N 表示数组长度。

5.2.4.2 自动变量与线程私有状态

设一个 kernel 中声明了 `int Row;` 和 `float Pvalue;`。每个线程都有自己的行索引和累加器：线程 A 更新其 `Pvalue`，不会因此更新线程 B 的 `Pvalue`。这正是用一个线程独立计算一个输出元素所需的私有状态。

当一个线程块中声明了共享变量，该块内的线程共同使用这份存储；另一个块中的同名共享变量属于另一个独立实例。因而，变量名相同与物理上访问同一份数据之间的关系，取决于变量所属的存储空间和作用域。

判断变量的三个问题

先确定谁需要访问它，再确定它的典型物理位置，最后检查它需要存在多久。线程私有工作值、块内共享中间结果、设备可见的大型输入输出，在这三个问题上分别对应不同的选择。

5.3 基本矩阵乘法：从公式到线程映射

5.3.1 一个输出元素对应一个点积

5.3.1.1 矩阵、下标与行主序地址

设输入矩阵 M, N 和输出矩阵 P 均为 $W \times W$ ，其中 W 对应代码中的 **Width**。采用与 C/CUDA 代码一致的零起始下标，行、列和求和下标分别为 i, j, k ，取值范围均为 $0, \dots, W-1$ 。矩阵乘法定义为

$$P_{ij} = \sum_{k=0}^{W-1} M_{ik} N_{kj}. \quad (5.1)$$

一个 P_{ij} 取决于 M 的第 i 行和 N 的第 j 列：将两者逐项相乘，再把这些乘积相加。数学公式与数组寻址均使用零起始下标。

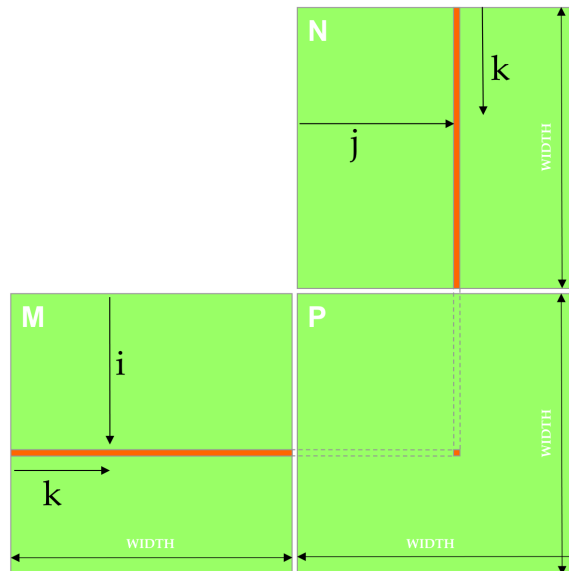


图 5.4 沿 M 的一行和 N 的一列取数，计算它们在 P 中交会位置的元素。

行主序 (*row-major order*) 将同一行的元素连续存储。在每行包含 W 个元素时，跳过前 i

行需要跳过 iW 个元素，再在当前行向右移动 j 个元素，因此二维位置与一维偏移之间的对应关系为

$$(i, j) \longleftrightarrow iW + j. \quad (5.2)$$

对公式(5.1)中的三个矩阵，这给出

$$M_{ik} \leftrightarrow M[i * \text{Width} + k], \quad N_{kj} \leftrightarrow N[k * \text{Width} + j],$$

$$P_{ij} \leftrightarrow P[i * \text{Width} + j].$$

这些表达式的下标以元素为单位；数组元素为 `float` 时，访存字节量另按每个元素 4 B 计算。

5.3.1.2 CPU 三重循环

固定 i 与 j 后，内层循环用一个私有累加器计算点积。外层的两重循环则遍历所有输出位置。

代码 5.1 基本 CPU 矩阵乘法，保留行主序寻址和三重循环结构。

```

1 void MatrixMul(float *M, float *N, float *P, int Width)
2 {
3     for (int i = 0; i < Width; ++i) {
4         for (int j = 0; j < Width; ++j) {
5             float sum = 0.0f;
6             for (int k = 0; k < Width; ++k) {
7                 float a = M[i * Width + k];
8                 float b = N[k * Width + j];
9                 sum += a * b;
10            }
11            P[i * Width + j] = sum;
12        }
13    }
14 }
```

5.3.1.3 并行化输出元素

不同的 P_{ij} 使用不同的累加器，并写入不同的输出位置；它们共同读取 M, N ，但计算一个输出元素不依赖另一个输出元素的完成。因此，可以将外层的 (i, j) 遍历映射为并行线程，让一个线程负责一个输出元素，而该线程内部仍顺序执行 k 循环。

把内层求和本身并行化会涉及归约 (*reduction*)。浮点加法通常不满足结合律，改变求和顺序可能改变舍入结果；每个线程内部的顺序累加保持固定的求和次序。

CPU 循环与 GPU 线程的对应关系

CPU 的两个外层循环负责“选择输出位置”，GPU 用线程的二维索引完成这项选择。内层循环负责“计算该位置上的完整点积”，仍由被分配到该位置的线程执行。

5.3.2 把输出矩阵划分给二维线程块

5.3.2.1 线程块、输出分块与网格大小

令 T 为 `TILE_WIDTH`，即一个方形线程块在两个方向上的宽度。一个 $T \times T$ 的线程块包含 T^2 个线程，负责输出矩阵中的一个 $T \times T$ 子区域，称为一个**分块**(tile)。

当 W 能被 T 整除时，两个方向各需要 W/T 个块。一般情况下需要向上取整，使边缘位置也被覆盖：

$$G_x = G_y = \left\lceil \frac{W}{T} \right\rceil, \quad N_{\text{blocks}} = G_x G_y = \left\lceil \frac{W}{T} \right\rceil^2. \quad (5.3)$$

这里的分块首先用于分配输出计算。共享内存分块矩阵乘法进一步把 M, N 的输入子矩阵显式装入共享内存，以组织输入数据复用。

5.3.2.2 三个网格配置例子

矩阵宽 W	块宽 T	每块线程数	网格尺寸	总块数
4	2	$2^2 = 4$	2×2	4
8	2	$2^2 = 4$	4×4	16
8	4	$4^2 = 16$	2×2	4

表 5.3 输出分块大小决定单块工作量及网格中线程块的数量。

P _{0,0}	P _{0,1}	P _{0,2}	P _{0,3}	P _{0,4}	P _{0,5}	P _{0,6}	P _{0,7}
P _{1,0}	P _{1,1}	P _{1,2}	P _{1,3}	P _{1,4}	P _{1,5}	P _{1,6}	P _{1,7}
P _{2,0}	P _{2,1}	P _{2,2}	P _{2,3}	P _{2,4}	P _{2,5}	P _{2,6}	P _{2,7}
P _{3,0}	P _{3,1}	P _{3,2}	P _{3,3}	P _{3,4}	P _{3,5}	P _{3,6}	P _{3,7}
P _{4,0}	P _{4,1}	P _{4,2}	P _{4,3}	P _{4,4}	P _{4,5}	P _{4,6}	P _{4,7}
P _{5,0}	P _{5,1}	P _{5,2}	P _{5,3}	P _{5,4}	P _{5,5}	P _{5,6}	P _{5,7}
P _{6,0}	P _{6,1}	P _{6,2}	P _{6,3}	P _{6,4}	P _{6,5}	P _{6,6}	P _{6,7}
P _{7,0}	P _{7,1}	P _{7,2}	P _{7,3}	P _{7,4}	P _{7,5}	P _{7,6}	P _{7,7}

(a) $W = 8, T = 2$: 16 个块。

P _{0,0}	P _{0,1}	P _{0,2}	P _{0,3}	P _{0,4}	P _{0,5}	P _{0,6}	P _{0,7}
P _{1,0}	P _{1,1}	P _{1,2}	P _{1,3}	P _{1,4}	P _{1,5}	P _{1,6}	P _{1,7}
P _{2,0}	P _{2,1}	P _{2,2}	P _{2,3}	P _{2,4}	P _{2,5}	P _{2,6}	P _{2,7}
P _{3,0}	P _{3,1}	P _{3,2}	P _{3,3}	P _{3,4}	P _{3,5}	P _{3,6}	P _{3,7}
P _{4,0}	P _{4,1}	P _{4,2}	P _{4,3}	P _{4,4}	P _{4,5}	P _{4,6}	P _{4,7}
P _{5,0}	P _{5,1}	P _{5,2}	P _{5,3}	P _{5,4}	P _{5,5}	P _{5,6}	P _{5,7}
P _{6,0}	P _{6,1}	P _{6,2}	P _{6,3}	P _{6,4}	P _{6,5}	P _{6,6}	P _{6,7}
P _{7,0}	P _{7,1}	P _{7,2}	P _{7,3}	P _{7,4}	P _{7,5}	P _{7,6}	P _{7,7}

(b) $W = 8, T = 4$: 4 个块。图 5.5 相同的 8×8 输出矩阵可以使用不同大小的输出分块；粗线表示块之间的边界。

对这两个 $W = 8$ 的整除例子，总线程数都是 64，每个线程仍计算一个输出元素。改变 T 调整了线程分组，但基本 kernel 的每个有效线程仍然执行长度为 W 的点积。

5.3.3 线程坐标如何确定输入与输出地址

5.3.3.1 从块索引与块内索引得到行列

记块索引为 (b_x, b_y) ，块内线程索引为 (t_x, t_y) ，块尺寸为 (B_x, B_y) 。它们分别对应 `blockIdx`、`threadIdx` 与 `blockDim` 的两个分量。线程负责的全局行、列为

$$\text{Row} = b_y B_y + t_y, \quad \text{Col} = b_x B_x + t_x. \quad (5.4)$$

x 方向沿矩阵的列变化， y 方向沿行变化。乘以块尺寸给出当前块的起点，再加上块内偏移，得到整个输出矩阵中的位置。

当 $T = 2$ 且 $(b_x, b_y) = (0, 0)$ 时，该块负责行 0,1 和列 0,1，四个线程与输出的对应关系如下。

(t_x, t_y)	Row	Col	负责的输出
(0, 0)	0	0	$P_{0,0}$
(1, 0)	0	1	$P_{0,1}$
(0, 1)	1	0	$P_{1,0}$
(1, 1)	1	1	$P_{1,1}$

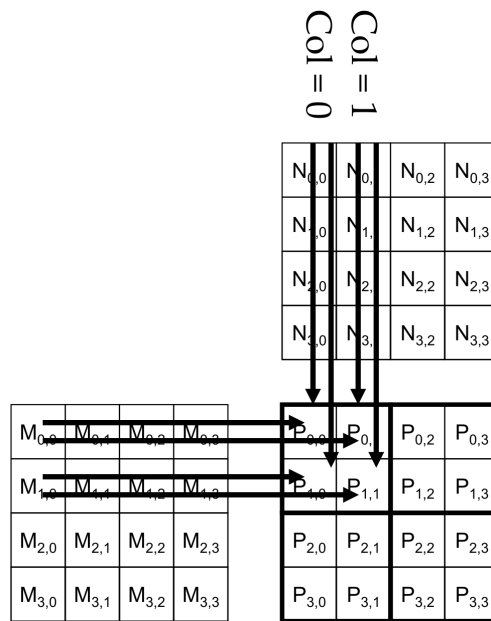


图 5.6 左上角线程块中的四个线程读取 M 的前两行与 N 的前两列，各自形成一个完整点积。

向右相邻的块满足 $(b_x, b_y) = (1, 0)$ ，其行仍为 $0, 1$ ，列变为 $2, 3$ 。CUDA 块坐标按 (b_x, b_y) 排列， b_x 表示水平方向， b_y 表示垂直方向。

5.3.3.2 固定一个线程，追踪整个点积

取 $W = 4$ 、 $\text{Row} = 1$ 、 $\text{Col} = 2$ 。线程最终写入 $P_{1,2}$ ，对应一维位置 $1 \times 4 + 2 = 6$ 。内层循环的四次访问为

k	M 的元素与偏移	N 的元素与偏移	对累加器的贡献
0	$M_{1,0}$ ，偏移 4	$N_{0,2}$ ，偏移 2	$M_{1,0}N_{0,2}$
1	$M_{1,1}$ ，偏移 5	$N_{1,2}$ ，偏移 6	$M_{1,1}N_{1,2}$
2	$M_{1,2}$ ，偏移 6	$N_{2,2}$ ，偏移 10	$M_{1,2}N_{2,2}$
3	$M_{1,3}$ ，偏移 7	$N_{3,2}$ ，偏移 14	$M_{1,3}N_{3,2}$

沿 k 增长时， M 的地址每次增加 1， N 的地址每次增加 W 。这是一个线程内部沿一行和一系列取数所产生的地址序列。每次读入两个元素并相乘累加，四次循环结束后，只需将累加器写入一次输出数组。

5.3.4 kernel、启动配置与边界保护

5.3.4.1 完整的基本 kernel

代码 5.2 一个线程计算一个输出元素的基本矩阵乘法 kernel。

```

1  __global__
2  void MatrixMulKernel(float *d_M, float *d_N,
3                      float *d_P, int Width)
4  {
5      int Row = blockIdx.y * blockDim.y + threadIdx.y;
6      int Col = blockIdx.x * blockDim.x + threadIdx.x;
7
8      if ((Row < Width) && (Col < Width)) {
9          float Pvalue = 0.0f;
10         for (int k = 0; k < Width; ++k) {
11             Pvalue += d_M[Row * Width + k]
12                     * d_N[k * Width + Col];
13         }
14         d_P[Row * Width + Col] = Pvalue;
15     }
16 }

```

Row、Col、k 与 Pvalue 均为线程自己的状态。d_M、d_N 和 d_P 指向设备数组，其中循环内两处下标访问读取全局数据，循环后的下标访问写回输出。Pvalue 在整个循环中保留累加结果，避免每做一次乘加就把部分和写回输出数组。

5.3.4.2 主机侧启动配置

令 TILE_WIDTH 为预先定义的正整数常量，三个设备数组已经准备好，Width 为矩阵宽度。启动片段如下；ceil 对应向上取整，C++ 中可在包含 <cmath> 后使用 std::ceil。

代码 5.3 二维网格与二维线程块的启动片段。

```

1  dim3 dimGrid(std::ceil((1.0 * Width) / TILE_WIDTH),
2              std::ceil((1.0 * Width) / TILE_WIDTH), 1);
3  dim3 dimBlock(TILE_WIDTH, TILE_WIDTH, 1);
4
5  MatrixMulKernel<<<dimGrid, dimBlock>>>>(
6      d_M, d_N, d_P, Width);

```

$1.0 * Width$ 使除法在浮点数范围内进行，从而在向上取整前保留余数对应的小数部分。对正整数 W, T ，同一数量也可用整数算术表达为

$$\left\lceil \frac{W}{T} \right\rceil = \left\lfloor \frac{W + T - 1}{T} \right\rfloor,$$

因此可以用 $(\text{Width} + \text{TILE_WIDTH} - 1) / \text{TILE_WIDTH}$ 计算每个方向的块数。两个表达式都表示“有余数时再增加一个块”。

5.3.4.3 向上取整后为什么需要边界判断

取 $W = 5, T = 2$ ，每个方向需要 $\lceil 5/2 \rceil = 3$ 个块，共 9 个块、36 个线程。它们覆盖一个 6×6 的线程坐标区域，而有效输出只包含 $5 \times 5 = 25$ 个位置。因此，有 11 个线程对应超出矩阵边缘的坐标。

`if ((Row < Width) && (Col < Width))` 同时检查行与列，使这些边缘线程跳过整个点积和输出写入。有效线程内部有 $0 \leq k < W$ ，因此 M 的列索引与 N 的行索引也位于范围内。

任一有效位置 (r, c) 都可以唯一分解为

$$b_y = \lfloor r/T \rfloor, \quad t_y = r \bmod T, \quad b_x = \lfloor c/T \rfloor, \quad t_x = c \bmod T.$$

这说明向上取整的网格覆盖了所有有效输出，并且每个输出都被唯一一个线程负责。不同输出可以重复读取相同输入，但写入目标彼此不同。

5.4 访存量、算术强度与 Roofline 性能分析

5.4.1 基本矩阵乘法为什么容易受带宽限制

5.4.1.1 逐次迭代统计操作数与运算量

基本矩阵乘法将输出位置分配给大量线程，但每个线程在每次 k 循环中都要读取 `d_M[Row * Width + k]` 和 `d_N[k * Width + Col]`。两个 `float` 元素各为 4 B，因此每次迭代请求 8 B 输入数据，同时执行一次浮点乘法和一次浮点加法。

浮点运算 (*floating-point operation, FLOP*) 是这里的运算计数单位：一次乘法与一次加法合计 2 FLOP。将数据需求除以运算量，得到

$$R_{\text{req}} = \frac{8 \text{ B}}{2 \text{ FLOP}} = 4 \text{ B/FLOP}. \quad (5.5)$$

其倒数是每读取一字节所完成的运算量，即**算术强度** (*arithmetic intensity*)。用 I 表示，基本

kernel 在不计缓存复用、忽略最终写回的口径下有

$$I_{\text{req}} = \frac{2}{8} = 0.25 \text{ FLOP/B.} \quad (5.6)$$

设总浮点运算量为 F ，源代码请求的总字节量为 Q_{req} 。保留最后一次输出写回时，计数可逐层展开如下。

计数范围	浮点运算量	请求的字节量
一个线程的一次 k 迭代	2 FLOP	8 B 输入
一个有效线程的完整点积	$2W$ FLOP	$8W + 4 \text{ B}$
整个 $W \times W$ 输出	$F = 2W^3$ FLOP	$Q_{\text{req}} = 8W^3 + 4W^2 \text{ B}$

由此得到更完整的请求级算术强度

$$I_{\text{req}} = \frac{2W^3}{8W^3 + 4W^2} = \frac{W}{4W + 2} \xrightarrow{W \rightarrow \infty} \frac{1}{4} \text{ FLOP/B.} \quad (5.7)$$

输出写入在总请求流量中的比例随 W 增大而减小，因此大矩阵分析中忽略它，可以得到简洁的 0.25 FLOP/B 估计。

请求流量与设备内存流量

上述 Q_{req} 统计各线程在代码中请求的操作数。缓存复用可以让多个请求由已缓存数据响应，使实际到达所建模存储层的字节量 Q 减少。用设备内存带宽估计性能时，算术强度应按该层的 $I = F/Q$ 计算；忽略缓存复用时，才把上述请求计数作为该层流量的近似。

5.4.1.2 150 GB/s 如何得到 37.5 GFLOP/s

考虑给定的示例处理器：计算吞吐上限为 1000 GFLOP/s，全局内存带宽为 150 GB/s。若每执行 1 FLOP 都需要从该层取得 4 B 数据，则内存每秒能支撑的运算量为

$$\frac{150 \text{ GB/s}}{4 \text{ B/FLOP}} = 37.5 \text{ GFLOP/s.} \quad (5.8)$$

也可以写成 $150 \text{ GB/s} \times 0.25 \text{ FLOP/B}$ 。字节单位消去后，结果为运算吞吐率。

这解释了瓶颈：计算单元每秒最多能够完成 1000 G 次浮点运算，但当前的数据需求让带宽只能支撑其中的 37.5 G 次。大量输出线程提供了并行任务，却没有减少每单位计算需要搬运的数据。

在这一示例中，37.5 GFLOP/s 是按给定访存模型得到的带宽上限；原材料给出的执行表

现约为 25 GFLOP/s，并解释为内存并非始终处于满负荷状态。提高性能的方向是减少重复的全局数据访问，使每次搬入的数据参与更多运算。上述数值均属于该教学示例。

5.4.2 从两个峰值建立 Roofline 模型

5.4.2.1 运算吞吐率与内存带宽

浮点运算吞吐率 (*FLOPS rate*) 表示每秒能够完成多少浮点运算，单位为 FLOP/s。内存带宽 (*memory bandwidth*) 表示每秒能够提供多少数据，单位为 B/s。前者度量计算供给，后者度量数据供给。

用 C_{peak} 表示峰值计算吞吐率， B_{peak} 表示所建模存储层的峰值带宽；设一个 kernel 的总工作量为 F FLOP，在该层传输 Q B 数据，执行时间为 t 。则算术强度和实际运算吞吐率分别为

$$I = \frac{F}{Q} \quad (\text{FLOP/B}), \quad \mathcal{P} = \frac{F}{t} \quad (\text{FLOP/s}). \quad (5.9)$$

F 是运算总量， $\mathcal{P}$ 是运算速率；相应地， Q 是数据总量， Q/t 是平均数据传输速率。计算例子中的 G 和 T 按十进制量级 10^9 、 10^{12} 使用。

5.4.2.2 性能上限与时间下限

计算单元提供了 C_{peak} 的吞吐上限；内存以 B_{peak} B/s 供给数据，每字节可支持 I FLOP，因而给出 $B_{\text{peak}}I$ 的另一条上限。两项限制必须同时满足：

$$\mathcal{P} \leq \min(C_{\text{peak}}, B_{\text{peak}}I). \quad (5.10)$$

这就是 **Roofline 模型 (roofline model)** 的基本关系。

用执行时间表达同一限制，计算所需时间至少为 F/C_{peak} ，数据传输所需时间至少为 Q/B_{peak} ，因此

$$t \geq t_{\min} = \max\left(\frac{F}{C_{\text{peak}}}, \frac{Q}{B_{\text{peak}}}\right). \quad (5.11)$$

这里取两项的最大值，表达两条下限必须同时成立；该式没有把两个阶段假定为必然串行。即便达到较小的一项，仍须满足较大的时间需求。

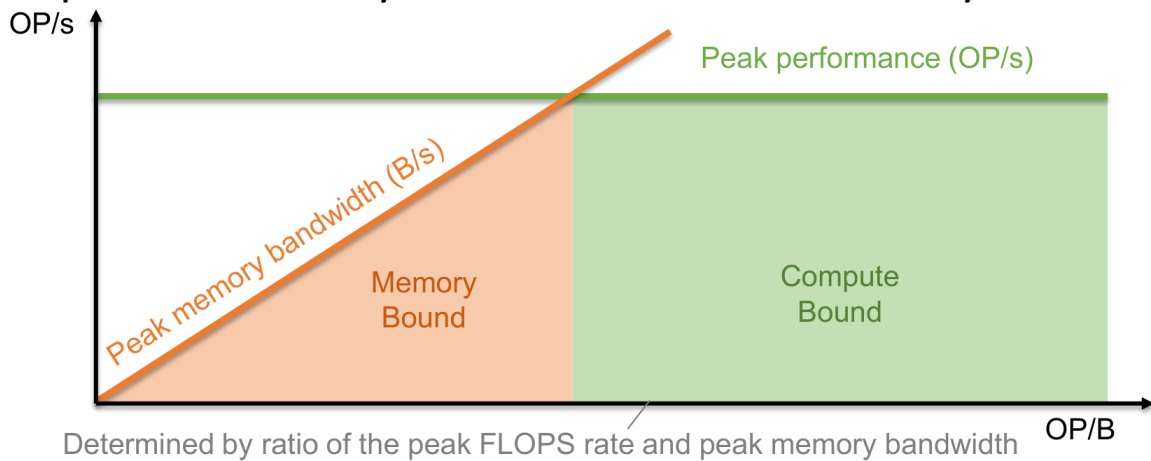


图 5.7 横轴为算术强度，纵轴为运算吞吐率。斜线给出带宽限制，水平线给出计算限制；可达到的性能位于两条限制的下方。

5.4.2.3 屋脊点与受限类型

两条上限相交的位置称为**屋脊点 (ridge point)**。令 $C_{\text{peak}} = B_{\text{peak}} I_*$ ，得到

$$I_* = \frac{C_{\text{peak}}}{B_{\text{peak}}}. \quad (5.12)$$

当 $I < I_*$ 时，带宽提供的性能上限较低，对应**内存受限 (memory-bound)** 区域：计算核心可能等待内存提供数据。当 $I > I_*$ 时，计算吞吐上限较低，对应**计算受限 (compute-bound)** 区域。这里讨论的是 **Roofline** 模型中的限制关系，实际执行性能可以位于上限之下。

沿斜线区域向右移动，表示每字节数据参与更多运算，带宽所能支撑的 FLOP/s 随之提高；到达屋脊后，继续提高算术强度也不会抬高计算单元本身的吞吐上限。

表 5.4 给定 GPU 峰值参数与屋脊点。单位分别为 GFLOP/s、GB/s 和 FLOP/B。

GPU	微架构	C_{peak}	B_{peak}	I_*
K40	Kepler	5,040	288	17.5
M40	Maxwell	6,844	288	23.8
P100	Pascal	9,340	732	12.8
V100	Volta	14,028	900	15.6
A100	Ampere	19,500	1,555	12.5
H100	Hopper	66,900	3,352	20.0

同一个 I 在不同硬件上，需要与不同的 I_* 比较。峰值参数是给定计算条件下的性能上限，不能直接用作任意 kernel 的预期实测速率。

以 H100 的示例参数为例，采用近似值计算，有

$$I_* \approx \frac{67 \text{ TFLOP/s}}{3.35 \text{ TB/s}} \approx 20 \text{ FLOP/B.}$$

基本矩阵乘法的请求级估计仅约 0.25 FLOP/B，在不计缓存复用的带宽模型中远低于这个屋脊点。

5.4.3 向量加法与矩阵乘法的对照

5.4.3.1 向量加法：每个输出都要读二写一

设向量长度为 n ，每个元素为 4 B 的单精度浮点数，计算为

```
z[i] = x[i] + y[i];
```

每个输出需要读取两个输入元素、写入一个输出元素，共 $4 + 4 + 4 = 12 \text{ B}$ ，只进行一次浮点加法。因而

$$F = n, \quad Q = 12n, \quad I_{\text{vec}} = \frac{1}{12} \approx 0.083 \text{ FLOP/B.} \quad (5.13)$$

这里输出写入占每个元素数据流量的三分之一，必须计入。矩阵乘法每个线程做 W 次乘加而只写一次输出，向量加法则每做一次加法就写一次输出，两者写回成本在总流量中的比重不同。

对 $n = 10^9$ ，运算量为 1 GFLOP，数据量为 12 GB。使用 H100 示例中的 3.35 TB/s 带宽，以及舍入后的 $I = 0.083 \text{ FLOP/B}$ ，可得理论极限时间

$$t_{\min} \approx \frac{1 \times 10^9 \text{ FLOP}}{(0.083 \text{ FLOP/B})(3.35 \times 10^{12} \text{ B/s})} \approx 3.60 \text{ ms.} \quad (5.14)$$

直接用未舍入的数据量计算 $12 \times 10^9 / (3.35 \times 10^{12})$ ，得到约 3.58 ms；差别来自将 1/12 提前舍入为 0.083。

理论极限性能 (*speed of light*) 提供比较实测结果的参照。若测得 kernel 时间 t ，则该模型下的平均有效带宽为 Q/t ，与带宽上限的比值为

$$\eta_B = \frac{Q/t}{B_{\text{peak}}} = \frac{Q/B_{\text{peak}}}{t}.$$

对这一内存受限例子，它等于带宽时间下限与实测时间之比。比较时应沿用相同的数据量、存储层和计时范围。

5.4.3.2 矩阵乘法：理想复用下的算术强度

矩阵乘法包含 W^2 个点积，每个点积执行 W 次乘法与 W 次加法，因此

$$F = 2W^3.$$

在**理想复用**假设下，两个输入矩阵在所考察存储层各读入一次，输出矩阵写回一次。每个矩阵有 W^2 个 4 B 元素，于是

$$Q_{\text{ideal}} = 3 \times W^2 \times 4 = 12W^2 \text{ B},$$

$$I_{\text{ideal}} = \frac{2W^3}{12W^2} = \frac{W}{6} \approx 0.167W \text{ FLOP/B}. \quad (5.15)$$

计算量按 W^3 增长，而理想数据量按 W^2 增长，因此算法具有随规模增大的复用潜力。以 $I_* \approx 20 \text{ FLOP/B}$ 为例，理想强度与屋脊相交的条件为 $W/6 \approx 20$ ，即 $W \approx 120$ 。这个交点属于理想复用模型，实际 kernel 的算术强度仍由实现形成的数据流量决定。

5.4.3.3 相同算法为何得到不同的强度

表 5.5 矩阵乘法的算法级复用潜力与基本实现的请求级计数。

分析口径	计数的访存行为	算术强度
理想复用	两个输入各读一次，输出写一次	$W/6 \text{ FLOP/B}$
基本 kernel	每次乘加重新请求两个输入元素	约 0.25 FLOP/B
计入缓存复用	按实际到达所建模层的字节量计数	F/Q ，取决于复用

固定 M_{ik} ，它会参与同一输出行的所有 P_{ij} ，即被不同的 j 使用；固定 N_{kj} ，它会参与同一输出列的所有 P_{ij} ，即被不同的 i 使用。因此，一个输入元素在数学上可以服务多个输出点积。

基本 **kernel** 将这些点积分给不同线程，但仍在每个线程内部独立请求操作数。缓存可以自然保留部分数据，显式组织复用则进一步利用上述行列关系。共享内存分块的方向是把一组输入先搬入块内可共享的存储，再让多个线程使用这组数据完成各自的计算。

两类例子指向的性能结论

向量加法的强度约为 0.083 FLOP/B，基本矩阵乘法的请求级估计约为 0.25 FLOP/B，都远低于 H100 示例的约 20 FLOP/B 屋脊点。矩阵乘法额外具有显著的输入复用潜力；实现这种复用，可以减小所建模内存层的数据流量，使算术强度向屋脊点移动。

5.5 例题解析与知识归纳

5.5.1 三道例题的条件与推导

5.5.1.1 256 行、4 列矩阵需要多少个线程块

题目。输入矩阵有 256 行、4 列，线程块为 16×16 ，每个线程处理一个输入元素。需要启动多少个线程块？

在忽略原矩阵二维几何形状、通过展平或重映射 (*flattening / remapping*) 自行定义索引的假设下，需要 4 个块。其数量计算为

$$\text{元素数} = 256 \times 4 = 1024, \quad \text{每块线程数} = 16 \times 16 = 256,$$

$$N_{\text{blocks}} = 1024/256 = 4.$$

一个实现这种假设的映射为：将 4 个块沿网格的 x 方向排布，块内仍使用 16×16 个线程。先将块内二维线程索引展平，再映射回原矩阵：

$$\begin{aligned} \ell &= t_y \times 16 + t_x, & 0 \leq \ell < 256, \\ g &= b_x \times 256 + \ell, & 0 \leq g < 1024, \\ \text{row} &= \lfloor g/4 \rfloor, & \text{col} = g \bmod 4. \end{aligned}$$

对应的索引计算片段为

代码 5.4 由原题的展平假设展开得到的索引片段。

```
1 int localIndex = threadIdx.y * blockDim.x + threadIdx.x;
2 int threadsPerBlock = blockDim.x * blockDim.y;
3 int index = blockIdx.x * threadsPerBlock + localIndex;
4
5 int row = index / 4;
6 int col = index % 4;
```

二维几何覆盖的对照。若继续使用矩阵乘法中 x 对应列、 y 对应行的直接二维映射，则

$$G_x = \left\lceil \frac{4}{16} \right\rceil = 1, \quad G_y = \left\lceil \frac{256}{16} \right\rceil = 16,$$

需要 $1 \times 16 = 16$ 个块。每块只有 4 列线程对应有效元素，其余 12 列需由边界判断跳过。这样共启动 4096 个线程，其中 1024 个有效。

块数随索引方案变化

展平重映射下，4 个块可让全部 1024 个线程各处理一个有效元素。直接二维覆盖下，需要 16 个块覆盖 256 行，并产生无效边缘线程。判断块数时，需要先确定线程如何对应数据坐标。

5.5.1.2 同一线程块中的线程如何交换数据

题目。某些线程需要读取同一个线程块中其他线程写入的数据。在寄存器、共享内存和全局内存中，哪一种最适合且访问最快？

答案：共享内存。所需存储必须同时满足“同一个块内的线程能够访问同一份数据”和“具有较低访问代价”。共享内存的作用域为线程块，典型位置为 **SM**，正好匹配这两项需求。

寄存器用于每个线程的私有工作值，不能作为这道题中的普通块内共享存储；全局内存支持跨线程访问，但在给定的存储层次中具有更高的访问代价。因此，在三个选项中，共享内存最合适。

5.5.1.3 两个线程块写同一地址会得到什么结果

题目。设备端分配一个 `char` 元素，初始值为 3。执行下列 kernel 后，`*dst` 的可能值是什么？

代码 5.5 两个线程块各用一个线程写入同一全局地址。

```
1 __global__ void kernel(char *dst)
2 {
3     dst[0] = blockIdx.x;
4 }
5
6 // dst[0] was initialized to 3 on the device.
7 kernel<<<2, 1>>>(dst);
```

按独立写入先后解释的结果：**0 或 1**。启动配置包含两个线程块，每个块只有一个线程。块 0 向 `dst[0]` 写入 0，块 1 向同一地址写入 1；二者都覆盖原来的值 3。由于块的执行顺序没有保证，在这一写入顺序模型中，最终写入者可以不同。

多个线程块在缺少确定次序的情况下写同一个位置，形成**数据竞争 (data race)**。全局内存的设备可见性允许线程访问该地址，访问权限本身没有确定谁先写、谁后写。

基本矩阵乘法的输出映射让每个有效位置只有一个写入线程，因此其输出写入不具有这道题的同址竞争。判断并行代码时，需要同时检查输入可以共享读取，以及输出是否被多个线程重复写入。

5.5.2 关键公式与术语

5.5.2.1 从地址映射到性能上限

下表集中给出前述推导中的关系。 W 为方阵宽度， T 为方形线程块宽度； F 为浮点运算量， Q 为所建模存储层的传输字节量。

问题	关系与条件
二维行主序寻址	$(r, c) \mapsto rW + c$ 。
线程负责的位置	$\text{Row} = b_y B_y + t_y$; $\text{Col} = b_x B_x + t_x$ 。
方阵网格大小	$G_x = G_y = \lceil W/T \rceil$; 总块数为 $G_x G_y$ 。
基本矩阵乘法计数	$F = 2W^3$; $Q_{\text{req}} = 8W^3 + 4W^2 B$ 。
算术强度	$I = F/Q$, 单位为 FLOP/B。
基本矩阵乘法强度	$I_{\text{req}} = W/(4W + 2) \approx 0.25 \text{ FLOP/B}$ 。
向量加法强度	读二写一, $I = 1/12 \approx 0.083 \text{ FLOP/B}$ 。
矩阵乘法理想强度	输入各读一次、输出写一次, $I_{\text{ideal}} = W/6$ 。
Roofline 吞吐上限	$\mathcal{P} \leq \min(C_{\text{peak}}, B_{\text{peak}} I)$ 。
屋脊点	$I_* = C_{\text{peak}}/B_{\text{peak}}$ 。
执行时间下限	$t_{\text{min}} = \max(F/C_{\text{peak}}, Q/B_{\text{peak}})$ 。

存储空间决定数据由哪些线程访问、典型位置在哪里；索引映射决定线程实际读写哪些元素；访存量和算术强度把实现与硬件上限连接起来。分析一个 **kernel** 时，这三个层次应使用相互一致的数据与计数口径。

5.5.2.2 中英文术语对照

中文	英文	对应含义
寄存器 / 寄存器堆	Register / register file	线程当前工作值所在的低延迟存储。
流式多处理器	Streaming multiprocessor (SM)	包含运算核心及片上存储资源。
共享内存	Shared memory	同一线程块共同访问的存储。
全局内存	Global memory	保存设备可见的大型输入输出。
常量内存	Constant memory	设备线程只读，适于一致访问。
局部内存	Local memory	线程私有，由设备内存支持。
缓存	Cache	保留数据副本以支持后续访问。
作用域 / 生命周期	Scope / lifetime	谁能访问，以及存储存在多久。
行主序	Row-major order	同一行的元素连续存储。
分块	Tile / tiling	将矩阵划分为线程块负责的区域。
归约	Reduction	将多个值汇合为一个结果，例如求和。
展平 / 重映射	Flattening / remapping	在数据与线程坐标之间重新建立对应。
浮点运算 / 吞吐率	FLOP / FLOPS rate	运算次数 / 每秒运算次数。
内存带宽	Memory bandwidth	每秒可传输的数据字节数。
算术强度	Arithmetic intensity	每传输一字节对应的运算量。
屋脊点	Ridge point	计算上限与带宽上限的交点。
内存受限 / 计算受限	Memory-bound / compute-bound	由数据供给 / 计算供给主导的限制。
理论极限性能	Speed of light	给定模型下最好可能达到的性能。
数据竞争	Data race	原题中多个线程块无确定次序地写同一地址。

第 6 讲

Data Locality and Tiled Matrix Multiply

6.1 课程事项与核心内容

6.1.1 学习目标与课程事项

学习目标为：为 Lab 3 的分块矩阵乘法做准备，处理分块算法的边界条件，以及评估分块带来的收益。技术内容围绕同一个计算问题展开：将矩阵乘法中的输入复用转化为较少的全局内存访问，同时保证线程协作的正确性。

6.1.1.1 作业与阅读安排

以下课程事项对应 2026 年 9 月 10 日。Lab 1 在当周周五截止，要求先完成 Lab 0，且 Lab 1 不计入最终成绩。阅读材料为教材第 5 章；待办事项包括获取 Delta 账号、设置 GitHub，以及提交 Labs 0 和 1。

实验和项目的通常截止时间为周五 20:00，默认提供三天宽限期：周一 20:00 前提交仍可获得完整分数，无需提前通知。若需要超过三天的宽限期，必须在周五 20:00 前，通过邮件向负责相应作业的助教提出申请；是否批准取决于具体特殊情况，超过这一申请时间的请求不予考虑。最低分的一次实验会从总评中剔除；在上述延期政策之外，实验和项目不再接受逾期提交。

6.1.1.2 概念关系

全局内存带宽限制了数据供给速度；共享内存使线程块可以显式保存并复用一小块输入。矩阵乘法沿内积维度逐块累加，形成“协作加载—同步—局部计算—同步”的循环。边界处理维持这一循环的统一结构，性能分析则比较数据复用与线程块资源消耗。

6.1.2 Delta 命令行支持工具

6.1.2.1 启动与用途

命令行支持代理可协助排查 Delta 使用问题，例如实验无法运行、SLURM 作业状态查询，也可辅助实验代码编写。使用前查看 lab1 目录下的 hpcGPT.md。启动命令为：

```
module load hpc-gpt/1.18.23-1
opencode
```

该模块版本对应 2026 年 9 月 10 日的支持工具入口。

内存分配示例涉及三个数组：两个输入和一个输出都需要对应的设备端空间。分配量由元素个数乘以 sizeof(float) 得到，单位是字节；CUDA 分配函数需要设备指针的地址，以便把分配结果写回该指针。

6.1.2.2 内置命令

输入 / 可以调出内置命令列表。下表按用途归类保留命令及其含义。

用途	命令与含义
会话与目录	/new 新会话；/sessions 切换会话；/move 更换项目目录；/exit 退出。
代理与连接	/agents 切换代理；/models 切换模型；/connect 连接提供方；/mcps 切换 MCP。
作业与状态	/jobs 显示或隐藏侧栏 SLURM 作业；/jobs-refresh 立即刷新作业；/status 查看状态。
编辑与检查	/editor 打开编辑器；/diff 查看差异；/review 检查变更，支持 commit、branch、pr，默认检查未提交变更。
配置与帮助	/init 引导设置 AGENTS.md；/skills 查看技能；/themes 主题；/help 帮助。
诊断与反馈	/debug 调试信息；/feedback 附带当前对话提交反馈；/report 向 Delta 支持团队报告问题。

6.2 数据局部性与分块矩阵乘法

6.2.1 从内积计算到输入复用

6.2.1.1 矩阵、线程与下标约定

先考虑两个 $N \times N$ 矩阵相乘， $C = AB$ 。采用与数组一致的零起始下标，有

$$C_{r,c} = \sum_{j=0}^{N-1} A_{r,j} B_{j,c}, \quad 0 \leq r, c < N. \quad (6.1)$$

每个输出元素是 A 的一行与 B 的一列的内积。基本线程映射是：一个线程负责一个 $C_{r,c}$ ，在线程私有的累加变量中保存部分和，内积完成后写回一次。

令 T 表示方形分块的边长，对应代码中的 `TILE_DIM` 或 `TILE_WIDTH`。一个线程块含 $T \times T$ 个线程，负责一个同样大小的输出分块。线程块坐标为 (b_x, b_y) ，块内线程坐标为 (t_x, t_y) ；横坐标对应列，纵坐标对应行：

$$r = b_y T + t_y, \quad c = b_x T + t_x, \quad 0 \leq t_x, t_y < T. \quad (6.2)$$

后续使用 q 表示沿内积方向的分块编号， k 表示某一分块内部的内积下标，因此完整内积下标为 $j = qT + k$ 。这三个层次分别回答“哪个输出块”“哪个输出元素”和“当前使用哪一段输入”。

6.2.1.2 同一个输入服务于多个输出

固定 r, j ，元素 $A_{r,j}$ 参与 C 的第 r 行全部 N 个输出；固定 j, c ，元素 $B_{j,c}$ 参与 C 的第 c 列全部 N 个输出。因此，输入矩阵中每个元素都具有重复使用的机会。

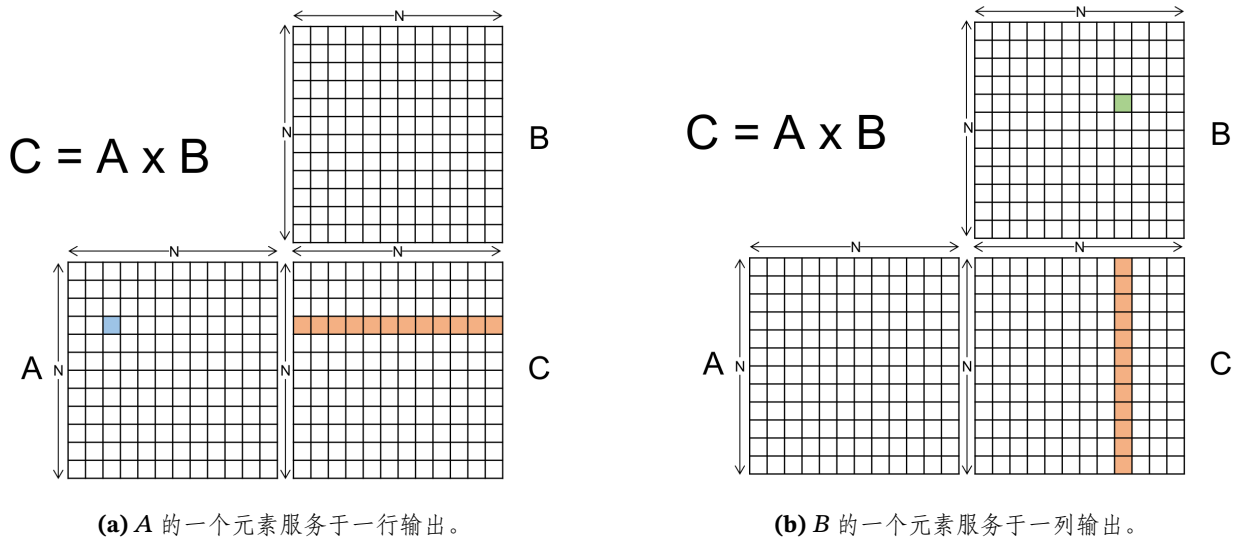


图 6.1 矩阵乘法的输入复用关系。

当一个线程块负责 $T \times T$ 个输出时，同一行的 T 个线程会使用相同的 A 元素，同一列的 T 个线程会使用相同的 B 元素。将这些共同需要的输入保存在块内共享的存储区域，就能让一次全局加载支撑多次计算。

6.2.2 局部性与共享内存

6.2.2.1 带宽为什么会限制运算速度

全局内存 (global memory) 由 DRAM 实现。线程读取的数据有时已经位于 L1 缓存中；也可能在另一个线程再次使用之前被逐出。**共享内存 (shared memory)** 为线程块提供一个显式管理的、较小而较快的数据区域，可以直接组织所需的复用。

数据局部性 (data locality) 在这里体现为：把一组即将反复访问的数据集中起来，在它们仍然可用时完成尽可能多的相关计算。按一次乘法和一次加法合计两个浮点运算计数，每个内积步读取两个 `float` 输入，需要 8 字节，得到

$$\frac{8 \text{ B}}{2 \text{ FLOP}} = 4 \text{ B/FLOP}. \quad (6.3)$$

暂时忽略输出写回和缓存复用。对于计算峰值 1000 GFLOP/s、内存带宽 150 GB/s 的示例 GPU，内存只能支撑

$$\frac{150 \text{ GB/s}}{4 \text{ B/FLOP}} = 37.5 \text{ GFLOP/s}. \quad (6.4)$$

计算单元即使具有更高峰值，也需要足够快的数据供给才能发挥作用。这组参数说明了带宽

对数据供给的限制。

6.2.2.2 分块使数据复用成为显式协作

分块 (tiling) 将数据划分为能够放入共享内存的子集。线程块协作把当前子集从全局内存搬入共享内存，随后反复使用它完成计算，再处理下一组输入。在基本分块矩阵乘法中，每个线程每轮加载一个 A 元素和一个 B 元素，整个线程块共同填满两个 $T \times T$ 的共享数组。

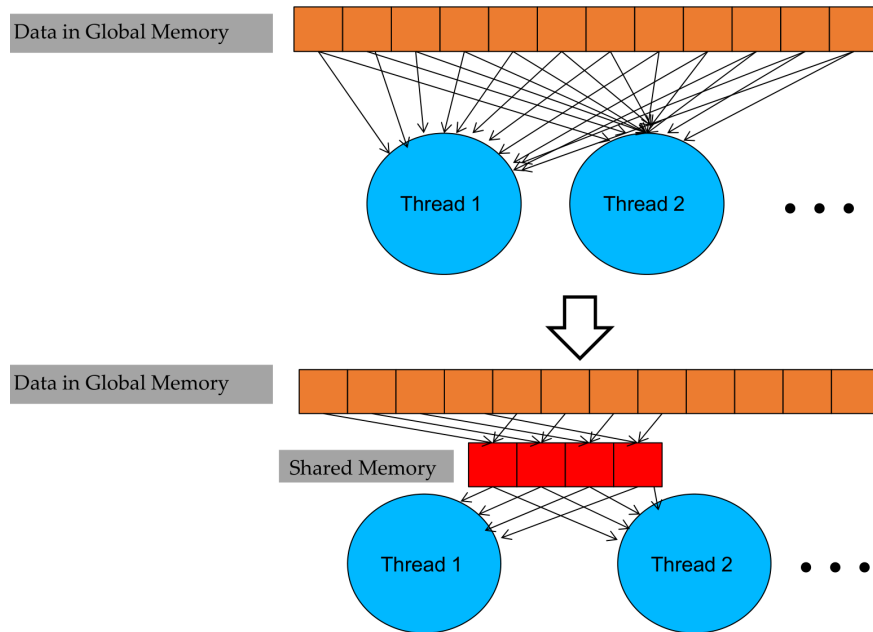


图 6.2 多线程直接重复读取全局数据，以及经共享内存组织复用的两种访问方式。

数据的存放位置

A 、 B 的完整输入和 C 的完整输出位于全局内存；当前两个输入分块位于共享内存；每个线程的部分和 sum 位于寄存器中。一个输出块的结果由该块全部线程的私有部分和共同构成。计算结束时，各线程把自己的最终结果直接写回 C 。

6.2.3 沿内积维度推进分块

6.2.3.1 固定输出块，依次更换输入块

先假定 N 是 T 的整数倍。输出块的 (b_x, b_y) 在整个核函数执行中保持不变，而输入分块随 $q = 0, 1, \dots, N/T - 1$ 推进。第 q 轮使用 A 的列区间 $[qT, (q+1)T)$ 和 B 的行区间 $[qT, (q+1)T)$ 。

把式 (6.1) 的求和下标写成 $j = qT + k$ ，得到与分块循环直接对应的等式：

$$C_{r,c} = \sum_{q=0}^{N/T-1} \left(\sum_{k=0}^{T-1} A_{r,qT+k} B_{qT+k,c} \right). \quad (6.5)$$

内层求和产生一个部分内积，外层求和累加所有输入分块的贡献。此重排用于说明分块算法与原矩阵乘法的对应关系。

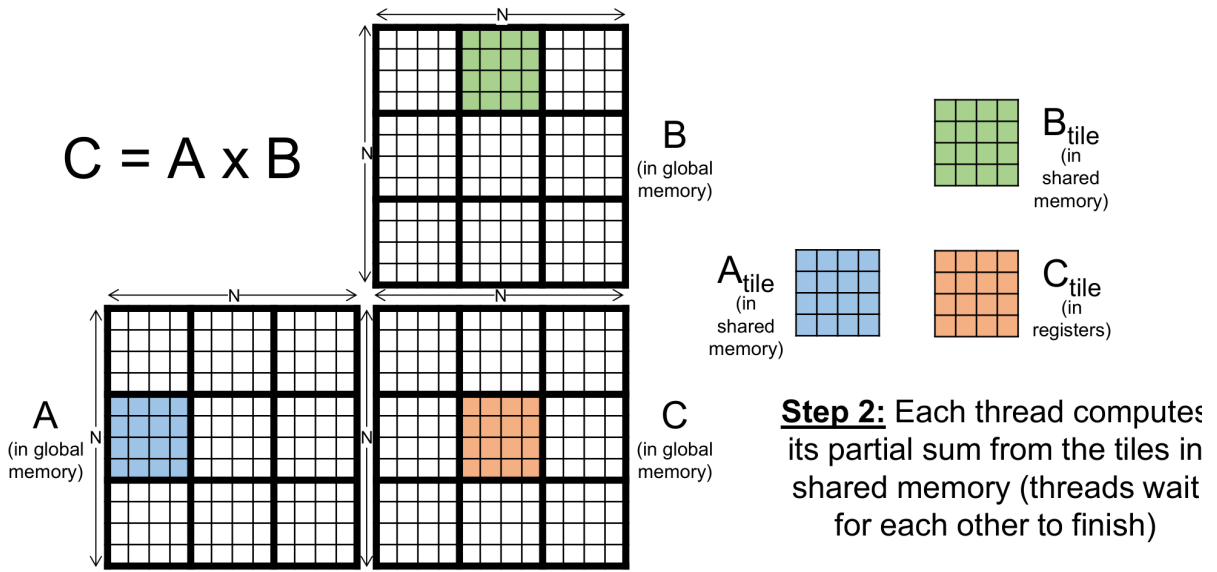


图 6.3 一个输出块对应一对当前输入块；输入块位于共享内存，输出部分和位于各线程寄存器。

6.2.3.2 协作加载的二维索引

线程 (t_x, t_y) 把两个输入元素分别写入共享数组的同一个局部位置 $[t_y][t_x]$ ：

$$A_s[t_y][t_x] \leftarrow A[r][qT + t_x], \quad (6.6)$$

$$B_s[t_y][t_x] \leftarrow B[qT + t_y][c]. \quad (6.7)$$

对于 A ，全局行号固定在当前输出行，沿列方向推进分块；对于 B ，全局列号固定在当前输出列，沿行方向推进分块。因此， A 的内积方向由 t_x 定位， B 的内积方向由 t_y 定位。

第 0 轮分别读取 $A[r][t_x]$ 、 $B[t_y][c]$ ；第 1 轮分别读取 $A[r][T+t_x]$ 、 $B[T+t_y][c]$ 。块内所有线程联合覆盖整对输入分块。加载阶段还应使同一 warp 中的访问便于合并访问 (coalesced access)。

6.2.3.3 行主序的一维地址

行主序 (row-major order) 将每一行连续存放。对于列数为 N 的数组，二维位置 (i, j) 的一维偏移为 $iN + j$ 。式 (6.7) 因而变为：

```
A_s[ty][tx] = A[row * N + q * TILE_DIM + tx];
B_s[ty][tx] = B[(q * TILE_DIM + ty) * N + col];
```

A 中的 $qT + t_x$ 已经是列偏移，无需再乘以行宽； B 中的 $qT + t_y$ 是行号，必须先乘以行宽。共享数组在声明时具有二维形状，可以继续使用二维下标。

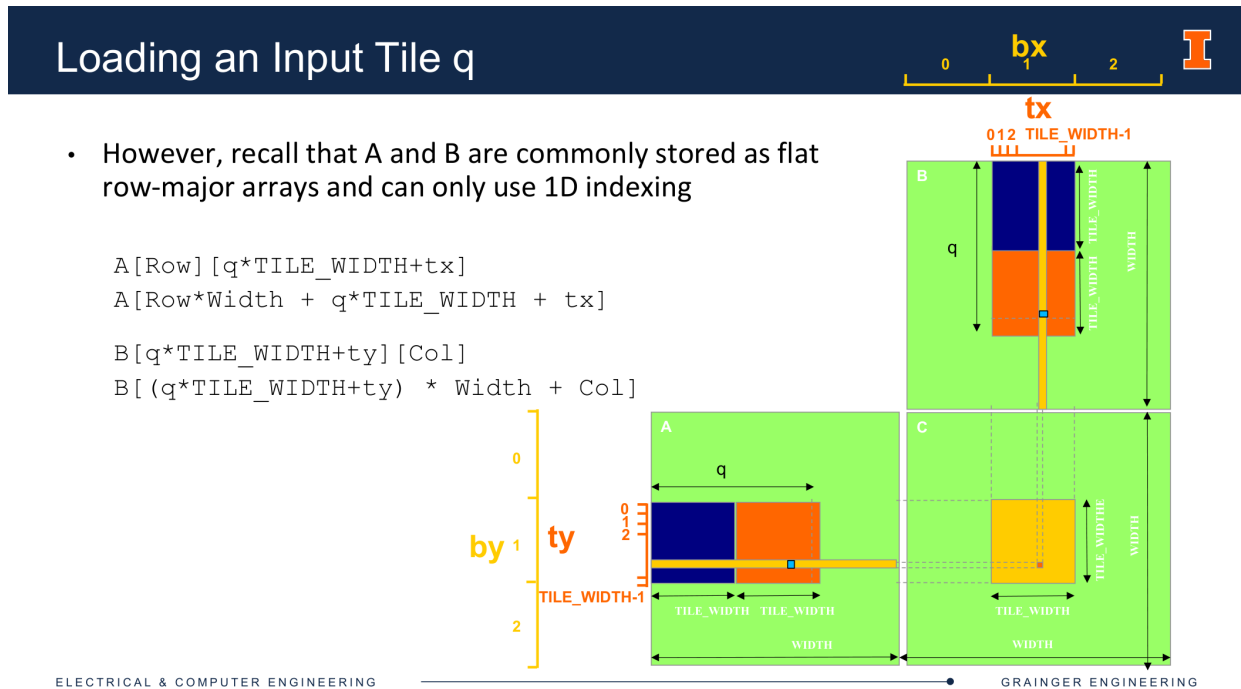


图 6.4 输出块坐标固定， A 的输入块向右移动， B 的输入块向下移动；两者由同一分块编号 q 控制。

6.2.3.4 计算时读取共享行与共享列

分块加载完成后，线程 (t_x, t_y) 按自己的输出位置读取共享数组的一行与一列：

```
for (int k = 0; k < TILE_DIM; ++k) {
    sum += A_s[ty][k] * B_s[k][tx];
}
```

将加载定义代入可验证，每一步使用的恰好是

$$A_s[t_y][k]B_s[k][t_x] = A_{r,qT+k}B_{qT+k,c}.$$

加载时，线程负责 $[t_y][t_x]$ 这个存储位置；计算时，它需要由多个线程加载的共享行和共享列。正是这一区分使协作加载产生复用，也使线程间同步成为必要条件。

例：一个线程在加载与计算中的不同职责

取 $N = 4$ 、 $T = 2$ ，考虑线程块 $(b_x, b_y) = (0, 1)$ 中的线程 $(t_x, t_y) = (1, 0)$ 。由式 (6.2)，它负责 $C_{2,1}$ 。

在 $q = 0$ 时，该线程加载 $A_{2,1}$ 与 $B_{0,1}$ ，但其部分内积为

$$A_{2,0}B_{0,1} + A_{2,1}B_{1,1}.$$

其中 $A_{2,0}$ 和 $B_{1,1}$ 由其他线程加载。在 $q = 1$ 时，该线程加载 $A_{2,3}$ 与 $B_{2,1}$ ，并累加

$$A_{2,2}B_{2,1} + A_{2,3}B_{3,1}.$$

两轮得到完整的 $C_{2,1}$ 。该例用于逐项解释加载索引和计算索引，并沿用每线程一个输出的基本算法。

6.3 线程块同步与正确执行顺序

6.3.1 共享数据带来的两类依赖

6.3.1.1 使用之前，必须完成当前分块加载

每个线程只加载当前输入分块的一小部分，却会在内积循环中读取其他线程负责的元素。例如，线程 (t_x, t_y) 在第 k 步读取的 $A_s[t_y][k]$ ，由线程 (k, t_y) 加载； $B_s[k][t_x]$ ，由线程 (t_x, k) 加载。开始计算之前，必须保证这些加载已经完成。

6.3.1.2 覆盖之前，必须完成上一分块使用

两轮分块复用相同的共享数组 A_s 、 B_s 。较快的线程如果提前进入下一轮，就可能把第 $q+1$ 轮数据写到仍被其他线程读取的位置。这会让某些乘积混用两轮输入。进入下一轮加载之前，需要确认当前轮的读取和计算已经结束。

这两类依赖分别保护“生产完成后再消费”和“消费完成后再覆盖”。只等待加载完成，不能保护共享数组的再次使用。

6.3.2 屏障与整体同步执行

6.3.2.1 屏障的含义

屏障 (barrier) 是线程集合共同到达的同步点。先到达的线程等待其余线程；参与线程全部到达后，才能继续执行屏障之后的代码。

整体同步执行 (bulk synchronous execution) 将并行计算划分为若干阶段：每个阶段内部，各线程并行完成自己的工作；阶段之间由屏障建立清晰的先后关系。线程在一个阶段内部的执行进度可以不同。

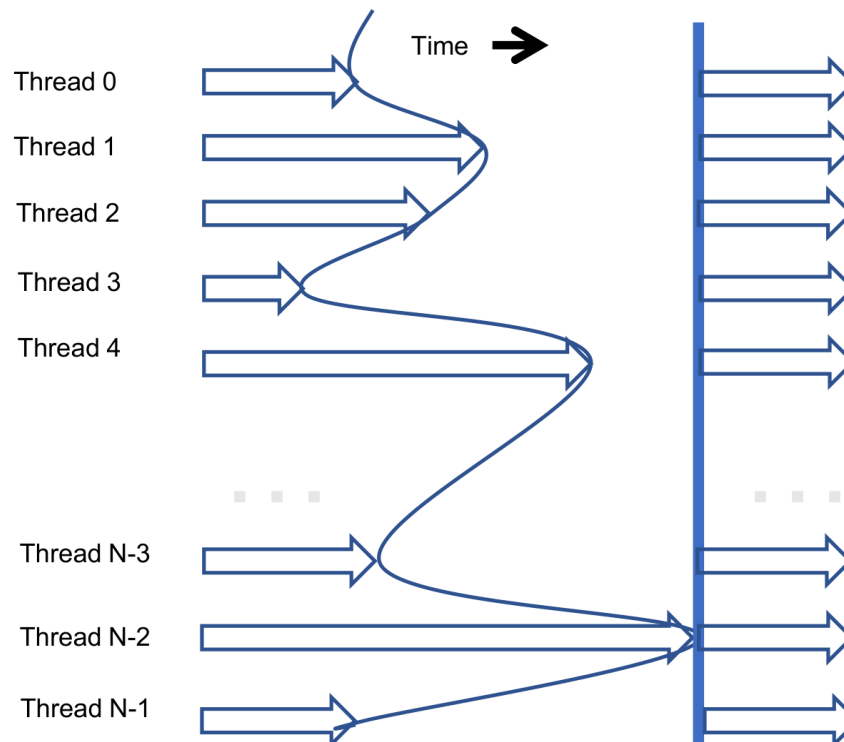


图 6.5 线程以不同进度到达屏障，在共同的同步点之后继续执行。

6.3.2.2 CUDA 的块内同步

`__syncthreads()` 是 CUDA 的设备端内置同步操作，用于同一线程块中的线程。线程块内的所有线程需要到达相应同步点，才能继续执行。

同步调用可以位于条件分支内，前提是该条件对整个参与线程块一致。如果只有一部分线程执行同步，执行可能挂起或产生错误行为。边界线程处理尤其需要保持这一点：线程的输入加载是否有效可以不同，但参与分块循环和同步的结构应保持一致。

每一轮的执行顺序

协作加载 $A_s, B_s \rightarrow \text{__syncthreads}()$

当前分块的部分内积 $\rightarrow \text{__syncthreads}()$

然后进入下一轮加载。

第一个屏障保证“数据已准备好”，第二个屏障保证“旧数据已使用完”。屏障作用于当前线程块；不同输出块具有各自的输入缓冲区和输出区域。

6.3.3 整除情形的完整核函数

下面将加载、两次同步、部分内积和写回组合起来。假设 $N > 0$ 且 T 整除 N ，所有矩阵均按行主序存储；线程块大小固定为 $T \times T$ ，每线程负责一个输出。代码将 `TILE_DIM` 设为 16，这一取值仅用于具体化示例。

代码 6.1 带两次块内同步的方阵分块核函数。

```

1  #include <cuda_runtime.h>
2  #define TILE_DIM 16
3
4  __global__ void matmul_tiled_square(
5      const float* A, const float* B, float* C, int N)
6  {
7      __shared__ float A_s[TILE_DIM][TILE_DIM];
8      __shared__ float B_s[TILE_DIM][TILE_DIM];
9      const int tx = threadIdx.x;
10     const int ty = threadIdx.y;
11     const int row = blockIdx.y * TILE_DIM + ty;
12     const int col = blockIdx.x * TILE_DIM + tx;
13     float sum = 0.0f;
14
15     for (int q = 0; q < N / TILE_DIM; ++q) {
16         A_s[ty][tx] = A[row * N + q * TILE_DIM + tx];
17         B_s[ty][tx] = B[(q * TILE_DIM + ty) * N + col];
18         __syncthreads(); // Wait for all tile producers.
19
20         for (int k = 0; k < TILE_DIM; ++k) {
21             sum += A_s[ty][k] * B_s[k][tx];
22         }
23         __syncthreads(); // Wait for all tile consumers.
24     }
25     C[row * N + col] = sum;
26 }
```

对应的主机端启动代码为：

```

dim3 block(TILE_DIM, TILE_DIM);
dim3 grid(N / TILE_DIM, N / TILE_DIM);
matmul_tiled_square<<<grid, block>>>(d_A, d_B, d_C, N);
```

其中 `d_A`、`d_B`、`d_C` 是已经分配的设备端矩阵指针。启动片段只说明线程配置，沿用已完成输入准备和设备内存分配的前提。

6.3.3.1 正确性的阶段不变量

以下不变量用于形式化说明代码 6.1 的执行过程。第 q 轮开始时，线程 (t_x, t_y) 的累加值为

$$\text{sum} = \sum_{j=0}^{qT-1} A_{r,j} B_{j,c}.$$

第一次同步之后，当前所需的共享行与共享列均已加载。内积循环增加

$$\sum_{k=0}^{T-1} A_{r,qT+k} B_{qT+k,c},$$

所以本轮结束时，累加范围恰好扩展为 $0 \leq j < (q+1)T$ 。第二次同步保证当前输入不会被过早覆盖。全部 N/T 轮结束后，累加范围覆盖 $0 \leq j < N$ ，因而 sum 等于 $C_{r,c}$ 。

6.4 边界条件与一般尺寸矩阵

6.4.1 从矩阵形状推导三类有效性条件

6.4.1.1 一般矩阵尺寸与行宽

采用一般矩阵形状

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N}. \quad (6.8)$$

其中 M 是输出行数， N 是输出列数， K 是内积长度。方阵情形对应 $M = K = N$ 。线程映射仍为 $r = b_y T + t_y$ 、 $c = b_x T + t_x$ 。

A 的行宽为 K ， B 和 C 的行宽为 N 。因此，一维数组偏移分别为

$$A_{r,j} \longleftrightarrow rK + j, \quad B_{j,c} \longleftrightarrow jN + c, \quad C_{r,c} \longleftrightarrow rN + c.$$

各矩阵的行宽决定地址偏移；加载条件和输出条件需要分别使用相应矩阵的维度。

6.4.1.2 加载有效性性与输出有效性分别判断

同一个线程在一轮中有三个独立的任务：加载一个 A 元素、加载一个 B 元素，以及更新自己的输出部分和。前两项是否对应有效输入，需要根据各自的输入坐标判断。

操作	目标位置	有效性条件
加载 A	$A_{r,qT+t_x}$	$r < M$ 且 $qT + t_x < K$ 。
加载 B	$B_{qT+t_y,c}$	$qT + t_y < K$ 且 $c < N$ 。
写回 C	$C_{r,c}$	$r < M$ 且 $c < N$ 。

所有下标由非负线程坐标生成，因此这里仅列上界条件。

一个输出列越界的线程，仍可能负责加载同块其他线程需要的 A 元素；一个输出行越界的线程，仍可能负责加载有效的 B 元素。输出有效性只能决定是否写回当前输出，不能直接代替输入加载条件。

6.4.1.3 三种方向的分块数量

输出网格覆盖 $M \times N$ ，所以纵横方向的线程块数分别为

$$n_x = \left\lceil \frac{N}{T} \right\rceil, \quad n_y = \left\lceil \frac{M}{T} \right\rceil. \quad (6.9)$$

每个输出都需要遍历长度为 K 的内积方向，因此每个线程块执行

$$Q = \left\lceil \frac{K}{T} \right\rceil \quad (6.10)$$

轮输入加载。网格尺寸由输出矩阵决定，循环轮数由内积长度决定。

对于正整数 D ，整数向上取整可以写成

$$\left\lceil \frac{D}{T} \right\rceil = \left\lfloor \frac{D-1}{T} \right\rfloor + 1. \quad (6.11)$$

这对应 $(D - 1) / \text{TILE_DIM} + 1$ 。若 D 为 T 的倍数，减一使商减少一，再加一恢复；若存在余数，则在完整分块数上加一，覆盖最后的不完整分块。公式用于 $D > 0$ 的情形。

6.4.2 零填充维持统一的计算循环

6.4.2.1 缺失输入在加载时写零

零填充 (zero padding) 在加载阶段处理边界：目标输入有效时正常加载，越界时向该线程负责的共享数组位置写入 0。随后，所有线程都可以执行相同的 T 步内积循环。超出内积长度的项因乘以零而不产生贡献。

每一轮都必须写入该共享位置，无论写入的是输入还是零。共享数组会在多轮之间复用；若越界分支仅跳过赋值，该位置可能保留上一轮的数据，使不应参与计算的项继续产生贡献。

零填充的三个动作

加载 A ：使用 A 自身的边界条件，越界写零。

加载 B ：使用 B 自身的边界条件，越界写零。

写回 C ：仅当输出坐标有效时写入全局内存。

两个同步点仍由整个线程块共同执行。

6.4.2.2 例一：输出有效，但最后一轮输入不完整

取 $M = K = N = 3$ 、 $T = 2$ 。线程块 $(b_x, b_y) = (0, 0)$ 负责左上角四个有效输出，共执行 $Q = 2$ 轮。第二轮 $q = 1$ 对应全局内积下标 $j = 2, 3$ ，其中 $j = 3$ 超出有效输入范围。共享数组应为

$$A_s = \begin{bmatrix} A_{0,2} & 0 \\ A_{1,2} & 0 \end{bmatrix}, \quad B_s = \begin{bmatrix} B_{2,0} & B_{2,1} \\ 0 & 0 \end{bmatrix}. \quad (6.12)$$

在 $k = 0$ 时，四个线程分别增加 $A_{r,2}B_{2,c}$ ；在 $k = 1$ 时，共享行的第二个元素与共享列的第二个元素均为零，贡献为零。以 $C_{0,1}$ 为例，全部两轮累加为

$$\underbrace{A_{0,0}B_{0,1} + A_{0,1}B_{1,1}}_{q=0} + \underbrace{A_{0,2}B_{2,1} + 0 \cdot 0}_{q=1}.$$

这个线程的输出位置始终有效，但它仍需要对最后一轮输入进行边界处理。

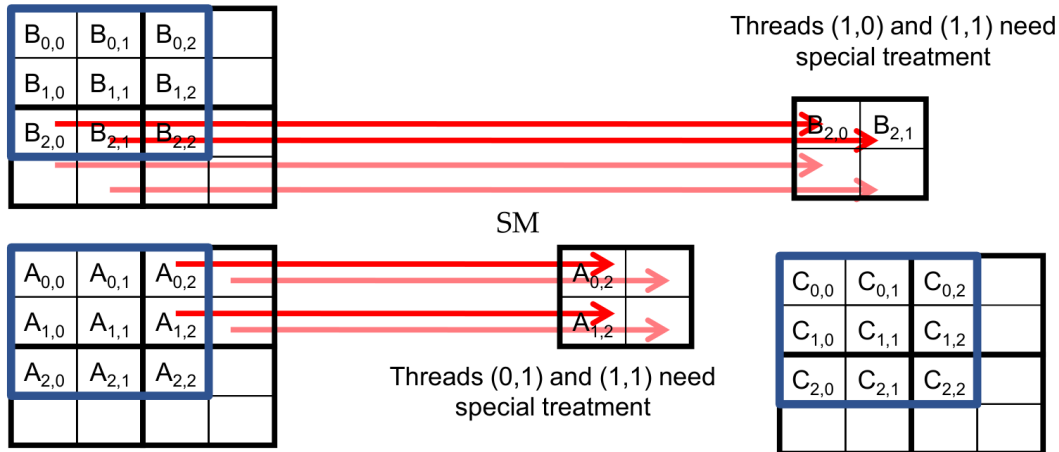


图 6.6 3×3 矩阵使用 2×2 分块时，第二轮输入块包含缺失位置。图中的线程编号采用（行，列）次序。

6.4.2.3 例二：输出越界的线程仍承担加载任务

仍取 3×3 矩阵与 2×2 分块。线程块 $(b_x, b_y) = (1, 1)$ 覆盖输出坐标 $r, c \in \{2, 3\}$ ，只有 $C_{2,2}$ 有效。在第一轮 $q = 0$ ，四个线程的任务如下；线程坐标均按 (t_x, t_y) 书写。

线程	输出位置	加载 A	加载 B
(0,0)	$C_{2,2}$ 有效	$A_{2,0}$	$B_{0,2}$
(1,0)	$C_{2,3}$ 越界	$A_{2,1}$	越界，写零
(0,1)	$C_{3,2}$ 越界	越界，写零	$B_{1,2}$
(1,1)	$C_{3,3}$ 越界	越界，写零	越界，写零

于是共享数组为

$$A_s = \begin{bmatrix} A_{2,0} & A_{2,1} \\ 0 & 0 \end{bmatrix}, \quad B_s = \begin{bmatrix} B_{0,2} & 0 \\ B_{1,2} & 0 \end{bmatrix}.$$

唯一负责有效输出的线程 (0,0)，需要线程 (1,0) 加载的 $A_{2,1}$ 和线程 (0,1) 加载的 $B_{1,2}$ ，才能计算

$$A_{2,0}B_{0,2} + A_{2,1}B_{1,2}.$$

第二轮再增加 $A_{2,2}B_{2,2}$ ，最终得到完整的 $C_{2,2}$ 。因此，应当保留输出越界线程的协作加载与同步，只在写回时排除其输出。

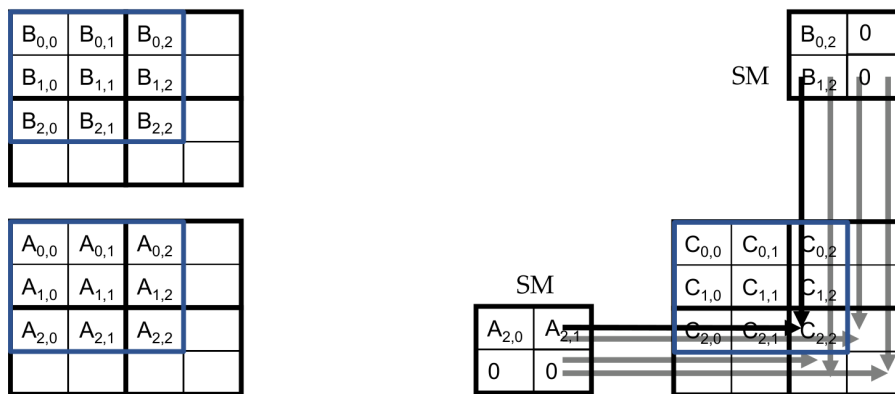


图 6.7 右下角输出块只含一个有效输出，但该输出仍需要其他线程协作加载输入。

6.4.3 支持一般尺寸的完整实现

6.4.3.1 核函数与启动配置

代码 6.2 把三类边界判断、零填充和向上取整组合起来。假设 M, K, N 为正整数，矩阵使用行主序，输入和输出数组已经按式 (6.8) 分配。

代码 6.2 一般尺寸的分块矩阵乘法；加载与同步由全部线程参与。

```

1  #include <cuda_runtime.h>
2  #define TILE_DIM 16
3
4  __global__ void matmul_tiled(
5      const float* A, const float* B, float* C,
6      int M, int K, int N)
7  {
8      __shared__ float A_s[TILE_DIM][TILE_DIM];
9      __shared__ float B_s[TILE_DIM][TILE_DIM];
10     const int tx = threadIdx.x;
11     const int ty = threadIdx.y;
12     const int row = blockIdx.y * TILE_DIM + ty;
13     const int col = blockIdx.x * TILE_DIM + tx;
14     const int numTiles = (K - 1) / TILE_DIM + 1;
15     float sum = 0.0f;
16
17     for (int q = 0; q < numTiles; ++q) {
18         const int aCol = q * TILE_DIM + tx;
19         const int bRow = q * TILE_DIM + ty;
20         A_s[ty][tx] = (row < M && aCol < K)
21             ? A[row * K + aCol] : 0.0f;
22         B_s[ty][tx] = (bRow < K && col < N)
23             ? B[bRow * N + col] : 0.0f;
24         __syncthreads();
25
26         for (int k = 0; k < TILE_DIM; ++k) {
27             sum += A_s[ty][k] * B_s[k][tx];
28         }
29         __syncthreads();
30     }
31     if (row < M && col < N) {
32         C[row * N + col] = sum;
33     }
34 }

```

启动时，网格横向覆盖 N 列，纵向覆盖 M 行：

```

dim3 block(TILE_DIM, TILE_DIM);
dim3 grid((N - 1) / TILE_DIM + 1,
          (M - 1) / TILE_DIM + 1);
matmul_tiled<<<grid, block>>>(d_A, d_B, d_C, M, K, N);

```

参数顺序为 M, K, N ，与 A 的行数、内积长度、 B 的列数一致。

6.4.3.2 为什么零填充保持结果正确

对输入在原矩阵范围外定义零扩展 $\tilde{A}$ 、 $\tilde{B}$ 。当 $0 \leq r < M$ 、 $0 \leq c < N$ 时，代码实际计算

$$\sum_{q=0}^{Q-1} \sum_{k=0}^{T-1} \tilde{A}_{r,qT+k} \tilde{B}_{qT+k,c} = \sum_{j=0}^{QT-1} \tilde{A}_{r,j} \tilde{B}_{j,c} \quad (6.13)$$

$$= \sum_{j=0}^{K-1} A_{r,j} B_{j,c} = C_{r,c}. \quad (6.14)$$

由于 $QT \geq K$ ，有效内积项全部被覆盖；多出来的项恰好为零。此推导解释了为何内积循环仍可固定执行 T 步。零扩展只是在共享加载阶段实现的逻辑含义，无需为完整矩阵额外分配补零后的全局数组。

6.4.3.3 可选的计算判断与同步位置

对于无有效输出的线程，零填充使其相应共享行或共享列为零，部分和不影响任何有效输出。可以继续执行内积循环，也可以仅让有效输出线程执行乘加，以减少这些线程的无用运算。该调整的实际性能收益取决于分支和省去运算的开销。

若增加计算条件，正确的局部结构如下：

```
__syncthreads();
if (row < M && col < N) {
    for (int k = 0; k < TILE_DIM; ++k) {
        sum += A_s[ty][k] * B_s[k][tx];
    }
}
__syncthreads();
```

两个同步点都在按线程变化的条件之外。将输出越界线程提前移出分块循环，会丢失它们仍可能承担的输入加载任务；将同步放入仅部分线程执行的分支，也不满足块内同步要求。

6.4.3.4 边界分支的工作量范围

边界条件可能造成分支分歧 (*branch divergence*)。输出的末行、末列分块会包含输出越界线程；当 K 不能被 T 整除时，每个输出块的最后一轮也会处理内积方向的缺失输入。这一分类由三类边界谓词直接得到。

在矩阵各维度相对 T 都很大时，边缘区域和最后一轮输入只占较小比例；小矩阵或高度

不规则的尺寸则可能有明显边界开销。该判断延续大矩阵边界影响较小的定性分析，具体占比仍由 M, N, K, T 决定。

6.5 分块收益与资源约束

6.5.1 用数据流量解释计算收益

6.5.1.1 一个分块轮次的加载量与计算量

以下推导展开分块减少全局访问的数量关系。采用 $N \times N$ 方阵，且 T 整除 N ；按代码中的全局输入加载计数，暂时忽略缓存效果、输出流量、同步和其他执行开销。

一个线程块的一轮计算，需要加载两个 $T \times T$ 输入块，共 $2T^2$ 个 `float`，即 $8T^2$ 字节。该轮计算 T^2 个输出部分和，每个包含 T 次乘加；按每次乘加两个浮点运算计，计算量为 $2T^3$ FLOP。于是，计算强度 (*arithmetic intensity*) 为

$$I_T = \frac{2T^3 \text{ FLOP}}{8T^2 \text{ B}} = \frac{T}{4} \text{ FLOP/B.} \quad (6.15)$$

等价地，每个浮点运算需要 $4/T$ 字节的全局输入数据。 T 增大时，同一输入被更多输出复用。

对比不使用共享复用的情况： T^2 个线程各自为 T 个内积步加载两次输入，共有 $2T^3$ 次输入元素加载；协作加载只需 $2T^2$ 次。因此，当前线程块这一轮的输入加载量减少为原来的 $1/T$ 。

6.5.1.2 整个矩阵乘法的流量

不使用共享复用时， N^2 个输出各需 $2N$ 次输入加载，总数为 $2N^3$ 。分块后，有 $(N/T)^2$ 个输出块，每块执行 N/T 轮，每轮加载 $2T^2$ 个输入元素，得到

$$L_T = \left(\frac{N}{T}\right)^2 \frac{N}{T} 2T^2 = \frac{2N^3}{T}. \quad (6.16)$$

当前方法的复用发生在线程块内部；不同输出块仍分别加载它们需要的输入块。上述计数保留了这一点。

将输出写回加入流量模型

每个有效输出最终写回一次，故输出流量为 $4N^2$ 字节。把输入与输出合并，可得到更完整的计算强度近似：

$$I_T^{\text{in+out}} = \frac{2N^3}{8N^3/T + 4N^2} = \frac{NT}{4N + 2T} \text{ FLOP/B.}$$

当 N 相对 T 很大时，该式趋近 $T/4$ 。这一补充用于说明忽略输出写回的近似条件，其余缓存和执行开销仍未包含。

6.5.2 带宽瓶颈如何转向计算瓶颈

令 P_{peak} 表示计算峰值， B_{mem} 表示全局内存带宽。用计算峰值和带宽支持能力作为两项上界，可写成

$$P_T \leq \min(P_{\text{peak}}, B_{\text{mem}} I_T). \quad (6.17)$$

这是一种用于解释瓶颈转移的简化模型。代入示例的 $P_{\text{peak}} = 1000 \text{ GFLOP/s}$ 、 $B_{\text{mem}} = 150 \text{ GB/s}$ ，有

$$B_{\text{mem}} I_T = 37.5T \text{ GFLOP/s.}$$

T	输入字节/FLOP	带宽支持能力	同时考虑计算峰值
1	4	37.5 GFLOP/s	37.5 GFLOP/s
16	0.25	600 GFLOP/s	600 GFLOP/s
32	0.125	1200 GFLOP/s	1000 GFLOP/s

16×16 分块把带宽支持能力提高到 600 GFLOP/s 。 32×32 分块的输入供给可以支持 1200 GFLOP/s ，已经超过示例 GPU 的计算峰值，所以该简化模型的限制转为计算能力。

这些数值描述带宽可以支撑的运算量及理论上界。实际执行还受到输出流量、缓存效果、同步、占用率和寄存器压力影响；这些因素正是从流量模型走向实际性能时需要补充的项。

6.5.3 分块尺寸与驻留资源

6.5.3.1 每个线程块需要多少资源

每线程一个输出、两个共享输入块的基本实现中，线程数和共享内存用量为

$$n_{\text{threads}} = T^2, \quad S_{\text{block}} = 2T^2 \text{sizeof(float)} = 8T^2 \text{ B.} \quad (6.18)$$

较大的分块带来更多复用，也同时增大线程块和共享内存占用。资源分配会限制一个流多处理器(*streaming multiprocessor, SM*)上可同时驻留的线程块数量，进而影响占用率(*occupancy*)。

占用率描述驻留 warp 相对于硬件可容纳 warp 数的比例。对这里的 16×16 、 32×32 规则线程块，可通过驻留线程数与线程上限比较其资源占用水平。资源容量给出驻留数量的上界，实际驻留状态还取决于核函数的资源需求。

6.5.3.2 示例：共享容量限制与线程容量限制

采用第一代 Maxwell 的示例条件：每个 SM 有 64 kB 共享内存，每线程块最多使用 48 kB，每个 SM 最多驻留 2048 个线程。以下存储容量换算使用 $1 \text{ kB} = 1024 \text{ B}$ 。

资源或计数	$T = 16$	$T = 32$
每块线程数 T^2	256	1024
每块共享内存 $8T^2$	2048 B = 2 kB	8192 B = 8 kB
仅按共享容量限制的块数	$64/2 = 32$	$64/8 = 8$
仅按线程容量限制的块数	$2048/256 = 8$	$2048/1024 = 2$
上述两项合并的块数上限	8	2
相应驻留线程数上限	$8 \times 256 = 2048$	$2 \times 1024 = 2048$
每线程两次加载的计数	$8 \times 256 \times 2 = 4096$	$2 \times 1024 \times 2 = 4096$

两个分块尺寸在这一估算中都暴露出 4096 个待加载操作。 32×32 分块虽然允许驻留的线程块更少，但每块线程更多，合计线程数相同；其输入复用倍数则更高。这里的加载数是按驻留线程和每线程两次加载得到的并行性估算。

6.5.3.3 寄存器与其他上限共同决定驻留数量

共享内存和线程数是两项直接可见的限制，寄存器数量及每 SM 的线程块数量上限也会约束驻留。令 S_{SM} 为每 SM 共享内存容量， R_{SM} 为寄存器数量， R_{thread} 为每线程寄存器用量， n_{max} 与 B_{max} 分别为每 SM 的线程数和块数上限。可用下面的理想化上界概括最紧资源约束：

$$B_{\text{active}} \leq \min \left\{ B_{\text{max}}, \left\lfloor \frac{n_{\text{max}}}{T^2} \right\rfloor, \left\lfloor \frac{S_{\text{SM}}}{8T^2} \right\rfloor, \left\lfloor \frac{R_{\text{SM}}}{R_{\text{thread}} T^2} \right\rfloor \right\}. \quad (6.19)$$

该式将资源容量除以每块需求，表达“由最先耗尽的资源限制驻留”的原则；实际资源分配粒度未包含在内。每块资源需求还必须满足设备对单个线程块的限制。

表 6.1 GPU 资源规格示例；M40 属于第二代 Maxwell。

GPU 型号	K40	M40	P100	V100	A100	H100
微架构	Kepler	Maxwell	Pascal	Volta	Ampere	Hopper
计算能力	3.5	5.2	6.0	7.0	8.0	9.0
SM 数量	15	24	56	80	108	132
每 SM 核心数	192	128	64	64	64	128
每 SM 最大线程数	2048	2048	2048	2048	2048	2048
每 SM 最大块数	16	32	32	32	32	32
每块最大线程数	1024	1024	1024	1024	1024	1024
每 SM 寄存器数	64K	64K	64K	64K	64K	64K
每 SM 共享内存	48KB	96KB	64KB	96KB	164KB	228KB

6.5.3.4 如何理解分块尺寸的取舍

分块边长 T 同时影响两组数量：复用次数随 T 增长，而每块线程数与共享内存需求随 T^2 增长。选择尺寸时，需要把输入复用、可驻留线程块数量、寄存器消耗和同步代价放在同一个执行模型中考虑。

基本实现中， 16×16 对应 256 个线程， 32×32 对应 1024 个线程；后者已经达到表 6.1 中列出的每块最大线程数。提高复用的空间因此也受到线程映射方式约束。具体核函数在实际设备上的较优尺寸仍需由测量确定。

6.6 共享内存接口与分块思想的推广

6.6.1 动态共享内存

6.6.1.1 将容量配置推迟到核函数启动时

静态共享数组的大小写在声明中，例如 `A_s[TILE_DIM][TILE_DIM]`。当所需容量需要在启动时确定，可以使用**动态共享内存** (*dynamic shared memory*):

```
extern __shared__ float A_s[];
```

核函数启动配置的第三个参数 `smemPerBlock` 指定每个线程块申请的动态共享内存字节数。配置形式为 `<<<nBlocks, nThreadsPerBlock, smemPerBlock>>>`。该容量按线程块分别提供，单位是字节。

6.6.1.2 将一段共享区域划分为两个输入块

以下局部片段用于把动态共享内存接口连接到两个输入块的存储需求。设运行时的分块边长为 T ，可以在核函数内部声明一段共享区域，再把它分成前后两段：

```
extern __shared__ float shared[];  
float* A_s = shared;  
float* B_s = shared + T * T;
```

两个输入块各占 T^2 个 `float`。采用一维共享指针后，局部位置 (t_y, t_x) 使用偏移 $t_y T + t_x$ ；计算时的两个偏移分别为 $t_y T + k$ 和 $kT + t_x$ 。对应的主机端容量计算为：

```
size_t smemPerBlock = 2 * T * T * sizeof(float);
```

分区方式只改变共享数组的表示和容量配置；协作加载、两次同步、零填充和输出写回的职责保持相同。该局部片段为存储布局说明，完整矩阵乘法实现采用代码 6.2 的静态共享数组版本。

6.6.2 Hopper 的相关功能

6.6.2.1 分布式共享内存

分布式共享内存(*distributed shared memory*)允许同一线程块集群(*thread block cluster*)内的线程块访问彼此的共享内存。它从 Hopper 架构开始提供,将访问范围扩展到集群中的线程块。前面的基本分块核函数使用各线程块自己的共享数组。

6.6.2.2 张量内存加速器

张量内存加速器(*Tensor Memory Accelerator, TMA*)支持全局内存与共享内存之间的数据异步传输。它可用于分块算法的输入搬运阶段。

6.6.3 CPU 缓存分块

6.6.3.1 相同的局部性,不同的存储管理方式

CPU 上也可以通过分块重排计算顺序。缓存分块(*cache blocking*)使一小组数据在较短的执行区间内被反复使用,从而建立时间局部性(*temporal locality*),提高数据仍在缓存中时被再次访问的机会。

GPU 基本实现由程序把输入块显式放入共享内存,并通过线程块同步协调使用;CPU 示例依靠缓存复用,通过循环顺序集中对当前分块的访问。二者都将计算组织为“小块数据,多次使用”。

6.6.3.2 三层分块循环与三层元素循环

CPU 示例对输出行、输出列和内积方向分别划分分块。外层三重循环选择输出块与当前输入块，内层三重循环计算该输入块对输出块的贡献。代码 6.3 是该循环结构的函数实现；假设 $N > 0$ 、 $T > 0$ ，且 T 整除 N 。

代码 6.3 CPU 缓存分块矩阵乘法，保持每个内积块的部分和累加顺序。

```

1 void matmul_cpu_tiled(
2     const float* A, const float* B, float* C, int N, int T)
3 {
4     for (int rt = 0; rt < N / T; ++rt) {
5         for (int ct = 0; ct < N / T; ++ct) {
6             for (int qt = 0; qt < N / T; ++qt) {
7                 for (int row = rt * T;
8                     row < (rt + 1) * T; ++row) {
9                     for (int col = ct * T;
10                        col < (ct + 1) * T; ++col) {
11                         float sum = 0.0f;
12                         for (int j = qt * T;
13                             j < (qt + 1) * T; ++j) {
14                             sum += A[row * N + j]
15                                 * B[j * N + col];
16                         }
17                         if (qt == 0) {
18                             C[row * N + col] = sum;
19                         } else {
20                             C[row * N + col] += sum;
21                         }
22                     }
23                 }
24             }
25         }
26     }
27 }

```

6.6.3.3 初始化与部分和的归属

对于固定的 $C_{r,c}$ ，每个内积块产生一个新的局部变量 **sum**。第一块通过赋值初始化 $C_{r,c}$ ，后续块通过 **+=** 累加。因此，初始的 C 数组内容无需参与计算。

GPU 基本核函数把一个输出的 **sum** 保持在线程寄存器中，跨越全部输入分块，最后统一写回；这里的 CPU 循环先计算当前内积块对整个输出块的贡献，跨内积块的部分和保存在 C 中。两种写法都实现式 (6.5)，但部分和保留的位置与循环组织不同。

6.7 关键结论与公式汇总

6.7.1 机制之间的联系

6.7.1.1 复用决定分块的收益

矩阵乘法的同一个输入会服务于多个输出。一个 $T \times T$ 线程块先协作加载 $2T^2$ 个输入，再计算 $2T^3$ 个浮点运算，将块内输入复用转化为约 T 倍的全局加载量缩减。

6.7.1.2 同步决定共享缓冲区的正确使用

共享数组在空间上由多个线程共同使用，在时间上由多个分块轮次复用。第一次屏障保护当前数据的完整加载，第二次屏障保护当前数据的完整使用。两次屏障分别对应这两类依赖。

6.7.1.3 边界处理决定协作任务能否保留

每个线程的 A 加载、 B 加载和 C 写回具有不同的有效性条件。加载阶段写零，使内积循环保持统一；输出阶段做有效性判断，使越界线程保留协作职责而不写入非法输出位置。

6.7.1.4 资源约束决定复用能否转化为性能

分块增大使计算强度线性增长，同时使每块线程数和共享内存需求按平方增长。带宽、计算峰值、驻留线程块数量以及寄存器需求，共同影响最终的执行速度。

6.7.2 关键公式

以下使用 $A \in \mathbb{R}^{M \times K}$ 、 $B \in \mathbb{R}^{K \times N}$ 、 $C \in \mathbb{R}^{M \times N}$ ，分块边长为 T 。

对象	表达式
线程对应输出	$r = b_y T + t_y, \quad c = b_x T + t_x。$
输出网格尺寸	$n_x = \lceil N/T \rceil, \quad n_y = \lceil M/T \rceil。$
内积方向轮数	$Q = \lceil K/T \rceil。$
A 加载地址与条件	$rK + qT + t_x; \quad r < M \text{ 且 } qT + t_x < K。$
B 加载地址与条件	$(qT + t_y)N + c; \quad qT + t_y < K \text{ 且 } c < N。$
共享内积	$\sum_{k=0}^{T-1} A_s[t_y][k] B_s[k][t_x]。$
输出地址与条件	$rN + c; \quad r < M \text{ 且 } c < N。$
每块线程与共享容量	T^2 个线程, $8T^2$ 字节共享内存。
理想输入计算强度	$I_T = T/4 \text{ FLOP/B}。$
简化性能上界	$P_T \leq \min(P_{\text{peak}}, B_{\text{mem}} I_T)。$

计算强度的单精度假设、浮点计数与流量模型见第 6.5 节。

第 7 讲

DRAM Bandwidth and other Performance Considerations

7.1 课程安排与核心内容

7.1.1 课程安排与阅读材料

以下课程事项对应 2026 年 9 月 15 日。

Lab 2 提交要求 Lab 2 在授课当周的星期五截止。需要完成全部要求的代码，包括主机内存的分配与释放；将工作推送到 GitHub；代码完成之后再提交 Lab 2 quiz。

课程辅助资源 课程聊天机器人可用于询问设备内存分配、主机到设备的数据复制、GPU 内核 (*kernel*) 的调用、共享内存数组声明，以及向量加法代码的逐行解释。复习时也可要求它围绕 Lecture 2 和 Lecture 3 逐题提问、等待回答并检查正确性。阅读任务是教材第 6 章和 CUDA Best Practices Guide 的 *Memory Optimizations*；待办事项是提交 Lab 2。

7.1.2 学习目标与知识范围

学习目标 理解动态随机存取存储器的组织方式；理解突发模式和多个存储体如何提高数据传输率；理解连接 GPU kernel 性能与 DRAM 组织的合并访存；掌握多种 kernel 优化手段及其相互权衡。

优化机制之间的关系 每块线程数、每块共享内存量和每线程寄存器数共同影响**占用率 (occupancy)**；减少控制分歧可提高 SIMD 效率，共享内存分块和寄存器分块则减少内存流量。访存与执行优化包括合并访存、面向内存延迟的占用率调节、线程粗化、循环展开和双缓冲。私有化与 warp 级原语分别用于减少访问冲突和线程间协调开销。

全局内存的标称带宽与程序实际获得的有效数据率之间可能存在差距。并发访问若争用同一资源，会产生串行化与延迟；为计算核心提供足够多的线程，还需要让这些线程以合适的方式请求数据。

三种互补的改进方向

数据复用减少需要搬运的数据量；合并访存提高必要传输中有效数据的比例；并发请求使内存系统在等待某次访问时仍能处理其他工作。线程粗化、指令调度与双缓冲进一步调整每线程工作量和等待关系。

7.2 DRAM 组织与带宽

7.2.1 从存储单元到二维阵列

7.2.1.1 电容保存比特，晶体管控制连接

随机存取存储器 (*Random Access Memory, RAM*) 允许直接访问任意地址，无须先顺序经过前面的存储位置。动态随机存取存储器 (*Dynamic RAM, DRAM*) 用电容上的电荷保存一个比特，并通过晶体管把该电容接到位线 (*bit line*) 上进行读写。选择信号 (*SELECT*) 决定这个连接是否导通。

电容会漏电，保存的信息因此需要周期性刷新 (*refresh*)；典型时间尺度为数十毫秒。每个比特只需要一个晶体管和一个电容，因此可以在有限面积内布置大量存储单元。

7.2.1.2 高密度与慢访问来自同一组织方式

在示意组织中，一根位线连接约 1,000 个单元，地址译码选择其中一个单元与位线接通。存储单元的电容很小，长位线的电容却很大；微小电荷改变位线状态需要时间，因此还要使用感应放大器 (*sense amplifier*) 来加速信号检测。

一个 DRAM 存储体 (*bank*) 由二维单元阵列组成：先激活一行，再按列选择数据。一条行选择线连接约 1,000 根位线，这与每根位线上的约 1,000 个单元一起形成约 10^6 比特量级的示意阵列。这些数值用于说明组织规模。

各部件的分工 行译码器 (*row decoder*) 根据行地址选择一行；感应放大器检测单元数据；列锁存器 (*column latches*) 保存已取得的行数据；多路选择器 (*multiplexer, Mux*) 再根据列地址选择输出。访问阵列和从已取得的数据中选择输出，构成快慢不同的两个阶段。

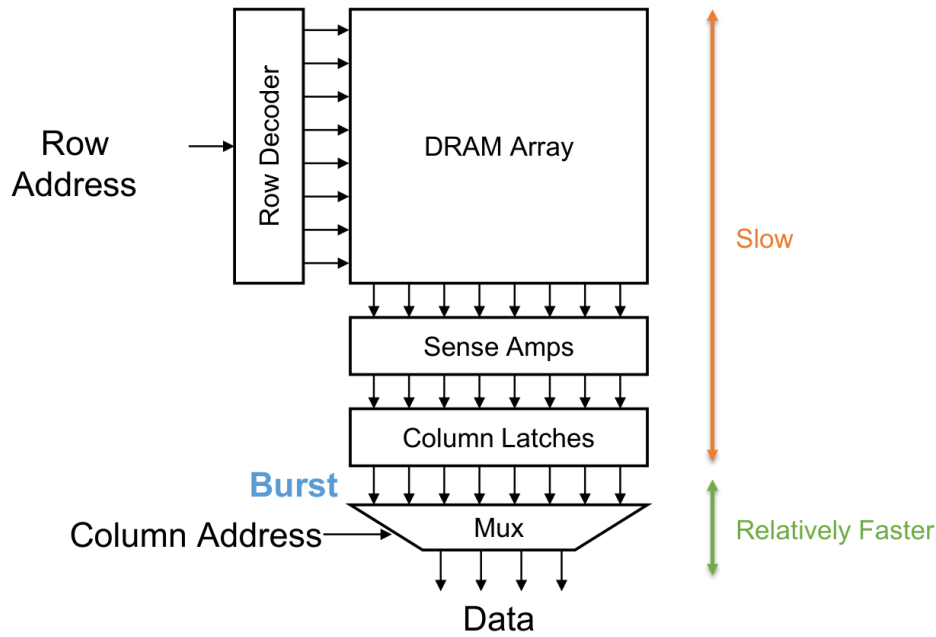


图 7.1 DRAM bank 的行译码、阵列、感应放大器、列锁存器与多路选择器。上部阵列访问较慢，已取得数据的列选择与输出相对较快。

7.2.2 选行、选列与突发传输

7.2.2.1 一次访问的三个阶段

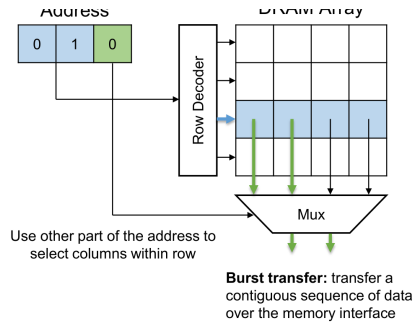
行激活 (*row activation*) 把选定行的数据打开到行缓冲区 (*row buffer*)；列访问 (*column access*) 从已打开的行中选择所需数据；突发传输 (*burst transfer*) 通过内存接口传送一串连续数据。

行地址与列地址承担不同的选择任务。行访问先承担较慢的阵列操作，后续数据可以从已经取得的内容中选出。这样，一次较慢的准备工作可支持多个相邻数据的输出。

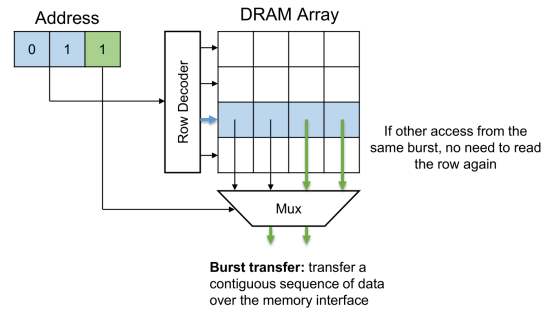
7.2.2.2 8×2 比特小存储体示例

“ 8×2 bit” 表示 8 个可寻址位置、每个位置 2 bit，总计 16 bit。按图中的地址划分，地址的前两位控制行选择，最后一位控制行内数据选择。示意阵列画成 4 行、每行 4 bit。

地址从 010 变为 011 时，仅最后一位改变。两次访问复用已取得的行数据，只改变后续的选择与输出，从而共用较慢的前期工作。



(a) 地址 010: 选行后选择一组数据。



(b) 地址 011: 保持行选择, 改变行内选择。

图 7.2 同一批已取得的数据支持相邻地址的输出。地址前两位以蓝色标记, 最后一位以绿色标记。

7.2.2.3 把慢阶段的成本分摊到多个数据

小存储体的时序模型用橙色表示较慢的阵列访问, 用绿色表示较快的数据输出。访问同一 burst 中的数据, 可以共用前面的橙色阶段; 图中不同 burst 的示意则重复了这个阶段。

示意时序的形式化 设 L 为一次慢阶段的时间, t 为一次快输出的时间, q 为所需输出的数据个数。在“每个数据单独重复慢阶段”和“ q 个数据共用慢阶段”这两个示意情形下, 分别有

$$T_{\text{separate}} = q(L + t), \quad (7.1)$$

$$T_{\text{burst}} = L + qt, \quad (7.2)$$

$$T_{\text{separate}} - T_{\text{burst}} = (q - 1)L. \quad (7.3)$$

L 和 t 分别表示模型中的慢阶段与输出阶段耗时; 时间式通过比较两阶段的重复次数解释 burst 的收益。

突发传输的带宽含义

连续数据能够分摊一次较慢的准备成本。若一次传输带来的数据大部分都会被使用, 接口传输的数据就能更多地转化为程序需要的数据; 若只使用其中少量位置, 其余传输占用的带宽就没有直接贡献。

7.2.3 多个存储体与内存级并行

7.2.3.1 通过重叠操作填补传输间隙

单个 bank 即使使用 burst，两次突发传输之间仍可能存在等待时间。多个 bank 分别处理独立请求时，控制器可以把它们的内部操作重叠起来，使一个 bank 准备数据的时间与另一个 bank 输出数据的时间交错，从而保持数据接口忙碌。

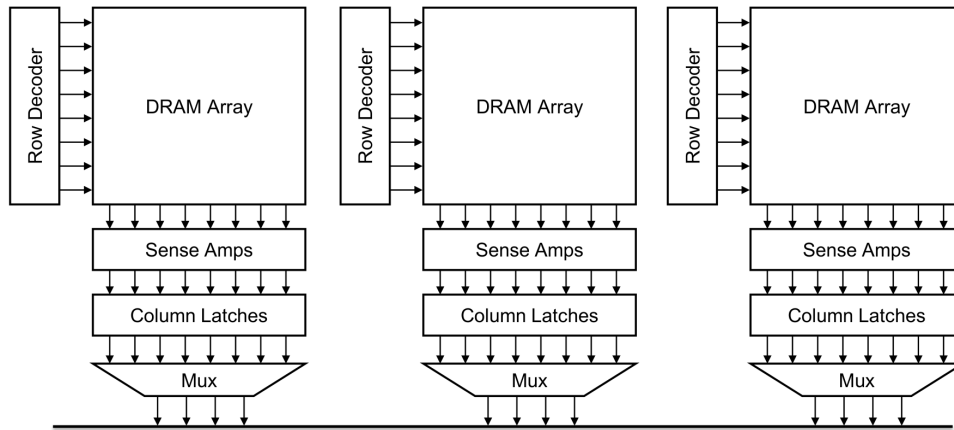


图 7.3 多个 DRAM bank 具有各自的阵列、译码与数据选择部件，并连接到下方的数据接口。不同 bank 上的独立请求提供可重叠的内部操作。

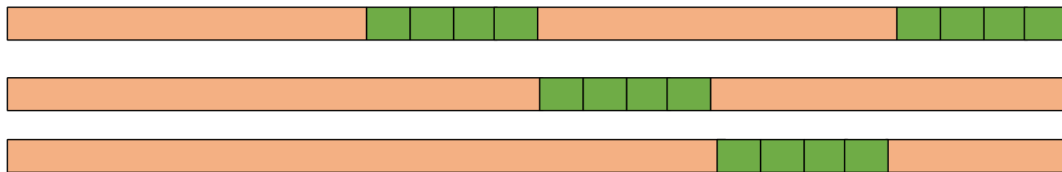


图 7.4 三个 bank 的慢阶段与突发输出交错排列。绿色输出段依次出现，使整体数据传输能比单个 bank 更连续。

7.2.3.2 硬件并行需要足够多的独立请求

内存级并行 (*memory-level parallelism, MLP*) 是多个独立内存操作同时处于进行中的状态。多个 bank 提供并行处理的条件，程序则需要持续发出足够多的请求，让这些 bank 有工作可做。

较高的流多处理器 (*Streaming Multiprocessor, SM*) 占用率可以提供更多线程，从而增加可并发的内存访问。这与使用多个线程束 (*warp*) 隐藏计算核心中的流水线延迟具有相同的基本思路：一部分工作正在等待时，仍有其他工作可以推进。

与单次访存效率的关系 合并访存改善一次 warp 访问的数据分布，MLP 改善不同请求在时间上的重叠程度。共享内存分块则通过复用减少需要发出的请求。三者分别作用于传输量、传输效率和请求并发程度。

占用率的权衡 为了提高占用率，需要同时考虑每块线程数、每块共享内存和每线程寄存器。更多驻留线程有助于隐藏延迟，也会共享并竞争这些有限资源；优化目标是提供足够的独立工作，同时保留每个线程或线程块需要的复用空间。

7.3 Warp 访存与共享内存分块

7.3.1 合并访存的判断单位

7.3.1.1 固定一条访存指令，比较同一 warp 的地址

合并访存 (memory coalescing) 是把同一线程束 (warp) 内多个线程对相邻位置的访问组合起来服务。连续访问能够以较少的内存事务 (memory transaction) 传送所需数据；分散到不同位置的请求往往需要更多事务，也会增加 warp 等待数据的时间。这类分散访问也称为内存分歧 (memory divergence)。

判断时需要固定同一次访存，再观察 warp 内各线程的地址。例如，在矩阵乘法内积循环中固定 k ，列出线程 $T_0, T_1, \dots$ 此时分别访问哪里。一个线程随 k 变化的访问顺序，与多个线程在固定 k 时的地址分布，是两个不同的观察方向。

访存分析的三个要素

线程块形状决定一个 warp 中哪些线程坐标相邻；地址表达式把线程坐标映射成元素下标；元素类型再把下标差换算成字节距离。确定这三项之后，才能判断同一 warp 的地址是连续、相同，还是带有步长。

7.3.1.2 二维线程块如何组成 warp

对二维线程块，线程首先沿 x 维连续排列，再进入下一条 y 行。用 $B_x = \text{blockDim.x}$ 、 $B_y = \text{blockDim.y}$ 表示块形状，用 t_x, t_y 表示块内坐标，可将这个排列规则写为

$$t = t_y B_x + t_x, \quad t_x = t \bmod B_x, \quad t_y = \left\lfloor \frac{t}{B_x} \right\rfloor. \quad (7.4)$$

式中 t 为块内线性线程编号。每 32 个相邻线性线程构成一个 warp；用 $\ell = 0, \dots, 31$ 表示 warp 内的线程槽位(lane)，某个完整 warp 的线程编号可写成 $t = 32w + \ell$ ，其中 w 为该 warp 在块内的编号。这是对给定排列规则的索引展开。

因此，`dim3(32, 8)` 中的 warp 可以沿完整的一条 x 行排列；`dim3(1, 32)` 的 warp 沿 y 变化；`dim3(16, 16)` 的一个 warp 则由两条各含 16 个线程的 x 行组成。

7.3.1.3 四种典型地址分布

下面的地址均为字节地址。连续访问中的每个线程读取一个 4 字节元素。

表 7.1 同一次访存中，不同 lane 的地址关系。

模式	lane 0、1、2、3 等的地址	主要特征
连续合并 <i>coalesced</i>	0, 4, 8, 12, ..., 124	相邻线程请求相邻元素，形成连续的 128 字节需求。
广播 <i>broadcast</i>	40, 40, 40, 40, ...	多个线程请求同一数据，表现为同址复用。
步长访问 <i>strided</i>	0, 16, 32, 48, ...	每次只取相隔 16 字节的位置，地址间留有空隙。
分散访问 <i>scattered</i>	84, 0, 132, 16, ...	地址分散，需逐项检查其覆盖的传输区域。

广播可以满足多个线程对同一数据的需求；连续合并则让多个不同元素一起被有效传输。广播与连续合并对应不同的数据关系，分析复用收益时应分别处理。

在简化模型中，一组连续访问可由一次 burst 或事务服务。更一般地，合并访存减少满足同一 warp 请求所需的事务数量；具体数量还取决于硬件事务粒度与地址对齐。

7.3.2 向量访问与行主序地址

7.3.2.1 向量加法：每条指令内的地址都连续

对长度为 n 的单精度向量，线程的全局下标为

$$i = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}. \quad (7.5)$$

核心运算为 $z[i] = x[i] + y[i]$ ，边界判断保证只有有效下标参与计算。

代码 7.1 向量加法：相邻线程处理相邻元素。

```

1 __global__ void vectorAdd(const float* x, const float* y,
2                           float* z, unsigned int n) {
3     unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
4     if (i < n) {
5         z[i] = x[i] + y[i];
6     }
7 }

```

设一个完整 warp 的第一个全局下标为 i_0 ，则对数组 x 的读取地址为

$$a_{x,\ell} = a_x + 4(i_0 + \ell), \quad \ell = 0, \dots, 31, \quad (7.6)$$

其中 a_x 是数组起始字节地址。相邻 lane 的地址差是 4B，整个 warp 需要 32 个 float，即 128 B 的连续数据。对 y 的读取与对 z 的写入也分别具有同样的关系。三组数组访问分别分析。

7.3.2.2 二维 C 数组按行线性存储

行主序 (row-major order) 把同一行的元素连续放入内存，再接下一行。设矩阵列数为 $W = \text{Width}$ ，行列下标分别为 r, c ，则元素下标与字节地址为

$$\text{index}(r, c) = rW + c, \quad a(r, c) = a_0 + s(rW + c), \quad (7.7)$$

其中 a_0 为矩阵起始地址， s 为每个元素的字节数。固定 r 而令 c 增加 1，地址增加 s ；固定 c 而令 r 增加 1，地址增加 sW 。这是对行主序图的形式化。

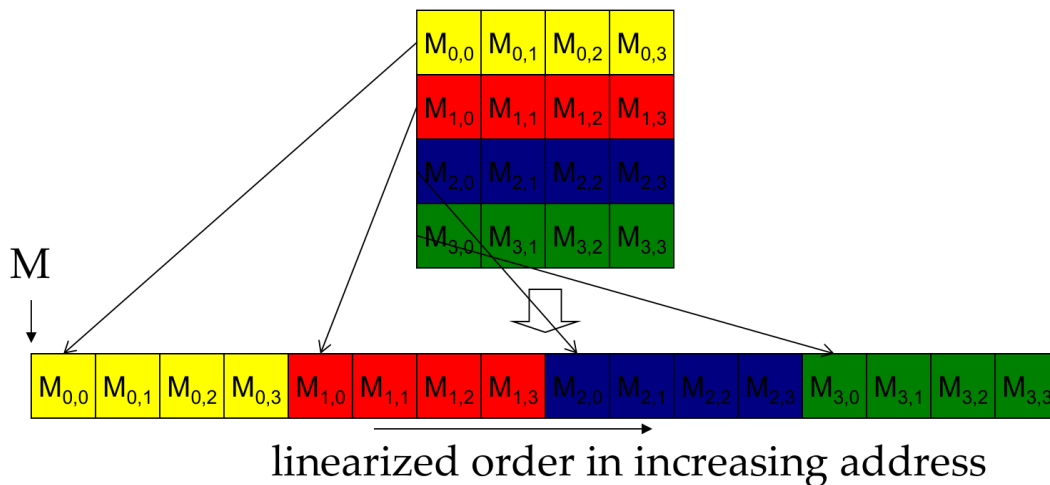


图 7.5 四行矩阵依次线性存储。每一行以同一颜色表示，内存地址沿下方箭头增大。

矩阵乘法中的 $M[\text{row} * \text{Width} + k]$ 沿单个线程的 k 循环读取一行, $N[k * \text{Width} + \text{col}]$ 沿单个线程的 k 循环读取一列。是否合并还取决于同一 warp 内的 row 和 col 如何变化。

7.3.3 矩阵乘法：连续访问、广播与跨行步长

7.3.3.1 固定输出元素与固定内积迭代

设 M, N, P 均为 $W \times W$ 矩阵, $P = MN$, 则

$$P_{r,c} = \sum_{k=0}^{W-1} M_{r,k} N_{k,c}. \quad (7.8)$$

每个线程负责一个输出元素, threadIdx.x 影响输出列, threadIdx.y 影响输出行。下列片段在 kernel 内使用, M, N 为输入指针, Width 为 W ; 假设该线程的行列坐标有效。

代码 7.2 矩阵乘法的内积访问, Width 表示矩阵维度。

```
1 int row = blockDim.y * blockIdx.y + threadIdx.y;
2 int col = blockDim.x * blockIdx.x + threadIdx.x;
3 float sum = 0.0f;
4 for (int k = 0; k < Width; ++k) {
5     sum += M[row * Width + k] * N[k * Width + col];
6 }
```

7.3.3.2 沿同一输出行排列的线程

考虑 $\text{blockDim.x}=32$, 一个 warp 位于同一输出行 r , 列下标为 $c_0 + \ell$ 。固定 k 后, 两个输入的元素下标分别为

$$I_M(\ell) = rW + k, \quad (7.9)$$

$$I_N(\ell) = kW + c_0 + \ell. \quad (7.10)$$

I_M 与 ℓ 无关, 32 个线程读取同一个 $M_{r,k}$, 对应广播或复用; I_N 随 ℓ 每次增加 1, 线程读取相邻的 N 元素, 对应经典连续合并访问。

沿着单个线程观察, N 的读取会随着 k 增加而跳到下一行; 沿着同一次 k 的 warp 观察, 线程却一起读取 N 的同一行。这解释了“单个线程纵向访问, 但 warp 仍可合并”的现象。

7.3.3.3 跨输出行排列的线程

若所比较的线程具有 $r = r_0 + \ell$ ，则固定 k 时

$$I_M(\ell) = (r_0 + \ell)W + k, \quad I_M(\ell + 1) - I_M(\ell) = W. \quad (7.11)$$

相邻线程访问不同行的同一列，相差 W 个元素。单精度数据的字节距离为 $4W$ ，这就是跨行步长访问。

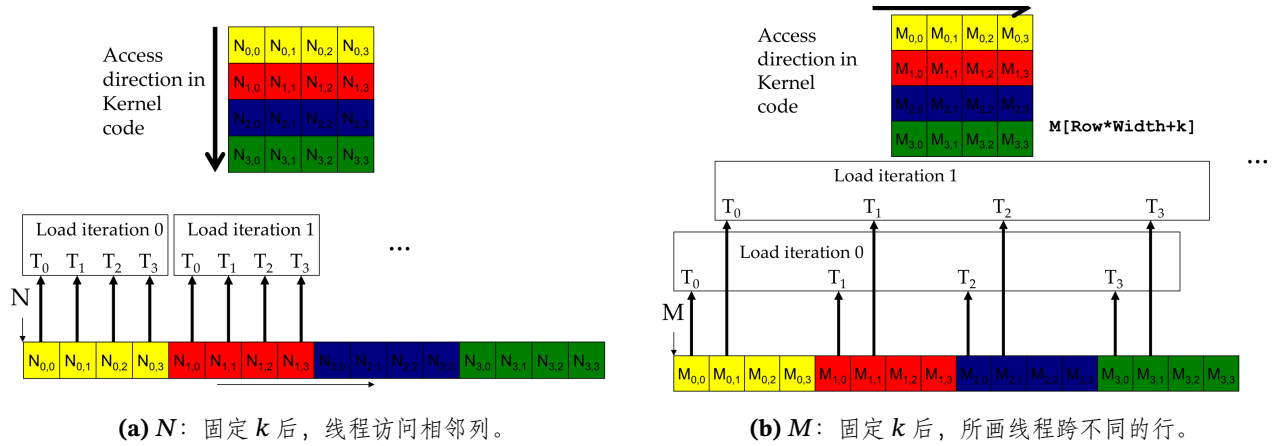


图 7.6 两个访问方向示意。上方箭头描述单线程沿 k 的移动，下方连线描述固定 k 时线程之间的地址关系。

线程映射决定访问模式 跨输出行的线程读取 M 时形成跨行步长；同一输出行中的线程则对 M 广播。判断访存模式时，需要先明确所比较线程的坐标关系。

块宽为 16 时的索引展开 一个 warp 包含两条输出行、每行 16 个线程。固定 k 时，对 M 的请求形成两组同址访问；对 N 的请求则是同一组 16 个连续列各被访问两次。这可直接由式(7.4)和内积代码得到，进一步说明应按实际 warp 布局判断。

7.3.4 共享内存分块：协作加载与数据复用

7.3.4.1 将数据搬运方式与计算访问方式分开

共享内存分块(*shared-memory tiling*) 让一个线程块协作加载输入子块，再在共享内存中反复使用这些值。全局内存加载按便于合并的顺序分工，乘法运算则按内积所需的行列关系读取共享内存。

设子块边长为 $T = \text{TILE_DIM}$ ，阶段编号为 p ，块内线程坐标为 t_x, t_y 。每个线程在阶段 p 加载

$$M_s[t_y][t_x] \leftarrow M[rW + pT + t_x], \quad (7.12)$$

$$N_s[t_y][t_x] \leftarrow N[(pT + t_y)W + c]. \quad (7.13)$$

在同一条块内 y 行上， r 与 t_y 固定， t_x 连续变化， c 也随 t_x 连续变化，因此两个输入的加载地址都连续。

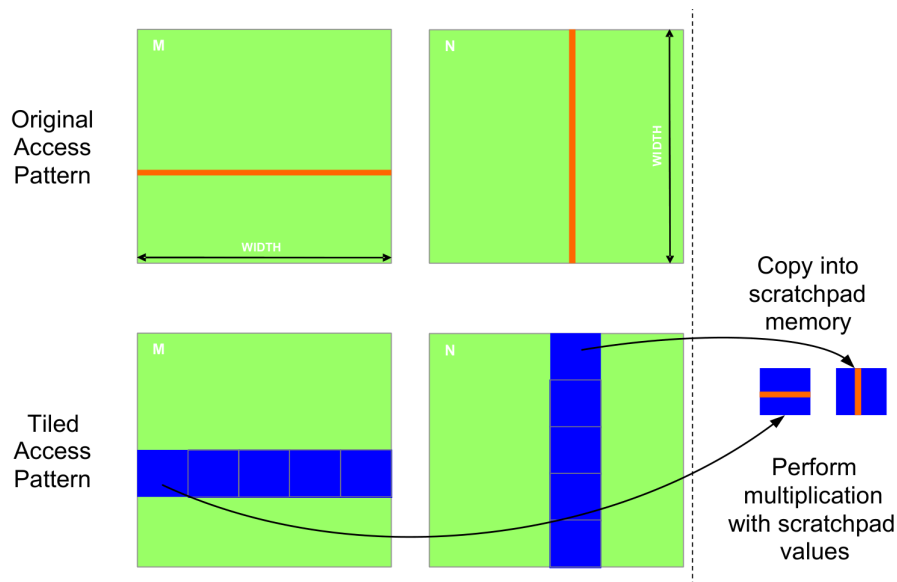


图 7.7 输入子块先复制到暂存区(*scratchpad memory*)，再使用暂存值执行乘法。蓝色区域表示分阶段加载的子块，橙色线表示内积所需的行列方向。

7.3.4.2 完整计算阶段与同步位置

方阵分块核函数依次执行协作加载、同步、计算和再次同步。 T 为编译期常量， $Width$ 是 T 的正整数倍；线程块为 $T \times T$ ，网格为 $(Width/T) \times (Width/T)$ 。这些约定使所有加载下标有效。

代码 7.3 包含协作加载、两次同步和输出写回的分块矩阵乘法。

```

1  template<int T>
2  __global__ void matmulTiled(const float* M, const float* N,
3                             float* P, int Width) {
4      __shared__ float M_s[T][T];
5      __shared__ float N_s[T][T];
6      int tx = threadIdx.x;
7      int ty = threadIdx.y;
8      int row = blockIdx.y * T + ty;
9      int col = blockIdx.x * T + tx;
10     float sum = 0.0f;
11
12     for (int tile = 0; tile < Width / T; ++tile) {
13         M_s[ty][tx] = M[row * Width + tile * T + tx];
14         N_s[ty][tx] = N[(tile * T + ty) * Width + col];
15         __syncthreads();
16
17         for (int k = 0; k < T; ++k) {
18             sum += M_s[ty][k] * N_s[k][tx];
19         }
20         __syncthreads();
21     }
22     P[row * Width + col] = sum;
23 }

```

第一次同步确保两个输入子块已经由所有线程加载完成；第二次同步确保所有线程都用完当前子块，随后才允许下一阶段覆盖这两块共享内存。

7.3.4.3 合并收益与复用收益分别计算

加载顺序 在 $T = 32$ 的示意下，warp 中的 $t_x = 0, \dots, 31$ ，两个加载式分别请求 32 个连续元素。在简化事务模型中，这类连续加载可由一次事务服务。块宽变化后仍按实际 warp 布局分析，例如 $T = 16$ 时每个 warp 包含两段各 16 个元素的行内加载。

同一输出行直接读取 M 时，整个 warp 取得一个共同元素；协作加载阶段则一次取得一段不同的 M 元素，随后供块内重复使用。这样既提高传输中不同有效数据的利用程度，也保留后续的复用机会。

源代码层面的读取次数 一个 $T \times T$ 输出子块在一个内积阶段需要 T^3 次乘法。逐输出直接读取两项输入的写法，对应 $2T^3$ 次逻辑元素读取；协作加载两个 $T \times T$ 子块，则只在加载阶段执行 $2T^2$ 次全局元素读取。于是

$$\frac{\text{逐输出读取次数}}{\text{协作加载次数}} = \frac{2T^3}{2T^2} = T. \quad (7.14)$$

这项计数用于解释复用机会：同一输入元素在块内参与多次乘法。广播和缓存会影响实际 DRAM 流量，所以该比值描述代码中的请求计数，不直接等于实际带宽或加速比。

资源代价 代码中两个共享数组共占 $2T^2$ 个 float。按每个 float 为 4 字节，共享内存需求为

$$S_{\text{block}} = 8T^2 \text{ B}. \quad (7.15)$$

例如 $T = 16$ 时为 2,048 B， $T = 32$ 时为 8,192 B；对应线程数分别为 256 与 1,024。这是由代码得到的资源计数。更大的子块增加复用机会，同时消耗更多每块资源，因而会与占用率产生权衡。

7.4 线程粒度、指令调度与同步

7.4.1 从细粒度线程到线程粗化

7.4.1.1 细粒度划分与透明可扩展性

细粒度线程划分 (*fine-grain thread granularity*) 把尽可能小的可并行工作单元分给每个线程：向量加法中一个线程处理一个向量元素，RGB 转灰度或模糊操作中一个线程处理一个输出像素，矩阵乘法中一个线程处理一个输出元素。

这种划分向硬件暴露尽可能多的线程。资源充足时，可以同时推进较多工作；线程数超过硬件当时能容纳的数量时，其余工作可由硬件以较低开销分批执行。未来设备具有更多资源时，同一工作划分还能直接提取更多并行性，这就是**透明可扩展性** (*transparent scalability*)。

过细划分的代价 若把工作拆给更多线程会额外引入重复读取、重复计算或同步，这些开销会随着细粒度划分一起出现。只有一部分线程能同时执行时，程序可能已经承担了更多并行化开销，却仍需要等待多批工作依次完成。

7.4.1.2 一个线程承担多个工作单元

线程粗化 (*thread coarsening*) 把多个可并行工作单元交给同一线程。粗化因子 (*coarsening factor*) 表示每个线程负责的工作单元数。例如，粗化因子为 4 时，每个线程处理四个工作单元。

下列片段在 `kernel` 内使用，`foo(i)` 表示已经定义好的第 i 个独立工作单元的处理函数。未粗化时，每个线程只调用一次：

代码 7.4 细粒度映射。

```
1 unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
2 foo(i);
```

粗化之后，每个线程顺序处理四个单元：

代码 7.5 连续四单元分配给同一线程。有效下标范围在该概念片段中作为前提。

```
1 unsigned int iStart =
2     4 * (blockIdx.x * blockDim.x + threadIdx.x);
3 for (unsigned int c = 0; c < 4; ++c) {
4     unsigned int i = iStart + c;
5     foo(i);
6 }
```

粗化把一部分跨线程的工作关系改为线程内部的顺序关系，为复用数据或中间结果、减少重复工作和同步提供机会；具体收益取决于原来为并行化付出了什么开销。

7.4.2 粗化后的地址布局与资源代价

7.4.2.1 连续分配给线程可能产生跨线程步长

设一个 `warp` 中相邻线程的全局编号为 $q_0 + \ell$ 。在上面的连续四单元分配中，固定粗化循环下标 c 后，第 ℓ 个 `lane` 的元素下标为

$$i_{\ell,c} = 4(q_0 + \ell) + c, \quad i_{\ell+1,c} - i_{\ell,c} = 4. \quad (7.16)$$

若 `foo(i)` 访问数组第 i 个 `float`，相邻 `lane` 相隔 4 个元素，即 16 B。每个线程自己的四次访问相邻，但同一次循环迭代中，`warp` 的地址带有步长。

7.4.2.2 按线程块宽度跨步，保留每轮连续访问

另一种映射让相邻线程在同一轮处理相邻元素，让同一线程在不同轮之间跨过一个线程块宽度。令 $B = \text{blockDim.x}$ ，代码为：

代码 7.6 保持每次迭代内 warp 连续访问的四倍粗化。

```

1 unsigned int base =
2     blockIdx.x * (4 * blockDim.x) + threadIdx.x;
3 #pragma unroll
4 for (unsigned int c = 0; c < 4; ++c) {
5     unsigned int i = base + c * blockDim.x;
6     if (i < n) {
7         foo(i);
8     }
9 }

```

用 b 表示线程块编号， t 表示块内线程编号，则该映射为

$$i(b, t, c) = 4bB + t + cB, \quad 0 \leq t < B, \quad 0 \leq c < 4. \quad (7.17)$$

固定 b, c ，相邻线程的下标差为 1；固定 b, t ，同一线程下一轮的下标增加 B 。一个块覆盖从 $4bB$ 到 $4(b+1)B-1$ 的连续 $4B$ 个位置。

例如取 $b = 0, B = 32$ ，一个 warp 的映射如下。表中的数字为元素下标：

表 7.2 四轮粗化中的线程分工，由式(7.17)计算。

循环下标 c	lane 0	lane 1	lane 2	lane 31
0	0	1	2	31
1	32	33	34	63
2	64	65	66	95
3	96	97	98	127

因此，对 n 个工作单元，线程块数量应相应改成

$$G = \left\lceil \frac{n}{4B} \right\rceil. \quad (7.18)$$

最后一个块中， $i < n$ 使超过数组长度的位置不执行处理。

7.4.2.3 粗化因子与占用率一起选择

粗化可能减少冗余内存访问、冗余计算、同步开销或控制分歧；同时，一个线程也可能需要更多寄存器或其他资源，进而降低占用率。粗化因子过大，还会使可并发线程过少而无法充分利用设备。

相同的粗化因子在不同设备上可能表现不同，因此需要重新调节。影响性能的关系包括：每线程增加了多少有效工作、减少了多少重复开销、每线程资源增加多少，以及仍有多少独立线程或指令可以隐藏延迟。

粗化的两个联合检查

一方面检查工作分配：每轮相邻 lane 是否仍访问相邻元素，网格是否覆盖全部工作；另一方面检查资源代价：增加的线程内工作是否值得以更少的驻留线程来交换。

7.4.3 循环展开、指令调度与寄存器使用

7.4.3.1 展开循环体，减少循环控制

循环展开 (*loop unrolling*) 把循环体复制若干次，同时按相同倍数减少循环迭代次数。循环体执行的总工作量保持一致，循环计数、判断和跳转出现的次数则减少。

设 $A(i)$ 、 $B(i)$ 是已定义的操作，原循环为：

代码 7.7 原始循环执行 16 次，每次处理一个下标。

```
1 for (unsigned int i = 0; i < 16; ++i) {
2     A(i);
3     B(i);
4 }
```

按因子 4 展开之后，每次循环处理四个下标，外层循环只执行 4 次：

代码 7.8 按因子 4 展开的循环。

```
1 for (unsigned int i = 0; i < 16; i += 4) {
2     A(i);    B(i);
3     A(i + 1); B(i + 1);
4     A(i + 2); B(i + 2);
5     A(i + 3); B(i + 3);
6 }
```

循环展开减少停顿的两条路径是：减少循环分支，以及暴露更多独立指令供后续调度。当缺少分支预测、循环分支具有较长延迟时，减少分支也能减少相应等待。

7.4.3.2 拉开依赖指令之间的距离

指令调度 (instruction scheduling) 在保持程序依赖关系的条件下重排指令，把相互依赖的指令隔得更远，从而让中间穿插的独立指令填补等待时间。

示例明确假设： $B(i)$ 依赖 $A(i)$ ，而不同 i 之间没有冲突依赖或副作用。也就是每个下标内部存在 $A(i) \rightarrow B(i)$ ，不同下标的操作链相互独立。于是展开后的指令可以排列为：

代码 7.9 在各迭代独立的前提下先安排 A，再安排依赖它们的 B。

```
1 for (unsigned int i = 0; i < 16; i += 4) {  
2     A(i);  
3     A(i + 1);  
4     A(i + 2);  
5     A(i + 3);  
6     B(i);  
7     B(i + 1);  
8     B(i + 2);  
9     B(i + 3);  
10 }
```

这样， $A(i)$ 与 $B(i)$ 之间插入了另外三个独立的 A 操作。循环展开扩大了可见的指令序列，指令调度再利用其中的独立性。这种单个线程内部可利用的指令并行性称为**指令级并行 (instruction-level parallelism, ILP)**。

若不同迭代之间存在必须保留的依赖，能够重排的范围就由这些依赖限定。例如，示意中的调度依赖于“不同迭代没有冲突”这一明确假设。

7.4.3.3 编译器指令与编译期常量

循环展开和指令调度通常由编译器自动完成，也可以使用指令控制展开。例如，使用 `##pragma unroll 4`：

代码 7.10 使用编译器指令指定展开因子。

```
1 #pragma unroll 4  
2 for (unsigned int i = 0; i < 16; ++i) {  
3     A(i);  
4     B(i);  
5 }
```

固定循环范围有利于编译器识别迭代次数。线程粗化常会引入这种短循环；粗化因子为编译期常量时，粗化循环可以完全展开。

7.4.3.4 常量下标使局部数组更容易提升为寄存器

粗化之后，一个线程可能用小数组保存多个工作单元的状态。例如，从一个标量变为四个元素：

代码 7.11 具有变量下标的线程局部数组。

```
1 int x[4] = {0, 0, 0, 0};  
2 for (unsigned int c = 0; c < 4; ++c) {  
3     foo(x[c]);  
4 }
```

完全展开后，各次访问的下标成为常量：

代码 7.12 展开后各次访问具有固定下标。

```
1 int x[4] = {0, 0, 0, 0};  
2 foo(x[0]);  
3 foo(x[1]);  
4 foo(x[2]);  
5 foo(x[3]);
```

固定下标使每个数组元素可分别对应一个寄存器值，从而提供**寄存器提升 (register promotion)** 的机会。变量下标可能使数组需要可寻址的存储，常量下标则便于编译器把各元素分别分配到寄存器；实际分配取决于编译器优化和资源限制。

三种变换的关系 线程粗化改变“一个线程负责多少工作”；循环展开改变“同一线程怎样表达这些工作”；指令调度改变“在依赖允许范围内，先执行哪条指令”。三者可以结合，以减少重复开销、循环控制和依赖等待，同时仍需考虑增加的寄存器需求。

7.4.4 双缓冲与同步依赖

7.4.4.1 单缓冲为何需要两次屏障

双缓冲 (double buffering) 用一个缓冲区读取当前数据，用另一个缓冲区写入新数据，消除因共享同一存储位置而产生的**假依赖 (false dependence)**。

设一个线程块内的线程编号为 `tid`, `other` 是该线程要读取的有效块内下标。每轮从其他线程对应的旧状态取得数据, 再更新自己的状态。用已定义函数 `updateValue` 表示状态更新, 单缓冲循环可写为:

代码 7.13 单缓冲迭代: 读完旧值后才能覆盖, 写完新值后下一轮才能读取。

```
1 for (int j = 0; j < steps; ++j) {  
2     float next = updateValue(buffer[other]);  
3     __syncthreads();  
4     buffer[tid] = next;  
5     __syncthreads();  
6 }
```

第一次屏障: 保护尚未完成的旧值读取 一些线程可能已经读完并准备写入, 而另一些线程仍在读取旧数据。第一次 `__syncthreads()` 要求整个块先完成旧值读取, 随后再覆盖同一数组。它保护的是**写后于读** (*write-after-read, WAR*) 的顺序。这个约束源于读写复用了同一存储位置, 因此属于假依赖。

第二次屏障: 确保下一轮读到已完成的新值 下一轮需要使用本轮其他线程产生的新数据, 所以所有写入都要完成, 线程才能进入下一轮读取。这个**读后于写** (*read-after-write, RAW*) 的关系来自计算的数据流, 属于**真依赖** (*true dependence*)。

假依赖仍决定单缓冲实现的正确性。改变存储安排、让旧值与新值分别保存之后, 才具备移除第一道屏障的条件。

7.4.4.2 两个缓冲区交替承担输入与输出

假设 `buffer0`、`buffer1` 是两个大小相同、彼此分离的块内共享缓冲区，初始状态已放入 `buffer0`，并已完成必要的初始化同步。所有参与线程执行相同轮数。上述初始化前提用于明确执行语义，指针交换在代码中显式展开。

代码 7.14 双缓冲迭代：只保留每轮末尾的屏障。两个指针交换输入与输出缓冲区的角色。

```

1 float* inBuffer = buffer0;
2 float* outBuffer = buffer1;
3 for (int j = 0; j < steps; ++j) {
4     float next = updateValue(inBuffer[other]);
5     outBuffer[tid] = next;
6     __syncthreads();
7
8     float* tmp = inBuffer;
9     inBuffer = outBuffer;
10    outBuffer = tmp;
11 }
```

读取 `inBuffer` 时，其他线程写入的是 `outBuffer`，因此写入新数据不会覆盖本轮仍需读取的旧值。每轮结束的屏障仍然保留，确保所有新数据都已完成写入。交换两个指针后，刚写好的数组成为下一轮输入，原输入数组成为下一轮输出。

表 7.3 双缓冲的角色随迭代交替，由指针交换代码直接得到。

迭代编号	读取旧状态	写入新状态	交换后的输入指针
$j = 0$	<code>buffer0</code>	<code>buffer1</code>	指向 <code>buffer1</code>
$j = 1$	<code>buffer1</code>	<code>buffer0</code>	指向 <code>buffer0</code>
$j = 2$	<code>buffer0</code>	<code>buffer1</code>	指向 <code>buffer1</code>

循环状态的检查 每次交换之后，`inBuffer` 都指向最新结果，`outBuffer` 指向可供下一轮覆盖的旧缓冲区。交换只改变指针代表的角色，不搬运数组内容。最后一轮也完成交换，因此循环退出后的最新状态仍由 `inBuffer` 指向。以上结论直接来自代码中的赋值顺序。

7.4.4.3 减少屏障与增加存储之间的交换

按上述两个循环计数，执行 $J = \text{steps}$ 轮时，单缓冲的循环内屏障调用次数为 $2J$ ，双缓冲为 J ；初始化同步不计入这项比较。减少的是用于约束旧值读取与覆盖写入的那一道屏障。

若一个缓冲区占用 S 字节，两块缓冲区需要 $2S$ 字节。在共享内存中实现时，增加的每块存储需求还要与占用率一起考虑。同步开销在总时间中所占比例，以及额外存储对并行执行的影响，共同决定实际收益。

双缓冲保留的数据依赖

旧状态与新状态分别存储，使本轮读取和本轮写入可以不经中间屏障直接衔接；本轮写入与下一轮读取之间仍有真依赖，所以每轮末尾的屏障保留。

7.5 优化权衡与例题

7.5.1 常见优化的功能清单

7.5.1.1 计算利用率

计算利用率优化主要处理可执行工作不足、循环和依赖停顿，以及 SIMD 执行中的空闲线程。

表 7.4 计算利用率优化的收益与实现方向。

优化	对计算与访存的收益	实现方向
占用率调节	用更多工作隐藏流水线延迟；用更多并行内存请求隐藏 DRAM 延迟。	调节每块线程数、每块共享内存及每线程寄存器等 SM 资源用量。
循环展开	减少循环分支；暴露独立指令；可能促进局部数组进入寄存器而减少内存访问。	利用编译器展开，在适用处使用编译期可知的固定循环范围。
减少控制分歧	提高 SIMD 效率，减少统一执行过程中空闲的计算资源。	重新组织线程与工作、数据的映射，或调整数据布局。

7.5.1.2 内存利用率

内存利用率优化分别减少无效传输、跨层重复取数和冲突串行化。共享内存分块主要减少全局内存与 SM 之间的重复传输；寄存器分块进一步减少共享内存到寄存器的重复取数。

表 7.5 内存利用率优化。私有化用于减少共享访问冲突。

优化	对计算与访存的收益	实现方向
合并全局访存	减少等待全局数据的停顿；减少无效流量，提高 burst / 缓存行利用率。	调整线程映射或数据布局；合并地搬入共享内存，再在其中执行所需访问，例如 corner turning 。
共享内存分块	减少全局内存流量及相应等待。	把线程块内重复使用的数据保存在共享内存，减少跨全局内存层的重复传输。
寄存器分块	减少共享内存流量及相应等待。	把线程或 warp 内复用的数据放入寄存器，减少共享内存到寄存器的重复取数。
私有化 <i>privatization</i>	减少原子更新引起的等待、竞争与串行化。	先在私有副本上做局部更新，完成后更新公共副本。

7.5.1.3 同步延迟与一般工作粒度

Warp 级原语 (*warp-level primitives*) 先在 **warp** 内完成需要协调的操作，再在块内合并各 **warp** 的结果，可减少块级屏障和共享内存流量。

双缓冲 把写入和此前的读取放到不同缓冲区，消除写后于读的假依赖，从而删除用于保护该假依赖的屏障。

线程粗化 把多个并行单元分配给同一线程，用于减少不必要的并行化开销。计算和内存两方面的收益均取决于被减少的开销类型。

7.5.2 优化顺序、资源权衡与瓶颈

7.5.2.1 从数据搬运到线程内工作

优化目标可以按以下顺序组织。这个顺序先检查传输量，再检查传输效率与并发程度，最后结合每线程工作和实际测量。

1. **减少不必要的数据流量。**通过复用和分块，避免反复搬运已经取得的数据。
2. **提高必要流量的传输效率。**通过合并访存，让一次传输覆盖尽可能多的有效数据。
3. **暴露足够多的并行工作。**通过延迟隐藏和 **MLP**，使不同请求能够重叠处理。
4. **增加每线程的有效工作。**结合线程粗化和 **ILP**，减少重复开销并提高独立指令的可见性。

5. 比较优化前后的测量结果。依据应用在目标设备上的表现评价变换。

7.5.2.2 优化会交换不同资源

占用率、数据复用与每线程工作量共享同一组有限的 SM 资源。增加驻留线程可以隐藏更多延迟，也会增加寄存器、共享内存和缓存的竞争；增加共享内存子块可以提高复用，但可能减少同时驻留的线程块；增加粗化因子可以减少并行化开销，但提高每线程资源需求。

这些关系使单个指标的最大值与实际性能之间存在权衡。合理的配置需要同时保留足够独立的工作，以及支撑每线程有效计算和数据复用的资源。

7.5.2.3 以应用在目标设备上的瓶颈为依据

瓶颈 (bottleneck) 是限制应用在某个设备上继续提高性能的约束。它同时取决于应用与设备：相同代码在不同设备上的限制可能不同，同一设备处理不同应用也可能受到不同资源约束。

优化通过增加一种资源的使用，缓解另一种资源的限制。诊断出主要瓶颈之后，才有依据判断应优先减少数据量、改善访存布局、增加并行请求，还是减少同步和线程内停顿。

性能上限与停止条件 将应用性能与对应的**速度极限 (speed-of-light performance)** 比较，有助于判断还有多大的改进空间，以及何时停止继续优化。比较时需使用相同设备和计算条件下的性能上限。

7.5.3 例题一：线程块形状改变访存优势

7.5.3.1 题目与已知信息

下面两个写法使用相同的全局线程坐标。比较它们的全局内存访问效率：

代码 7.15 两种行列下标映射。A、B 表示数组名；注释中的 A/B 表示待比较的两种写法。

```

1 int tx = blockIdx.x * blockDim.x + threadIdx.x;
2 int ty = blockIdx.y * blockDim.y + threadIdx.y;
3
4 // Form A
5 B[ty * Width + tx] = 2 * A[ty * Width + tx];
6
7 // Form B
8 B[tx * Width + ty] = 2 * A[tx * Width + ty];

```

只给出地址表达式时，答案是尚不能判断，因为线程块的启动形状没有给出。两种写法的输入读取和输出写入都使用各自相同的下标，因此分析该下标在一个 warp 内如何变化即可。

7.5.3.2 32×8 与 1×32 的相反结果

记 $W = \text{Width}$ 。忽略所有线程共有的起始偏移，只比较相邻 lane 之间的下标变化。

线程块为 $\text{dim3}(32, 8)$ 一个 warp 内的 ty 固定，tx 随 lane 增加 1。写法 A 的下标增加 1，写法 B 的下标增加 W ，因此原例给出的预期优势是 A：

$$I_A(\ell) = I_A(0) + \ell, \quad (7.19)$$

$$I_B(\ell) = I_B(0) + \ell W. \quad (7.20)$$

线程块为 $\text{dim3}(1, 32)$ 一个 warp 内的 tx 固定，ty 随 lane 增加 1。此时写法 A 的下标增加 W ，写法 B 的下标增加 1，原例给出的预期优势变为 B：

$$I_A(\ell) = I_A(0) + \ell W, \quad (7.21)$$

$$I_B(\ell) = I_B(0) + \ell. \quad (7.22)$$

同一段下标表达式的表现，会随着哪一个线程坐标在 warp 内连续变化而改变。

7.5.3.3 16×16 : 分别展开两组 16 个线程

考察线程块内第一个 warp，去掉全局起点偏移后，前 16 个 lane 的坐标为 $(t_x, t_y) = (0, 0), \dots, (15, 0)$ ，后 16 个 lane 为 $(0, 1), \dots, (15, 1)$ 。因此两种写法的相对元素下标分别为：

$$I_A : \underbrace{0, 1, \dots, 15}_{t_y=0}, \underbrace{W, W+1, \dots, W+15}_{t_y=1}, \quad (7.23)$$

$$I_B : \underbrace{0, W, \dots, 15W}_{t_y=0}, \underbrace{1, W+1, \dots, 15W+1}_{t_y=1}. \quad (7.24)$$

写法 A 形成两段各 16 个元素的行内连续访问；写法 B 在每组 16 个线程内部具有步长 W ，两组之间在对应位置形成相邻的两个元素。具体传输区域的覆盖还要代入矩阵宽度与起点，因此源例要求分别分析两组线程，而不是直接套用“整个 warp 都沿 x 变化”的情形。

表 7.6 启动形状与 warp 地址分布的对应关系。

线程块形状	warp 内坐标变化	原例的判断
dim3(32, 8)	同一行内 x 连续变化	A 形成连续行访问。
dim3(1, 32)	x 固定, y 连续变化	交换索引后的 B 形成连续访问。
dim3(16, 16)	两条 x 行, 各 16 个线程	对两个半 warp 分别展开地址。

7.5.4 例题二：物理带宽与有效输入吞吐率

7.5.4.1 保留题目中的全部假设

每个 float 为 4 字节；输入起点按 512 字节对齐；流式输入数据起初不在缓存中；两次加载所需的每个 512 字节区域总共只读取一次；并发程度足以持续达到 240 GB/s 的物理读取流量；忽略其他流量。求理想的有效输入吞吐率。

代码 7.16 每个线程使用四个连续元素中的前两个。

```
1 int i = blockIdx.x * blockDim.x + threadIdx.x;
2 float temp = A[4 * i] + A[4 * i + 1];
```

512 字节区域与 240 GB/s 均是这个题目的给定条件。有效吞吐率只统计上面两次加载真正使用的输入字节。

7.5.4.2 先在四元素组内计算有效比例

线程 i 涉及的四元素组为

$$A[4i], A[4i + 1], A[4i + 2], A[4i + 3]. \quad (7.25)$$

实际只使用前两个元素，因此每 $4 \times 4 = 16$ B 的地址区间中，使用 $2 \times 4 = 8$ B，另有 8 B 未用于该运算。

表 7.7 一个 16 字节组中的有效输入位置。

元素	$A[4i]$	$A[4i + 1]$	$A[4i + 2]$	$A[4i + 3]$
相对字节范围	0-3	4-7	8-11	12-15
是否用于 temp	是	是	否	否

7.5.4.3 再用一个 512 字节区域核对

取 $i = 0, \dots, 31$ 。32 个线程覆盖 32 组、每组 16 字节，共

$$D_{\text{physical}} = 32 \times 16 = 512 \text{ B}. \quad (7.26)$$

每个线程使用两个 float，对应的有效输入为

$$D_{\text{useful}} = 32 \times 2 \times 4 = 256 \text{ B}. \quad (7.27)$$

于是有效字节比例为

$$\eta = \frac{D_{\text{useful}}}{D_{\text{physical}}} = \frac{256}{512} = \frac{1}{2}. \quad (7.28)$$

这里的两次加载共同使用同一个区域，题目规定这个区域只读取一次，所以物理字节数按 512 B 计。

7.5.4.4 将有效字节比例乘以物理带宽

设 B_{physical} 为物理读取带宽， B_{useful} 为有效输入吞吐率，则在题设条件下

$$B_{\text{useful}} = \eta B_{\text{physical}} = \frac{1}{2} \times 240 \text{ GB/s} = \boxed{120 \text{ GB/s}}. \quad (7.29)$$

这个结果保留了题目的“理想有效输入吞吐率”口径：物理接口仍按每秒 240 GB 读取，但其中一半字节用于指定的两个输入元素。

两道例题的共同计算方法

先根据实际线程布局展开同一次访问的地址，再对照题目给出的传输区域统计有效字节。第一题强调 **launch** 决定 **warp** 坐标关系；第二题强调所用字节数与所传字节数共同决定有效输入吞吐率。

第 8 讲

Convolution Concept; Constant Cache

8.1 课程安排与核心内容

8.1.1 核心内容与学习目标

授课日期为 2026 年 9 月 17 日。卷积(convolution)是信号、图像和视频处理中的常见并行计算模式，也与科学计算中的模板计算(stencil computation)及神经网络有关。学习目标是理解卷积，并利用缓存提高 GPU 数据访问效率。一维与二维邻域加权和确定了输入访问模式，常量缓存与共享内存分别利用系数一致性和输入窗口重叠提高数据复用。

8.1.2 课程安排与阅读材料

以下安排对应上述授课日期。

事项	安排
Lab 2	授课当周的星期五截止。
Midterm 1 时间	2026 年 10 月 6 日，19:00—22:00。
地点与形式	具体考场分配发布在 Canvas；纸笔考试。
考试范围	Lectures 1—11，Labs 1—4。
时间冲突安排	有有效时间冲突者须在 2026 年 9 月 30 日前邮件联系教师，另行安排考试。
复习与阅读	复习资料位于 Canvas 的 Files/MT1；指定阅读为教材第 7 章。

课程安排与阅读材料。

核心关系

每个输出依赖一个输入邻域，而不同输出使用相同的滤波器。前一种重叠带来输入数据的复用机会，后一种一致性适合常量存储器。分块时，输出覆盖范围确定了需要载入的输入核心区与两侧 halo；线程如何分担这些载入工作，决定了三种策略的区别。

8.2 卷积的计算结构与边界处理

8.2.1 邻域加权和与滤波器方向

一个输出元素由其附近输入元素的加权和得到。权重数组称为**卷积掩模**(*convolution mask*)或**滤波器**(*filter*)，通常也称卷积核；为避免与 CUDA 核函数混淆，以下用“滤波器”指权重数组，用“核函数”指设备端并程序。同一个滤波器通常用于整个输入数组。相邻位置使用不同的输入窗口，但使用同一组权重，因此改变滤波器就能改变对局部信号的处理方式。

连续函数与离散序列的数学卷积分别写为

$$(f * g)(x) = \int_{-\infty}^{\infty} f(\tau)g(x - \tau) d\tau, \quad (f * g)[x] = \sum_{k=-\infty}^{\infty} f[k]g[x - k]. \quad (8.1)$$

其中 x 指定输出位置， τ 或 k 遍历参与计算的输入位置。表达式中的 $x - \tau$ 、 $x - k$ 对应其中一个函数或序列的翻转与平移。

互相关(*cross-correlation*) 采用不翻转滤波器的滑动匹配。对实值函数，可写为 $(f \star g)(t) = \int f(\tau)g(t + \tau) d\tau$ 。卷积常用于物理系统与滤波，互相关常用于模式匹配与机器学习。CNN 中通常计算互相关，但历史上仍沿用“卷积”的名称；当权重通过训练获得时，是否预先规定翻转可通过学习到的权重方向吸收。

后续数组与 CUDA 示例采用**滤波器按原顺序读取**的约定。这个约定直接决定代码中使用 $M[j]$ ，也便于对应每一个乘加项。给定一组非对称权重时，翻转与不翻转通常会得到不同结果；下面一维示例的权重是对称的，因此该示例的数值无法单独区分这两种方向。

8.2.2 一维窗口、中心与零填充

输入数组记为 N ，输出数组记为 P ，长度均为 W ；滤波器记为 M ，宽度为 K 。代码使用 `Width`、`Mask_Width` 表示这两个宽度。取奇数 $K = 2r + 1$ ，则半径 $r = (K - 1)/2$ 表示输出中心左右各涉及多少个输入元素；代码中的整数除法 `Mask_Width / 2` 在这个条件下等于 r 。

为形式化说明边界判断，定义零延拓输入 $\tilde{N}[q]$ ：有效下标 $0 \leq q < W$ 时等于 $N[q]$ ，其

他位置等于零。按滤波器原顺序读取，一个输出为

$$P[i] = \sum_{j=0}^{K-1} \tilde{N}[i-r+j]M[j], \quad 0 \leq i < W. \quad (8.2)$$

这个表达式把三个下标作用分开： i 决定输出位置， j 决定滤波器内的位置， $i-r+j$ 决定当前乘加读取哪个输入。 $j=0$ 对应窗口最左端， $j=r$ 对应输入中心 $N[i]$ ， $j=K-1$ 对应最右端。

以 $N = [1, 2, 3, 4, 5, 6, 7]$ 、 $M = [3, 4, 5, 4, 3]$ 为例， $K = 5$ 、 $r = 2$ 。计算 $P[2]$ 时读取 $N[0]$ 到 $N[4]$ ，而计算 $P[3]$ 时整个窗口向右移动一格：

$$\begin{aligned} P[2] &= 1 \cdot 3 + 2 \cdot 4 + 3 \cdot 5 + 4 \cdot 4 + 5 \cdot 3 = 57, \\ P[3] &= 2 \cdot 3 + 3 \cdot 4 + 4 \cdot 5 + 5 \cdot 4 + 6 \cdot 3 = 76. \end{aligned} \quad (8.3)$$

每一个输出都需要五次乘加，但这两个窗口共同使用了 $N[1]$ 到 $N[4]$ 。这种重叠正是后续共享内存分块的依据。

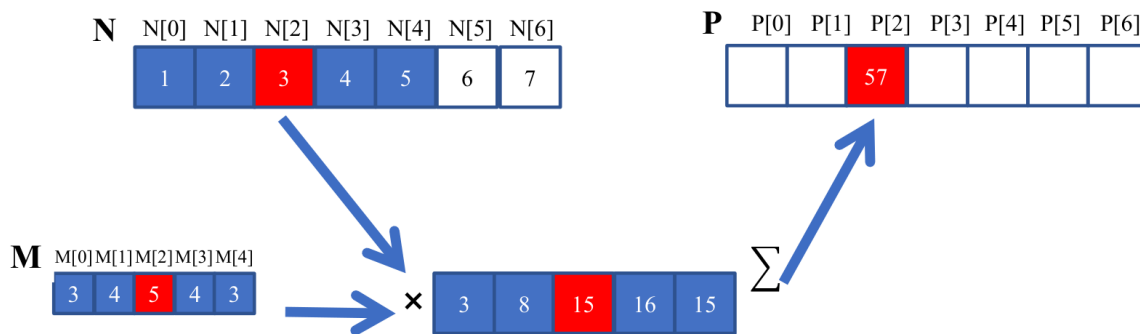


图 8.1 一维窗口与滤波器逐项相乘，再归约到输出 $P[2] = 57$ 。

当窗口跨出数组边界时，会出现没有对应实际数组元素的位置，称为**幽灵元素 (ghost elements)**。边界规则可以采用零填充，也可以复制边缘值；本讲代码采用零填充。输出 $P[1]$ 的输入窗口为 $[0, 1, 2, 3, 4]$ ，因此 $P[1] = 0 \cdot 3 + 1 \cdot 4 + 2 \cdot 5 + 3 \cdot 4 + 4 \cdot 3 = 38$ 。

零填充可以通过条件判断实现：遇到无效输入下标就跳过乘加。由于该项应为零，跳过与显式读取一个零具有相同的数学作用，因此无须为基本核函数实际分配数组外的元素。按式(8.2)复算全部位置，可得到 $P = [22, 38, 57, 76, 95, 90, 74]$ ，可作为边界实现的参考结果。

8.2.3 二维邻域与图像平滑

二维计算将一维窗口扩展为一个矩形邻域。设输入有 H 行、 W 列，滤波器为 $K \times K$ ，两个方向的半径均为 r 。在与一维相同的读取约定下，零延拓输入为 $\tilde{N}$ ，输出行列为 (y, x) ，则

$$P[y, x] = \sum_{u=0}^{K-1} \sum_{v=0}^{K-1} \tilde{N}[y-r+u, x-r+v] M[u, v]. \quad (8.4)$$

两个求和分别遍历滤波器的行与列；每个输出将整个局部窗口压缩为一个数，输入窗口与滤波器先逐项相乘，再将全部乘积相加。

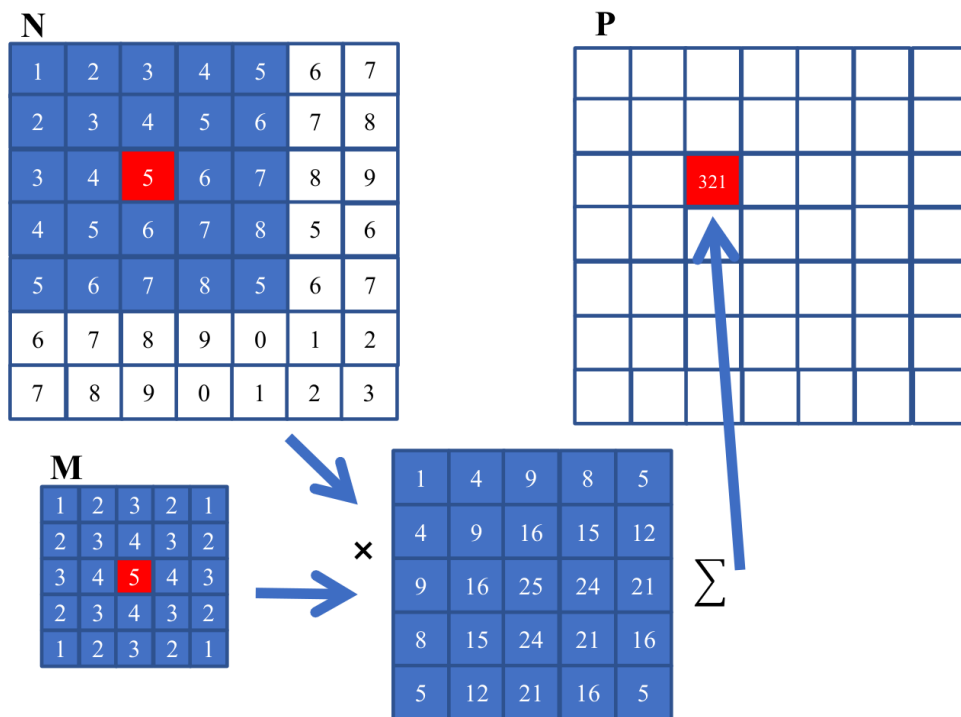


图 8.2 5×5 输入邻域与权重逐项相乘，得到二维内部输出 $P[2, 2] = 321$ 。

在图中的零起始坐标下，中心为 $(2, 2)$ ，使用输入的前五行、前五列。乘积矩阵各行之和依次为 27, 56, 95, 84, 59，合计 321。在边界例中，输出 $(1, 0)$ 的窗口跨过上边界和左边界，填零后五行乘积之和为 0, 16, 34, 34, 28，合计 112。这些行和来自图中数值，可用于把二维索引与实际求和对应起来。

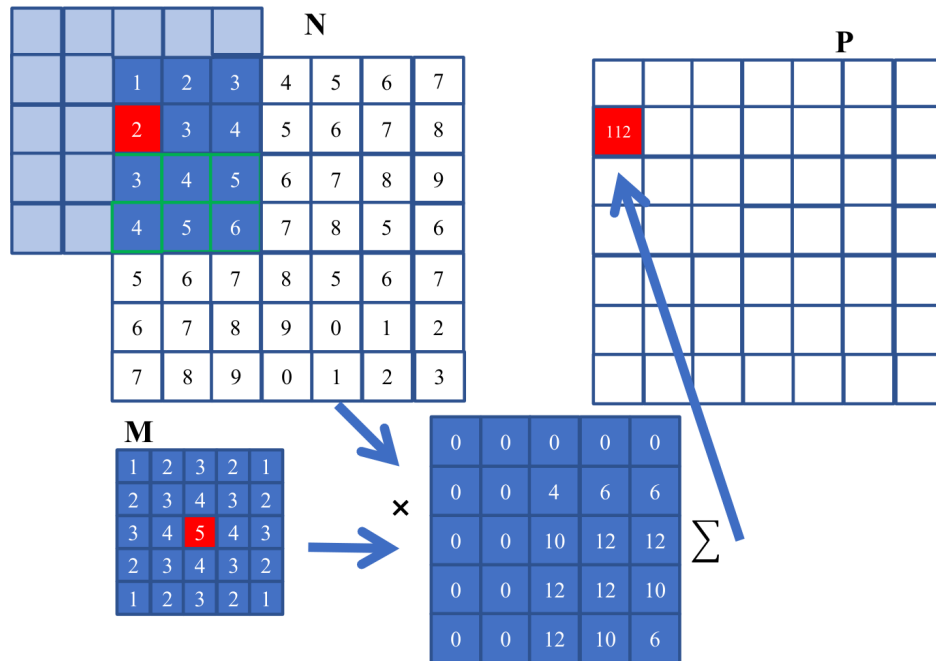


图 8.3 二维零填充：窗口上侧与左侧的无效位置取零，得到边界输出 112。

幽灵单元也称 **apron cells** 或 **halo cells**。例如，另一个零填充边界窗口的五行分别为 $[0, 0, 0, 0, 0]$ 、 $[0, 3, 4, 5, 6]$ 、 $[0, 2, 3, 4, 5]$ 、 $[0, 3, 5, 6, 7]$ 、 $[0, 1, 1, 3, 1]$ 。使用图中相同的整数权重，五行乘积之和依次为 0, 52, 54, 61, 12，输出为 179。这些名称都与邻域的外围数据有关；进入分块实现后，需要进一步区分“落在全局数组外的补零位置”与“仍在全局数组内、但落在当前块核心区外的邻域元素”。后一类是真实输入，后续分块必须读取其值。

一个具体的图像滤波器为

$$M = \frac{1}{273} \begin{bmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 16 & 26 & 16 & 4 \\ 7 & 26 & 41 & 26 & 7 \\ 4 & 16 & 26 & 16 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{bmatrix}. \quad (8.5)$$

平滑作用与边界条件。所有系数非负，矩阵元素之和为 $17 + 66 + 107 + 66 + 17 = 273$ ，归一化后权重和为 1。每个输出是附近像素的加权平均，中心像素的权重较大、远处像素的权重较小，因此它起到平滑、模糊局部变化的作用。若整个窗口内的像素都等于常数 c ，则输出仍为 c 。采用零填充时，图像边缘窗口含有零，因而这一“保持常数”结论限于完整有效窗口；边界值可能被拉低。

8.2.4 直接 CUDA 实现：一个线程计算一个输出

输出之间只共享输入读取，没有相互依赖的输出写入，所以可让每个线程独立计算一个 $P[i]$ 。一维网格中的全局下标为 $i = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x}$ 。最后一个线程块可能超过数组长度，因此先检查输出下标，再在循环中检查输入下标。

代码 8.1 直接一维卷积：每个线程计算一个输出，越界输入按零处理。

```

1  __global__ void convolution_1D_basic_kernel(
2      const float* N, const float* M, float* P,
3      int Mask_Width, int Width)
4  {
5      int i = int(blockIdx.x) * int(blockDim.x)
6          + int(threadIdx.x);
7      if (i >= Width) return;
8      float Pvalue = 0.0f;
9      int N_start_point = i - Mask_Width / 2;
10     for (int j = 0; j < Mask_Width; ++j) {
11         int q = N_start_point + j;
12         if (q >= 0 && q < Width) {
13             Pvalue += N[q] * M[j];
14         }
15     }
16     P[i] = Pvalue;
17 }

```

两个范围检查各自保护不同的数组：入口处的 $i \geq \text{Width}$ 保护输出 P ，循环内的检查保护输入 N 。即使 i 合法， $i - r + j$ 仍可能越界，因此两者都需要。局部标量 $Pvalue$ 是每个线程自己的累加器；每个有效线程恰好写一个输出。

实现说明。代码将只读输入写为 `const float*`，并显式把 CUDA 索引转为有符号整数，便于表达减去半径后可能出现的负值。直接版没有共享内存协作，因此无效输出线程可在入口直接返回。分块版的线程还承担输入搬运任务，返回位置需要与这些任务一起分析。

假设每块 B 个线程且 $W > 0$ ，网格块数取 $\lceil W/B \rceil$ 。在整数代码中可以写作 $1 + (\text{Width} - 1) / B$ 。随后，每个线程沿着滤波器执行顺序乘加，线程之间沿输出坐标并行；这同时解释了计算并行性和输入访问的重叠。

8.3 缓存层次与常量存储器

8.3.1 从主存访问到缓存命中

GPU 的计算需要持续获得输入。如果每次读取都访问全局存储器中的底层数据，带宽就可能限制执行速度。分块矩阵乘法通过共享内存显式复用输入；**缓存 (cache)** 则在硬件管理下保留近期访问数据，为后续读取提供另一条复用路径。

缓存由若干**缓存行 (cache line)** 组成，每行可存放一段连续地址的数据。当请求的数据尚未在缓存中时，硬件把包含该地址的一行数据带入缓存；缓存中还保存**标签 (tag)**，记录该行对应的内存地址。后续请求找到对应数据时称为缓存命中，否则需要继续访问后面的存储层次。缓存数据是原数据的副本。

连续地址成组传输，使一次读取也可能为后续的邻近读取准备好数据。例如，一次突发传输约 1024 bit，即 128 B 时，相邻数据可以随请求位置一起进入缓存。这个示例以一次突发对应一行说明成组搬运；具体缓存行与事务大小取决于存储层次和硬件实现。

空间局部性与时间局部性。空间局部性 (*spatial locality*) 指相近的时间里访问连续或邻近地址；时间局部性 (*temporal locality*) 指一个数据在短时间内被多次访问。对于固定线程，一维卷积沿着窗口依次读取 $N[i-r], N[i-r+1], \dots$ ，体现空间局部性；相邻输出窗口重叠，使一个输入可能在较短时间内再次被使用，体现时间局部性。两种局部性都能提高缓存发挥作用的机会。

缓存容量小于全局存储器，不能保留全部输入。当需要为新缓存行腾出空间时，已有行会被替换；一种典型的替换思路是**最近最少使用 (least recently used, LRU)**，即优先替换最久没有被访问的缓存行。能否命中还取决于相关数据在再次访问前是否仍留在缓存中，因此“存在复用机会”与“每次都能缓存命中”需要分开判断。

8.3.2 共享内存、L1/L2 与常量缓存

CUDA 的存储空间在可见范围、读写方式及用途上不同。下表给出用于直观比较的近似访问周期。实际延迟取决于器件、缓存命中及访问模式，这些示例量级不构成统一的运行时间预测模型。

存储空间	可见范围	线程权限	示例访问周期
寄存器	单线程	读、写	~ 1 cycle
共享内存	同一线程块	读、写	~ 5 cycles
全局存储器	网格内线程	读、写	~ 500 cycles
常量存储器	网格内线程	只读	缓存条件下 ~ 5 cycles

表 8.1 CUDA 存储空间的程序员视图与教学访问量级。

共享内存与缓存都利用靠近 SM 的片上存储减少主存压力，但**内容由谁管理**决定了它们的编程方式。共享内存是程序显式管理的暂存区 (scratchpad)：程序决定把什么值写入哪个位置，并在需要时同步线程。缓存的内容则由微体系结构根据内存访问自动选择和替换，程序通常不显式指定某个数据应放入哪一条缓存行。共享内存还可以保存中间结果，不要求内容始终对应全局存储器中的一个副本。

在 Volta 及之后的相关设计中，共享内存和 L1 可以使用统一的物理资源，并在这些用途之间分配。物理资源可以共享，访问与管理接口仍保持各自特点。图 8.4 中的局部片上存储靠近各个 SM，L2 位于多个 SM 与全局存储器之间。

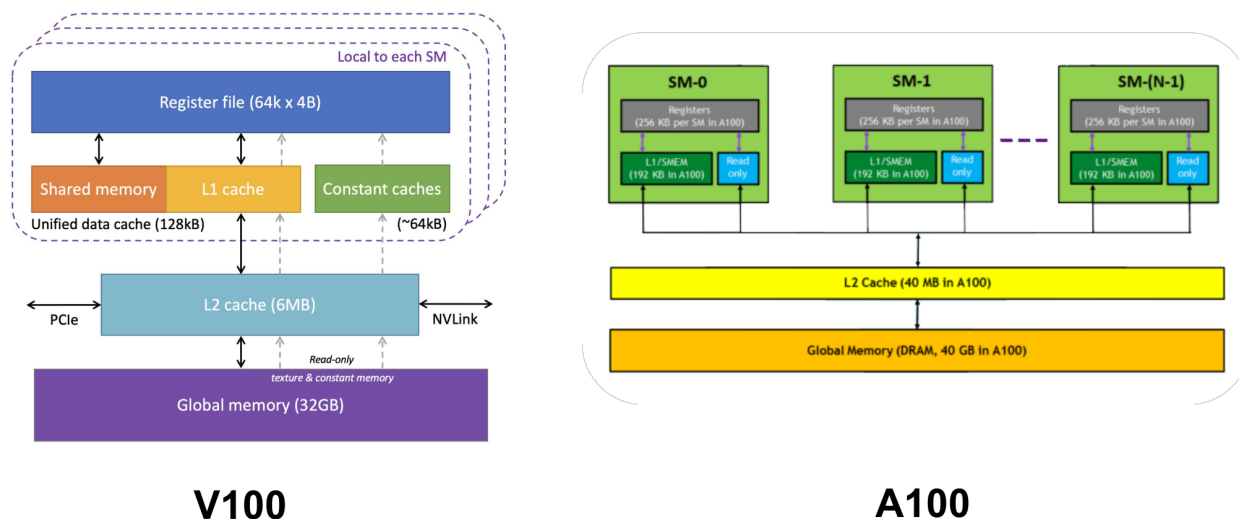


图 8.4 V100 与 A100 的缓存层次示例：SM 附近的局部存储、共享 L2 及其后的全局存储器。图中容量保留对应示例器件的标注。

对于可修改的缓存数据，硬件需要处理修改状态及与后备内存的一致性。**常量缓存 (constant cache)** 服务于核函数执行期间保持不变的数据，读取路径可以针对这类模式优化。常量和纹理的只读访问路径，与一般全局访问使用的 L1 路径具有不同用途。滤波器系数的实现使用常量存储器。

8.3.3 滤波器的访问一致性

滤波器 M 有三个相关性质：计算所有输出都需要它，它在一次网格执行期间保持不变，而且各线程按照相同的顺序读取系数。在循环第 j 次迭代，输出位置不同的线程读取各自的 $N[i - r + j]$ ，同时读取同一个 $M[j]$ 。小而重复使用、且访问顺序一致的滤波器因此适合放入常量存储器 (constant memory)。

同一轮乘加的两种地址关系

对相邻输出线程 $i, i + 1, \dots$ ，固定 j 时，输入地址为 $i - r + j, i + 1 - r + j, \dots$ ，沿数组相邻分布；滤波器地址均为 j 。输入复用与系数复用具有不同结构，分别对应后续共享内存分块与常量存储器优化。

同地址访问与广播。同一 warp 的线程访问同一常量地址时可以共享该次读取；访问多个不同地址时，请求按不同地址串行处理，代价随不同地址数增加。因此 $Mc[j]$ 的一致访问有利，而按线程编号访问各不相同的常量位置就不具备同样的优势。

输入 N 在本次卷积中也保持只读，但往往比滤波器大得多。常量存储器总容量上限为 64 KB，因此是否只读只是选择条件之一；还要考虑容量与访问结构。64 KB 指常量地址空间的容量约束，不能从这个数字直接推出每个 SM 的常量缓存大小。常量缓存未命中时仍需取得后备数据。

8.3.4 声明、初始化与常量版核函数

常量数组通过 `__constant__` 在所有主机函数与设备核函数之外声明。主机用 `cudaMemcpyToSymbol` 将权重复制到这个符号；主机端数据可以运行时生成，设备端执行网格期间按只读方式使用这些系数。

代码 8.2 常量滤波器声明及主机初始化；五个系数沿用一维数值例。

```

1  constexpr int MASK_WIDTH = 5;
2  __constant__ float Mc[MASK_WIDTH];
3  // Inside a host function, before launching the kernel:
4  float Mask[MASK_WIDTH] = {3, 4, 5, 4, 3};
5  cudaError_t status = cudaMemcpyToSymbol(
6      Mc, Mask, MASK_WIDTH * sizeof(float));
7  if (status != cudaSuccess) {
8      // Report or propagate the error; do not launch on failure.
9      return;
10 }
```

复制长度的单位是字节，所以必须乘以 `sizeof(float)`。这次调用采用默认偏移 0 与

默认方向 `cudaMemcpyHostToDevice`。权重也可以由随机数生成器产生；核函数启动之前，都需要先完成权重初始化和复制。

常量版只把原先的 `M[j]` 改为 `Mc[j]`，窗口起点、边界条件与求和次序均保持不变：

代码 8.3 使用常量滤波器的一维卷积。

```

1  __global__ void convolution_1D_constant_kernel(
2      const float* N, float* P, int Mask_Width, int Width)
3  {
4      int i = int(blockIdx.x) * int(blockDim.x)
5          + int(threadIdx.x);
6      if (i >= Width) return;
7      float Pvalue = 0.0f;
8      int N_start_point = i - Mask_Width / 2;
9      for (int j = 0; j < Mask_Width; ++j) {
10         int q = N_start_point + j;
11         if (q >= 0 && q < Width) {
12             Pvalue += N[q] * Mc[j];
13         }
14     }
15     P[i] = Pvalue;
16 }

```

参数与初始化条件。常量版直接读取设备符号 `Mc[j]`，因此核函数参数只包含输入、输出及两种宽度，无须传入滤波器指针。调用应传入已经初始化的有效宽度，满足 $0 < K \leq \text{MASK_WIDTH}$ ；后续完整示例固定采用奇数 $K = 5$ 。

假设 `Nd`、`Pd` 已完成设备端分配、`Nd` 已存有输入， $W > 0$ ，取每块 B 个线程，则调用形式为

代码 8.4 与常量版核函数匹配的启动参数。

```

1  dim3 dimBlock(B);
2  dim3 dimGrid(1 + (Width - 1) / B);
3  convolution_1D_constant_kernel<<<dimGrid, dimBlock>>>>(
4      Nd, Pd, MASK_WIDTH, Width);

```

常量版改善了系数的读取路径，但相邻线程仍会反复请求重叠的输入窗口。共享内存分块进一步利用 N 的复用。

8.4 分块的数据复用与三种策略

8.4.1 访问计数、算术强度与复用机会

对于不接触边界的一个一维输出，直接实现读取 K 个输入与 K 个系数，共 $2K$ 次标量读取，并执行 K 次乘法和 K 次加法，共 $2K$ 次浮点运算。二维 $K \times K$ 滤波器对应 $2K^2$ 次标量读取与 $2K^2$ 次浮点运算。因此增加滤波器宽度时，直接算法的读取量和计算量一起增长，其复用前的算术强度较低。

算术强度 (arithmetic intensity) 用计算量与数据搬运量之比描述计算对带宽的需求。按每个 `float` 为 4 B，将标量读取次数换成字节：一维内部输出读入 $8K$ B、写出 4 B。忽略复用时，仅以读入量为分母得到 $2K/(8K) = 0.25$ FLOP/B；把输出写入也计入，得到

$$I_{\text{direct,1D}} = \frac{2K}{8K + 4}, \quad I_{\text{direct,2D}} = \frac{2K^2}{8K^2 + 4}. \quad (8.6)$$

较低的算术强度说明直接算法容易受到带宽限制；常量缓存与共享内存的作用，就是减少同一份数据在较慢存储层次上的重复获取。

计数口径。这里的“读取次数”首先指代码层面的标量访问。缓存命中、常量同地址访问以及成组内存事务会改变实际 DRAM 流量，所以不能直接把 $2K$ 次读取等同于 $2K$ 次独立 DRAM 事务。边界线程又会在直接核函数中跳过无效项，此时实际执行的读取与乘加比内部输出少。后面的精确例题始终先说明统计的是输入请求、字节还是乘加次数。

8.4.2 输出分块、输入覆盖范围与 halo

设一个线程块生成 B 个连续输出，块号为 b ，输出起点为 $s = bB$ 。输出范围是 $[s, s+B-1]$ 。最左输出需要从 $s-r$ 开始读取，最右输出需要读到 $s+B-1+r$ ，因此整个块需要的输入范围为

$$[s-r, s+B+r-1], \quad T_{\text{in}} = B + 2r = B + K - 1. \quad (8.7)$$

这里 B 是输出块宽度， T_{in} 是包含边缘邻域的输入块宽度。输入核心区与输出对齐，左右各多出的 r 个位置称为**晕区 (halo)**。

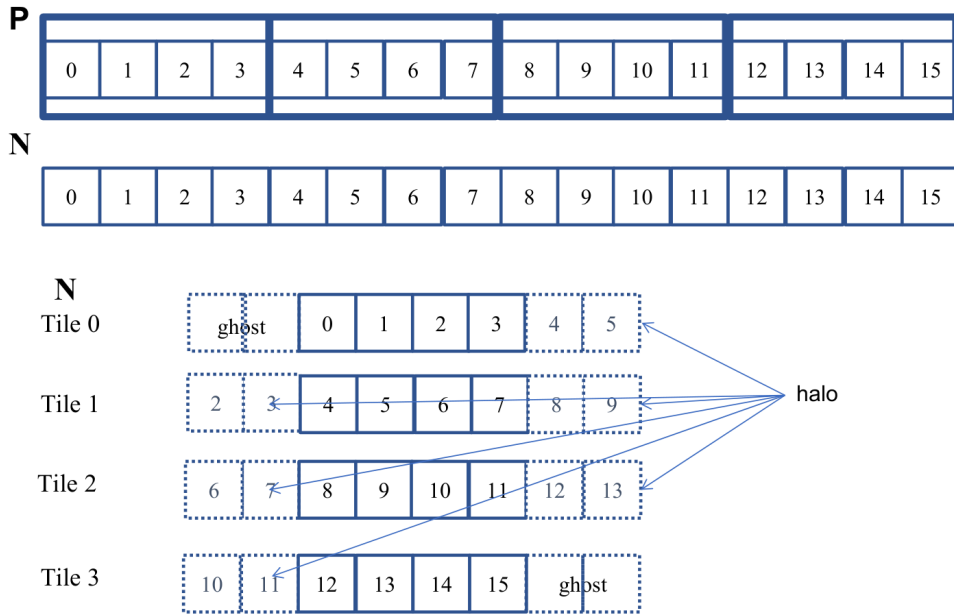


图 8.5 $W = 16$ 、 $B = 4$ 、 $K = 5$ 时的四个输出块及其输入邻域。中间块的 halo 是真实输入，首末块还包含越界的幽灵位置。

以 $b = 1$ 为例，这个块生成 $P[4]$ 到 $P[7]$ ，输入需要 $N[2]$ 到 $N[9]$ 。核心区是 $N[4 : 7]$ ，左 halo 是 $N[2 : 3]$ ，右 halo 是 $N[8 : 9]$ 。相邻块的输出不重叠、输入可以重叠，所以线程块可以独立写各自的输出，但边缘输入可能被多个块读取。第一个块需要的 $N[-2]$ 与 $N[-1]$ 才属于全局数组外的幽灵位置，按边界规则填零。

块内复用的数量。对于 $B = 4, K = 5$ ，四个输出分别需要五个输入，共产生 20 次输入使用，但覆盖的不同输入只有八个。 $N[2]$ 到 $N[9]$ 的块内使用次数依次为 1, 2, 3, 4, 4, 3, 2, 1，总和为 20。一旦把这八个值放入共享内存，块内后续使用就可以从共享内存读取。

对于远离全局边界的完整块，直接方法的输入请求数为 BK ；把完整输入块载入一次则为 $B + 2r$ 。因此只比较 N 的块内读取请求，可写出复用因子

$$R_N = \frac{BK}{B + 2r} = \frac{BK}{B + K - 1}. \quad (8.8)$$

这给出示例中的 $20/8 = 2.5$ 。固定 K 而增大 B 时，halo 相对占比 $2r/B$ 下降， R_N 趋近于 K 。这些是输入覆盖关系推得的请求量比较，没有计入系数缓存、同步或硬件资源对实际速度的影响。

8.4.3 线程数量与 halo 放置的组合

分块之后仍有两个决定：线程数与输出块匹配，还是与输入块匹配；halo 是否也放入共享内存。由此形成三种主要策略。

比较项	策略 1	策略 2	策略 3
线程块覆盖	B 个输出	$B + 2r$ 个输入	B 个输出
每块线程数	B	$B + 2r$	B
共享内存内容	核心区与两侧 halo	核心区与两侧 halo	仅核心区
载入方式	部分线程额外载入	每线程载入一个输入	每线程载入一个核心输入
计算方式	每线程计算一个输出	仅部分线程计算输出	每线程计算一个输出
计算时 halo	从共享内存读取	从共享内存读取	从全局地址读取
主要取舍	载入阶段任务不均	计算阶段部分线程闲置	计算循环选择两种读取路径

表 8.2 以相同的 B 个输出为比较对象，线程数与输入存放方式的差异。

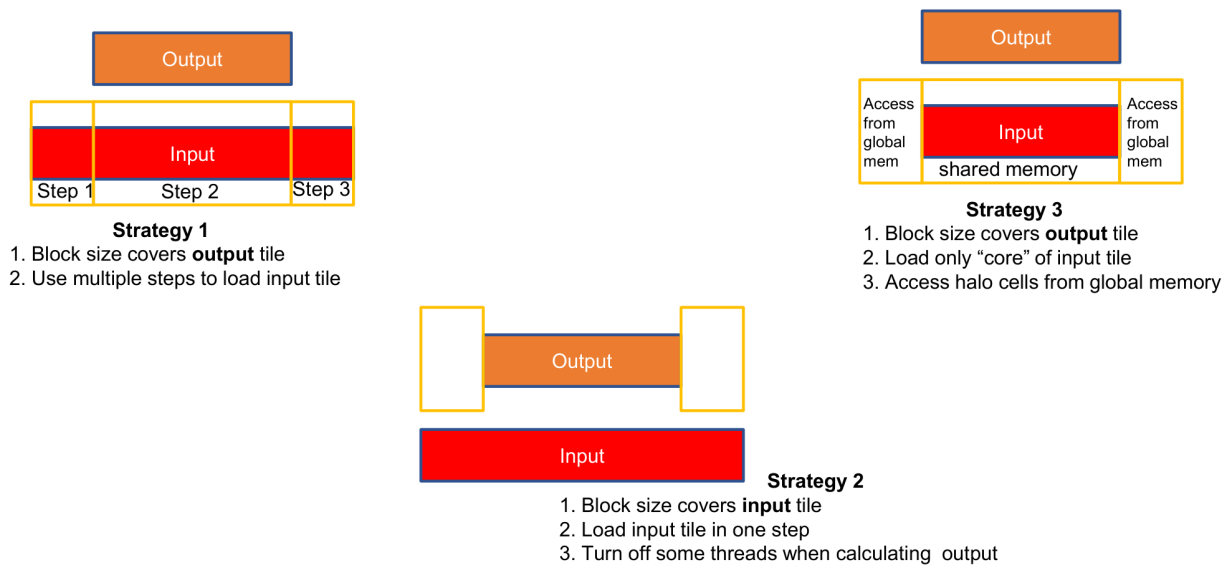


图 8.6 三种分块策略的输入、输出和线程工作范围。策略 1 分步载入完整输入块；策略 2 为输入配置线程；策略 3 将 halo 留给全局读取路径。

策略 1 的取舍。将 halo 一并载入共享内存后，卷积计算循环的输入都来自同一种位置，不需要在每次乘加时选择共享或全局路径。连续位置可以按组载入，也有利于合并访问。代价是共享内存由 B 个元素增加到 $B + 2r$ 个元素，而且部分线程在载入阶段必须多做工作。载入任务按线程编号分配时，可能出现部分线程执行分支、其他线程不执行的情况。

策略 3 的取舍。只缓存核心区，把共享内存优先用于块内复用较高的输入。计算每一个乘加项时判断输入是否位于本块核心区：位于核心区就读共享内存，位于 halo 就按全局地

址读取。同一 warp 内的线程可能选择不同路径；窄 halo 的访问也可能只利用一次成组内存传输的一小部分。

按 $r \leq B$ 的完整内部块展开计数，左侧各输出对左 halo 的读取数为 $r, r-1, \dots, 1$ ，右侧对称，因此两侧一共 $r(r+1)$ 次全局 halo 请求。再加 B 次核心区载入，策略 3 对 N 的请求数为 $B + r(r+1)$ 。在 $B = 4, r = 2$ 的例子中，这个数为 10，策略 1 则是 8。该比较按程序请求计数；策略 3 的 halo 请求可能由缓存满足，不能只靠这两个数判定实测速度。

策略 2 的取舍。让每个线程载入一个输入，载入阶段无需由少量线程额外承担 halo；随后仅有与输出对应的线程执行乘加。按相同输出宽度 B 比较，计算输出的线程比例为 $B/(B+2r)$ 。这种分配减少高延迟载入阶段的工作分歧，同时让部分线程不参与较低延迟的计算阶段。全局数组首末处的边界填零仍然需要处理。

8.5 分块核函数与索引推导

8.5.1 策略 1：完整输入块与三段载入

策略 1 为一个输出块配置 B 个线程。局部线程号记为 t ，块起点为 $s = bB$ ，线程的输出下标为 $i = s + t$ 。共享数组 N_ds 需要容纳 $B + 2r$ 个元素。在这个共享数组中，最左位置从整个输入窗口的左端开始，因而有统一映射

$$N_{ds}[q] = \tilde{N}[s - r + q], \quad 0 \leq q < B + 2r. \quad (8.9)$$

右侧的 $\tilde{N}$ 已包含零填充约定。这个关系同时描述了左 halo、核心区与右 halo，是核对所有搬运代码的共同依据。

线程 t 计算的输出为 $P[s + t]$ ，其第 j 项需要 $\tilde{N}[s + t - r + j]$ 。代入式(8.9)可知，该项的共享内存下标恰好是 $q = t + j$ 。因此乘加使用 $N_ds[\text{threadIdx.x} + j]$ ，不再额外减去半径：半径偏移已经包含在共享数组起点的选择中。

三段载入的分工。左 halo 由局部编号最大的 r 个线程搬运，即 $t \geq B - r$ 的线程。其全局下标为 $(b-1)B + t = s - B + t$ ，共享下标为 $t - (B - r)$ 。当 $t = B - r$ 时，这两个下标分别为 $s - r$ 与 0；当 $t = B - 1$ 时，分别为 $s - 1$ 与 $r - 1$ ，正好覆盖左 halo。

所有线程将核心值 $N[s + t]$ 放入共享位置 $r + t$ 。右 halo 则由局部编号最小的 r 个线程搬运：全局下标为 $(b+1)B + t = s + B + t$ ，共享下标为 $r + B + t$ 。三个目标区间 $[0, r-1]$ 、 $[r, r+B-1]$ 、 $[r+B, r+B+r-1]$ 连续且不重叠，共同填满 $B + 2r$ 个位置。

载入区域	负责的局部线程 t	全局输入下标	共享数组下标
左 halo	$B - r, \dots, B - 1$	$s - B + t$	$t - (B - r)$
核心区	$0, \dots, B - 1$	$s + t$	$r + t$
右 halo	$0, \dots, r - 1$	$s + B + t$	$r + B + t$

表 8.3 策略 1 的三段搬运映射；要求 $r \leq B$ ，并对无效全局下标填零。

在 $b = 1, B = 4, r = 2$ 的例子中，线程 2,3 把 $N[2], N[3]$ 放入共享位置 0,1；所有线程把 $N[4], \dots, N[7]$ 放入位置 2, ..., 5；线程 0,1 把 $N[8], N[9]$ 放入位置 6,7。相同线程可以同时承担核心区和 halo 的载入，且这些工作写入不同的共享地址。

完整的三段载入核函数如下。滤波器存放在常量数组 **Mc** 中，宽度固定为 $K = 5$ ，由 **MASK_WIDTH** 指定。块宽 B 为模板参数，启动时每块必须恰好配置 B 个线程。共享数组在核函数内声明；有符号索引与双侧范围检查保证左右边界按零填充处理。

代码 8.5 策略 1：核心区与两侧 halo 分段载入。

```

1  template<int B>
2  __global__ void convolution_1D_tiled_three_loads(
3      const float* N, float* P, int Width)
4  {
5      constexpr int radius = MASK_WIDTH / 2;
6      static_assert(MASK_WIDTH % 2 == 1, "Odd mask required");
7      static_assert(B >= radius, "B must cover each halo");
8      __shared__ float N_ds[B + MASK_WIDTH - 1];
9      int t = int(threadIdx.x);
10     int s = int(blockIdx.x) * B;
11     int i = s + t;
12     if (t >= B - radius) {
13         int q = s - B + t;
14         N_ds[t - (B - radius)] =
15             (0 <= q && q < Width) ? N[q] : 0.0f;
16     }
17     N_ds[radius + t] = (i < Width) ? N[i] : 0.0f;
18     if (t < radius) {
19         int q = s + B + t;
20         N_ds[radius + B + t] =
21             (0 <= q && q < Width) ? N[q] : 0.0f;
22     }
23     __syncthreads();
24     float Pvalue = 0.0f;
25     for (int j = 0; j < MASK_WIDTH; ++j) {
26         Pvalue += N_ds[t + j] * Mc[j];
27     }
28     if (i < Width) P[i] = Pvalue;
29 }

```

为何载入后需要同步。一个线程计算时会读取其他线程写入的共享位置。屏障 `__syncthreads()` 将共同载入与后续读取分开，使参与线程在开始乘加前可见已完成的共享内存写入。三段载入本身写入互不重叠的位置，中间不需要互相消费刚写出的数据，因此一次位于载入之后的屏障即可。

输出越界不等于搬运任务消失。最后一个块中，某个线程的 i 即使超过 $W - 1$ ，它负责的共享位置仍可能被其他有效输出使用，包括对幽灵位置写零。因此完整代码先让所有线程完成分工、经过屏障，再用 $i < W$ 保护写出。直接版入口处的提前返回不能原样搬到这段代码的载入前；具体缺失哪些位置，可在第8.6节的末尾块例子中核对。

代码采用“全部线程计算、只保护输出写入”的结构，所以末尾无有效输出的线程也会执行乘加。若将计算限制为有效输出线程，需要把这一限制放在共同载入与屏障之后，且不得跳过仍被其他线程需要的搬运任务。

8.5.2 策略 1 的连续载入写法

三段写法按“左、核心、右”分配输入。另一种写法直接把完整共享数组看成一段连续的长度 $B + 2r$ 的区间：第一轮由所有线程填入前 B 个位置，第二轮由部分线程填入剩余 $2r = K - 1$ 个位置。两种写法最终都满足式(8.9)，所以计算循环完全相同。

第一轮让线程 t 读取 `start = i - radius`，写入 `N_ds[t]`。第二轮令 `start += B`，由 $t < 2r$ 的线程读取该地址，写入 `N_ds[B+t]`。这里第二轮补齐的是“完整输入块的末尾”，其中可能同时包含核心区的一部分和右 halo。

例如 $B = 4, r = 2, s = 4$ 时，第一轮载入 $N[2 : 5]$ ，即左 halo 加核心区的前半部分；第二轮载入 $N[6 : 9]$ ，即核心区后半部分加右 halo。与三段写法相比，区别在于每个线程负责哪些搬运，而非最终共享数组的内容。

代码 8.6 策略 1: 两轮连续载入; 需要 $K - 1 \leq B$ 。

```

1  template<int B>
2  __global__ void convolution_1D_tiled_two_loads(
3      const float* N, float* P, int Width)
4  {
5      constexpr int radius = MASK_WIDTH / 2;
6      static_assert(MASK_WIDTH % 2 == 1, "Odd mask required");
7      static_assert(B >= MASK_WIDTH - 1, "Two loads need B >= K-1");
8      __shared__ float N_ds[B + MASK_WIDTH - 1];
9      int t = int(threadIdx.x);
10     int i = int(blockIdx.x) * B + t;
11     int start = i - radius;
12     N_ds[t] = (0 <= start && start < Width)
13         ? N[start] : 0.0f;
14     if (t < MASK_WIDTH - 1) {
15         start += B;
16         N_ds[t + B] = (0 <= start && start < Width)
17             ? N[start] : 0.0f;
18     }
19     __syncthreads();
20     float Pvalue = 0.0f;
21     for (int j = 0; j < MASK_WIDTH; ++j) {
22         Pvalue += N_ds[t + j] * Mc[j];
23     }
24     if (i < Width) P[i] = Pvalue;
25 }

```

两轮载入的覆盖条件。第二轮最多有 B 个线程各填一个位置，而剩余位置数为 $2r$ ，因此要求 $2r \leq B$ 。条件 $\text{MASK_WIDTH} - 1 > \text{threadIdx.x}$ 在这个条件下可以覆盖所有剩余位置；当 $K - 1 > B$ 时，仅有两轮就无法填满共享数组。这个限制由载入分工推得，不影响卷积计算公式本身。

由同一映射导出的循环形式。需要支持更多载入轮次时，让线程 t 依次负责 $q = t, t + B, t + 2B, \dots$ ，直到 $q \geq B + K - 1$ 。每个合法共享下标都有唯一的模 B 余数，因此恰好由一个线程负责：

代码 8.7 连续载入的索引推广；替换载入阶段，之后仍需相同屏障与计算循环。

```

1  int s = int(blockIdx.x) * B;
2  for (int q = t; q < B + MASK_WIDTH - 1; q += B) {
3      int g = s - radius + q;
4      N_ds[q] = (0 <= g && g < Width) ? N[g] : 0.0f;
5  }
6  __syncthreads();

```

该循环将两轮载入推广到任意所需轮数。当输入块比线程块更宽时，各线程以步长 B 分摊载入任务，仍保持共享位置恰好初始化一次。

8.5.3 策略 3：核心区缓存与按需 halo 读取

策略 3 的共享数组只容纳 B 个核心输入。线程 t 将 $N[s+t]$ 放入 $N_ds[t]$ ，所以共享映射变为 $N_{ds}[q] = \tilde{N}[s+q]$ ， $0 \leq q < B$ 。载入完成后经过屏障，再进入卷积循环。

对输出 $i = s+t$ 的第 j 项，输入全局下标为 $g = i - r + j$ 。先判断它是否属于全局有效区间 $[0, W)$ ；有效时再判断是否属于本块核心区 $[s, s+B)$ 。位于核心区的输入，其共享下标为 $g - s = t - r + j$ ；位于 halo 的输入直接使用全局下标 g 读取。

代码 8.8 策略 3：缓存核心区，并检查全局输入范围。

```

1  template<int B>
2  __global__ void convolution_1D_core_shared(
3      const float* N, float* P, int Width)
4  {
5      constexpr int radius = MASK_WIDTH / 2;
6      static_assert(MASK_WIDTH % 2 == 1, "Odd mask required");
7      __shared__ float N_ds[B];
8      int t = int(threadIdx.x);
9      int s = int(blockIdx.x) * B;
10     int i = s + t;
11     N_ds[t] = (i < Width) ? N[i] : 0.0f;
12     __syncthreads();
13     float Pvalue = 0.0f;
14     for (int j = 0; j < MASK_WIDTH; ++j) {
15         int N_index = i - radius + j;
16         if (0 <= N_index && N_index < Width) {
17             if (s <= N_index && N_index < s + B) {
18                 Pvalue += N_ds[t - radius + j] * Mc[j];
19             } else {
20                 Pvalue += N[N_index] * Mc[j];
21             }
22         }
23     }
24     if (i < Width) P[i] = Pvalue;
25 }

```

共享下标安全性。进入共享读取分支时，已有 $s \leq g < s+B$ ，因此 $0 \leq g-s < B$ ，等价于 $0 \leq t-r+j < B$ 。尽管表达式中含有减去半径，分支条件保证实际读取时下标合法。外层条件负责全局数组边界，内层条件负责选择数据来源，两者不能相互替代。

以 $s=4, B=4, r=2$ 的线程 $t=0$ 为例，它计算 $P[4]$ ，依次读取 $N[2]$ 到 $N[6]$ 。前两项不属于核心区，从全局地址读取；后三项属于核心区，从共享位置 0,1,2 读取。与此同时，计算 $P[6]$ 的线程可能在相同的循环迭代已经使用共享内存，因而同一 warp 内可能出现不同读取分支。

两种共享数组起点决定两种计算下标

策略 1 的共享位置 0 对应全局位置 $s-r$ ，因此读取下标为 $t+j$ 。策略 3 的共享位置 0 对应全局位置 s ，因此读取下标为 $t-r+j$ ，且仅在输入属于核心区时使用。这一差异完全来自共享数组表示的输入范围。

核心区载入同样需要检查边界。 $(i < \text{Width}) ? N[i] : 0.0f$ 在第一次全局读取之前处理了末尾块；仅在最后给 $P[i]$ 加条件，并不能保护此前已经执行的 $N[i]$ 读取。策略 3 与策略 1 都要完成共同载入和屏障，随后才进入各自的计算路径。

8.6 综合算例与关键结论

8.6.1 末尾不足一个完整块时的计算

沿用一维输入和滤波器，逐项分析末尾块的载入与计算。取 $W = 7$ 、 $B = 4$ 、 $K = 5$ 、 $r = 2$ ，输入 $N = [1, 2, 3, 4, 5, 6, 7]$ ，滤波器 $M = [3, 4, 5, 4, 3]$ 。网格需要两块，分别负责输出下标 0—3 与 4—7，其中 $P[7]$ 没有对应的有效输出位置。

第二块从 $s = 4$ 开始。策略 1 的共享数组覆盖全局位置 2 到 9，依次填入 $[3, 4, 5, 6, 7, 0, 0, 0]$ 。前五个值来自真实输入，后三个值按零填充规则生成。三段载入中，线程 $t = 3$ 虽然对应无效输出 $i = 7$ ，仍负责把 $N[3] = 4$ 放入共享位置 1，并把其核心位置写为零。

如果线程 $t = 3$ 在搬运前提前返回，共享位置 1 就没有被它初始化。有效线程 $t = 0$ 计算 $P[4]$ 时需要共享位置 0 到 4，其中就包含这一位置。因此有效输出会依赖未完成的搬运；只查看“哪些线程要写输出”无法判断哪些线程可以提前结束。

局部线程	输出位置	策略 1 读取的共享值	结果与写入
$t = 0$	$P[4]$	$[3, 4, 5, 6, 7]$	$9 + 16 + 25 + 24 + 21 = 95$
$t = 1$	$P[5]$	$[4, 5, 6, 7, 0]$	$12 + 20 + 30 + 28 = 90$
$t = 2$	$P[6]$	$[5, 6, 7, 0, 0]$	$15 + 24 + 35 = 74$
$t = 3$	无有效输出	$[6, 7, 0, 0, 0]$	可执行乘加，但不写 $P[7]$

表 8.4 末尾块的完整窗口；零值也占据明确的共享位置。

两轮载入写法中，第二块第一轮载入 $[N[2], N[3], N[4], N[5]]$ ，第二轮载入 $[N[6], 0, 0, 0]$ ，最终共享内容与三段写法相同。在不同写法中负责同一数据的线程可能不同，但只要式(8.9)成立，乘加结果就一致。

对策略 3，同一块的共享内容只有 $[N[4], N[5], N[6], 0] = [5, 6, 7, 0]$ 。计算 $P[4]$ 时从全局地址取得 $N[2], N[3]$ ，从共享数组取得 $N[4], N[5], N[6]$ 。不同数据放置方式由不同下标转换

衔接到同一个卷积窗口。

8.6.2 带边界条件的精确计数

对直接核函数，位置 i 的有效乘加项数就是窗口 $[i-r, i+r]$ 与数组 $[0, W-1]$ 的交集长度。将边界判断写成计数式，可得

$$v_i = \min(W-1, i+r) - \max(0, i-r) + 1, \quad 0 \leq i < W. \quad (8.10)$$

对于 $W=7, r=2$, v_i 依次为 3, 4, 5, 5, 5, 4, 3，总计 29。直接核函数每个有效项读一个输入和一个系数，因此一共执行 58 次标量读取，对应 232 B 的逻辑读入；有效输出写入 7 个 float，对应 28 B；乘法加法合计为 58 FLOPs。这些数字来自代码实际通过边界分支的项数。

若只比较策略 1 对输入 N 的请求，首块的输入覆盖 $[-2, 5]$ ，其中六个位置有效；第二块覆盖 $[2, 9]$ ，其中五个位置有效，共读入 11 个真实输入值，即 44 B。共享内存中总共写入 $2 \times 8 = 16$ 个位置，其余五个位置由线程直接写零。常量系数的读取和缓存事务没有包含在这 44 B 中。

完整输入块已在载入阶段填零，策略 1 的乘加循环就不再跳过零项。七个有效输出各执行五次乘加，需要 70 FLOPs；代码 8.5 与代码 8.6 采用全部线程计算的结构，八个已启动线程共执行 80 FLOPs，最后一个线程的结果不写出。减少输入请求可以同时增加零项或无效输出上的算术工作，这正是按计数对象分别核对的意义。

再考虑 $W=16, B=4, K=5$ 的布局。四个块的有效输入请求数依次为 6, 8, 8, 6，策略 1 合计请求 28 个输入值。直接核函数的有效输入使用次数为 $3+4+12 \times 5+4+3=74$ 。该例按整个网格统计的输入请求减少因子为 $74/28 \approx 2.64$ ，而式(8.8)的 2.5 比较的是一个远离边界的完整内部块。两者采用了不同的统计范围。

8.6.3 关键公式与实现关系

卷积计算、缓存与分块可以通过以下关系一起核对。公式中的 $K = 2r + 1$, B 表示每块输出数, $s = bB$ 表示输出块起点; 策略 2 若按输入块大小配置线程, 线程数应相应增加为 $B + 2r$ 。

对象	关系
输出与输入窗口	$P[i] = \sum_{j=0}^{K-1} \tilde{N}[i - r + j]M[j]$
一块所需输入数	$T_{in} = B + 2r = B + K - 1$
完整输入块的共享映射	$N_{ds}[q] = \tilde{N}[s - r + q]$
策略 1 的计算下标	$q = t + j$
核心区的共享映射	$N_{ds}[q] = \tilde{N}[s + q]$
策略 3 的共享计算下标	$q = t - r + j$, 仅当 $s \leq i - r + j < s + B$
内部块输入请求比较	直接法 BK ; 策略 1 为 $B + 2r$
策略 1 的线程覆盖条件	三段载入要求 $r \leq B$; 两轮载入要求 $2r \leq B$
共享内存容量	策略 1、2 为 $4(B + 2r)B$; 策略 3 为 $4B B$

表 8.5 卷积窗口、共享索引与资源用量。

从输出窗口出发确定完整输入范围, 再选择共享数组的起点, 即可推得载入与计算下标。常量存储器处理统一系数的读取, 共享内存处理邻域输入的复用; 二者连接到同一个卷积计算, 而优化的对象各不相同。